INSTITUTO FEDERAL FLUMINENSE
Curso de Engenharia de Controle e Automação

Sistemas SCADA e Supervisórios Industriais

Apostila Técnica Ilustrada

Supervisão Industrial, SCADA, HMI, IoT e IIoT

Versão 2.6
Atualizada com IIoT, edge computing, OPC UA, MQTT, Sparkplug, alarmes, dashboards, ThingsBoard, aquisição e historização de dados e analítica industrial

D.Sc. William da Silva Vianna

Licenciada sob Creative Commons Atribuição–CompartilhaIgual 4.0 Internacional

Esta apostila reúne os fundamentos e as tendências dos **Sistemas Supervisórios e SCADA** (*Supervisory Control and Data Acquisition*) aplicados à automação industrial. O material foi desenvolvido para oferecer uma progressão didática — do conceito à implementação — com linguagem acessível, diagramas explicativos e estudos de caso reais.

Os 20 capítulos estão organizados em quatro blocos:

- **Fundamentos (capítulos 1 a 4):** automação industrial, pirâmide de automação e níveis hierárquicos, sistemas supervisórios (SCADA e HMI), arquitetura e componentes.
- **Comunicação, integração e segurança (capítulos 5 a 9 e 13):** redes e protocolos industriais, IIoT, Node-RED, cibersegurança, Cloud SCADA e Digital Twin, OPC UA, MQTT e Sparkplug B.
- **Dados, alarmes e visualização (capítulos 10 a 12 e 14 a 16):** tags, variáveis, estados e eventos; aquisição e tratamento de dados; historiadores e séries temporais; gerenciamento de alarmes; dashboards; ThingsBoard.
- **Borda, nuvem e projeto (capítulos 17 a 20):** edge computing, integração SCADA + CLP + IoT + Cloud, projeto e implantação de um sistema supervisório, IA e analítica industrial.

As normas aparecem onde mudam a decisão de engenharia, não como apêndice teórico: o **gerenciamento de alarmes** segue ANSI/ISA-18.2, IEC 62682, EEMUA 191 e NAMUR NA 102, o **projeto de IHM de alta performance** segue a ISA-101, e a **cibersegurança industrial** se apoia na série IEC 62443.

Cada capítulo abre com objetivos de aprendizagem e fecha com questões de revisão — 159 no total. O corpo do texto traz 47 diagramas Mermaid, 64 tabelas legendadas e 79 posições de figura, 31 delas com reproduções do material de origem (telas de supervisão, arquiteturas de rede, painéis de alarmes, tendências e dashboards). Todos os 20 capítulos têm **estudo de caso**, com cenário, diagnóstico, decisão de engenharia e limitações do arranjo; os capítulos 10 a 13 trazem **atividades práticas**; e o **glossário técnico** em apêndice resolve a terminologia durante a leitura.

Ao final do percurso, o leitor deve ser capaz de especificar, integrar e operar um sistema supervisório completo — e de justificar cada escolha de arquitetura, protocolo, base de dados e política de alarmes.

Sumário

1 Introdução à Automação Industrial **1**
1.1 O que é Automação Industrial? . . . 1
1.2 Evolução Histórica . . . 2
1.3 Tipos de Processos Industriais . . . 3
1.3.1 Processos Contínuos . . . 3
1.3.2 Processos Discretos . . . 3
1.3.3 Processos em Batelada (*Batch*) . . . 4
1.4 Benefícios e Desafios da Automação . . . 4
1.4.1 Benefícios . . . 4
1.4.2 Desafios . . . 4
1.5 Visão Geral das Tecnologias de Automação . . . 5
1.6 Estudo de Caso — Da operação manual ao supervisório em uma estação de tratamento de água . . . 6
1.7 Resumo . . . 7
1.8 Questões de Revisão . . . 7
1.9 Referências . . . 8

2 Pirâmide de Automação e Níveis Hierárquicos **9**
2.1 A Pirâmide de Automação . . . 9
2.2 Nível 0 — Campo (Processo Físico) . . . 10
2.3 Nível 1 — Instrumentação de Campo . . . 10
2.3.1 Sensores e Transmissores . . . 10
2.3.2 Atuadores . . . 10
2.4 Níveis 1 e 2 — Detecção, atuação, monitoramento e controle . . . 11
2.4.1 CLP — Controlador Lógico Programável . . . 11
2.4.2 DCS — Sistema de Controle Distribuído . . . 11
2.4.3 RTU — Unidade Terminal Remota . . . 12
2.5 Nível 2 — Monitoramento e controle (inclui SCADA/HMI) . . . 12
2.6 Nível 3 — Operações de Manufatura (MES/MOM) . . . 13
2.7 Nível 4 — Planejamento e Logística do Negócio (ERP) . . . 13
2.8 Integração Vertical e Horizontal . . . 13
2.8.1 Integração Vertical . . . 13
2.8.2 Integração Horizontal . . . 14
2.9 A Pirâmide Revisitada: IIoT e a Convergência TI/OT . . . 14
2.10 Estudo de Caso — Duas plantas, duas formas de integrar . . . 15
2.11 Resumo . . . 16
2.12 Questões de Revisão . . . 16
2.13 Referências . . . 17

3 Sistemas Supervisórios — SCADA e HMI **18**
3.1 O que é SCADA? . . . 18
3.2 Funções Principais de um SCADA . . . 19
3.2.1 Aquisição de Dados . . . 20
3.2.2 Visualização (HMI Centralizado) . . . 20
3.2.3 Gerenciamento de Alarmes . . . 21
3.2.4 Controle Supervisório . . . 21
3.2.5 Historização . . . 21
3.2.6 Relatórios . . . 21
3.3 SCADA × DCS × HMI Local . . . 21
3.4 Gerações do SCADA . . . 22
3.5 Principais Plataformas SCADA . . . 24
3.5.1 Plataformas Comerciais . . . 24
3.5.2 Software livre e ferramentas adjacentes . . . 25
3.6 Requisitos para Implantação de um SCADA . . . 25
3.7 Estudo de Caso — Migração de um SCADA de geração 3 para geração 4 . . . 26
3.8 Resumo . . . 27
3.9 Questões de Revisão . . . 27
3.10 Referências . . . 28

4 Arquitetura e Componentes de um Sistema SCADA **29**
4.1 Componentes de uma Arquitetura SCADA . . . 29
4.2 RTU — Unidade Terminal Remota . . . 31
4.2.1 Módulo de I/O remoto não é necessariamente uma RTU . . . 31
4.2.2 RTU vs. CLP em telecontrole . . . 31
4.3 MTU — Unidade Terminal Mestre . . . 31
4.3.1 Redundância da MTU . . . 32
4.4 Servidor de Historização (Historiador) . . . 32
4.4.1 Por que não usar SQL convencional? . . . 32
4.4.2 Soluções de Historização . . . 33
4.5 Estações de Operação e Thin Clients . . . 33
4.6 Infraestrutura de Comunicação . . . 34
4.6.1 Meios Físicos . . . 34
4.6.2 Topologias de Rede . . . 34
4.7 Comparativo de Arquiteturas SCADA . . . 35
4.8 Estudo de Caso — SCADA para Sistema de Abastecimento de Água . . . 39
4.9 Resumo . . . 40
4.10 Questões de Revisão . . . 40
4.11 Referências . . . 41

5 Redes e Protocolos de Comunicação Industrial **42**
5.1 Redes Industriais versus Redes Corporativas . . . 42
5.2 Modelo OSI Aplicado à Automação . . . 43
5.3 Protocolos Clássicos . . . 44
5.3.1 Modbus . . . 44
5.3.2 Profibus . . . 45
5.3.3 DNP3 . . . 45
5.3.4 Foundation Fieldbus . . . 46
5.4 Protocolos Modernos . . . 46

5.4.1 OPC e OPC UA . . . 46
5.4.2 MQTT . . . 48
5.4.3 PROFINET . . . 49
5.4.4 EtherNet/IP . . . 49
5.5 Comparativo de Protocolos . . . 50
5.6 Estudo de Caso — Integração Modbus + OPC UA + MQTT . . . 50
5.7 Resumo . . . 52
5.8 Questões de Revisão . . . 52
5.9 Referências . . . 53

6 IIoT e Tecnologias Emergentes 54
6.1 IoT versus IIoT . . . 54
6.2 Arquitetura IIoT: Edge, Fog e Cloud . . . 55
6.2.1 Edge Computing . . . 57
6.2.2 Fog Computing . . . 57
6.3 Plataformas IIoT em Nuvem . . . 57
6.4 Digital Twin . . . 58
6.4.1 Tipos de Digital Twin . . . 58
6.4.2 Aplicações . . . 58
6.5 Analytics Industrial e Big Data . . . 59
6.5.1 Stack Típica de Analytics Industrial . . . 60
6.5.2 Machine Learning Aplicado à Indústria . . . 61
6.6 Estudo de Caso — IIoT em Planta de Celulose . . . 61
6.7 Resumo . . . 62
6.8 Questões de Revisão . . . 62
6.9 Referências . . . 62

7 Node-RED como Interface Supervisória 64
7.1 O que é Node-RED? . . . 64
7.2 Instalação e Configuração . . . 66
7.2.1 Instalação via npm (Linux / Raspberry Pi) . . . 66
7.2.2 Instalação via Docker . . . 66
7.2.3 Acessando o Editor . . . 66
7.3 Conceitos Fundamentais . . . 66
7.3.1 Nodes (Nós) . . . 66
7.3.2 Flows e Wires . . . 67
7.3.3 Context (Contexto) . . . 67
7.4 Aquisição de Dados via Modbus TCP . . . 67
7.4.1 Instalando o Node Modbus . . . 67
7.4.2 Flow: Leitura de Temperatura via Modbus TCP . . . 67
7.5 Comunicação MQTT no Node-RED . . . 68
7.5.1 Configurando Conexão com Broker MQTT . . . 68
7.5.2 Flow: Assinatura de Tópico MQTT e Alarme . . . 68
7.6 Dashboard — Interface de Supervisão Web . . . 69
7.6.1 Instalação . . . 69
7.6.2 Widgets Disponíveis . . . 69
7.6.3 Exemplo de Flow Completo com Dashboard . . . 70
7.7 Integração com InfluxDB e Grafana . . . 70
7.7.1 Instalação . . . 70

7.7.2 Node `influxdb out` . . . 71
7.8 Node-RED versus SCADA Comercial . . . 71
7.9 Estudo de Caso — Node-RED como camada de integração em uma planta-piloto . . . 72
7.10 Resumo . . . 73
7.11 Questões de Revisão . . . 73
7.12 Referências . . . 74

8 Segurança em Sistemas SCADA **75**
8.1 OT Security: Por que é Diferente? . . . 75
8.2 Linha do Tempo de Ataques a Infraestruturas Críticas . . . 76
8.3 Principais Vetores de Ataque . . . 76
8.4 Norma IEC 62443 . . . 77
8.4.1 Zonas de Segurança e Condutos . . . 77
8.4.2 Security Levels (SL) . . . 79
8.5 Boas Práticas de Segurança em SCADA . . . 79
8.5.1 Segmentação de Redes . . . 79
8.5.2 Controle de Acesso Remoto . . . 79
8.5.3 Gestão de Vulnerabilidades e Patches . . . 80
8.5.4 Backup e Recuperação . . . 80
8.5.5 Treinamento e Cultura de Segurança . . . 80
8.6 Monitoramento e Resposta a Incidentes (SOC OT) . . . 81
8.6.1 Monitoramento Passivo da Rede OT . . . 82
8.6.2 Plano de Resposta a Incidentes OT . . . 82
8.7 Estudo de Caso — Segmentação por zonas e condutos em uma planta de médio porte . . . 82
8.8 Resumo . . . 83
8.9 Questões de Revisão . . . 84
8.10 Referências . . . 84

9 Tendências: Cloud SCADA e Digital Twin **85**
9.1 Cloud SCADA: Conceito e Motivações . . . 85
9.1.1 Por que migrar para a nuvem? . . . 85
9.1.2 Arquitetura Cloud SCADA . . . 87
9.2 Plataformas de Cloud SCADA . . . 88
9.2.1 Ignition Cloud Edition (Inductive Automation) . . . 88
9.2.2 AWS IoT SiteWise . . . 88
9.2.3 Azure IoT Hub + Azure Digital Twins . . . 88
9.2.4 AVEVA Insight . . . 88
9.3 Digital Twin nos Sistemas SCADA . . . 89
9.3.1 Como o SCADA alimenta o Digital Twin . . . 89
9.3.2 Caso de Uso: Otimização de Destilação . . . 89
9.4 Inteligência Artificial Aplicada a Sistemas Supervisórios . . . 90
9.4.1 Manutenção Preditiva (PdM) . . . 90
9.4.2 Controle Avançado com IA . . . 90
9.4.3 Processamento de Linguagem Natural (NLP) em SCADA . . . 90
9.5 Low-Code / No-Code em Supervisórios . . . 90
9.6 O Futuro da Profissão: Convergência TI/OT . . . 91
9.7 Estudo de Caso — Piloto de Cloud SCADA em uma frota de elevatórias . 91

9.8 Resumo . . . 92
9.9 Questões de Revisão . . . 93
9.10 Referências . . . 93

10 Gerenciamento de Alarmes **95**
10.1 O problema que o gerenciamento de alarmes resolve . . . 95
10.2 Conceitos e taxonomia . . . 96
10.2.1 Prioridade e severidade não são a mesma coisa . . . 97
10.2.2 Ajustes que definem o comportamento . . . 97
10.3 Normas e o escopo de cada uma . . . 98
10.4 Ciclo de vida . . . 99
10.5 Racionalização na prática . . . 101
10.5.1 Um exemplo completo . . . 102
10.6 Apresentação ao operador e desempenho . . . 103
10.6.1 Como o alarme chega ao operador . . . 103
10.6.2 Métricas de desempenho . . . 104
10.7 Alarmes em SCADA e em plataformas IIoT . . . 105
10.7.1 No SCADA clássico . . . 105
10.7.2 Na borda e na plataforma . . . 106
10.8 Estudo de caso — alarmes de uma estação de bombeamento . . . 108
10.9 Atividade prática . . . 108
10.10Resumo . . . 109
10.11Questões de Revisão . . . 109
10.12Referências . . . 110

11 Dashboards e Visualização de Dados **111**
11.1 Do dado ao indicador . . . 111
11.2 Hierarquia de telas . . . 113
11.3 Tipos de painel e quando usar cada um . . . 115
11.3.1 Cor, escala e precisão . . . 116
11.4 Comparando plataformas de dashboard . . . 117
11.5 Grafana na prática . . . 118
11.5.1 Fonte de dados: InfluxDB . . . 118
11.5.2 Variáveis de dashboard . . . 119
11.5.3 Alertas: a limitação que pega todo mundo . . . 119
11.5.4 Provisionamento como código . . . 119
11.6 Estudo de caso — dashboard de pressão, temperatura, vazão e nível . . . 119
11.7 Atividade prática . . . 122
11.8 Resumo . . . 123
11.9 Questões de Revisão . . . 123
11.10Referências . . . 123

12 ThingsBoard **125**
12.1 O que é o ThingsBoard e onde ele se encaixa . . . 125
12.2 Modelo de entidades . . . 126
12.3 Telemetria e atributos . . . 127
12.3.1 MQTT: tópicos e formato . . . 128
12.3.2 HTTP e REST . . . 128
12.4 Rule engine . . . 129

12.4.1 Alarme como entidade . . . 131
12.5 RPC — comando e consulta . . . 132
12.6 Dashboards e visualização . . . 132
12.7 Integração com CLP e com Node-RED . . . 132
12.7.1 Direto do CLP . . . 132
12.7.2 Por gateway de borda . . . 133
12.7.3 Com Node-RED . . . 133
12.8 Estudo de caso — monitoramento de frota de transmissores . . . 134
12.9 Atividade prática . . . 134
12.10Resumo . . . 135
12.11Questões de Revisão . . . 135
12.12Referências . . . 136

13 OPC UA, MQTT e Sparkplug B **137**
13.1 O problema: conectar é fácil, entender é difícil . . . 137
13.2 OPC UA . . . 138
13.2.1 Modelo de informação . . . 139
13.2.2 Serviços . . . 140
13.2.3 Segurança . . . 140
13.2.4 Cliente-servidor e PubSub . . . 141
13.3 MQTT . . . 142
13.3.1 Tópicos e curingas . . . 142
13.3.2 Recursos que definem o comportamento . . . 142
13.4 Sparkplug B: MQTT com regras . . . 143
13.4.1 Estrutura de tópicos . . . 143
13.4.2 O que o Sparkplug **não** resolve . . . 144
13.5 Combinando as três peças . . . 145
13.6 Estudo de caso — integração de uma frota de gateways . . . 146
13.7 Atividade prática . . . 147
13.8 Resumo . . . 147
13.9 Questões de Revisão . . . 148
13.10Referências . . . 148

14 Tags, Variáveis, Estados e Eventos **150**
14.1 O tag é o contrato entre o campo e a tela . . . 150
14.2 Anatomia de um cadastro de tag . . . 152
14.3 Tipos, resolução e escalonamento . . . 153
14.3.1 Da faixa bruta à unidade de engenharia . . . 154
14.4 O tag na tela: representação de discretos e analógicos . . . 155
14.5 Amostragem, varredura e qualidade do dado . . . 156
14.5.1 Qualidade: o valor sozinho não basta . . . 156
14.6 Estados: quando o valor não conta a história . . . 157
14.7 Eventos e alarmes: a diferença que evita a sobrecarga . . . 159
14.8 Comparativo: tag clássico, nó OPC UA e métrica IoT . . . 161
14.9 Estudo de Caso — Mapa de tags de uma estação de bombeamento . . . 162
14.10Resumo . . . 163
14.11Questões de Revisão . . . 164
14.12Referências . . . 164

15 Aquisição e Tratamento de Dados **166**
15.1 A cadeia de aquisição . . . 166
15.2 Tipos de sinal e forma de conexão . . . 168
15.3 Digitalização: resolução, exatidão e ruído . . . 169
15.4 Varredura: quem decide o ritmo . . . 170
15.4.1 O evento que se perde entre as varreduras . . . 171
15.5 Tratamento: o que se faz com o dado antes de mostrá-lo . . . 171
15.6 Registro da primeira falha . . . 173
15.7 Comparativo das arquiteturas de aquisição . . . 175
15.8 Estudo de Caso — A planta que parou e a tela não mostrou por quê . . . 176
15.9 Resumo . . . 177
15.10Questões de Revisão . . . 177
15.11Referências . . . 178

16 Historiadores e Séries Temporais **179**
16.1 A pergunta que o histórico responde . . . 179
16.2 Tendência real e tendência histórica . . . 180
16.3 Os parâmetros que decidem o que você vai ver depois . . . 181
16.4 Por que banco relacional não é historiador . . . 182
16.5 Compressão: o que o histórico decide esquecer . . . 183
16.6 Retenção, agregação e conformidade . . . 184
16.7 Histórico de processo × histórico de eventos . . . 185
16.8 Comparativo das soluções de historização . . . 186
16.9 Estudo de Caso — O pico que o histórico não viu . . . 187
16.10Resumo . . . 188
16.11Questões de Revisão . . . 189
16.12Referências . . . 189

17 Edge Computing na Indústria **191**
17.1 Por que a borda existe . . . 191
17.2 O que uma borda faz — e o que acontece quando falta . . . 193
17.3 O gateway na prática . . . 193
17.3.1 Store and forward: o teste que separa projeto de improviso . . . 194
17.4 Reduzir na borda: filtro, agregação e evento . . . 195
17.5 Edge analytics: o que cabe e o que não cabe . . . 197
17.6 Operar uma frota de gateways . . . 198
17.7 Segurança e a borda . . . 199
17.8 Estudo de Caso — A borda que não guardava . . . 199
17.9 Resumo . . . 200
17.10Questões de Revisão . . . 200
17.11Referências . . . 201

18 Integração SCADA + CLP + IoT + Cloud **202**
18.1 Quatro mundos, um dado só . . . 202
18.2 Onde cada decisão deve morar . . . 203
18.3 A sequência de dados, passo a passo . . . 205
18.3.1 Válvula solenoide comandada por CLP . . . 205
18.3.2 CLP → SCADA . . . 206
18.3.3 CLP → MQTT → plataforma IIoT . . . 206

18.3.4 Sensor → Node-RED → MQTT → plataforma . . . 206
18.3.5 SCADA + historiador + dashboard . . . 207
18.4 Onde a integração costuma quebrar . . . 207
18.5 Como provar que a integração funciona . . . 208
18.6 Estudo de Caso — A integração que duplicou os alarmes . . . 208
18.7 Resumo . . . 209
18.8 Questões de Revisão . . . 210
18.9 Referências . . . 210

19 Projeto e Implantação de um Sistema Supervisório 212
19.1 A parte difícil não é a tela . . . 212
19.2 O ciclo de vida em sete fases . . . 212
19.3 O levantamento que antecede a tela . . . 214
19.4 Ergonomia: o operador trabalha na tela . . . 216
19.5 Hierarquia de navegação e layout . . . 217
19.6 Implementação, testes e aceitação . . . 218
19.7 Manutenção: onde o projeto se prova . . . 219
19.8 Estudo de Caso — A IHM que a operação rejeitou . . . 219
19.9 Resumo . . . 220
19.10Questões de Revisão . . . 221
19.11Referências . . . 221

20 IA e Analítica Industrial 223
20.1 O que a indústria realmente pede à IA . . . 223
20.2 O pré-requisito: dado com contexto . . . 225
20.3 Manutenção preditiva . . . 225
20.4 Detecção de anomalia . . . 226
20.5 Sensor virtual e controle avançado . . . 228
20.6 O ciclo de vida do modelo em operação . . . 228
20.7 Limites e riscos . . . 229
20.8 Estudo de Caso — O modelo que acertava e ninguém usava . . . 230
20.9 Resumo . . . 231
20.10Questões de Revisão . . . 231
20.11Referências . . . 231

A Glossário Técnico — Sistemas SCADA e Supervisórios Industriais 233
A.1 A . . . 233
A.2 B . . . 234
A.3 C . . . 234
A.4 D . . . 235
A.5 E . . . 236
A.6 F . . . 236
A.7 G . . . 237
A.8 H . . . 237
A.9 I . . . 238
A.10 J–L . . . 239
A.11 M . . . 239
A.12 N . . . 240
A.13 O . . . 240

A.14 P–Q 241
A.15 R 241
A.16 S 242
A.17 T 243
A.18 V–Z 244

Referências **245**

Lista de Figuras

Figura 2.5 – Pirâmide tradicional de automação industrial 15

Figura 3.1 – Janela de IHM de um sistema supervisório 19
Figura 3.3 – Sala de controle com painel de instrumentos convencionais . . 23

Figura 4.1 – Arquitetura básica de um sistema SCADA 30
Figura 4.3 – Arquitetura de rede de campo com PROFIBUS e supervisão remota 34
Figura 4.4 – Arquitetura SCADA com uso de CLP 37
Figura 4.5 – Arquitetura com controladores single-loop e multi-loop 38
Figura 4.6 – Arquitetura com sistema de aquisição de dados 38
Figura 4.7 – Tempo de resposta de um sistema SCADA 39

Figura 5.2 – Métodos de acesso ao meio: mestre-escravo e token passing . . 45
Figura 5.3 – Arquitetura com Foundation Fieldbus 46
Figura 5.6 – Integração de múltiplos protocolos em um servidor de supervisão 52

Figura 7.1 – Interface do editor Node-RED 65
Figura 7.3 – Dashboard Node-RED: painel de supervisão 70

Figura 10.3 – Sinalização de estado no sinótico, com o alarme fora da tela de processo 103
Figura 10.4 – Diagrama de estados de uma variável de alarme 105
Figura 10.5 – Registro de eventos de alarme com estado, prioridade, valor e limite 106

Figura 11.1 – Navegação entre telas: barra de navegação e link horizontal no sinótico 115
Figura 11.2 – Tendência de processo com escala fixa e legenda da variável . 116
Figura 11.3 – Fluxograma P&I do processo usado no estudo de caso 122

Figura 13.1 – Arquitetura lógica do OPC clássico: servidor, grupos e itens 139

Figura 14.1 – Dicionário de tagnames: cadastro com tipo, contexto, modo de leitura e ponto inicial 153
Figura 14.2 – Representação de uma variável discreta: a mesma bomba ligada e desligada 155
Figura 14.3 – Taxa de amostragem inadequada: a curva plotada difere da variação real 156
Figura 14.4 – Eventos de processo, de operação e de sistema que disparam scripts 159

Figura 15.1 – Tempo de resposta de uma tarefa: o intervalo entre a solicitação e a resposta realizada 170
Figura 15.3 – Lógica em Ladder do registrador de primeiro evento, com bit de bloqueio e blocos Move 174

Figura 16.1 – Tendência real com duas variáveis de nível, escala fixa e legenda 180
Figura 16.2 – Parâmetros de configuração de um gráfico de tendência . . . 181

Figura 18.1 – Camada de abstração: onde o software de supervisão se separa do hardware de controle 205

Figura 19.2 – Gerenciador de aplicações: criação e abertura de um projeto de supervisão 218
Figura 19.3 – Criação de uma janela e definição de suas propriedades . . . 218

Lista de Tabelas

Capítulo 1

Introdução à Automação Industrial

Objetivos de Aprendizagem

Ao final deste capítulo, o leitor será capaz de:

- Compreender o conceito de automação industrial e sua evolução histórica.
- Distinguir os tipos de processos industriais: contínuos, discretos e em batelada.
- Identificar os principais benefícios e desafios da automação.
- Reconhecer as tecnologias que compõem um sistema automatizado moderno.

1.1 O que é Automação Industrial?

Automação industrial é o conjunto de tecnologias, equipamentos e sistemas que permitem realizar processos produtivos com o mínimo de intervenção humana direta. Não se trata apenas de substituir mão de obra: o objetivo principal é aumentar a **produtividade**, melhorar a **qualidade**, garantir a **segurança operacional** e reduzir desperdícios.

Um sistema automatizado é composto, em essência, por três elementos fundamentais:

1. **Sensores** — capturam variáveis do processo (temperatura, pressão, nível, vazão, posição).
2. **Controladores** — processam as informações e tomam decisões (CLPs, DCS, microcontroladores).
3. **Atuadores** — executam as ações de controle (válvulas, motores, relés, inversores de frequência).

Nota

Nota: A palavra "automação" vem do grego *automatos* ("que age por si mesmo"). O conceito, no entanto, é mais amplo: sistemas automatizados não agem de forma totalmente autônoma — eles seguem lógicas de controle programadas por engenheiros.

Figura

[**Figura 1.1 – Elementos de um sistema automatizado**] *Diagrama de blocos simples mostrando o ciclo: Processo → Sensores → Controlador → Atuadores → Processo. Usar cores distintas para cada bloco. Incluir exemplos concretos em cada*

elemento (ex: sensor de temperatura PT-100, CLP Siemens S7-1200, válvula de controle Fisher).

1.2 Evolução Histórica

A automação industrial passou por ondas de transformação que se aceleraram progressivamente:

Era	Período	Tecnologia-Chave	Marco
Mecânica	séc. XVIII–XIX	Máquinas a vapor, teares automáticos	Revolução Industrial (1ª)
Eletromecânica	1870–1960	Relés, contatores, motores elétricos	Linha de montagem Ford (1913)
Eletrônica	1960–1980	Transistores, CIs, primeiros CLPs	1º CLP: Modicon 084 (1969)
Digital	1980–2000	Microprocessadores, redes industriais, SCADA	Sistemas DCS (Honeywell TDC 2000, 1975)
Conectada	2000–2015	Ethernet industrial, OPC, MES, ERP	Integração TI/OT
Inteligente (Industry 4.0)	2015–hoje	IIoT, Cloud, IA, Digital Twin, 5G	Plataformas de analytics industriais

Figura

[Figura 1.2 – Linha do tempo da automação industrial] *Infográfico horizontal com os marcos históricos acima. Cada era representada por uma cor e um ícone representativo. Destacar o surgimento do CLP (1969), do SCADA e do IIoT como pontos de inflexão.*

O primeiro **CLP** (*Controlador Lógico Programável*) comercial — o Modicon 084 — foi desenvolvido em 1969 por Dick Morley para substituir painéis de relés na indústria automobilística. A adoção do CLP marcou o início da era digital na automação de chão de fábrica.

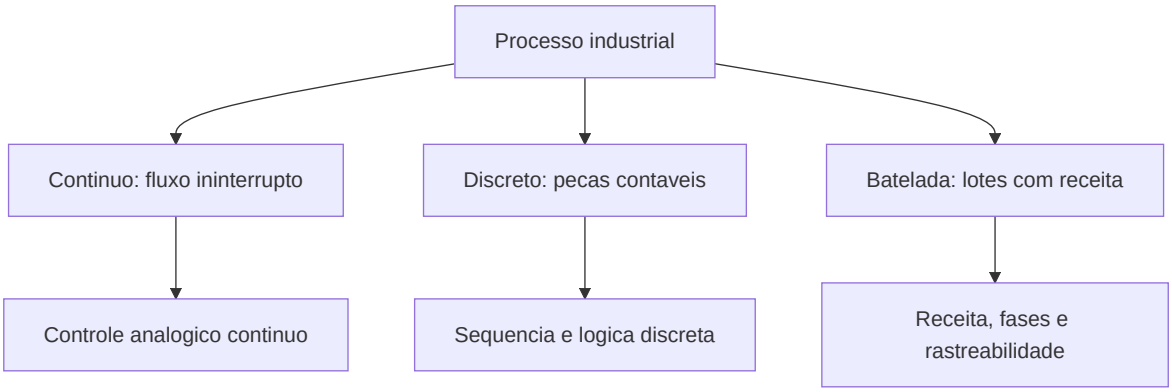
A partir dos anos 1980, sistemas **SCADA** (*Supervisory Control and Data Acquisition*) emergiram para centralizar a supervisão de grandes instalações — usinas, oleodutos, subestações — onde os operadores precisavam monitorar centenas de variáveis simultaneamente a partir de uma sala de controle.

Dica

Dica: Ao estudar a evolução da automação, observe que cada onda tecnológica não substituiu a anterior — ela a complementou. CLPs modernos ainda convivem com relés de segurança; sistemas SCADA legados coexistem com plataformas IIoT na mesma planta.

1.3 Tipos de Processos Industriais

Compreender o tipo de processo é fundamental para escolher a tecnologia de automação adequada. A escolha entre CLP, DCS e sistema supervisório começa aqui.



1.3.1 Processos Contínuos

Em processos **contínuos**, o produto flui ininterruptamente durante a operação. As variáveis de controle (temperatura, pressão, vazão, composição) variam continuamente e precisam ser mantidas dentro de faixas precisas.

Exemplos: refinarias de petróleo, usinas de papel e celulose, indústria química, tratamento de água e efluentes, estações de bombeamento.

Tecnologia típica: DCS (*Distributed Control System*), CLP com módulos analógicos (AI/AO), controladores PID, instrumentação com sinais 4–20 mA ou digitais (HART, PROFIBUS PA).

Nota

Nota: Historicamente, DCS era a escolha padrão para processos contínuos devido à capacidade nativa de lidar com loops analógicos complexos. Contudo, CLPs modernos (Siemens S7-1200/1500, Allen-Bradley CompactLogix, Schneider M221) também implementam controle PID e leitura de sinais analógicos com excelente precisão. A escolha entre CLP e DCS hoje depende mais da complexidade da rede, redundância exigida e integração com níveis superiores (MES/ERP) do que do tipo de processo.

1.3.2 Processos Discretos

Processos **discretos** produzem unidades individuais e contáveis. As operações são sequenciais: peças passam por etapas definidas até completar o produto.

Exemplos: montagem de veículos, fabricação de eletrônicos, embalagem, usinagem CNC.

Tecnologia típica: CLP (*Controlador Lógico Programável*), robótica industrial, visão computacional.

1.3.3 Processos em Batelada (*Batch*)

Processos **em batelada** combinam características dos dois anteriores: lotes de produto passam por etapas sequenciais em equipamentos compartilhados. A norma **ISA-88** define os modelos de receita e controle para esses processos.

Exemplos: indústria farmacêutica, fabricação de tintas, produção de cerveja e alimentos.

Tecnologia típica: DCS ou CLP com módulos batch, MES (*Manufacturing Execution System*).

Figura

[**Figura 1.3 – Comparativo dos tipos de processo industrial**] *Três ilustrações lado a lado: (1) fluxograma de uma refinaria e de uma estação de bombeamento (contínuo — ambos podem usar CLP ou DCS), (2) linha de montagem automotiva (discreto — CLP + robótica), (3) reator de batelada farmacêutico (batelada — CLP/DCS com sequenciamento ISA-88). Abaixo de cada uma, os atributos: variável de controle, tecnologia típica (destacando que contínuo pode ser CLP ou DCS), norma de referência.*

1.4 Benefícios e Desafios da Automação

1.4.1 Benefícios

- **Produtividade:** operação 24/7 sem pausas, maior velocidade de produção.
- **Qualidade e repetibilidade:** eliminação de variações humanas no processo.
- **Segurança:** afastamento do operador de ambientes insalubres ou perigosos (altas temperaturas, produtos tóxicos, explosivos).
- **Rastreabilidade:** registro histórico de todas as variáveis e eventos do processo.
- **Eficiência energética:** controle preciso reduz consumo de energia e insumos.

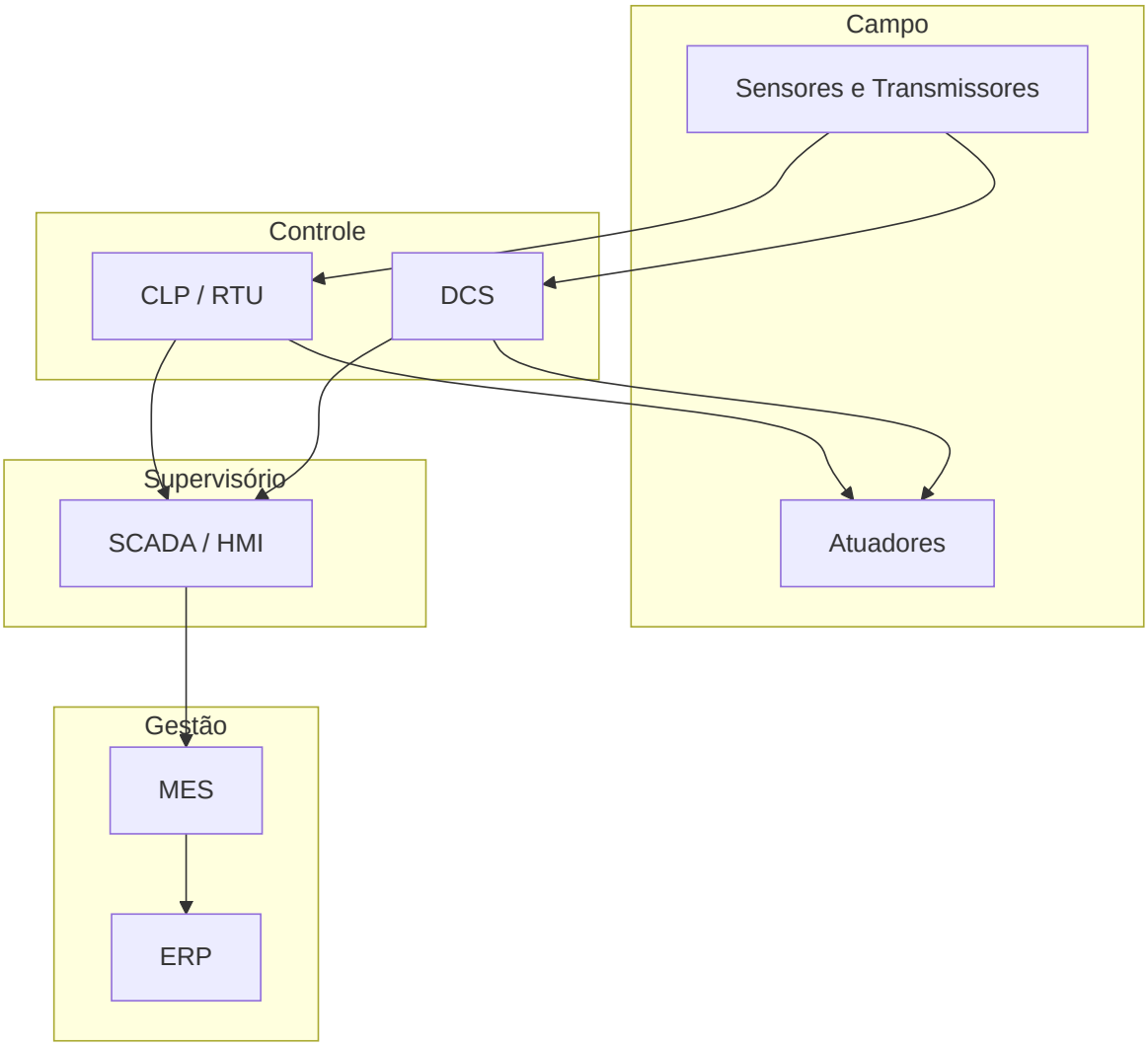
1.4.2 Desafios

- **Custo de implantação:** projetos de automação exigem investimento significativo em hardware, software e engenharia.
- **Qualificação da mão de obra:** demanda profissionais treinados em programação, redes e instrumentação.
- **Cibersegurança:** sistemas conectados estão expostos a ameaças digitais (ver Capítulo 8).
- **Integração de sistemas legados:** muitas plantas possuem equipamentos antigos sem interfaces digitais modernas.

Atenção

Atenção: A automação não elimina a necessidade do engenheiro de processo. Ela amplifica sua capacidade de monitorar e otimizar, mas a definição da lógica de controle, das condições de segurança e dos limites operacionais continua sendo responsabilidade humana.

1.5 Visão Geral das Tecnologias de Automação



Figura

[Figura 1.4 – Ecossistema de tecnologias de automação industrial] *Versão ilustrada e colorida do diagrama acima, com ícones representativos para cada tecnologia. Usar a hierarquia de baixo para cima: campo → controle → supervisório → gestão. Adicionar exemplos de produtos reais em cada nível.*

Tabela 1.1 – Principais tecnologias de automação e suas funções

Tecnologia	Sigla	Função Principal	Exemplos de Fabricantes
Controlador Lógico Programável	CLP	Controle de sequências lógicas e discretas	Siemens, Allen-Bradley, Schneider
Sistema de Controle Distribuído	DCS	Controle de processos contínuos	Honeywell, ABB, Yokogawa

Tecnologia	Sigla	Função Principal	Exemplos de Fabricantes
Unidade Terminal Remota	RTU	Aquisição de dados em campo remoto	Emerson, SEL, Advantech
Interface Homem-Máquina	HMI	Visualização local do processo	Siemens, Weintek, Proface
Supervisório / SCADA	SCADA	Supervisão centralizada de múltiplos pontos	AVEVA, Ignition, Wonderware
Sistema de Execução da Manufatura	MES	Gestão e rastreabilidade da produção	SAP ME, Rockwell Plex
Planejamento de Recursos Empresariais	ERP	Gestão financeira e logística	SAP, Oracle, TOTVS

1.6 Estudo de Caso — Da operação manual ao supervisório em uma estação de tratamento de água

Cenário. Estação de tratamento de água de uma cidade de 180 mil habitantes, com vazão de projeto de 250 L/s. A operação é manual: a dosagem de coagulante é ajustada por tentativa, as válvulas são acionadas no local, o nível do reservatório é lido em régua e o turno é registrado em planilha.

Diagnóstico. Três problemas dominam a rotina:

1. **Variabilidade da qualidade.** A dosagem deveria acompanhar a turbidez da água bruta, que varia com a chuva. Sem medição contínua, a correção chega tarde e o coagulante acaba aplicado em excesso "por segurança".
2. **Consumo de energia.** As bombas de recalque são acionadas por horário fixo, não pelo nível do reservatório — há partidas desnecessárias em horário de ponta.
3. **Sem rastro para investigar.** Quando o reservatório baixa, não existe registro de quem operou o quê nem de quando a bomba partiu.

Solução por ondas. A automação não é um projeto único: entra em etapas, e cada etapa resolve um problema específico. Nenhuma etapa depende da seguinte para dar resultado.

Onda	O que entra	Nível da pirâmide	Problema resolvido
1	Sensores de turbidez, nível e vazão, com CLP na casa de química	0 e 1	Medição contínua em vez de amostragem manual

Onda	O que entra	Nível da pirâmide	Problema resolvido
2	Malha de dosagem corrigida pela turbidez	2	Dosagem proporcional, sem excesso fixo
3	Supervisório com sinóticos, alarmes e histórico	3	Operação centralizada e rastreabilidade
4	Indicadores de consumo específico para a gestão	4	Decisão sobre horário de bombeamento

O que observar no resultado. A primeira onda já muda a operação **antes de existir qualquer tela**: passa a existir dado confiável. Projetos de automação costumam começar pela interface e pela promessa visual, quando a ordem correta é instrumentar, controlar e só então supervisionar.

Limitações do arranjo. Analisadores de turbidez exigem limpeza e calibração periódicas: sem plano de manutenção, o dado "confiável" deixa de ser confiável em poucos meses — e um supervisório alimentado por sensor sujo é pior do que nenhum, porque produz confiança injustificada. Além disso, a automação não substitui a decisão sobre a qualidade da água tratada, que permanece com quem opera a estação.

1.7 Resumo

Neste capítulo foram abordados:

- **Automação industrial** é o uso de tecnologias para executar processos com mínima intervenção humana, tendo como pilares sensores, controladores e atuadores.
- A evolução histórica vai das máquinas a vapor (séc. XVIII) à Industry 4.0, com o primeiro CLP lançado em 1969.
- Os **três tipos de processo** — contínuo, discreto e batelada — definem a escolha da tecnologia de controle.
- Os benefícios da automação incluem produtividade, qualidade, segurança e rastreabilidade; os desafios envolvem custo, qualificação e cibersegurança.
- O ecossistema tecnológico vai do nível de campo (sensores/atuadores) ao nível de gestão (ERP), passando por controle, supervisório e MES.

1.8 Questões de Revisão

Conceituais:

1. Quais são os três elementos fundamentais de qualquer sistema automatizado? Explique a função de cada um com um exemplo concreto.

2. Qual é a diferença entre um processo contínuo e um processo em batelada? Cite uma indústria típica de cada tipo.
3. Por que o surgimento do CLP em 1969 foi um marco na história da automação?

Práticas:

4. Uma indústria farmacêutica deseja automatizar a produção de comprimidos, onde lotes passam por mistura, compressão e revestimento em equipamentos compartilhados. Qual tipo de processo e qual tecnologia de controle você recomendaria? Justifique.
5. Liste três desafios que uma empresa enfrentaria ao automatizar uma planta que atualmente opera de forma manual. Proponha ações para mitigar cada desafio.

Desafio:

6. Pesquise o conceito de "Industry 4.0" e identifique pelo menos três tecnologias abordadas nesta apostila que são consideradas pilares dessa revolução industrial. Como elas se inter-relacionam?

1.9 Referências

- GROOVER, M. P. *Automação Industrial e Sistemas de Manufatura*. 3. ed. São Paulo: Pearson, 2011.
- ISA-5.1: *Instrumentation Symbols and Identification*. ISA, 2009.
- NATIONAL INSTRUMENTS. *What is Industrial Automation?* Disponível em: ni.com. Acesso: 2024.
- SIEMENS. *History of Automation*. Disponível em: `siemens.com/global/en/markets/machinebuilding/automation.html`.

Capítulo 2

Pirâmide de Automação e Níveis Hierárquicos

Objetivos de Aprendizagem

Ao final deste capítulo, o leitor será capaz de:

- Descrever os cinco níveis funcionais (0 a 4) do modelo ISA-95 e reconhecer extensões didáticas da pirâmide.
- Identificar as tecnologias e funções de cada nível.
- Compreender como se dá a integração vertical e horizontal entre sistemas.
- Analisar como o IIoT está transformando a hierarquia tradicional.

2.1 A Pirâmide de Automação

A **pirâmide de automação** é uma representação didática das funções industriais e de negócio. O modelo **ISA-95** (*Enterprise-Control System Integration*) define os níveis 0 a 4: processo físico, detecção/atuação, monitoramento e controle, operações de manufatura e planejamento/logística do negócio. Algumas representações acrescentam um nível corporativo acima do nível 4; essa extensão não deve ser apresentada como um nível adicional definido pela ISA-95.

Cada nível possui:

- **Função específica** no processo produtivo.
- **Velocidade de resposta** diferente: quanto mais próximo do campo, mais rápido deve ser o controle.
- **Volume de dados** processados: os níveis superiores lidam com informações consolidadas; os inferiores, com sinais brutos em tempo real.

Figura

[**Figura 2.1 – Pirâmide de Automação Clássica (ISA-95)**] *Pirâmide de cinco níveis, numerados de 0 a 4 conforme ISA-95: processo físico; sensores e atuadores; monitoramento e controle (CLP, DCS, SCADA conforme a função); operações de manufatura (MES/MOM); planejamento e logística do negócio (ERP). Mostrar a*

direção da integração sem sugerir que todo produto pertence exclusivamente a um único nível. Não incluir um nível 5 como se fosse parte da ISA-95.

2.2 Nível 0 — Campo (Processo Físico)

O **Nível 0** é a base da pirâmide: o processo físico em si — os equipamentos, tubulações, motores, reatores e as variáveis físicas que precisam ser medidas e controladas.

Componentes típicos: - Variáveis: temperatura, pressão, vazão, nível, posição, velocidade. - Equipamentos: bombas, compressores, válvulas, correias transportadoras, fornos.

Nota

Nota: Tecnicamente, o "Nível 0" representa o processo e não contém nenhum componente eletrônico de automação. Ele é o objeto a ser controlado pelos demais níveis.

2.3 Nível 1 — Instrumentação de Campo

O **Nível 1** engloba todos os dispositivos que fazem a interface direta com o processo: **sensores** (que medem variáveis) e **atuadores** (que executam ações de controle).

2.3.1 Sensores e Transmissores

Variável	Princípio de Medição	Exemplos de Sensores
Temperatura	Resistência elétrica (RTD), termopar (EMF)	PT-100, termopar tipo K/J
Pressão	Célula capacitiva, extensométrica	Rosemount 3051, Endress+Hauser PMC
Vazão	Pressão diferencial, efeito Coriolis, eletromagnético	Yokogawa ADMAG, Emerson Micro Motion
Nível	Ultrassônico, radar, pressão hidrostática	VEGA VEGAPULS, Siemens Sitrans LR
Posição/Velocidade	Encoder, resolver, sensor indutivo	Heidenhain, Balluff

2.3.2 Atuadores

- **Válvulas de controle:** ajustam vazão de fluidos (ex.: Fisher, Metso).
- **Motores elétricos:** acionados por inversores de frequência (ex.: WEG, ABB ACS).
- **Relés e contatores:** acionamento de cargas elétricas.
- **Sistemas pneumáticos:** cilindros e válvulas solenoides.

Figura

[**Figura 2.2 – Exemplos de instrumentos de campo**] *Montagem fotográfica com: (a) transmissor de pressão diferencial Rosemount; (b) válvula de controle com posicionador; (c) sensor de nível radar; (d) encoder industrial. Legendas indicando variável medida e protocolo de comunicação (4-20 mA, HART, Profibus PA).*

Dica

Dica: O protocolo **HART** (*Highway Addressable Remote Transducer*) permite a comunicação digital bidirecional sobre o sinal analógico 4-20 mA. Isso possibilita configurar remotamente e obter diagnósticos de um transmissor sem interromper o sinal de controle.

2.4 Níveis 1 e 2 — Detecção, atuação, monitoramento e controle

No **Nível 1**, os dispositivos fazem a interface com o processo e executam funções básicas de detecção e atuação. No **Nível 2**, sistemas de controle e supervisão monitoram o processo e executam ou apoiam o controle automático. A alocação exata de CLP, DCS, IHM e SCADA depende da arquitetura; a pirâmide representa funções, não uma regra física de localização.

2.4.1 CLP — Controlador Lógico Programável

O **CLP** é o equipamento dominante na automação discreta e em batelada. Suas características:

- Programação em linguagens da norma **IEC 61131-3**: Ladder Diagram (LD), Function Block Diagram (FBD), Structured Text (ST) e Sequential Function Chart (SFC). A Instruction List (IL), presente em edições antigas, foi removida da 4ª edição publicada em 2025; projetos legados ainda podem encontrá-la.
- Alta confiabilidade e robustez para ambientes industriais.
- Ciclos de varredura da ordem de **1 a 100 ms**.
- Comunicação via Modbus, Profibus, Profinet, EtherNet/IP, OPC UA.

Principais fabricantes: Siemens (S7-1200/1500), Allen-Bradley (ControlLogix), Schneider (Modicon M340/M580), Mitsubishi (MELSEC iQ-R).

2.4.2 DCS — Sistema de Controle Distribuído

O **DCS** (*Distributed Control System*) é projetado para processos contínuos de grande escala. Suas características:

- Controle distribuído em múltiplos controladores de campo, com banco de dados centralizado.
- Algoritmos PID (*Proportional-Integral-Derivative*) otimizados para controle de processo.

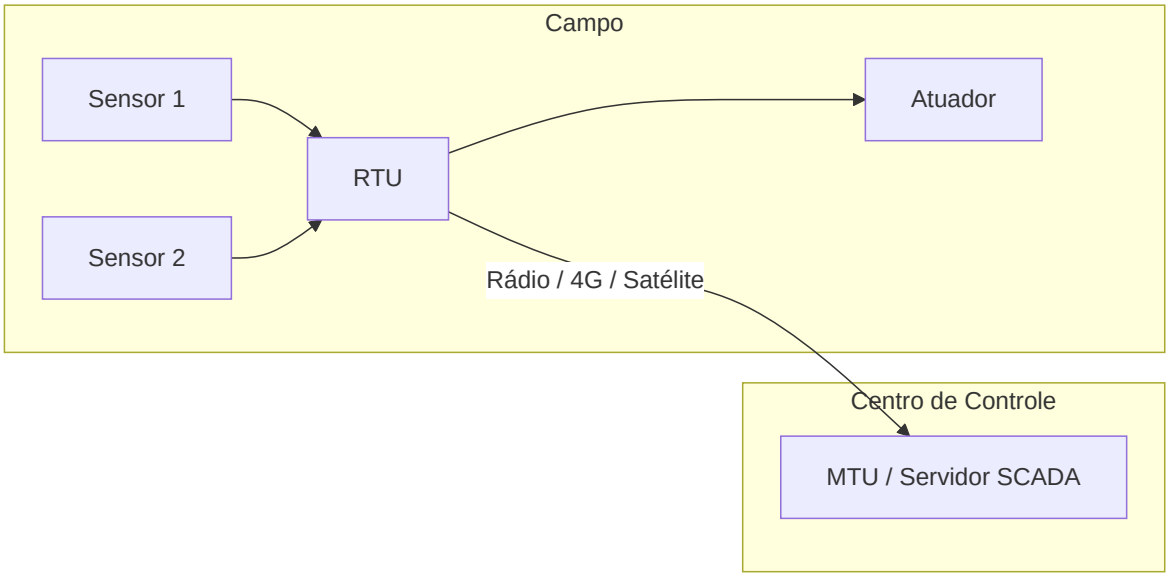
- Alta disponibilidade: arquitetura redundante (CPU, rede, fonte de alimentação).
- Integração nativa com instrumentação de campo (Foundation Fieldbus, HART).

Principais fabricantes: Honeywell (Experion PKS), ABB (Ability System 800xA), Yokogawa (CENTUM VP), Emerson (DeltaV).

2.4.3 RTU — Unidade Terminal Remota

A **RTU** (*Remote Terminal Unit*) é usada em aplicações geograficamente distribuídas, onde não há rede elétrica confiável ou conectividade local. Possui baixo consumo energético e comunicação via rádio, GPRS, 4G ou satélite.

Aplicações típicas: oleodutos, gasodutos, subestações de energia, estações de bombeamento de água, telemetria agrícola.



Figura

[**Figura 2.3 – Arquitetura RTU em campo remoto**] *Diagrama mostrando: campo remoto com RTU conectada a sensores e atuadores; link de comunicação sem fio (ícone de antena ou satélite) até o centro de controle com servidor SCADA. Exemplo real: estação de bombeamento de água em área rural.*

2.5 Nível 2 — Monitoramento e controle (inclui SCADA/HMI)

O **Nível 2** inclui funções de monitoramento e supervisão. Em muitas arquiteturas, é onde operam sistemas SCADA e estações de operação; controladores locais executam as malhas e sequências que precisam continuar funcionando sem depender da interface supervisória.

Essas funções são abordadas em detalhes nos Capítulos 3 e 4. Em resumo:

- **HMI local:** painel com tela touchscreen instalado próximo ao equipamento.

- **SCADA centralizado:** servidor com banco de dados, servidor de alarmes, cliente web e thin clients espalhados pela planta ou sala de controle.

2.6 Nível 3 — Operações de Manufatura (MES/MOM)

O **MES** (*Manufacturing Execution System*) ou **MOM** (*Manufacturing Operations Management*) coordena e registra as operações de manufatura, fazendo a interface entre o controle da produção e as funções de negócio. Suas funções incluem:

- **Rastreabilidade:** registro do histórico completo de cada lote produzido.
- **Qualidade:** coleta de dados de qualidade e gestão de não-conformidades.
- **Programação:** sequenciamento de ordens de produção.
- **OEE** (*Overall Equipment Effectiveness*): cálculo de disponibilidade, desempenho e qualidade dos equipamentos.
- **Gestão de energia:** monitoramento e otimização do consumo energético.

Exemplos: SAP ME, Rockwell Plex, Siemens Opcenter, TOTVS.

2.7 Nível 4 — Planejamento e Logística do Negócio (ERP)

O **ERP** (*Enterprise Resource Planning*) integra funções de negócio como finanças, recursos humanos, logística, vendas e planejamento. Troca informações com sistemas de operações de manufatura; essa integração não exige que todo dado de processo seja enviado diretamente ao ERP.

Integração ERP MES/MOM: a ISA-95 define modelos e atividades para a integração entre o domínio de negócio (Nível 4) e as operações de manufatura (Nível 3), além de terminologia e modelos relacionados.

2.8 Integração Vertical e Horizontal

2.8.1 Integração Vertical

A **integração vertical** conecta os níveis hierárquicos — do sensor ao ERP — permitindo que dados de processo sejam visíveis em tempo real para a gestão e que decisões corporativas influenciem a programação da produção.

Figura

[**Figura 2.4 – Integração vertical na pirâmide de automação**] *Seta bidirecional percorrendo todos os cinco níveis da pirâmide. Exemplos de dados que fluem para cima (telemetria, OEE, consumo) e para baixo (ordens de produção, receitas, metas de qualidade).*

2.8.2 Integração Horizontal

A **integração horizontal** conecta sistemas do mesmo nível — por exemplo, diferentes células de produção, linhas de manufatura ou plantas geograficamente distribuídas — compartilhando dados em tempo real.

Protocolos de integração comuns: OPC UA (nativo), REST/HTTP, banco de dados compartilhado, MQTT com broker centralizado.

2.9 A Pirâmide Revisitada: IIoT e a Convergência TI/OT

O modelo tradicional de pirâmide foi concebido em uma época em que os sistemas de **TI** (*Tecnologia da Informação*) e **OT** (*Operational Technology*) eram completamente separados. Com o advento do **IIoT** (*Industrial Internet of Things*), essa separação está sendo eliminada.

Principais mudanças:

- **Edge Computing:** processamento de dados ocorre diretamente em gateways de campo, sem passar pelos níveis intermediários.
- **Cloud SCADA:** o nível supervisório pode ser hospedado na nuvem, acessível de qualquer lugar.
- **Acesso direto sensor → nuvem:** dispositivos IIoT publicam dados diretamente em plataformas como AWS IoT ou Azure IoT Hub via MQTT, "pulando" os níveis da pirâmide.
- **Analytics e IA:** algoritmos de aprendizado de máquina operam sobre dados históricos de processo, retroalimentando decisões de controle.

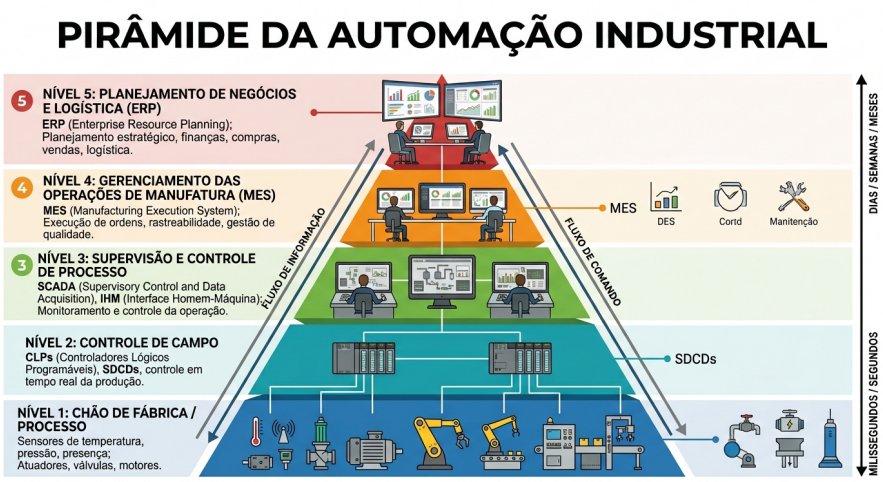


Figura 2.5 – Pirâmide tradicional de automação industrial. **Fonte:** arquivo `piramide_automacao.jpeg` fornecido no repositório; autoria e fonte original não informadas.

Nota

Nota: a imagem representa uma convenção tradicional em cinco níveis (1 a 5), não uma representação direta dos níveis ISA-95 (0 a 4) definidos neste capítulo. Ela não representa a arquitetura IIoT.

Atenção

Atenção: A convergência TI/OT traz desafios significativos de **cibersegurança**. Sistemas de controle que antes eram isolados fisicamente (*air gap*) passam a ter conectividade com redes externas, aumentando a superfície de ataque. Este tema é aprofundado no Capítulo 8.

2.10 Estudo de Caso — Duas plantas, duas formas de integrar

Cenário. Duas unidades do mesmo grupo industrial. A **planta A** opera desde 1998: controladores de gerações diferentes, barramento PROFIBUS no chão de fábrica, supervisório instalado em 2006 e um historiador que só recebe dados do supervisório. A **planta B** entrou em operação recentemente, com Ethernet industrial em toda a fábrica e *gateways* que publicam dados diretamente na nuvem.

O que a pirâmide explica bem — nas duas plantas.

- **Integração vertical** entre os níveis 3 e 4: a ordem de produção desce, o resultado da produção sobe.
- **Separação de responsabilidade**: quem controla o quê e com qual requisito de tempo.
- **Vocabulário comum**: quando alguém diz "nível 2", o escopo é o mesmo em qualquer unidade.

Onde a pirâmide deixa de descrever o que existe — na planta B.

Comportamento observado na planta B	Leitura pela pirâmide	Por que a leitura falha
Sensor de vibração publica na nuvem sem passar pelo CLP	Caminho direto entre campo e serviço externo	A pirâmide descreve níveis funcionais, não impõe um caminho único para os dados
Modelo analítico propõe um *setpoint* ao CLP	Integração entre analytics e controle	A proposta deve passar por validação e autoridade de controle local antes de ser aplicada
Dado bruto chega ao time de analytics antes do supervisório	Fluxos paralelos de dados	Os níveis funcionais não determinam a ordem de entrega de cada fluxo

Conclusão de engenharia. A pirâmide continua sendo o melhor **mapa de responsabilidades** — e é isso que ela deve ser usada para responder: onde mora a lógica de controle, quem decide o quê, qual requisito de tempo se aplica. O que ela não descreve mais é a **topologia física** do fluxo de dados. Projetos que confundem as duas coisas acabam criando arquiteturas em que a nuvem participa de um laço de controle crítico — erro que a Seção 2.9 já antecipa e o Capítulo 11 reforça.

Pergunta de verificação. Na planta B, se o link de internet cair às 3 h da manhã, o processo continua em segurança? Se a resposta depende de algum serviço fora da fábrica, o desenho está apoiado na pirâmide errada.

2.11 Resumo

- A ISA-95 descreve cinco níveis funcionais, numerados de 0 a 4: processo físico; detecção e atuação; monitoramento e controle; operações de manufatura; planejamento e logística do negócio.
- Cada nível tem função, tecnologia e velocidade de resposta específicas.
- **CLP**, **DCS** e **RTU** são as tecnologias dominantes no nível de controle, cada uma adequada a um tipo de processo.
- O **MES/MOM** conecta as operações de manufatura às funções de negócio; a ISA-95 fornece modelos e terminologia para essa integração.
- O **IIoT** está achatando a pirâmide, permitindo que dados de campo cheguem diretamente à nuvem e às ferramentas de analytics.

2.12 Questões de Revisão

Conceituais:

1. Quais são os cinco níveis da pirâmide de automação? Descreva a função principal de cada um.

2. Qual é a diferença fundamental entre um CLP e um DCS? Em qual tipo de processo cada um é mais indicado?
3. O que é uma RTU e em que situação ela é mais adequada do que um CLP convencional?

Práticas:

4. Uma empresa deseja integrar os dados de produção de três linhas automatizadas ao seu sistema ERP. Quais níveis da pirâmide precisam ser interconectados? Quais protocolos ou sistemas fariam essa integração?
5. Explique com suas palavras como o Edge Computing altera o fluxo de dados na pirâmide de automação tradicional.

Desafio:

6. Compare a pirâmide de automação clássica com o modelo de referência RAMI 4.0 (*Reference Architecture Model Industry 4.0*). Quais conceitos são equivalentes? Quais são as principais diferenças?

2.13 Referências

- ANSI/ISA-95.00.01: *Enterprise-Control System Integration — Models and Terminology*. ISA. Consultar a edição vigente no catálogo da ISA antes de especificações contratuais.
- IEC 61131-3:2025: *Programmable Controllers — Part 3: Programming Languages*. IEC, 4ª ed., 2025.
- NIST: *Guide to Industrial Control Systems (ICS) Security*. NIST SP 800-82 Rev. 2, 2015.
- KAGERMANN, H. et al. *Recommendations for implementing the strategic initiative Industrie 4.0*. Acatech, 2013.
- SIEMENS. *Totally Integrated Automation Portal (TIA Portal)*. `siemens.com/tia`.

Capítulo 3

Sistemas Supervisórios — SCADA e HMI

Objetivos de Aprendizagem

Ao final deste capítulo, o leitor será capaz de:

- Definir SCADA e distinguir suas funções principais.
- Diferenciar SCADA, DCS e HMI local, reconhecendo quando usar cada um.
- Identificar os principais softwares SCADA comerciais e open-source.
- Descrever a evolução do SCADA: das primeiras gerações ao Cloud SCADA.

3.1 O que é SCADA?

SCADA (*Supervisory Control and Data Acquisition* — Controle Supervisório e Aquisição de Dados) é um sistema de software e hardware que permite **monitorar e controlar remotamente processos industriais e infraestruturas críticas** a partir de uma central de operações.

O nome descreve bem suas funções essenciais:

- **Supervisory Control:** permite que operadores visualizem o estado do processo e intervenham quando necessário — ajustando setpoints, abrindo válvulas, desligando equipamentos.
- **Data Acquisition:** coleta continuamente dados dos equipamentos de campo (CLPs, RTUs, transmissores) e os armazena em banco de dados histórico.

Nota

Nota: O SCADA **não realiza controle em malha fechada diretamente** na maioria das arquiteturas. Quem controla o processo em tempo real são os CLPs e DCSs no Nível 2. O SCADA supervisiona, centraliza e historiza — e intervém de forma supervisória quando necessário.

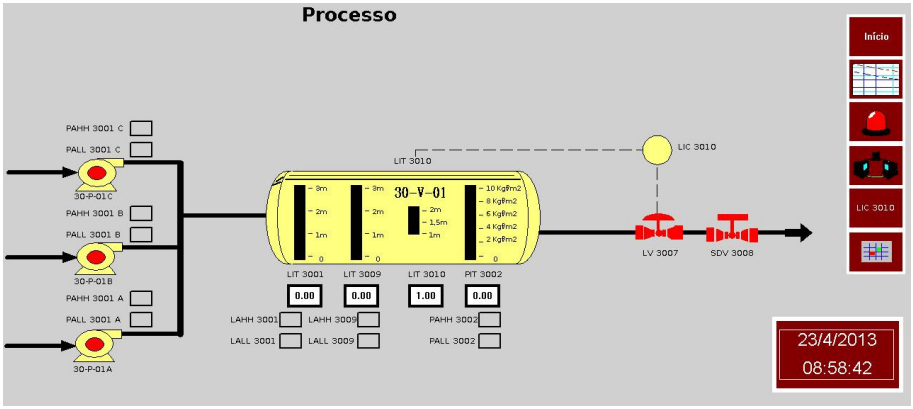
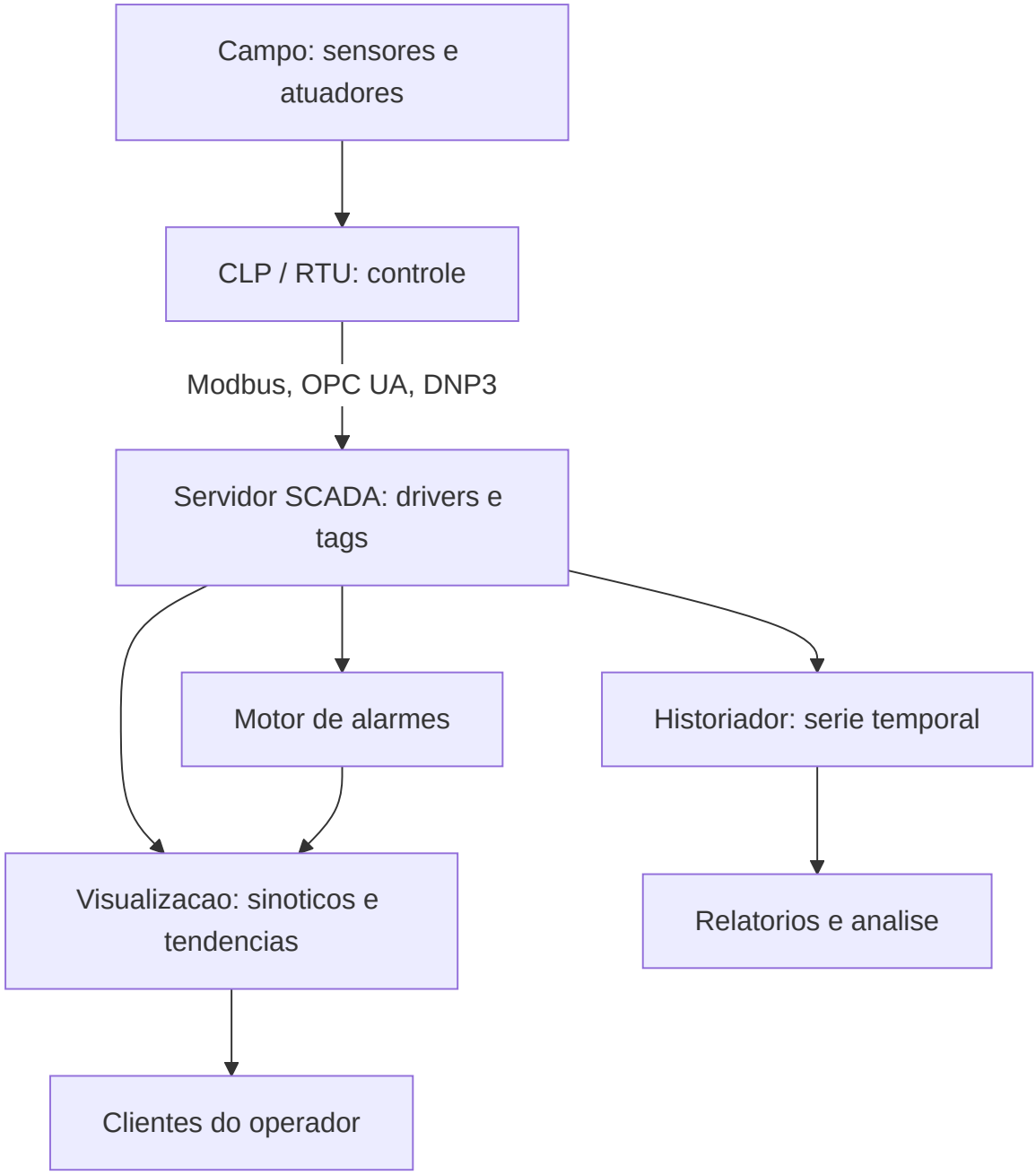


Figura 3.1 – Janela de IHM de um sistema supervisório. **Fonte:** *Livro SCADA — versão para análise*, p. 51 (Machado, Pontes e Vianna, reproduzida com crédito).

equipamento sobre o próprio desenho. Repare que a quantidade de informação é deliberadamente desenvolvido no Capítulo 11. A tela permite identificar elementos comuns de uma aplicação supervisória: sinótico, barra de navegação, indicação de estados e referência de data e hora. É uma captura de software de geração anterior e serve para reconhecer esses componentes, não como modelo de conformidade com a ISA-101 nem como recomendação visual atual. Os critérios de projeto de alta performance são discutidos nos Capítulos 10 e 11.

3.2 Funções Principais de um SCADA

Um sistema SCADA moderno oferece as seguintes funções. Elas não são independentes: todas dependem do mesmo conjunto de dados adquiridos do processo.



3.2.1 Aquisição de Dados

Coleta cíclica de variáveis analógicas (temperatura, pressão, vazão) e digitais (status de bombas, posição de válvulas, alarmes) a partir dos equipamentos de campo. A frequência de coleta pode variar de milissegundos a minutos, dependendo da criticidade da variável.

3.2.2 Visualização (HMI Centralizado)

Apresentação gráfica do processo em **sinóticos** animados, onde: - Válvulas mudam de cor conforme seu estado (aberta/fechada). - Tanques e reatores mostram nível preenchido dinamicamente. - Valores numéricos atualizam em tempo real. - Gráficos de tendência mostram a evolução da variável ao longo do tempo.

3.2.3 Gerenciamento de Alarmes

- Detecção automática de condições anormais (variável fora de faixa, falha de equipamento, perda de comunicação).
- Priorização de alarmes por criticidade (informativo, advertência, crítico, emergência).
- Registro de alarmes com timestamp, operador responsável e ação tomada.
- Relatórios de desempenho de alarmes (norma **EEMUA 191** e **ISA-18.2**).

Atenção

Atenção: Sistemas SCADA mal configurados podem gerar **inundação de alarmes** (*alarm flooding*) — situações em que centenas de alarmes disparam simultaneamente, dificultando a identificação da causa raiz. O gerenciamento racional de alarmes é uma disciplina de engenharia por si só.

3.2.4 Controle Supervisório

O operador pode enviar comandos ao processo através do SCADA: - Alteração de setpoints de controladores PID. - Abertura/fechamento de válvulas on/off. - Partida e parada de motores. - Execução de sequências de operação.

3.2.5 Historização

Armazenamento de séries temporais de dados de processo em banco de dados otimizado para esse fim. Permite: - Análise de tendências históricas. - Investigação de incidentes (reconstrução do evento). - Relatórios de produção. - Alimentação de sistemas de analytics e IA.

Bancos de dados de processo populares: OSIsoft PI System (agora AVEVA PI), Honeywell Uniformance, InfluxDB (open-source), TimescaleDB.

3.2.6 Relatórios

Geração automática de relatórios operacionais, de qualidade e de desempenho de equipamentos (OEE), em formatos PDF, Excel ou dashboards web.

Figura

[**Figura 3.2 – Funções de um SCADA moderno**] *Diagrama circular ou de hexágonos com as seis funções: Aquisição de Dados, Visualização, Alarmes, Controle, Historização e Relatórios — com ícones representativos para cada uma. No centro, o logo ou nome do sistema SCADA.*

3.3 SCADA × DCS × HMI Local

Uma dúvida comum entre estudantes e profissionais é a diferença entre SCADA, DCS e HMI. A **Tabela 3.1** resume as principais distinções:

Tabela 3.1 – Comparativo SCADA × DCS × HMI Local

Critério	HMI Local	SCADA	DCS
Escopo típico	Máquina ou célula	Planta, infraestrutura ou múltiplos sites	Processo integrado, frequentemente contínuo
Distribuição	Painel próximo ao equipamento	Controle local com supervisão central ou distribuída	Controladores e estações integrados em uma arquitetura de controle
Controle	Comandos e, conforme a aplicação, funções locais	Supervisão e comandos; malhas críticas ficam em controladores locais	Controle distribuído integrado; funções locais continuam essenciais
Conectividade	Rede local	LAN, WAN, rádio, fibra ou redes celulares	Redes de controle e integração; não necessariamente proprietárias
Histórico	Ausente ou de escopo limitado	Pode integrar historizador dedicado	Pode integrar historização, conforme arquitetura
Redundância	Depende do equipamento	Opção de projeto conforme disponibilidade requerida	Opção frequente em sistemas críticos, não garantia inerente
Aplicação típica	Máquina CNC, bomba, compressor	Oleoduto, ETAP, subestação	Refinaria, petroquímica, papel e celulose
Custo relativo	Baixo	Médio	Alto

Dica

Dica: As funções podem se sobrepor e produtos podem reunir recursos de supervisão e controle. A classificação como SCADA ou DCS depende da arquitetura implantada e de onde residem as funções de controle, não apenas do nome comercial da plataforma.

3.4 Gerações do SCADA

A evolução do SCADA ocorreu em quatro gerações bem definidas:

Geração 1 — Monolítico (anos 1970–1980) - Sistemas proprietários, sem conectividade entre sistemas. - Hardware dedicado de cada fabricante (protocolos fechados). - Computadores de grande porte (*mainframes*) ou minicomputadores.

Geração 2 — Distribuído (anos 1980–1990) - Redes locais (LANs) conectam múltiplas estações de trabalho. - Aparecimento dos primeiros padrões abertos (Modbus, RS-485). - Servidores Unix e estações Sun Sparc.

Geração 3 — Em Rede (anos 1990–2000) - Adoção da infraestrutura Ethernet e TCP/IP. - OPC (OLE for Process Control) como padrão de integração. - Interfaces

gráficas Windows, banco de dados SQL.

Tendências recentes — conectividade e serviços distribuídos (desde os anos 2010) - Integração nativa com IIoT, MQTT, OPC UA. - Opções de implantação locais, híbridas ou em nuvem; a nuvem não é adequada a toda função de controle. - Acesso por navegador e dispositivos móveis, sujeito a requisitos de disponibilidade, segurança e ergonomia. - Análise de dados e modelos de aprendizado de máquina podem complementar a supervisão.

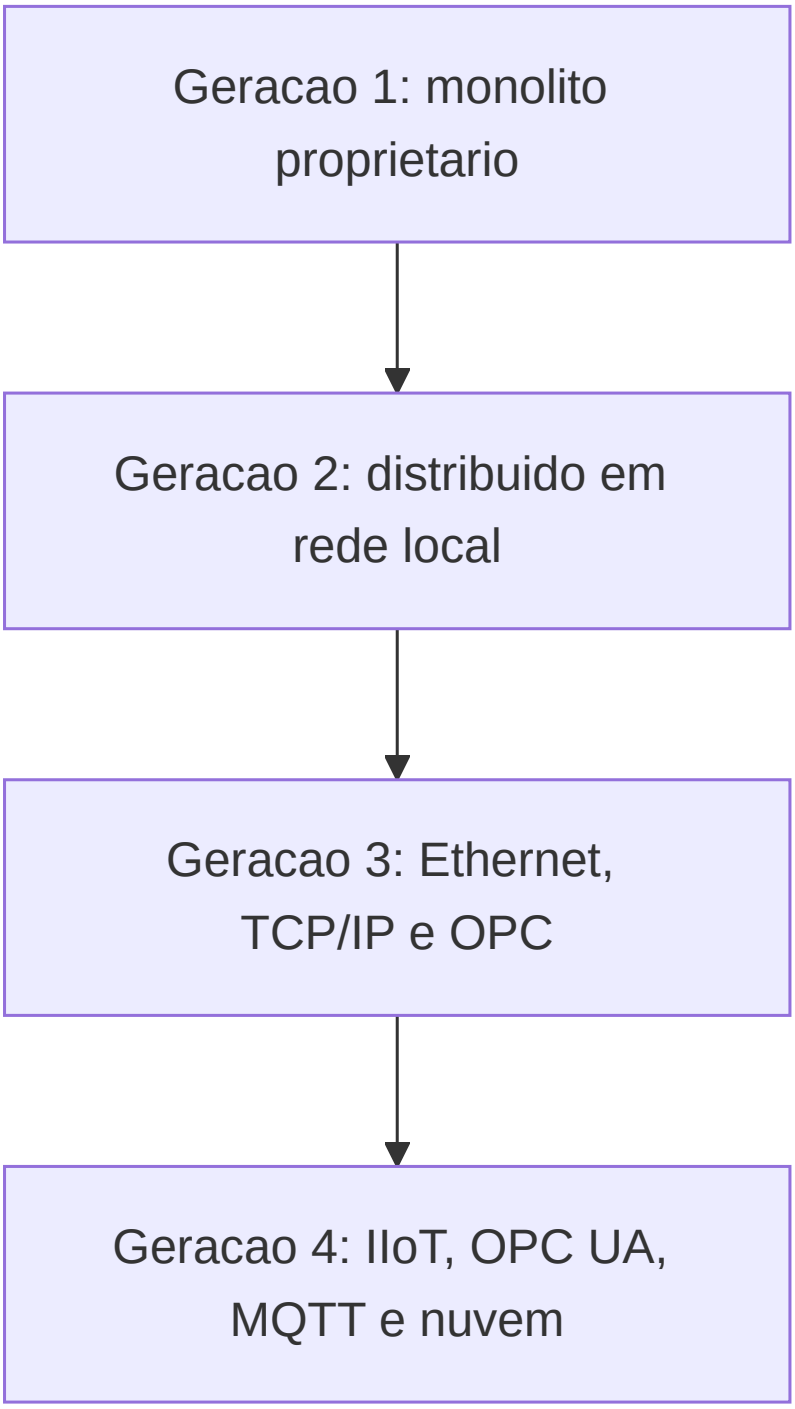
Essa evolução não é universal nem substitui automaticamente gerações anteriores: sistemas de diferentes períodos coexistem, e a escolha depende dos requisitos da aplicação.



Figura 3.3 – Sala de controle com painel de instrumentos convencionais. **Fonte:** *Livro SCADA — versão para análise*, p. 15 (fotografia de Solomon, 2005, adaptada pelos autores).

A cena corresponde à geração **anterior** ao SCADA: cada variável tem um mostrador dedicado no painel e o histórico do processo existe apenas na pena do registrador. Substituir esse painel por estações de supervisão reduziu o espaço físico da sala de controle e transferiu a informação para um banco de dados único — a mudança que a Seção 3.4 descreve em quatro gerações.

Cada geração resolveu o problema da anterior e criou o seu próprio:



3.5 Principais Plataformas SCADA

3.5.1 Plataformas Comerciais

AVEVA System Platform (ex-Wonderware) - Uma das plataformas mais difundidas globalmente. - Arquitetura de objetos reutilizáveis (Application Server). - Forte integração

com o AVEVA PI System para historização.

Ignition (Inductive Automation) - Licenciamento por servidor (não por tag), modelo altamente econômico. - Plataforma comercial, com designer web e clientes que reduzem a necessidade de instalação local. - Módulos para SCADA, IIoT (MQTT), MES e relatórios. - Comunidade open-source ativa.

Rockwell FactoryTalk View - Nativo do ecossistema Allen-Bradley/Rockwell Automation. - Excelente integração com CLPs ControlLogix e CompactLogix.

Siemens WinCC - Integrado ao TIA Portal. - Versões para HMI local (WinCC flexible) e SCADA (WinCC SCADA / WinCC OA).

Honeywell Experion PKS - Plataforma DCS+SCADA integrada para grandes processos contínuos.

3.5.2 Software livre e ferramentas adjacentes

Software	Categoria e características	Uso típico
ScadaBR	SCADA de código aberto, baseado em Java	Ensino e aplicações cuja validação atenda aos requisitos do projeto
OpenSCADA	Plataforma modular de código aberto	Integração e supervisão em ambientes compatíveis
Node-RED	Ferramenta de integração e programação visual de código aberto	IIoT, prototipagem e fluxos auxiliares
Grafana + InfluxDB	Visualização e armazenamento de séries temporais	Tendências históricas e análise; não substituem por si só um SCADA de operação

Nota

Nota: ferramentas de código aberto podem compor uma arquitetura supervisória, mas licenciamento aberto não comprova adequação para operação crítica. Avalie suporte, redundância, controle de acesso, auditoria, recuperação e validação operacional. O Node-RED é detalhado no Capítulo 7 e não é, sozinho, equivalente a um SCADA industrial.

3.6 Requisitos para Implantação de um SCADA

Ao especificar e implantar um sistema SCADA, os seguintes aspectos devem ser considerados:

Tabela 3.2 – Checklist de especificação de um SCADA

Aspecto	Questões a Responder
Escopo	Quantos pontos (tags)? Quantos sites?

Aspecto	Questões a Responder
Conectividade	Quais protocolos os CLPs/RTUs usam?
Disponibilidade	É necessária redundância de servidores?
Usuários	Quantos clientes simultâneos? Acesso remoto?
Histórico	Quais variáveis historizar? Por quanto tempo?
Alarmes	Quantos alarmes? Notificação por email/SMS?
Relatórios	Quais relatórios são obrigatórios (regulatórios)?
Segurança	Controle de acesso por perfil de usuário?
Integração	Interface com MES/ERP?
Normas	IEC 62264, ISA-18.2, LGPD (dados de operadores)?

3.7 Estudo de Caso — Migração de um SCADA de geração 3 para geração 4

Cenário. Distribuidora de água com 42 estações de bombeamento, elevatórias e reservatórios distribuídos em quatro municípios. O SCADA atual é de geração 3: rodou sobre Windows Server 2003, com OPC DA e drivers proprietários de dois fabricantes de RTU. O sistema funciona — o problema é que ninguém consegue mantê-lo.

Sintomas que dispararam o projeto.

- **Máquina virtual parada de atualizar** três anos, sem fornecedor para o sistema operacional.
- **Um engenheiro** na equipe domina o pacote antigo; as demais pessoas dependem dele.
- **Dados presos**: o historiador é proprietário e não aceita consulta externa, o que impede qualquer análise no Grafana ou no Excel.
- **Integração impossível**: o novo sistema corporativo de manutenção não fala OPC DA, e não há caminho para levar dados das RTUs até ele.

Estratégia adotada — em camadas, não em big bang.

Etapa	Movimento	Efeito imediato
1	*Gateway* OPC UA na frente dos drivers antigos	Moderniza a interface sem tocar nas RTUs
2	Novo servidor SCADA em paralelo, com OPC UA	Operação continua no sistema antigo
3	Espelhamento do histórico para banco de séries temporais	Dados deixam de ficar presos
4	Migração de telas em blocos, por área	Cada área migra e é validada separadamente

Etapa	Movimento	Efeito imediato
5	Desligamento do sistema antigo, mantido em leitura por 6 meses	Saída reversível

O que observar no resultado. A etapa 1 é a que destrava o projeto: um *gateway* de protocolo converte o que existe **sem exigir substituição de campo**. É o padrão descrito na Seção 5.4 e reaproveitado no Capítulo 13 — raramente a migração começa pelo equipamento mais caro.

Limitações do arranjo. O *gateway* é mais um ponto de falha e precisa de supervisão própria: se ele para, o dado para, mesmo com CLP e rede em pleno funcionamento. Além disso, migrar telas em blocos só funciona se as duas gerações coexistirem por um período — o que exige disciplina de numeração de tags e de *timestamps* entre os dois ambientes.

3.8 Resumo

- **SCADA** é um sistema de supervisão centralizada que combina aquisição de dados, visualização gráfica, gerenciamento de alarmes, controle supervisório e historização.
- **SCADA** difere do **DCS** (que controla diretamente o processo) e da **HMI local** (escopo restrito a um equipamento).
- O SCADA evoluiu em quatro gerações: de sistemas monolíticos proprietários (anos 70) a plataformas cloud-native com IIoT (hoje).
- Plataformas líderes incluem AVEVA, Ignition e Siemens WinCC; no segmento open-source, destacam-se ScadaBR e Node-RED.

3.9 Questões de Revisão

Conceituais:

1. Expanda a sigla SCADA e explique o significado de cada palavra.
2. Quais são as seis funções principais de um sistema SCADA moderno? Descreva duas delas.
3. Qual é a diferença entre um sistema DCS e um SCADA? Em qual situação você escolheria um DCS em vez de um SCADA?

Práticas:

4. Uma distribuidora de água deseja implantar um sistema para monitorar remotamente 50 estações de bombeamento distribuídas em um município, com histórico de consumo, alarmes de falhas e acesso web para a equipe de manutenção. Qual tipo de sistema (HMI local, SCADA ou DCS) e qual plataforma você recomendaria? Justifique.
5. O que é “alarm flooding” e quais práticas de engenharia podem ser adotadas para evitá-lo?

Desafio:

6. Compare o modelo de licenciamento do SCADA Ignition (por servidor, tags ilimitados) com o modelo tradicional por número de tags (ex.: AVEVA). Quais são as vantagens e desvantagens de cada modelo para uma empresa que está expandindo sua operação?

3.10 Referências

- BOYER, S. A. *SCADA: Supervisory Control and Data Acquisition*. 4. ed. ISA, 2010.
- ISA-18.2: *Management of Alarm Systems for the Process Industries*. ISA, 2016.
- EEMUA Publication 191: *Alarm Systems — A Guide to Design, Management and Procurement*. EEMUA, 2013.
- INDUCTIVE AUTOMATION. *Ignition SCADA Platform*. Disponível em: inductiveautomation.com.
- AVEVA. *System Platform Overview*. Disponível em: aveva.com.
- MACHADO, C. F. B.; PONTES, M. O.; VIANNA, W. S. *SCADA — Supervisory Control and Data Acquisition: sistemas de supervisão e aquisição de dados para automação*. (Material de origem desta apostila; figuras dos autores reproduzidas com crédito.)

Capítulo 4

Arquitetura e Componentes de um Sistema SCADA

Objetivos de Aprendizagem

Ao final deste capítulo, o leitor será capaz de:

- Identificar e descrever os componentes físicos e lógicos de uma arquitetura SCADA.
- Compreender as funções da MTU, RTU e servidor de dados.
- Distinguir arquiteturas monolíticas, distribuídas e redundantes.
- Dimensionar os elementos de uma arquitetura SCADA para um cenário típico.

4.1 Componentes de uma Arquitetura SCADA

Uma arquitetura SCADA típica combina dispositivos de campo e controle, redes, serviços de aquisição e supervisão, clientes de operação e, quando requerido, historiadores e serviços de integração. A figura resume uma configuração possível; os nomes MTU e RTU são mais comuns em telecontrole e não representam componentes obrigatórios em toda arquitetura moderna.

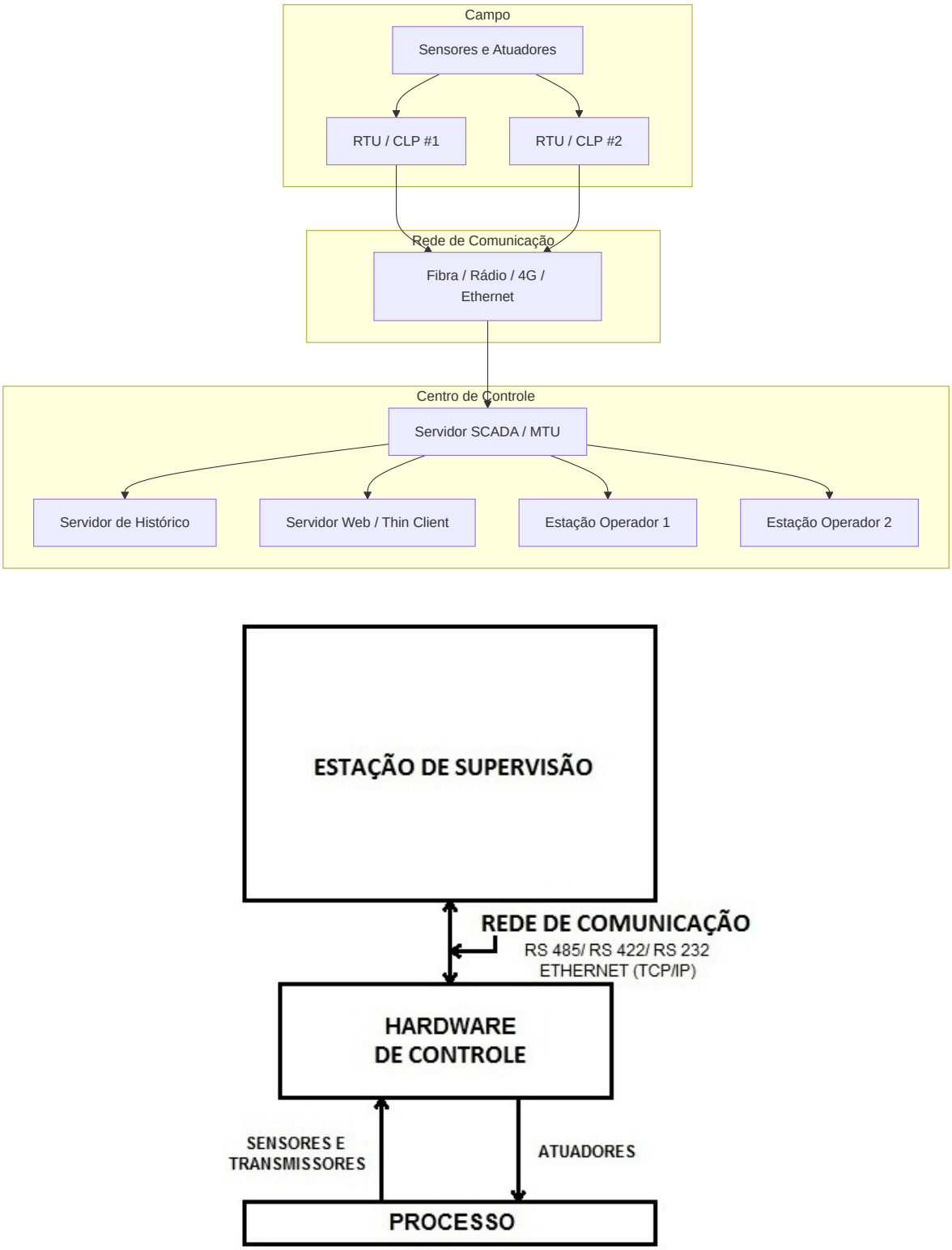


Figura 4.1 – Arquitetura básica de um sistema SCADA. **Fonte:** *Livro SCADA — versão para análise*, p. 22 (Machado, Pontes e Vianna, reproduzida com crédito).

A estação de supervisão comunica-se pelos meios RS-485, RS-422, RS-232 ou Ethernet e conversa com o hardware de controle, que por sua vez lê sensores e transmissores e aciona os atuadores sobre o processo.

4.2 RTU — Unidade Terminal Remota

A **RTU** (*Remote Terminal Unit*) é uma unidade de telecontrole instalada em local remoto. Em geral, reúne entradas/saídas, comunicação e capacidade de executar lógica local, mas os recursos variam por modelo e aplicação. Suas funções podem incluir:

1. **Coletar** dados de sensores e instrumentos.
2. **Executar** lógica local, quando prevista no projeto.
3. **Transmitir** dados ao centro por um protocolo de comunicação.
4. **Receber** comandos e encaminhá-los às saídas, conforme permissões e intertravamentos.

As quantidades de canais, faixas elétricas, protocolos, alimentação e temperatura admissível são características do modelo escolhido, não uma definição geral de RTU; devem ser conferidas na folha de dados.

4.2.1 Módulo de I/O remoto não é necessariamente uma RTU

Um **módulo de I/O remoto** (*remote I/O*) concentra canais distribuídos e os comunica a um controlador. Em geral, não executa por si só a lógica de controle nem desempenha as funções de telemetria e operação autônoma de uma RTU. Se apenas expande a E/S de um CLP, é um módulo remoto; se também adquire dados, comunica-se com o centro e executa funções locais, pode atuar como RTU. Há equipamentos híbridos: verifique a documentação e o comportamento durante a perda de comunicação.

4.2.2 RTU vs. CLP em telecontrole

CLPs podem atender a aplicações de telecontrole, assim como RTUs podem executar lógicas locais. A escolha deve comparar consumo do conjunto, autonomia, protocolos, manutenção, expansão de E/S, requisitos ambientais e comportamento durante falhas. Baixo consumo ou certificação ambiental não são garantidos apenas pelo rótulo RTU; confirme as características do modelo para o local de uso.

Dica

Dica: Em aplicações de telemetria de água e saneamento, é comum encontrar RTUs com comunicação GPRS/4G incorporada, alimentadas por painel solar e bateria, operando sem intervenção por meses. Fabricantes como Advantech, Emerson (Bristol) e SEL atendem bem esse nicho.

4.3 MTU — Unidade Terminal Mestre

A **MTU** (*Master Terminal Unit*) é um termo tradicional para a unidade mestre de um sistema de telecontrole. Em arquiteturas atuais, suas funções podem estar distribuídas entre servidores SCADA, front-ends de comunicação e outros serviços. Quando o termo é usado, costuma identificar o componente que coordena a comunicação com RTUs e concentra dados e comandos:

- **Polling:** interroga periodicamente cada RTU/CLP para coletar dados.

- **Banco de dados em tempo real (RTDB):** mantém na memória o estado atual de todas as variáveis do sistema.
- **Processamento de alarmes:** avalia condições de alarme e notifica os operadores.
- **Registro de eventos:** armazena com timestamp cada mudança de estado e alarme.
- **Execução de controle supervisório:** processa os comandos dos operadores e os encaminha às RTUs.
- **Interface com clientes HMI:** serve dados às estações de operação e thin clients.

4.3.1 Redundância da MTU

Para requisitos elevados de disponibilidade, a arquitetura pode incluir redundância **ativo-passivo** ou serviços **ativo-ativos**. O comportamento depende do produto, da topologia e do que está sincronizado; redundância de servidor, isoladamente, não garante continuidade de todos os serviços:

- **Ativo-passivo:** um nó atende às funções principais e outro fica pronto para assumir. Verifique o que é replicado, os critérios de detecção e o tempo de comutação especificado e testado.
- **Ativo-ativo:** os nós atendem simultaneamente a serviços ou cargas definidas pela arquitetura. A distribuição de clientes e a recuperação de sessão variam entre implementações.

Figura

[**Figura 4.2 – Configuração redundante de servidores SCADA**] *Diagrama mostrando dois servidores SCADA (Primário e Backup) conectados por link dedicado de sincronização. As RTUs de campo conectam-se aos dois servidores. Os clientes HMI recebem dados do servidor ativo. Seta pontilhada indicando o failover automático.*

4.4 Servidor de Historização (Historiador)

O **historiador de processo** é um banco de dados especializado em armazenar séries temporais de variáveis industriais com alta eficiência de compressão e acesso.

4.4.1 Por que não usar SQL convencional?

Bancos relacionais podem armazenar dados temporais e são adequados a várias aplicações industriais. Em altas taxas de ingestão, muitos anos de retenção ou consultas repetidas por intervalo, um historiador especializado ou banco de séries temporais pode reduzir o esforço de armazenamento, compressão e consulta. A escolha depende da escala, dos requisitos de integridade e das competências da equipe:

- Uma planta com 10.000 tags coletados a cada segundo gera **864 milhões de registros por dia**.
- Sem particionamento, compressão e índices adequados, o volume pode aumentar o custo de armazenamento e manutenção.
- Consultas temporais repetidas podem exigir modelagem e otimização que soluções especializadas já oferecem.

Isso não significa que todo banco SQL seja inadequado: particionamento, extensões para séries temporais e infraestrutura dimensionada podem atender ao caso. Compare com uma carga representativa e consultas reais.

4.4.2 Soluções de Historização

Produto	Tipo	Empresa	Diferencial
AVEVA PI System	Comercial	AVEVA	Padrão da indústria de O&G; ecossistema amplo (PI Vision, PI AF)
Honeywell Uniformance PHD	Comercial	Honeywell	Integrado ao Experion
InfluxDB	Open-source	InfluxData	Alta performance; linha do tempo nativa; integração com Grafana
TimescaleDB	Open-source	Timescale	Extensão PostgreSQL para séries temporais
Prometheus	Open-source	CNCF	Focado em métricas de infraestrutura; integra com Grafana

Nota

Nota: O **AVEVA PI System** é o historiador mais utilizado na indústria de petróleo, gás e energia elétrica no Brasil e no mundo. Conhecer sua estrutura de atributos (PI AF — Asset Framework) é uma competência valorizada no mercado.

4.5 Estações de Operação e Thin Clients

As **estações de operação** são os computadores onde os operadores monitoram o processo e executam comandos. Podem ser:

- **Fat clients (clientes completos):** PC com software SCADA instalado — alto desempenho, menor dependência de rede.
- **Thin clients:** estações sem disco local que executam a interface SCADA a partir de um servidor (Terminal Server ou Citrix). Mais fáceis de manter e atualizar.
- **Web clients:** acesso via navegador (Chrome, Edge) sem instalação. Tendência predominante em plataformas modernas como Ignition.
- **Mobile clients:** aplicativos iOS/Android para supervisão em campo ou acesso remoto.

4.6 Infraestrutura de Comunicação

A rede de comunicação é o sistema circulatório do SCADA. Suas características determinam a confiabilidade e o desempenho do sistema.

4.6.1 Meios Físicos

Meio	Vantagem	Limitação	Aplicação
Fibra ótica	Alta velocidade, imune a EMI	Custo de instalação	Redes industriais críticas
Ethernet par trançado	Baixo custo, fácil instalação	Distância ~100 m	Ambiente de controle local
Rádio UHF/VHF	Sem infraestrutura física	Sujeito a interferência	Áreas remotas sem cabeamento
4G/5G	Cobertura ampla, fácil implantação	Custo recorrente, latência variável	RTUs remotas, telemetria
Satélite	Cobertura global	Alta latência, custo elevado	Offshore, locais sem cobertura celular

4.6.2 Topologias de Rede

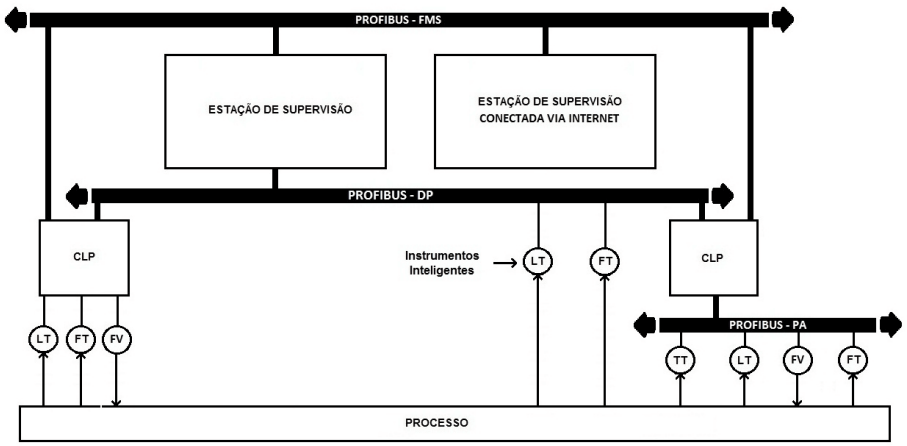


Figura 4.3 – Arquitetura de rede de campo com PROFIBUS e supervisão remota. **Fonte:** *Livro SCADA — versão para análise*, p. 30 (Machado, Pontes e Vianna, reproduzida com crédito).

Observe a topologia em barramento nos dois níveis: PROFIBUS DP entre CLPs e instrumentos, PROFIBUS PA no nível dos transmissores, e uma estação de supervisão que também é alcançada pela rede corporativa.

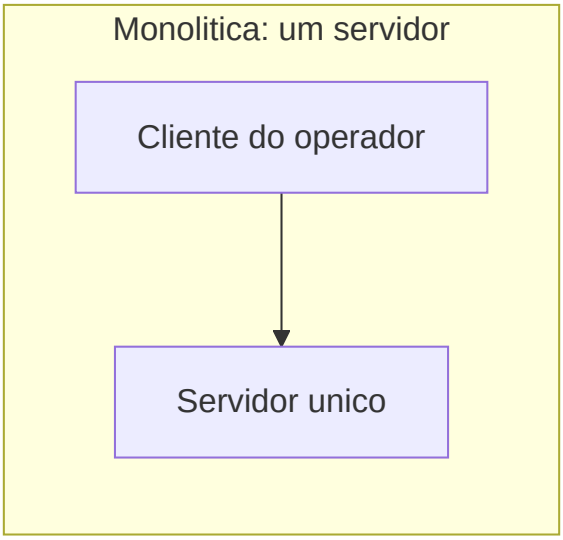
Atenção

Atualização tecnológica: o próprio material de origem anota, na página da figura, que ela **seria alterada**: na prática, o PROFIBUS DP passou a ser substituído por **Ethernet** industrial (PROFINET ou EtherNet/IP) no nível de controle, e o

PROFIBUS FMS foi descontinuado. O barramento PROFIBUS permanece relevante onde já está instalado — sobretudo na camada de instrumentação (PROFIBUS PA) —, não como escolha para projetos novos. A figura é mantida como registro de uma arquitetura real de campo, e não como recomendação de projeto.

4.7 Comparativo de Arquiteturas SCADA

A escolha da arquitetura é uma decisão de engenharia de disponibilidade: quanto mais crítico o processo, mais a redundância deixa de ser opcional.



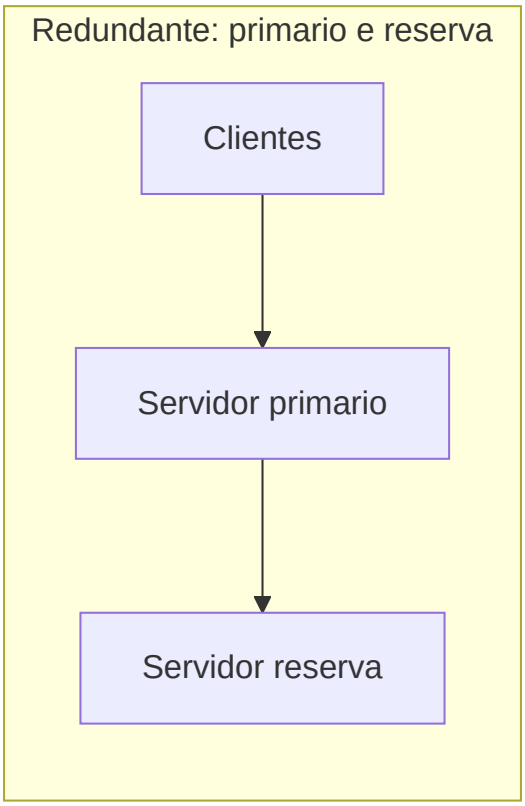


Tabela 4.1 – Arquiteturas SCADA e suas características

Arquitetura	Descrição	Vantagens	Desvantagens	Indicada para
Monolítica	Um único servidor SCADA	Simples, baixo custo	Ponto único de falha	Pequenas plantas, projetos piloto

Arquitetura	Descrição	Vantagens	Desvantagens	Indicada para
Cliente-Servidor	Servidor central + múltiplos clientes	Escalável, acesso multiusuário	Exige rede confiável	Plantas médias e grandes
Distribuída	Múltiplos servidores por área ou site	Falha isolada por área	Mais complexa de integrar	Múltiplos sites
Redundante	Servidor primário + backup automático	Alta disponibilidade (HA)	Custo dobrado	Infraestruturas críticas
Cloud SCADA	Servidor hospedado na nuvem	Acesso global, sem hardware local	Latência, segurança de dados	Supervisão distribuída, SaaS

As três variantes abaixo são as arquiteturas históricas que a tabela resume: com CLP, com controladores dedicados e com sistema de aquisição de dados. Elas continuam aparecendo em campo, mesmo quando o CLP é a escolha padrão.



Figura 4.4 – Arquitetura SCADA com uso de CLP. **Fonte:** *Livro SCADA — versão para análise*, p. 26 (Machado, Pontes e Vianna, reproduzida com crédito).

Com o CLP, o controle fica concentrado no controlador e a supervisão deixa de conversar diretamente com os instrumentos: ela fala com o CLP pela rede de comunicação.

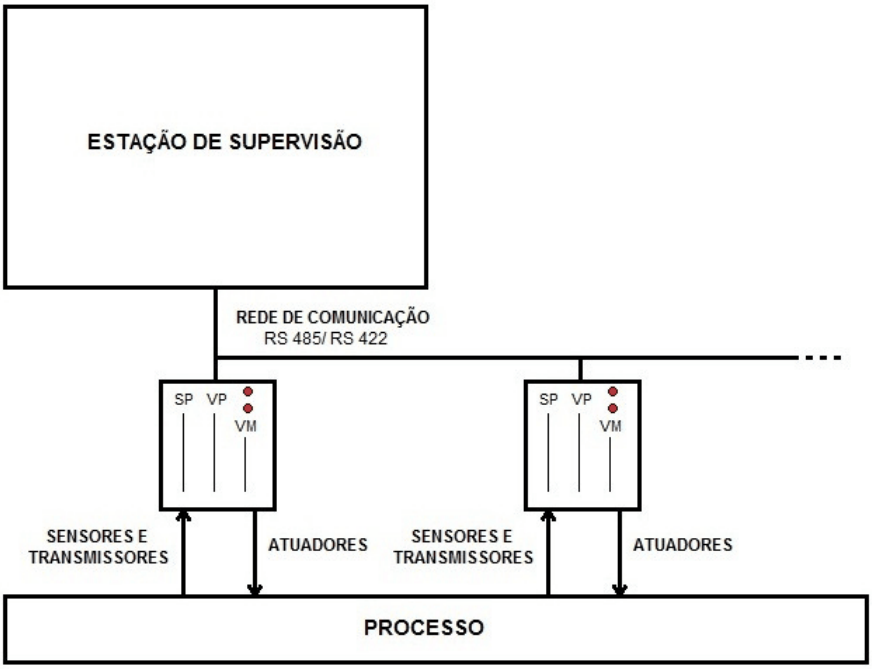


Figura 4.5 – Arquitetura com controladores single-loop e multi-loop. **Fonte:** *Livro SCADA — versão para análise*, p. 27 (Machado, Pontes e Vianna, reproduzida com crédito).

Cada controlador dedicado fecha uma ou mais malhas, e a supervisão lê e escreve setpoints sobre o barramento RS-485.



Figura 4.6 – Arquitetura com sistema de aquisição de dados. **Fonte:** *Livro SCADA — versão para análise*, p. 28 (Machado, Pontes e Vianna, reproduzida com crédito).

No DAQ não há controle pelo sistema supervisório: ele apenas adquire, apresenta e registra.

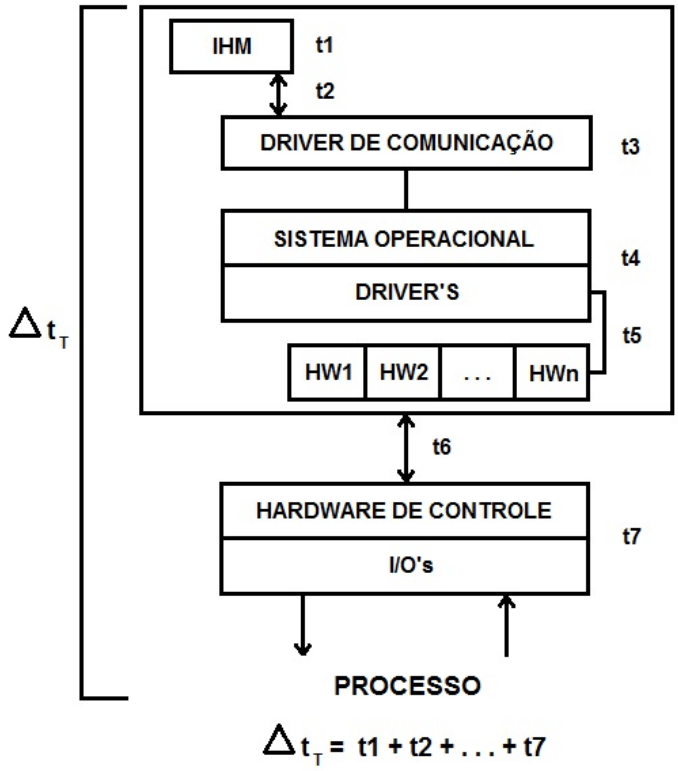


Figura 4.7 – Tempo de resposta de um sistema SCADA. **Fonte:** *Livro SCADA — versão para análise*, p. 33 (Machado, Pontes e Vianna, reproduzida com crédito).

O tempo total de resposta é a soma das parcelas: interface homem-máquina, driver de comunicação, sistema operacional, hardware de controle e E/S. É por isso que o tempo medido na tela nunca é igual ao tempo de varredura do CLP.

4.8 Estudo de Caso — SCADA para Sistema de Abastecimento de Água

Contexto: Uma companhia de saneamento opera 35 estações de bombeamento distribuídas em uma cidade de médio porte, com distâncias de até 80 km do centro de controle.

Desafio: Monitorar em tempo real nível dos reservatórios, pressão nas redes de distribuição e status das bombas; detectar vazamentos; gerar relatórios de consumo para a concessionária reguladora.

Solução adotada:

- **Campo:** RTUs alimentadas por energia solar + bateria, com medidores de vazão eletromagnéticos, transmissores de pressão e nível ultrassônico.
- **Comunicação:** Link 4G (operadora) com VPN criptografada; backup via rádio UHF para estações críticas.
- **SCADA:** Servidor central com ScadaBR (open-source) + módulo de relatórios. Interface web para acesso da equipe de operação e manutenção via tablets.
- **Histórico:** InfluxDB para séries temporais + Grafana para dashboards de consumo e OEE das bombas.

Resultados esperados no cenário didático: maior visibilidade do estado das estações, priorização mais rápida de falhas e disponibilidade de dados históricos para investigar desvios. Não são métricas de uma implantação real; estimativas de economia ou tempo de resposta exigiriam dados de referência e validação em campo.

Figura

[**Figura 4.8 – Arquitetura do SCADA de saneamento**] *Diagrama da arquitetura completa do estudo de caso: mapa geográfico esquemático da cidade com as 35 estações; link 4G até o centro de controle; servidor SCADA, InfluxDB e Grafana; estações de operação e acesso mobile.*

4.9 Resumo

- Uma arquitetura SCADA integra campo/controle, comunicação, aquisição e serviços de supervisão; MTU e RTU são termos tradicionais de telecontrole, não blocos obrigatórios em todos os sistemas.
- **RTUs** podem reunir E/S, comunicação e lógica local; módulos de I/O remoto normalmente expandem os canais de um controlador.
- A escolha entre historiador dedicado, banco de séries temporais ou relacional depende de volume, retenção, integridade, consulta e capacidade de operação.
- A arquitetura deve ser dimensionada conforme criticidade: desde monolítica (simples) até redundante com failover automático (infraestruturas críticas).
- A comunicação pode usar fibra, Ethernet, rádio ou 4G — cada meio com trade-offs de custo, velocidade e confiabilidade.

4.10 Questões de Revisão

Conceituais:

1. Qual é a diferença entre MTU e RTU em um sistema SCADA?
2. Por que bancos de dados relacionais convencionais são inadequados para historização de dados de processo industrial?
3. Quais são as diferenças entre as arquiteturas SCADA cliente-servidor e cloud SCADA?

Práticas:

4. Uma planta petroquímica opera 24 horas por dia, 7 dias por semana, com zero tolerância a paradas não programadas do sistema de supervisão. Qual arquitetura SCADA você recomendaria? Descreva os componentes de redundância necessários.
5. Para um sistema com 500 RTUs coletando 20 variáveis cada uma, a cada 10 segundos, calcule o volume de dados históricos gerado por dia e por ano. Qual solução de historização você escolheria?

Desafio:

6. Projete uma arquitetura SCADA para uma fazenda com 10 pivôs de irrigação localizados em área sem cobertura de fibra. Defina: meios de comunicação, equipamento de campo, servidor e interface de supervisão.

4.11 Referências

- BOYER, S. A. *SCADA: Supervisory Control and Data Acquisition*. 4. ed. ISA, 2010.
- INFLUXDATA. *InfluxDB Documentation*. Disponível em: docs.influxdata.com.
- AVEVA. *PI System Overview*. Disponível em: `aveva.com/en/products/pi-system/`.
- IEC 60870-5: *Telecontrol Equipment and Systems*. IEC, 2006.
- MACHADO, C. F. B.; PONTES, M. O.; VIANNA, W. S. *SCADA — Supervisory Control and Data Acquisition: sistemas de supervisão e aquisição de dados para automação*. (Material de origem desta apostila; figuras das arquiteturas dos autores, reproduzidas com crédito.)

Capítulo 5

Redes e Protocolos de Comunicação Industrial

Objetivos de Aprendizagem

Ao final deste capítulo, o leitor será capaz de:

- Identificar os principais protocolos de comunicação industrial e suas características.
- Distinguir protocolos clássicos (Modbus, Profibus) de protocolos modernos (OPC UA, MQTT).
- Compreender o papel de cada protocolo na arquitetura de um sistema SCADA.
- Aplicar critérios técnicos para seleção de protocolo em projetos de automação.

5.1 Redes Industriais versus Redes Corporativas

As redes industriais possuem requisitos distintos das redes corporativas de TI:

Tabela 5.1 – Comparativo: Rede Corporativa × Rede Industrial

Critério	Rede Corporativa (TI)	Rede Industrial (OT)
Prioridade	Confidencialidade, disponibilidade	Determinismo, tempo real
Latência	Tolerável (ms a s)	Crítica (μs a ms)
Disponibilidade	Alta (99,9%)	Muito alta (99,999%)
Equipamentos	PCs, servidores, switches	CLPs, RTUs, inversores, sensores
Ambiente	Sala climatizada	Calor, vibração, poeira, EMI
Ciclo de vida	3–5 anos	10–30 anos
Atualização	Frequente	Raramente (risco de parada)

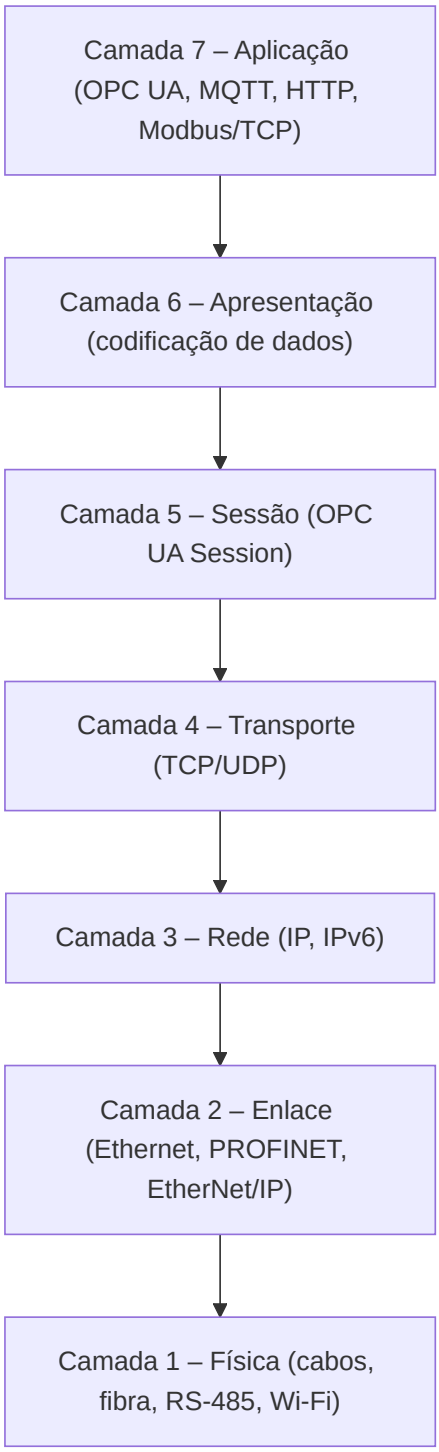
Atenção

Atenção: A convergência TI/OT é inevitável com o IIoT, mas deve ser feita com cuidado. Trazer protocolos e infraestrutura de TI (ex.: Wi-Fi público, VLAN sem

segmentação) para o ambiente OT sem as devidas proteções é a principal causa de incidentes de segurança industrial. Ver Capítulo 8.

5.2 Modelo OSI Aplicado à Automação

O modelo **OSI** (*Open Systems Interconnection*) de 7 camadas é frequentemente referenciado para classificar os protocolos industriais:



Figura

[**Figura 5.1 – Protocolos industriais mapeados no modelo OSI**] *Diagrama de 7 camadas OSI com protocolos industriais posicionados nas camadas correspondentes. Destacar: Modbus RTU (camadas 1-2), Modbus TCP (completo), OPC UA (7), MQTT (7), Profibus (1-2-7), PROFINET (1-2-7).*

5.3 Protocolos Clássicos

5.3.1 Modbus

O **Modbus** foi criado pela Modicon em 1979 e é um dos protocolos mais simples e amplamente suportados no mundo. Sua popularidade se deve à especificação aberta, gratuita e fácil de implementar.

Variantes:

- **Modbus RTU:** comunicação serial (RS-232 ou RS-485), codificação binária compacta. Máximo de 247 escravos por barramento RS-485.
- **Modbus ASCII:** comunicação serial, codificação hexadecimal. Raramente usado hoje.
- **Modbus TCP:** transporta mensagens do protocolo de aplicação Modbus sobre TCP/IP, normalmente na porta 502. Usa o cabeçalho MBAP e não encapsula simplesmente o quadro serial RTU; o alcance depende da infraestrutura de rede entre os nós.

Modelo de dados Modbus:

Área	Tipo	Acesso	Exemplo
Coils (0x)	Bit	Leitura/Escrita	Status de válvula, partida de motor
Discrete Inputs (1x)	Bit	Somente Leitura	Estado de botoeira, fim de curso
Holding Registers (4x)	Word 16 bits	Leitura/Escrita	Setpoint de temperatura, referência de velocidade
Input Registers (3x)	Word 16 bits	Somente Leitura	Valor de temperatura, corrente de motor

Dica

Dica: Para representar valores de ponto flutuante (ex.: 25,4°C) em Modbus, utiliza-se 2 registradores consecutivos de 16 bits no formato **IEEE 754** (32 bits). A ordem dos bytes (*byte order* ou *endianness*) varia por fabricante — sempre verifique na documentação do equipamento.

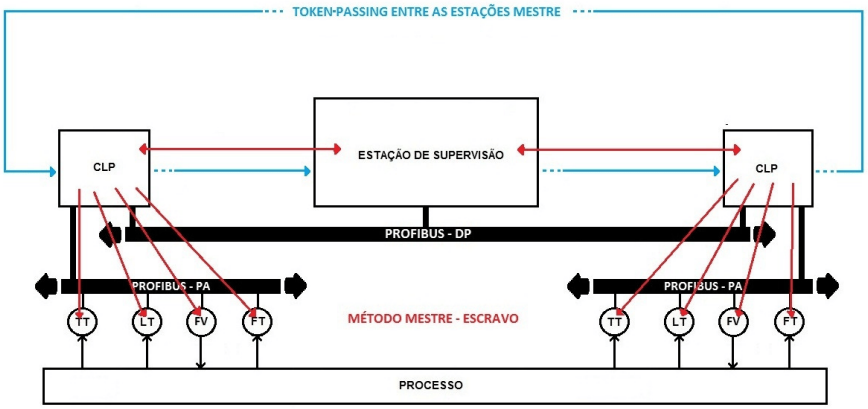


Figura 5.2 – Métodos de acesso ao meio: mestre-escravo e token passing. **Fonte:** *Livro SCADA — versão para análise*, p. 31 (Machado, Pontes e Vianna, reproduzida com crédito).

No método **mestre-escravo**, o mestre conduz a comunicação e cada instrumento responde quando interrogado — é o modelo do Modbus RTU. No **token passing**, o direito de transmitir circula entre as estações mestre por um quadro especial, o que evita colisão sem precisar de um mestre único.

5.3.2 Profibus

O **Profibus** (*Process Field Bus*) foi desenvolvido na Alemanha em 1987 e tornou-se o protocolo de fieldbus mais utilizado na Europa para automação discreta e de processo.

Variantes:

- **Profibus DP** (*Decentralized Peripherals*): otimizado para comunicação rápida com CLPs e módulos de I/O distribuído. Velocidades até 12 Mbps.
- **PROFIBUS PA** (*Process Automation*): para instrumentação de processo; dados e alimentação podem compartilhar o mesmo par. O uso em área classificada e a segurança intrínseca dependem do projeto do segmento, das barreiras/acopladores e dos parâmetros dos dispositivos.

Limitações: padrão aberto e publicado (séries IEC 61158/61784), com requisitos de conformidade e certificação definidos pelo ecossistema PROFIBUS; não trafega nativamente sobre IP — uma opção Ethernet industrial é o **PROFINET**. O diagnóstico depende das ferramentas e da integração disponíveis, não apenas de o protocolo usar ou não IP.

5.3.3 DNP3

O **DNP3** (*Distributed Network Protocol 3*) foi desenvolvido para sistemas SCADA de infraestrutura crítica — energia elétrica, água, óleo e gás. Características:

- Comunicação assíncrona: RTU reporta espontaneamente ao mestre quando uma variável muda (*unsolicited response*) — diferente do polling periódico do Modbus.
- Suporte a **time-stamping** preciso nos eventos — crucial para análise de incidentes em subestações.
- Suporte a comunicação com integridade de dados e autenticação (DNP3 Secure Authentication v5).
- Norma IEEE Std 1815-2012.

5.3.4 Foundation Fieldbus

O **Foundation Fieldbus** (FF) leva a lógica de controle para dentro do próprio barramento: os instrumentos são inteligentes, comunicam-se entre si e podem executar blocos de controle sem passar pelo CLP. Como não existe um mestre único, o direito de uso do meio é arbitrado pelo **LAS** (*Link Active Scheduler*), que distribui as janelas de tempo entre as estações.

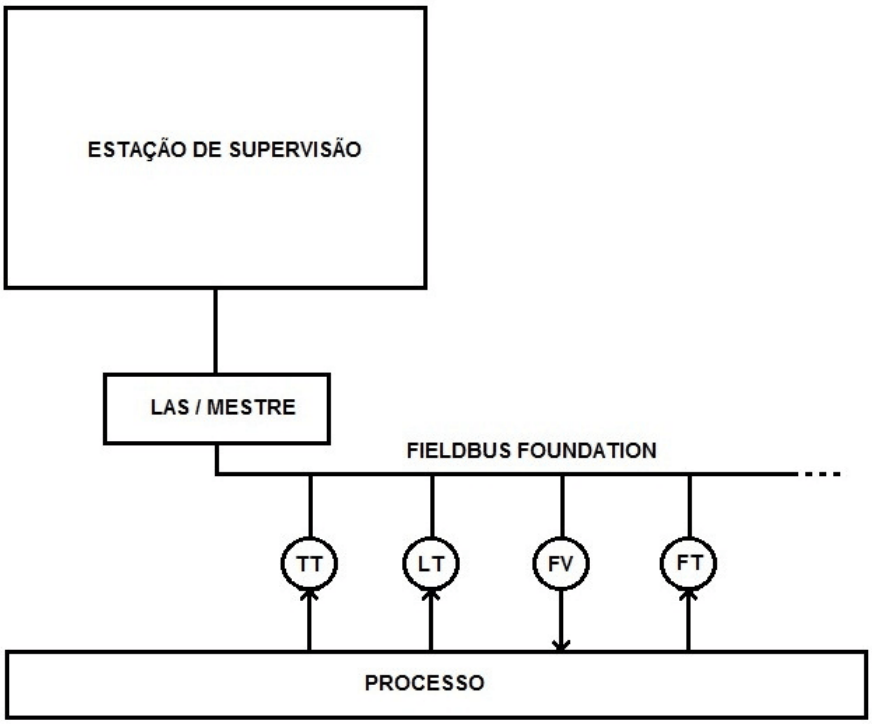


Figura 5.3 – Arquitetura com Foundation Fieldbus. **Fonte:** *Livro SCADA — versão para análise*, p. 29 (Machado, Pontes e Vianna, reproduzida com crédito).

O LAS **não** é um mestre no sentido do Modbus: ele agenda *quando* cada dispositivo pode falar, e outro instrumento habilitado pode assumir a função de LAS ativo se o original falhar. Os transmissores da figura (TT, LT, FV, FT) ligam-se ao mesmo par que os alimenta — característica que viabiliza o uso do FF em áreas classificadas, onde a alimentação intrinsecamente segura e a comunicação precisam compartilhar o cabo.

5.4 Protocolos Modernos

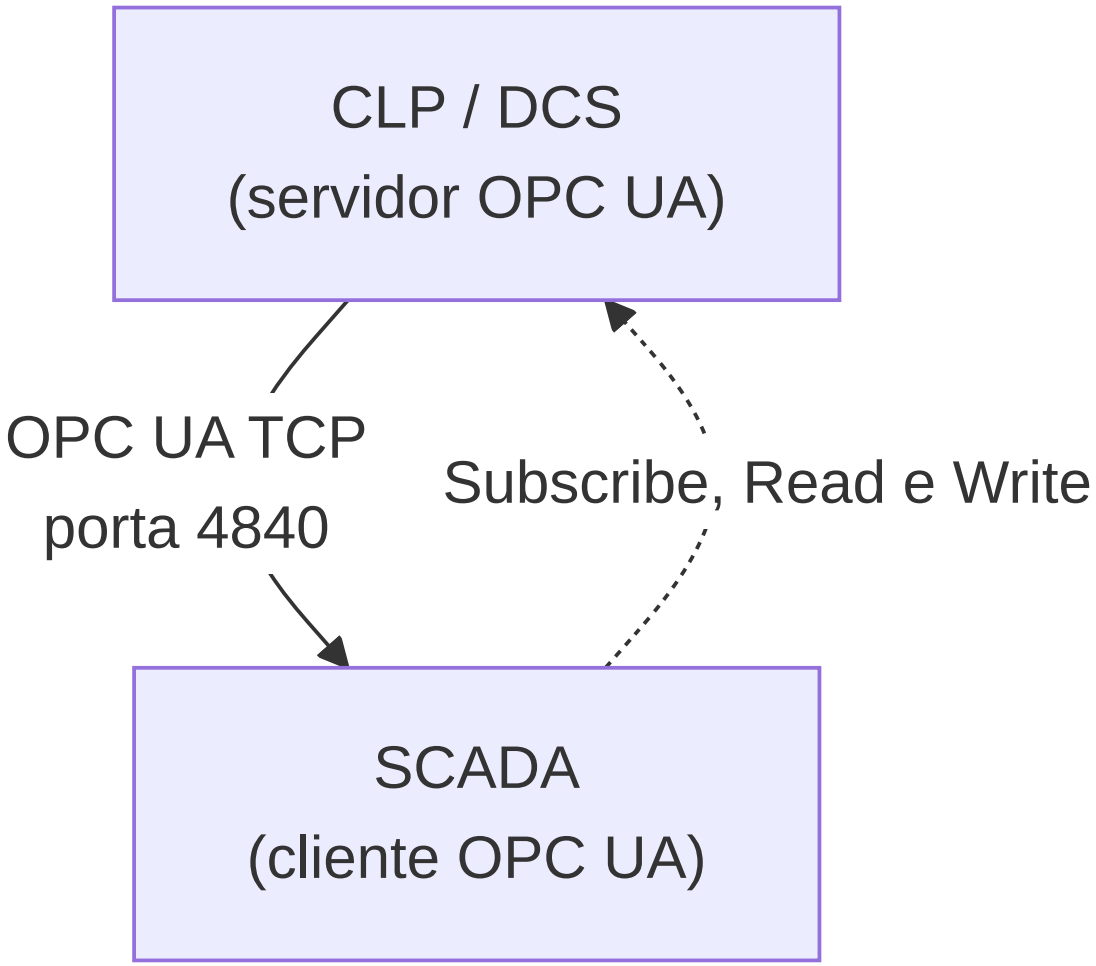
5.4.1 OPC e OPC UA

O **OPC** (*OLE for Process Control*) surgiu em 1996 como padrão de interoperabilidade entre softwares SCADA e CLPs no ambiente Windows. O problema: dependia fortemente de tecnologia Microsoft COM/DCOM, limitando seu uso a redes Windows.

O **OPC UA** (*Unified Architecture*), lançado em 2008 pela OPC Foundation, resolve essas limitações:

- **Plataforma-independente:** funciona em Linux, embarcados, CLPs modernos — não mais dependente de Windows.

- **Modelo de informação orientado a objetos:** dados são expostos como objetos com tipos, métodos e eventos — muito mais rico que endereços de registradores.
- **Segurança:** mecanismos definidos pelo OPC UA incluem certificados X.509, autenticação de aplicações e modos de segurança para assinatura e criptografia de mensagens. O perfil efetivo depende da política de segurança, do transporte e da configuração de cliente e servidor.
- **Transporte flexível:** TCP binário (eficiente) ou HTTPS/WebSocket (firewall-friendly).
- **Publisher-Subscriber:** adicionado no OPC UA Pub/Sub, permitindo comunicação via MQTT ou UDP multicast — integração nativa com IIoT.



Figura

[**Figura 5.4 – Arquitetura OPC UA: servidor no CLP, cliente no SCADA**] *Diagrama mostrando CLP com OPC UA Server integrado, conectado via rede Ethernet ao servidor SCADA (OPC UA Client). Indicar as operações: Read, Write, Subscribe, Browse. Destacar o certificado de segurança na conexão.*

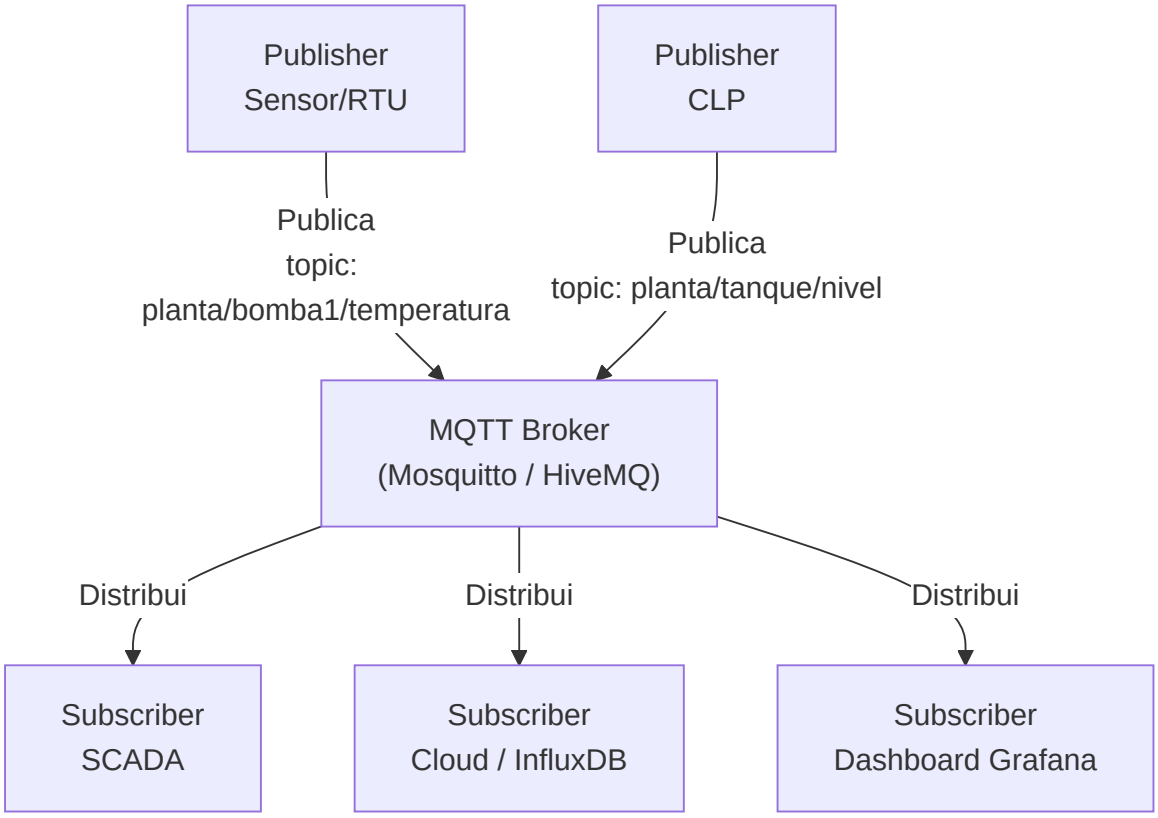
Nota

Nota: OPC UA é uma opção adequada quando a integração SCADA–CLP requer modelo de informação, descoberta e serviços padronizados. A compatibilidade deve ser confirmada por modelo, firmware, licenciamento e perfil suportado; não presuma que todo CLP de qualquer linha tenha servidor OPC UA habilitado.

5.4.2 MQTT

O **MQTT** (*Message Queuing Telemetry Transport*) foi criado em 1999 pela IBM para telemetria em oleodutos via satélite — com requisitos de baixo consumo de banda. Hoje é o protocolo dominante para IIoT.

Arquitetura Publish-Subscribe:



Figura

[**Figura 5.5 – Modelo Publish-Subscribe do MQTT**] *Diagrama colorido mostrando múltiplos publishers (CLPs, sensores IIoT, edge gateways) publicando em um broker central (mosquitto). Múltiplos subscribers (SCADA, banco de dados, dashboards, mobile app) recebendo dados por tópicos. Destacar a escalabilidade horizontal.*

Características técnicas:

- Leve: overhead mínimo (cabeçalho de 2 bytes) — ideal para dispositivos com memória e banda limitadas.

- **QoS** (*Quality of Service*): três níveis de entrega de mensagens: 0 (no máximo uma vez), 1 (pelo menos uma vez) e 2 (uma vez no intercâmbio MQTT). QoS 2 não garante que a aplicação execute um efeito de negócio exatamente uma vez após reconexões ou reprocessamento.
- **Retained messages:** o broker armazena o último valor de cada tópico; novos subscribers recebem imediatamente.
- **Last Will and Testament (LWT):** mensagem publicada automaticamente se o cliente desconectar inesperadamente — mecanismo de detecção de falha.
- Portas convencionais: 1883 para MQTT sobre TCP sem TLS e 8883 para MQTT sobre TLS. A segurança depende da configuração de TLS, autenticação e autorização no broker e nos clientes.
- **Sparkplug B:** especificação sobre MQTT com esquema de dados padronizado para IIoT industrial — define payloads, timestamps e gerenciamento de sessão.

Brokers MQTT populares:

- **Mosquitto** (Eclipse Foundation) — open-source, leve, amplamente usado.
- **HiveMQ** — enterprise, alta escalabilidade, clustering.
- **EMQX** — alta performance, suporte a MQTT 5.0.
- **AWS IoT Core, Azure IoT Hub** — brokers gerenciados na nuvem.

5.4.3 PROFINET

O **PROFINET** é um padrão de comunicação para Ethernet industrial desenvolvido pela PI (PROFIBUS & PROFINET International). É uma opção para aplicações que usam PROFIBUS ou outras redes de campo, mas não é uma substituição direta em todos os projetos.

- **PROFINET RT** (*Real-Time*): para I/O distribuído — ciclos de 1–4 ms.
- **PROFINET IRT** (*Isochronous Real-Time*): para motion control — ciclos de 250 μs com sincronismo de relógio.
- Compatível com infraestrutura Ethernet padrão (switches, cabos Cat5e/6).

5.4.4 EtherNet/IP

O **EtherNet/IP** (*Ethernet Industrial Protocol*) é o protocolo da Rockwell Automation e ODVA para automação discreta. Usa o protocolo CIP (*Common Industrial Protocol*) encapsulado em TCP/UDP/IP. Muito utilizado no ecossistema Allen-Bradley.

Esses protocolos atendem a funções diferentes e podem coexistir na mesma arquitetura. A seleção deve considerar aplicação, desempenho, interoperabilidade, segurança, ciclo de vida e capacidade de manutenção, não apenas se o protocolo é descrito como clássico ou moderno.

Uma mesma planta pode combinar protocolos de campo e de integração, usando gateways quando as interfaces não forem compatíveis. A tabela resume características típicas; a seleção final exige verificar versão, perfil, limites, diagnóstico e suporte no equipamento escolhido.

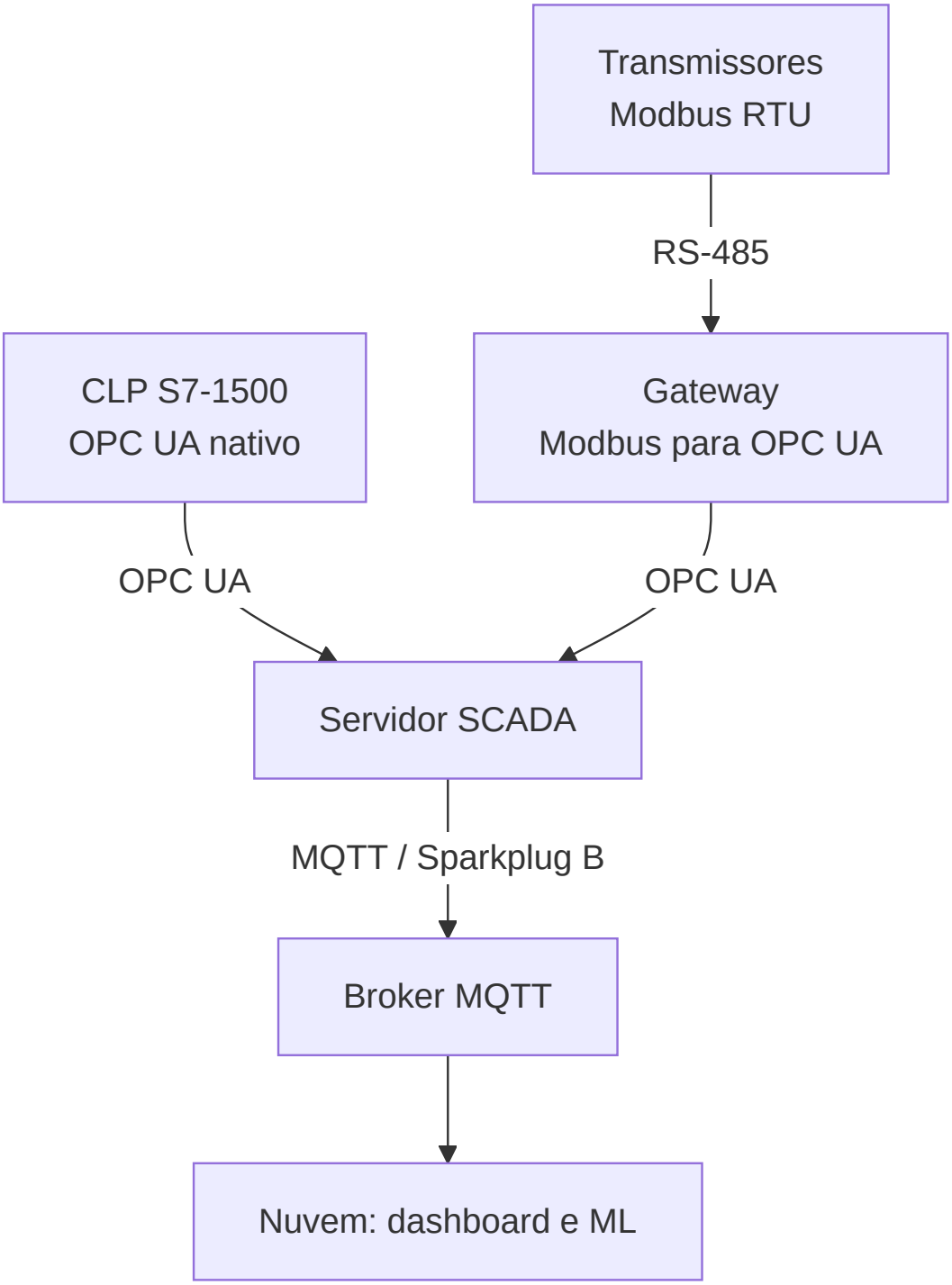
5.5 Comparativo de Protocolos

Tabela 5.2 – Principais protocolos industriais: comparativo

Protocolo	Camada Física	Topologia	Velocidade	Segurança	Aplicação Principal
Modbus RTU	RS-485	Barramento	9,6k–115,2 kbps	Nenhuma nativa	CLP instrumento
Modbus TCP	Ethernet / TCP/IP	Conforme a rede IP	Limitada pela rede	Modbus Security: TLS	CLP–SCADA
PROFIBUS DP	RS-485	Barramento	até 12 Mbps	Nenhuma nativa	Automação discreta
DNP3	Serial / TCP	Barramento / Estrela	variável	SAv5 opcional	SCADA energia/água
OPC DA	Ethernet (DCOM)	Estrela	variável	Windows DCOM	SCADA legado Windows
OPC UA	Ethernet / IP	Cliente-servidor ou Pub/Sub	variável	X.509; segurança conforme perfil	Integração semântica e troca de dados
MQTT	Ethernet / redes IP	Pub/Sub (broker)	variável	TLS e autorização no broker	IIoT, telemetria, nuvem
PROFINET	Ethernet	Estrela/Anel	100 Mbps	Opcional	I/O distribuído Siemens
EtherNet/IP	Ethernet	Estrela	100 Mbps / 1 Gbps	CIPsecurity	I/O Rockwell/AB

5.6 Estudo de Caso — Integração Modbus + OPC UA + MQTT

Contexto: Uma planta química possui CLPs Siemens S7-1500 (OPC UA nativo) e instrumentos de campo com Modbus RTU (transmissores antigos sem Ethernet).


Transmissores
Modbus RTU
RS-485
CLP S7-1500
OPC UA nativo
Gateway
Modbus para OPC UA
OPC UA
OPC UA
Servidor SCADA
MQTT / Sparkplug B
Broker MQTT
Nuvem: dashboard e ML

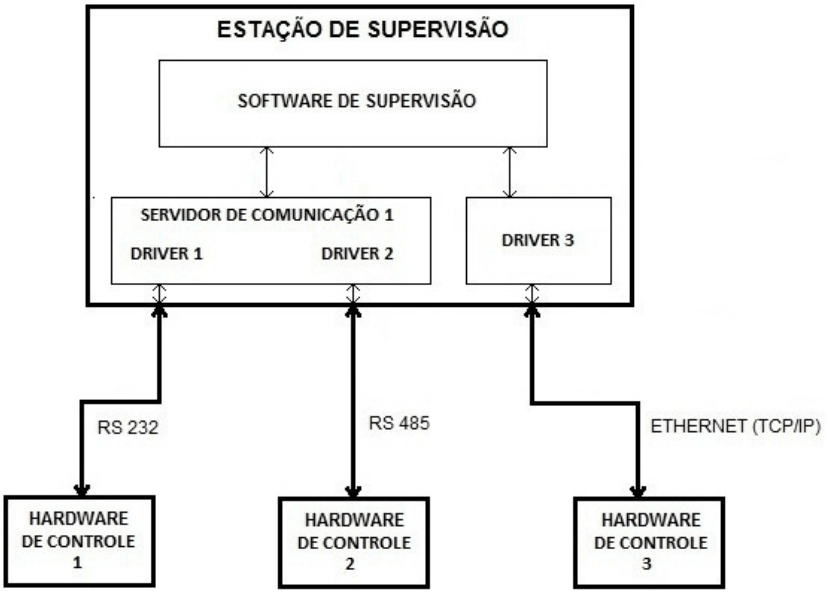


Figura 5.6 – Integração de múltiplos protocolos em um servidor de supervisão. **Fonte:** *Livro SCADA — versão para análise*, p. 25 (Machado, Pontes e Vianna, reproduzida com crédito).

O software de supervisão não fala "Modbus" nem "PROFIBUS": ele conversa com **drivers**. Na figura, três hardwares de controle com interfaces distintas (RS-232, RS-485 e Ethernet) chegam à mesma estação de supervisão por dois caminhos — dois deles compartilham um servidor de comunicação com dois drivers, e o terceiro exige um driver independente, o que obriga o supervisório a manter **dois links lógicos**. É por isso que o esforço de integração se mede pelo número de links e não pelo número de equipamentos.

Gateways de protocolo populares: Moxa MGate (Modbus→OPC UA), Kepware (OPC UA para praticamente qualquer protocolo), Ignition OPC UA Module, Prosys OPC UA.

5.7 Resumo

- Redes industriais priorizam **determinismo e disponibilidade**; redes corporativas priorizam **largura de banda e confidencialidade**.
- **Modbus** (1979) é o protocolo mais simples e difundido; ainda amplamente utilizado em instrumentação.
- **DNP3** é o padrão para SCADA de infraestrutura crítica (energia, água).
- **OPC UA** oferece serviços e modelos de informação úteis à integração SCADA–CLP; segurança efetiva requer perfil compatível, certificados e configuração adequada.
- **MQTT** domina o IIoT: leve, escalável, arquitetura pub/sub com broker central.
- A integração multiprotocolo via gateways e OPC UA é a realidade de plantas industriais com equipamentos de diferentes gerações.

5.8 Questões de Revisão

Conceituais:

1. Quais são as principais diferenças entre o Modbus RTU e o Modbus TCP?
2. Por que o OPC UA substituiu o OPC DA como padrão de interoperabilidade? Liste três vantagens do OPC UA.
3. Explique a arquitetura Publish-Subscribe do MQTT. Qual é o papel do broker?

Práticas:

4. Um inversor de frequência WEG CFW-11 e um transmissor de temperatura Endress+Hauser iTEMP TMT série são conectados via Modbus RTU a um CLP. O SCADA precisa ler a temperatura e a frequência de saída do inversor. Quantos registradores Modbus serão lidos (aproximadamente)? Qual é o endereço da função de leitura?
5. Cite três critérios que influenciam a escolha entre OPC UA e MQTT para a comunicação entre um CLP e um sistema SCADA.

Desafio:

6. Um operador de rede elétrica precisa integrar 200 subestações distribuídas em um estado. Cada subestação possui equipamentos com DNP3 e IEC 61850. Proponha uma arquitetura de comunicação, justificando a escolha dos protocolos em cada camada.

5.9 Referências

- MODBUS ORGANIZATION. *Modbus Application Protocol Specification V1.1b3*. Disponível em: modbus.org.
- OPC FOUNDATION. *OPC Unified Architecture Specification*. Disponível em: opcfoundation.org.
- OASIS. *MQTT Version 5.0 Specification*. Disponível em: docs.oasis-open.org.
- IEEE Std 1815-2012: *IEEE Standard for Electric Power Systems Communications — DNP3*.
- PI (PROFIBUS & PROFINET International). *PROFINET System Description*. Disponível em: profibus.com.
- MACHADO, C. F. B.; PONTES, M. O.; VIANNA, W. S. *SCADA — Supervisory Control and Data Acquisition: sistemas de supervisão e aquisição de dados para automação*. (Material de origem desta apostila; figuras dos autores reproduzidas com crédito.)

Capítulo 6

IIoT e Tecnologias Emergentes

Objetivos de Aprendizagem

Ao final deste capítulo, o leitor será capaz de:

- Distinguir IoT de IIoT e compreender seus requisitos industriais específicos.
- Descrever as arquiteturas de Edge, Fog e Cloud Computing e seu papel na automação.
- Compreender o conceito de Digital Twin e suas aplicações industriais.
- Identificar ferramentas de análise de dados industriais (Grafana, InfluxDB, Kafka).

6.1 IoT versus IIoT

A **IoT** (*Internet of Things* — Internet das Coisas) é o paradigma em que objetos físicos são conectados à internet, coletando e trocando dados. Aplicações populares: termostatos inteligentes, câmeras de segurança, rastreadores de fitness.

O **IIoT** (*Industrial Internet of Things* — Internet Industrial das Coisas) aplica esses conceitos ao ambiente industrial, mas com requisitos muito mais rigorosos:

Tabela 6.1 – IoT × IIoT: principais diferenças

Critério	IoT (Consumidor)	IIoT (Industrial)
Confiabilidade	Tolerável falha ocasional	Falha pode causar acidente ou parada de planta
Latência	Segundos a minutos	Milissegundos a segundos
Ciclo de vida	3–5 anos	15–30 anos
Ambiente	Ambientes domésticos	Calor, vibração, poeira, EMI, atmosfera explosiva
Segurança	Importante	Crítica — sistemas de segurança funcional
Integração	App + nuvem	CLP + SCADA + MES + ERP
Protocolos	HTTP, CoAP, MQTT, BLE	MQTT, OPC UA, Modbus, DNP3, Sparkplug B

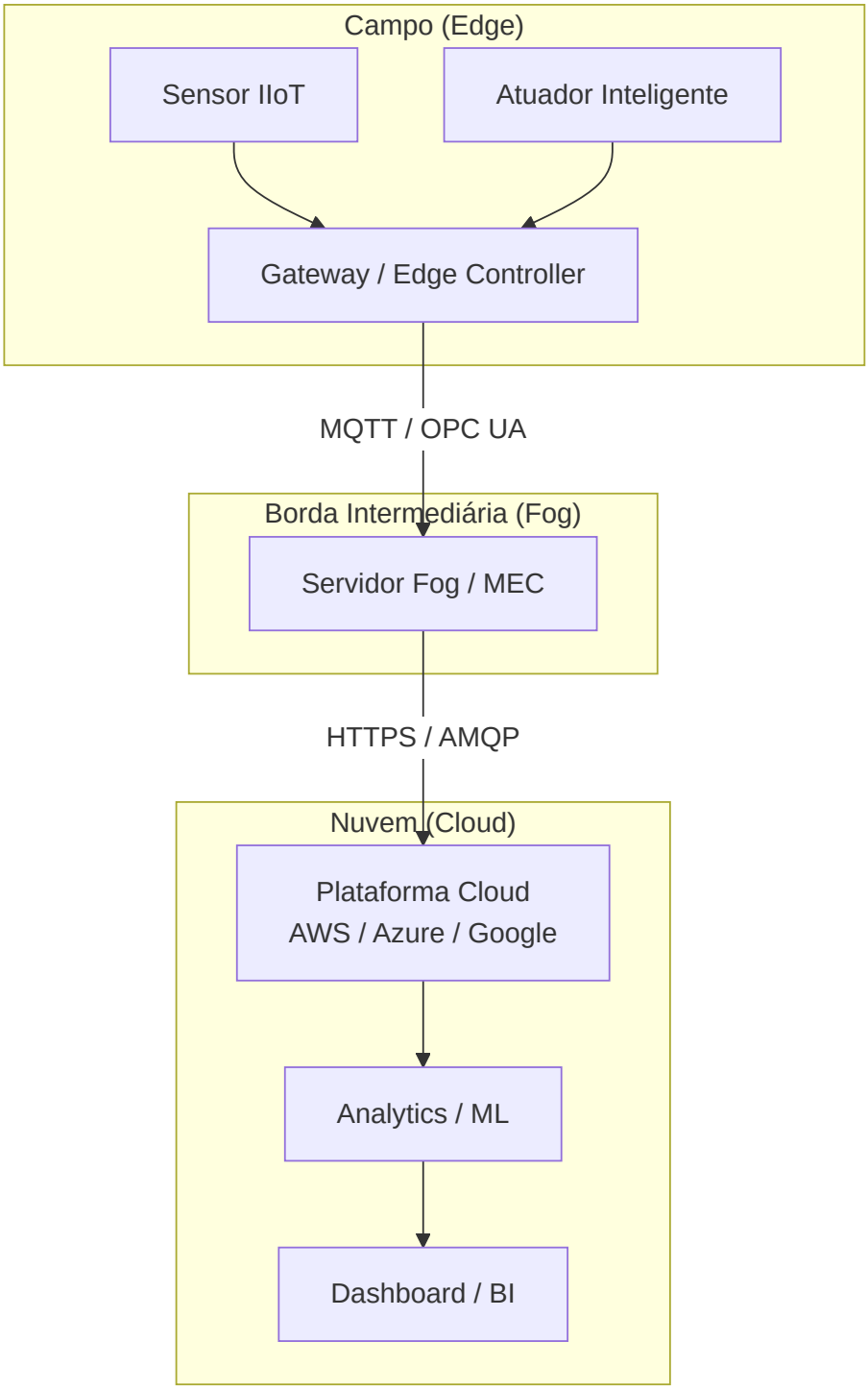
Critério	IoT (Consumidor)	IIoT (Industrial)
Normas	ETSI, IEEE	IEC 62443, ISA-99, IEC 61508

Figura

[**Figura 6.1 – IoT versus IIoT: ambientes e requisitos**] *Duas ilustrações lado a lado: (esquerda) casa inteligente com dispositivos IoT consumidor (termostato, lâmpada, câmera, smartwatch); (direita) planta industrial com sensores IIoT, gateway de borda, CLP e servidor SCADA. Destacar os requisitos distintos: segurança, latência, robustez.*

6.2 Arquitetura IIoT: Edge, Fog e Cloud

A arquitetura IIoT é tipicamente organizada em três camadas:



Figura

[Figura 6.2 – Arquitetura IIoT em três camadas: Edge, Fog e Cloud] *Diagrama em camadas horizontais com ícones: campo industrial (sensores, CLPs, RTUs) na base; gateway/edge controller no meio; e a nuvem no topo com plataformas AWS IoT, Azure IoT Hub e ferramentas de analytics. Indicar os protocolos em cada enlace.*

6.2.1 Edge Computing

O **Edge Computing** (computação na borda) leva o processamento o mais próximo possível da fonte de dados — no próprio equipamento ou em um gateway de campo.

Por que processar na borda? - **Latência:** ações de controle críticas não podem esperar a nuvem (viagem de ida e volta ~100 ms). - **Largura de banda:** pré-processar e filtrar dados na borda reduz drasticamente o volume a ser transmitido à nuvem. - **Operação offline:** o sistema continua funcionando mesmo sem conectividade com a nuvem. - **Privacidade:** dados sensíveis de produção não precisam sair da planta.

Exemplos de edge devices: - **Gateways industriais:** Moxa UC-8100, Advantech UNO-2372G, Siemens SIMATIC IPC. - **Edge controllers:** PLCnext (Phoenix Contact), Bedrock Automation, Beckhoff C6015. - **Microprocessadores embarcados:** Raspberry Pi (prototipagem), NVIDIA Jetson (IA na borda).

Dica

Dica: O **AWS IoT Greengrass** e o **Azure IoT Edge** são plataformas de software que transformam um gateway industrial em um nó de computação inteligente — executando funções Lambda/containers localmente, com sincronização automática com a nuvem quando conectado.

6.2.2 Fog Computing

O **Fog Computing** adiciona uma camada intermediária entre o edge e a nuvem — geralmente um servidor instalado na subestação, sala de controle da planta ou datacenter de borda (*MEC — Mobile Edge Computing*).

Funções do Fog: - Agregação de dados de múltiplos gateways de campo. - Processamento de analytics local (detecção de anomalias, correlação de eventos). - Cache de dados para resiliência a falhas de conectividade WAN.

6.3 Plataformas IIoT em Nuvem

As principais plataformas de nuvem oferecem serviços especializados para IIoT:

Tabela 6.2 – Plataformas cloud para IIoT

Plataforma	Empresa	Serviços IIoT	Diferencial
AWS IoT Core	Amazon	Broker MQTT gerenciado, Rules Engine, Greengrass	Ecossistema AWS; integração Lambda, S3, SageMaker
Azure IoT Hub	Microsoft	Device management, DPS, Stream Analytics	Integração com Azure Digital Twins e Power BI
Google Cloud IoT	Google	Pub/Sub, BigQuery, Vertex AI	Analytics e ML nativo

Plataforma	Empresa	Serviços IIoT	Diferencial
AVEVA Insight	AVEVA	Historização industrial em nuvem	Integrado ao PI System e System Platform
Ignition Cloud Edition	Inductive Automation	SCADA nativo em nuvem	Mesma experiência Ignition on-premises
Siemens MindSphere	Siemens	IoT industrial, analytics, apps	Integração nativa com Siemens TIA Portal

6.4 Digital Twin

O **Digital Twin** (*Gêmeo Digital*) é uma representação virtual dinâmica de um ativo físico, processo ou sistema que sincroniza em tempo real com seu equivalente real.

6.4.1 Tipos de Digital Twin

Tipo	Descrição	Exemplo
Digital Twin de Componente	Modelo de um único componente	Curva de desgaste de um rolamento
Digital Twin de Sistema	Modelo de um conjunto de equipamentos	Linha de produção completa
Digital Twin de Processo	Modelo do processo produtivo	Refinaria simulada em tempo real
Digital Twin de Planta	Modelo completo de uma instalação industrial	Fábrica virtual com layout 3D

6.4.2 Aplicações

- **Manutenção preditiva:** o modelo virtual prevê quando um componente atingirá o limite de desgaste com base nos dados reais operados.
- **Otimização de processo:** simular ajustes de setpoints no twin antes de aplicar na planta real — sem risco de parada.
- **Treinamento de operadores:** simuladores realistas sem risco à operação real.
- **Comissionamento virtual:** testar a lógica de controle no twin antes de ligar o equipamento físico.
- **Análise de falhas:** reconstruir o evento de uma falha usando os dados históricos do twin.

Figura

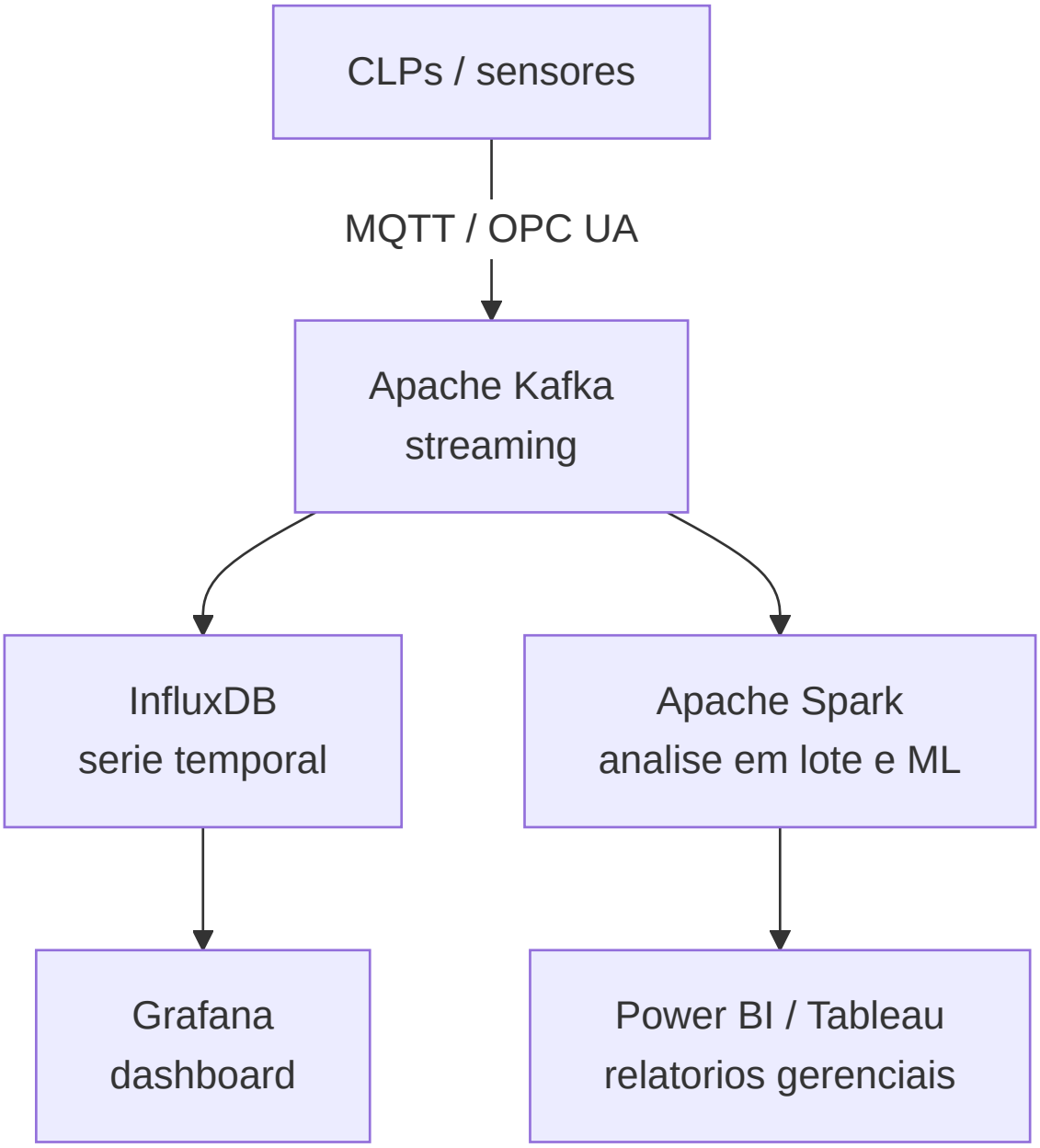
[**Figura 6.3 – Conceito de Digital Twin industrial**] *Diagrama mostrando: (esquerda) planta física com sensores enviando dados via SCADA/IIoT; (direita) representação virtual 3D da planta com os mesmos dados sincronizados em tempo real. Setas bidirecionais indicando o fluxo de dados e feedback de otimização.*

Plataformas de Digital Twin: Azure Digital Twins, AWS IoT TwinMaker, AVEVA E3D Digital Twin, Siemens NX / Tecnomatix, Bentley iTwin, ANSYS Twin Builder.

6.5 Analytics Industrial e Big Data

A coleta massiva de dados de processo gerou a necessidade de ferramentas especializadas:

6.5.1 Stack Típica de Analytics Industrial



Figura

[**Figura 6.4 – Stack de analytics industrial**] *Diagrama pipeline de dados: campo → Kafka (streaming) → InfluxDB (armazenamento) → Grafana (visualização operacional) e → Spark (processamento batch) → BI (relatórios gerenciais). Ícones oficiais de cada ferramenta.*

Apache Kafka: plataforma de streaming distribuído — ideal para processar fluxos de dados de centenas de equipamentos simultaneamente, com retenção configurável e tolerância a falhas.

InfluxDB + Grafana: a dupla open-source mais popular para monitoramento industrial. O InfluxDB armazena séries temporais com compressão eficiente; o Grafana

exibe dashboards interativos com alertas.

6.5.2 Machine Learning Aplicado à Indústria

Aplicação	Técnica de ML	Ferramenta
Manutenção preditiva	Classificação, anomalia (Isolation Forest, LSTM)	AWS SageMaker, Azure ML, Python/scikit-learn
Otimização de processo	Regressão, otimização bayesiana	Aspen GDOT, TIBCO Spotfire
Visão computacional (qualidade)	CNN (VGG, ResNet, YOLO)	NVIDIA Jetson + TensorRT
Detecção de intrusão (cibersegurança)	Anomalia de rede, clustering	Darktrace, Claroty

Atenção

Atenção: Modelos de ML para uso industrial devem ser **explicáveis** (*Explainable AI*). Um operador precisa entender por que o sistema recomendou uma ação — não pode confiar cegamente em uma "caixa-preta" em decisões que afetam segurança.

6.6 Estudo de Caso — IIoT em Planta de Celulose

Contexto: Uma fábrica de celulose no Brasil desejava reduzir as paradas não programadas de seus digestores (equipamentos críticos que operam a 170°C e 8 bar).

Solução implementada:

1. **Sensores adicionais:** acelerômetros em rolamentos das bombas de circulação + sensores de vibração nos agitadores dos digestores.
2. **Edge gateway:** coletou dados de vibração a 1 kHz; calculou localmente FFT (análise espectral) e transmitiu apenas os indicadores de frequência (não os dados brutos).
3. **Plataforma IIoT:** Azure IoT Hub recebeu os indicadores; modelo LSTM detectou padrões precursores de falha.
4. **Integração SCADA:** o modelo enviava alertas ao sistema SCADA (AVEVA System Platform) com antecedência de 72–120 horas antes da falha.

Resultados: redução de 68% nas paradas não programadas dos digestores; ROI atingido em 11 meses.

Figura

[**Figura 6.5 – Arquitetura IIoT da planta de celulose**] *Diagrama completo do estudo de caso: digestores com sensores de vibração → edge gateway com FFT → Azure IoT Hub → modelo LSTM → alerta no SCADA. Fotos ou ilustrações dos digestores industriais. Timeline mostrando detecção antecipada vs. falha real.*

6.7 Resumo

- O **IIoT** aplica o paradigma IoT à indústria com requisitos rigorosos de confiabilidade, latência e segurança.
- A arquitetura IIoT divide-se em três camadas: **Edge** (campo), **Fog** (intermediária) e **Cloud** (análise e gestão).
- O **Edge Computing** é essencial para controle em tempo real, redução de banda e operação offline.
- O **Digital Twin** permite simulação, otimização e manutenção preditiva sem risco à planta real.
- A stack de analytics industrial (Kafka + InfluxDB + Grafana) democratizou o acesso a ferramentas de Big Data para o setor industrial.

6.8 Questões de Revisão

Conceituais:

1. Cite três diferenças fundamentais entre IoT consumidor e IIoT industrial.
2. O que é Edge Computing? Por que processar dados na borda é vantajoso em aplicações industriais?
3. Explique o conceito de Digital Twin. Quais são os quatro tipos de Digital Twin descritos neste capítulo?

Práticas:

4. Uma empresa de mineração deseja monitorar remotamente 50 correias transportadoras, coletando vibração, temperatura de rolamentos e corrente dos motores. Proponha uma arquitetura IIoT com as camadas Edge, Fog e Cloud, especificando protocolos e ferramentas em cada camada.
5. Uma fábrica coleta 200 variáveis a cada segundo de um CLP. Calcule o volume de dados diário (em GB) sem compressão (assumindo float32 = 4 bytes por variável). Justifique por que o processamento na borda e a compressão de séries temporais são essenciais.

Desafio:

6. Elabore um projeto conceitual de Digital Twin para uma estação de bombeamento de água. Descreva: quais variáveis seriam sincronizadas, que modelo físico seria utilizado, como o twin poderia prever falhas e como ele se integraria ao SCADA existente.

6.9 Referências

- IIC (Industrial Internet Consortium). *Industrial Internet Reference Architecture (IIRA)*. IIC, 2019.

- MICROSOFT. *Azure IoT Reference Architecture*. Disponível em: `docs.microsoft.com/azure/architecture`.
- AMAZON. *AWS IoT Core Documentation*. Disponível em: `docs.aws.amazon.com/iot`.
- GRIEVES, M. *Digital Twin: Manufacturing Excellence through Virtual Factory Replication*. White Paper, 2014.
- INFLUXDATA. *InfluxDB and Grafana for Industrial IoT*. Disponível em: influxdata.com.

Capítulo 7

Node-RED como Interface Supervisória

Objetivos de Aprendizagem

Ao final deste capítulo, o leitor será capaz de:

- Descrever a arquitetura e os conceitos fundamentais do Node-RED.
- Criar flows para aquisição de dados via Modbus TCP e MQTT.
- Desenvolver dashboards de supervisão com o módulo node-red-dashboard.
- Avaliar quando Node-RED é adequado como solução supervisória e suas limitações.

7.1 O que é Node-RED?

O **Node-RED** é uma plataforma de programação visual *flow-based* desenvolvida pela IBM em 2013, inicialmente para conectar dispositivos de hardware, APIs e serviços online de forma rápida. Tornou-se open-source e hoje é mantido pela **OpenJS Foundation**.

Sua filosofia: **"wiring together"** — conectar entradas, processamentos e saídas arrastando e soltando nós (*nodes*) em um editor visual, sem escrever código complexo.

Por que Node-RED para automação?

- **Baixo custo:** open-source, sem licenciamento.
- **Prototipagem rápida:** um flow funcional em minutos.
- **Biblioteca ampla:** mais de 4.000 nodes na comunidade (Modbus, OPC UA, MQTT, HTTP, banco de dados).
- **Leve:** roda em Raspberry Pi, gateways industriais, servidores Linux.
- **Dashboard integrado:** módulo `node-red-dashboard` gera interfaces web sem HTML/CSS.

Na prática, o Node-RED costuma ocupar a posição de **camada de integração**: ele conversa com quem tem protocolo industrial, normaliza e entrega para quem consome dado.

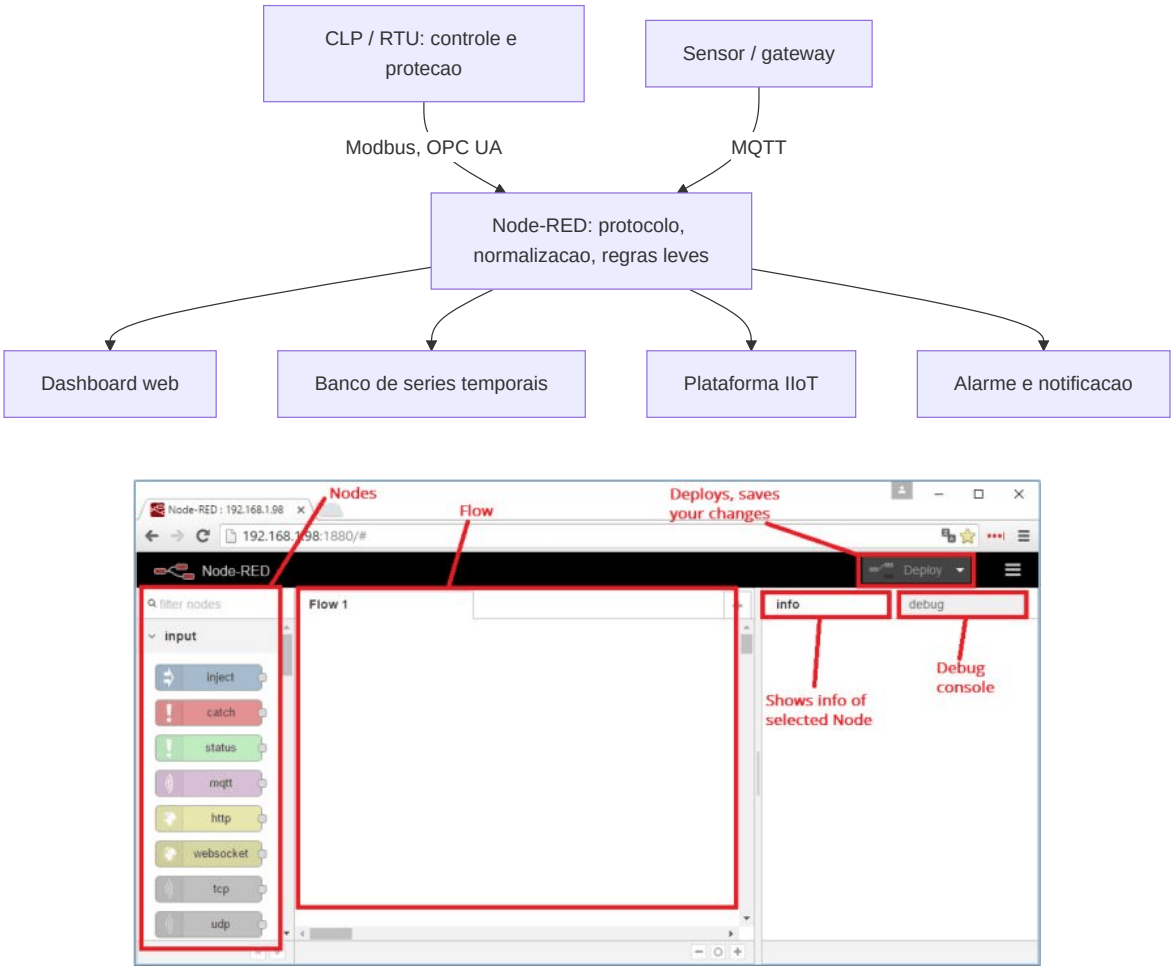


Figura 7.1 – Interface do editor Node-RED. **Fonte:** *Node-RED — Interface de Supervisão* (material do curso), p. 13.

O editor é uma única página web dividida em quatro regiões, cada uma com um papel definido:

Região	Posição	Função
Palette	esquerda	Catálogo de nodes disponíveis, agrupados por categoria
Canvas	centro	Área de desenho do flow; os fios ligam a saída de um node à entrada de outro
Sidebar	direita	Saída do node `debug`, ajuda do node selecionado, contexto e configurações
Deploy	topo	Publica o flow em edição para execução

A figura anota as três primeiras regiões. O botão **Deploy** é o que separa “desenho” de “sistema em operação”: enquanto não for acionado, o processo continua rodando com a

versão anterior do flow.

7.2 Instalação e Configuração

7.2.1 Instalação via npm (Linux / Raspberry Pi)

```
# Instalar Node.js (versão LTS recomendada)
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo bash -
sudo apt-get install -y nodejs

# Instalar Node-RED globalmente
sudo npm install -g --unsafe-perm node-red

# Iniciar Node-RED
node-red
```

7.2.2 Instalação via Docker

```
docker run -it -p 1880:1880 \
  -v node_red_data:/data \
  --name mynodered nodered/node-red
```

7.2.3 Acessando o Editor

Abra o navegador em: `http://localhost:1880`

Dica

Dica: Para uso em produção, configure o Node-RED com autenticação de usuários (arquivo `settings.js`), HTTPS e uma solução de inicialização automática como `pm2` ou `systemd`, para que o serviço reinicie automaticamente após reboot.

7.3 Conceitos Fundamentais

7.3.1 Nodes (Nós)

Os **nodes** são os blocos de construção do Node-RED. Cada node realiza uma função específica:

Categoria	Exemplos de Nodes	Função
Entrada (Input)	`mqtt in`, `modbus-read`, `http in`, `inject`	Recebe dados ou eventos
Processamento	`function`, `change`, `filter`, `json`	Transforma ou filtra dados
Saída (Output)	`mqtt out`, `modbus-write`, `http request`, `debug`	Envia dados ou executa ações
Dashboard	`gauge`, `chart`, `text`, `button`, `slider`	Exibe dados na interface web

Categoria	Exemplos de Nodes	Função
Banco de Dados	`influxdb in`, `influxdb out`, `mongodb`	Persistência de dados
Utilitários	`delay`, `trigger`, `rbe` (report-by-exception)	Controle de fluxo

7.3.2 Flows e Wires

Um **flow** é uma sequência de nodes conectados por **wires** (fios). Cada wire transporta uma mensagem (`msg`) do node anterior para o próximo.

A mensagem é um objeto JSON com campos padrão:

```
{
  "topic": "planta/bomba1/temperatura",
  "payload": 72.5,
  "timestamp": 1704067200000,
  "retain": false
}
```

- `msg.payload`: o dado principal — número, texto, objeto ou buffer.
- `msg.topic`: identificador do dado (muito usado com MQTT).
- Campos adicionais podem ser adicionados em qualquer node.

7.3.3 Context (Contexto)

O Node-RED permite armazenar dados persistentes em três escopos:

- **Node context:** disponível apenas para aquele node.
- **Flow context:** compartilhado entre todos os nodes do mesmo flow.
- **Global context:** compartilhado entre todos os flows da aplicação.

```
// Exemplo: incrementar contador no flow context
let count = flow.get('counter') || 0;
count++;
flow.set('counter', count);
msg.payload = count;
return msg;
```

7.4 Aquisição de Dados via Modbus TCP

7.4.1 Instalando o Node Modbus

No editor Node-RED, acesse **Menu** → **Manage Palette** → **Install** e instale:

```
node-red-contrib-modbus
```

7.4.2 Flow: Leitura de Temperatura via Modbus TCP

```
[Modbus Read] ---> [Function (converter)] ---> [MQTT Out]
     |                                              |
 (CLP/RTU)                                  (broker MQTT)
```

Configuração do node `Modbus Read`: - Server: IP do CLP (ex.: `192.168.1.100`), porta `502` - FC (Function Code): `3` (Read Holding Registers) - Address: `40001` (registrador de temperatura) - Quantity: `2` (float32 = 2 registradores) - Poll Rate: 1000 ms

Node `function` para converter dois registradores em float:

```
// Converter dois registradores Modbus em float IEEE 754
const buf = Buffer.alloc(4);
buf.writeUInt16BE(msg.payload.data[0], 0);
buf.writeUInt16BE(msg.payload.data[1], 2);
const temperature = buf.readFloatBE(0);
msg.payload = parseFloat(temperature.toFixed(2));
msg.topic = "planta/reator1/temperatura";
return msg;
```

Figura

[Figura 7.2 – Flow Node-RED: leitura Modbus TCP e publicação MQTT]
Screenshot do editor Node-RED mostrando o flow completo: node "Modbus Read" (cor laranja) → node "function" → node "mqtt out". Mostrar as configurações do node Modbus: IP do CLP, FC3, endereço e quantidade. Destacar o debug node mostrando o valor 72.5°C.

7.5 Comunicação MQTT no Node-RED

7.5.1 Configurando Conexão com Broker MQTT

Os nodes `mqtt in` e `mqtt out` compartilham uma configuração de broker:

- **Server:** `localhost` (Mosquitto local) ou endereço do broker externo.
- **Port:** `1883` (sem TLS) ou `8883` (com TLS).
- **Client ID:** identificador único do Node-RED no broker.
- **Username/Password:** credenciais (se o broker exigir autenticação).

7.5.2 Flow: Assinatura de Tópico MQTT e Alarme

```
// Node function: verificar temperatura e gerar alarme
const temp = msg.payload;
if (temp > 85) {
    msg.payload = {
        level: "CRITICO",
        value: temp,
        message: `Temperatura alta: ${temp} degC`,
        timestamp: new Date().toISOString()
    };
    msg.topic = "planta/reator1/alarmes";
    return msg;
}
return null; // não propagar se temperatura normal
```

7.6 Dashboard — Interface de Supervisão Web

O módulo `node-red-dashboard` transforma flows em interfaces web responsivas acessíveis pelo navegador.

7.6.1 Instalação

```
Menu -> Manage Palette -> Install -> node-red-dashboard
```

O dashboard fica acessível em: `http://localhost:1880/ui`

7.6.2 Widgets Disponíveis

Widget	Descrição	Uso típico
`gauge`	Velocímetro/medidor analógico	Temperatura, pressão, nível
`chart`	Gráfico de linhas/barras em tempo real	Tendência histórica de variável
`text`	Exibição de texto ou valor numérico	Status de equipamento
`slider`	Controle deslizante	Ajuste de setpoint
`button`	Botão de ação	Partida/parada, reconhecimento de alarme
`switch`	Toggle on/off	Habilitar/desabilitar saída
`notification`	Toast de notificação	Alertas de alarme
`table`	Tabela de dados	Log de eventos, lista de variáveis



Figura 7.3 – Dashboard Node-RED: painel de supervisão. **Fonte:** *Node-RED — Interface de Supervisão* (material do curso), p. 4.

O painel foi montado apenas com widgets do `node-red-dashboard`: medidores do tipo `gauge` para os valores instantâneos, `chart` para as tendências e `text` para os totais. Dois pontos merecem atenção didática. Primeiro, os blocos de *status* no topo repetem a mesma ideia de **visão geral** (nível 1) discutida no Capítulo 11. Segundo, o painel convive com uma imagem ao vivo de câmera no mesmo quadro — recurso comum em supervisão de acesso e logística, mas que **não** deve competir com as variáveis de processo pela atenção do operador.

7.6.3 Exemplo de Flow Completo com Dashboard

```
[mqtt in] --> [function: payload] --> [gauge: temperatura]
                                  --> [chart: tendencia]
                                  --> [function: alarme]
                                  --> [notification]
[button: resetar] --> [mqtt out: reset]
```

7.7 Integração com InfluxDB e Grafana

Para histórico e visualização avançada, o Node-RED integra-se facilmente com InfluxDB:

7.7.1 Instalação

```
node-red-contrib-influxdb
```

7.7.2 Node `influxdb out`

Configuração: - Host: `localhost` (ou IP do servidor InfluxDB) - Database: `industrial_data` - Measurement: `temperatura_reator1`

O node recebe uma mensagem com `msg.payload` contendo os campos a gravar:

```
// Preparar dados para InfluxDB (formato line protocol)
msg.payload = [
    {
        measurement: "temperatura",
        tags: { equipamento: "reator1", planta: "unidade_A" },
        fields: { valor: msg.payload },
        timestamp: new Date()
    }
];
return msg;
```

O **Grafana** se conecta ao InfluxDB como fonte de dados e exibe dashboards ricos, com alertas e anotações automáticas.

Figura

[Figura 7.4 – Pipeline Node-RED → InfluxDB → Grafana] *Diagrama do pipeline: SCADA/CLP → Node-RED (flow de coleta e transformação) → InfluxDB (séries temporais) → Grafana (dashboard). Screenshot do Grafana mostrando painel com múltiplas variáveis industriais, seletor de período e alerta configurado.*

7.8 Node-RED versus SCADA Comercial

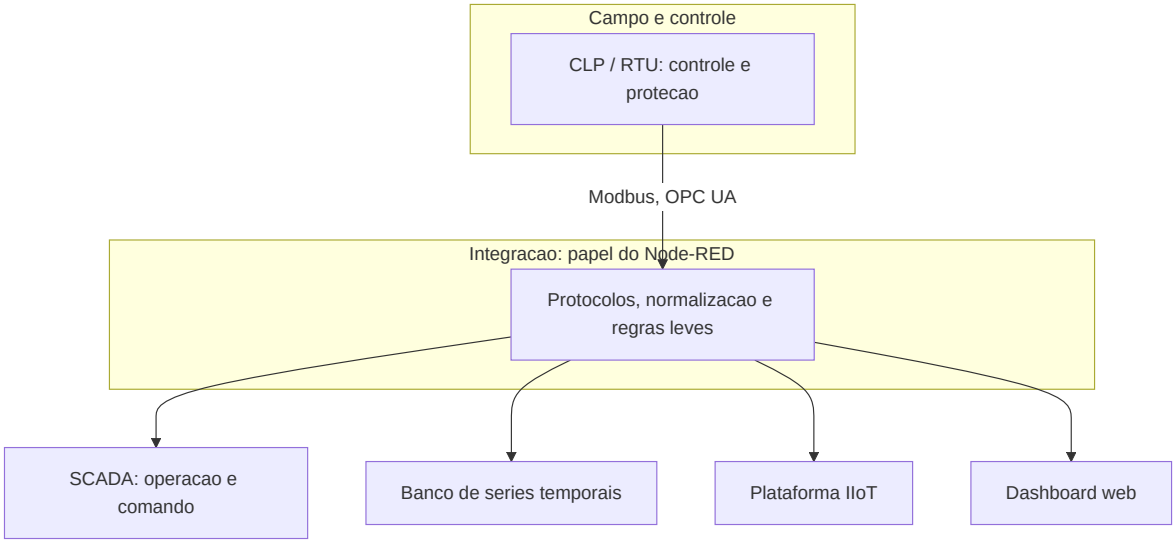
Tabela 7.1 – Comparativo Node-RED × SCADA Comercial

Critério	Node-RED	SCADA Comercial (ex.: Ignition, AVEVA)
Custo	Gratuito (open-source)	Licença cara (US$ 5k–500k+)
Curva de aprendizado	Baixa	Média a alta
Sinóticos gráficos	Limitado (dashboard básico)	Rico: P&IDs, animações, SVG
Segurança / acesso	Básica (plugin)	Controle de acesso robusto, LDAP
Historização	Via plugins externos	Nativa, otimizada
Alarmes	Manual (flow)	Motor de alarmes dedicado, ISA-18.2
Suporte	Comunidade	Suporte profissional 24/7
Certificação	Não certificado	IEC 62443, FDA 21 CFR Part 11
Redundância	Manual (infraestrutura)	Nativa e testada
Escalabilidade	Milhares de tags (com cuidado)	Centenas de milhares de tags

Quando usar Node-RED: - Prototipagem e provas de conceito. - Integrações e glue code entre sistemas. - Dashboards de monitoramento complementares. - Projetos educacionais. - Planta pequena com baixo requisito de disponibilidade.

Quando usar SCADA comercial: - Infraestruturas críticas (energia, água, óleo e gás). - Requisitos de certificação regulatória (FDA, ISO, ANEEL). - Operação 24/7 com SLA de disponibilidade. - Equipe de operação sem perfil técnico de programação.

O desenho que vem se consolidando na indústria não é "Node-RED **ou** SCADA": é cada um no seu papel, com o comando e a proteção no CLP e no SCADA, e a integração no Node-RED.



7.9 Estudo de Caso — Node-RED como camada de integração em uma planta-piloto

Cenário. Planta-piloto de uma indústria de alimentos, com um CLP de médio porte controlando um tanque de mistura e temperatura. O CLP não tem OPC UA nem MQTT: expõe Modbus TCP. A equipe de processo quer acompanhar a curva de temperatura sem esperar pelo projeto corporativo de SCADA, que está na fila há oito meses.

Arranjo montado. Um microcomputador na rede de controle roda o Node-RED. O fluxo tem três trechos, cada um com um papel claro:

1. **Coleta** — *nodes* `modbus-read` leem os registradores de temperatura e velocidade do agitador, em ciclo de 2 s.
2. **Tratamento** — um *node* `function` converte o inteiro de 16 bits em valor de engenharia (escala e *offset* do transmissor), aplica o *deadband* de 0,2 °C e marca a qualidade do dado quando o CLP não responde.
3. **Distribuição** — os mesmos dados vão para três destinos: `mqtt out` para o broker, `influxdb out` para o banco de séries temporais e `ui_gauge` / `ui_chart` para o dashboard web.

```
[modbus-read: temperatura]
      |
      v
[function: escala + deadband + qualidade]
      |
      +--> [mqtt out: planta/tanque/temperatura]
      +--> [influxdb out: processo]
      +--> [ui_chart: tendencia de 8 h]
```

Resultado. A equipe de processo passou a ter a curva de temperatura no navegador em três dias de trabalho. Quando o projeto corporativo de SCADA entrou em operação, o mesmo fluxo continuou rodando ao lado, agora consumindo OPC UA e servindo a plataforma de analytics.

Limitações que precisam ser ditas em voz alta.

- **Sem redundância.** Se o microcomputador reiniciar, a coleta para; o CLP continua controlando, mas o registro tem buraco. Para um piloto é aceitável; para um registro regulatório, não.
- **Sem trilha de auditoria.** Qualquer pessoa com acesso ao editor altera o fluxo; não há controle de versão nem aprovação. Versionar o JSON do fluxo é o mínimo.
- **Node-RED não substitui o motor de alarmes.** O alarme montado com `switch` + `notification` não tem reconhecimento, supressão nem *timestamp* de origem — os requisitos tratados no Capítulo 10 continuam valendo.
- **Comando não é papel do Node-RED.** Partida de bomba, abertura de válvula e intertravamento pertencem ao CLP e ao SCADA.

7.10 Resumo

- **Node-RED** é uma plataforma de programação visual flow-based, open-source, ideal para integrações e supervisão simples.
- Conceitos fundamentais: **nodes**, **wires**, **flows** e **context**.
- Módulos `node-red-contrib-modbus` e `mqtt in/out` permitem integração com CLPs e brokers MQTT.
- O módulo `node-red-dashboard` gera interfaces web de supervisão sem necessidade de HTML/CSS.
- A integração com **InfluxDB + Grafana** fornece histórico e visualização de alto nível.
- Node-RED **complementa** SCADA comercial; raramente o substitui em aplicações críticas.

7.11 Questões de Revisão

Conceituais:

1. O que é programação *flow-based*? Como ela difere da programação sequencial tradicional?
2. Qual é a estrutura de uma mensagem (`msg`) no Node-RED? Quais campos são padrão?

3. Cite três vantagens e três limitações do Node-RED como solução supervisória industrial.

Práticas:

4. Crie (descreva o flow) para: ler a temperatura de um CLP via Modbus TCP (registrador 40010, float32), exibir em um gauge no dashboard, gerar um alarme no painel se temperatura $> 90°C$ e gravar em InfluxDB a cada 5 segundos.
5. Como você configuraria o Node-RED para operar com autenticação de usuário e HTTPS? Quais configurações no arquivo `settings.js` seriam necessárias?

Desafio:

6. Implemente um flow Node-RED que: (a) subscreva ao tópico MQTT `fabrica/+/temperatura`; (b) calcule a média móvel das últimas 10 leituras; (c) publique a média em `fabrica/media/temperatura`; (d) exiba em um gráfico de tendência no dashboard. Descreva a lógica JavaScript do node `function`.

7.12 Referências

- NODE-RED. *Node-RED Documentation*. Disponível em: `nodered.org/docs`.
- OpenJS Foundation. *Node-RED GitHub Repository*. `github.com/node-red/node-red`.
- MQTT.ORG. *MQTT Essentials*. Disponível em: mqtt.org.
- INFLUXDATA. *Node-RED InfluxDB Integration*. Disponível em: docs.influxdata.com.
- MATERIAL DE ORIGEM: *Node-Red_InterfaceDeSupervisão* (material do curso) — telas do editor e do dashboard reproduzidas com crédito.

Capítulo 8

Segurança em Sistemas SCADA

Objetivos de Aprendizagem

Ao final deste capítulo, o leitor será capaz de:

- Compreender por que a cibersegurança em OT é diferente da segurança em TI.
- Identificar as principais ameaças e vetores de ataque a sistemas SCADA.
- Aplicar os conceitos da norma IEC 62443 para proteção de ambientes industriais.
- Descrever as melhores práticas de segmentação de redes TI/OT.

8.1 OT Security: Por que é Diferente?

A cibersegurança em **OT** (*Operational Technology*) segue princípios distintos da segurança em **TI** (*Information Technology*). A tríade clássica **CIA** (Confidencialidade, Integridade e Disponibilidade) continua relevante, mas não existe uma ordem universal de prioridade: os impactos sobre segurança de pessoas, meio ambiente, processo e continuidade precisam ser avaliados para cada sistema.

Tabela 8.1 – Ênfases de risco frequentes: TI × OT

Aspecto	TI (Corporativa)	OT (Industrial)
Impacto frequentemente enfatizado	Exposição de dados e privacidade	Segurança de pessoas, processo e continuidade operacional
Risco de integridade	Alteração de dados ou transações	Comandos ou valores manipulados podem afetar o processo físico
Risco de disponibilidade	Interrupção de serviços e negócios	Perda de supervisão ou controle pode causar consequências físicas e operacionais

Na OT, **parar um sistema pode significar** perda de produção, interrupção de serviço essencial ou riscos à segurança de pessoas e ao meio ambiente. A prioridade entre

confidencialidade, integridade e disponibilidade depende da análise de risco e do contexto operacional.

Outras diferenças críticas:

- **Ciclo de vida:** sistemas OT operam por décadas; patches e atualizações são raros ou impossíveis sem parada programada.
- **Tempo real:** latências introduzidas por firewalls e antivírus podem afetar o controle.
- **Protocolos legados:** Modbus, DNP3 e outros foram projetados sem autenticação ou criptografia.
- **Air gap histórico:** sistemas OT eram fisicamente isolados; a convergência TI/OT eliminou essa barreira.

Atenção

Atenção: Um antivírus mal configurado em um servidor SCADA pode consumir CPU durante um scan e causar timeouts de comunicação — resultando em falsos alarmes ou perda de controle do processo. Soluções de segurança para OT devem ser **específicas para o ambiente industrial**.

8.2 Linha do Tempo de Ataques a Infraestruturas Críticas

Figura

[**Figura 8.1 – Linha do tempo de ataques a sistemas SCADA e infraestruturas críticas**] *Infográfico com os principais incidentes de segurança em OT: Stuxnet (2010), Ukraine Power Grid (2015, 2016), Triton/Trisis (2017), Colonial Pipeline (2021), Oldsmar Water Treatment (2021). Para cada evento: ano, país, setor afetado, impacto e vetor de ataque.*

Stuxnet (2010): malware sofisticado criado para sabotar centrífugas de enriquecimento de urânio no Irã, manipulando CLPs Siemens S7-315/417. Primeiro ataque cibernético documentado a causar dano físico real.

Ukraine Power Grid (2015): ataque via phishing comprometeu estações SCADA de distribuidoras de energia, resultando em blackout para 230.000 pessoas.

Triton/Trisis (2017): ataque direcionado a sistemas de segurança funcional (SIS — Safety Instrumented System) em uma refinaria petroquímica no Oriente Médio. O objetivo era desabilitar a proteção que impede explosões.

Colonial Pipeline (2021): ransomware no sistema de TI da maior operadora de oleodutos dos EUA. A empresa desligou preventivamente o SCADA — causando escassez de combustível na Costa Leste americana.

8.3 Principais Vetores de Ataque

Vetor	Descrição	Mitigação
Phishing	Email falso induz operador a instalar malware	Treinamento, MFA, filtro de email
USB / mídia removível	Malware em pen drives usados na rede OT	Política zero USB, portas bloqueadas
Acesso remoto inseguro	VPN sem MFA, RDP exposto	MFA obrigatório, jump server, PAM
Vulnerabilidades em sistemas legados	CLPs/HMIs sem patch por anos	Segmentação, monitoramento passivo
Supply chain	Firmware malicioso em hardware comprado	Verificação de integridade, fornecedores certificados
Movimento lateral TI→OT	Atacante compromete TI e se move para OT	Segmentação, DMZ industrial

8.4 Norma IEC 62443

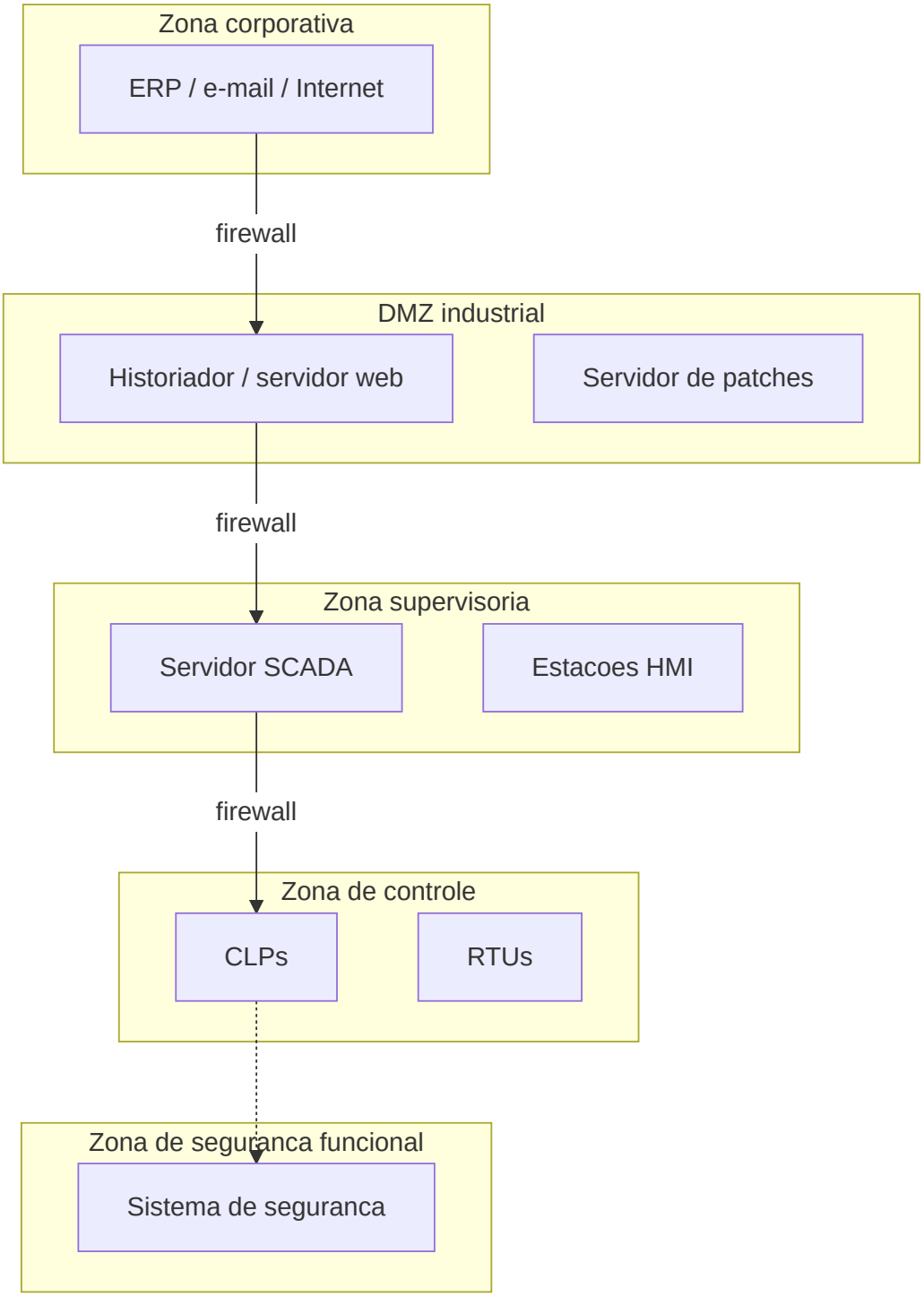
A **IEC 62443** (*Security for Industrial Automation and Control Systems*) é a família de normas internacional para cibersegurança em sistemas de automação e controle industrial. É estruturada em quatro séries:

Série	Título	Conteúdo
IEC 62443-1	Geral	Terminologia, conceitos, modelos
IEC 62443-2	Políticas e Procedimentos	Requisitos para operadores e integradores
IEC 62443-3	Sistema	Avaliação de risco, zonas e condutos
IEC 62443-4	Componentes	Requisitos para produtos (CLPs, SCADA)

8.4.1 Zonas de Segurança e Condutos

O conceito central da IEC 62443-3-2 é dividir o sistema em **zonas** (*Security Zones*) e **condutos** (*Conduits*):

- **Zona de Segurança:** agrupamento de ativos com requisitos de segurança semelhantes e nível de confiança equivalente.
- **Conduto:** canal de comunicação que conecta duas zonas, com controles de segurança aplicados ao tráfego.



Figura

[Figura 8.2 – Modelo de zonas e condutos IEC 62443] *Diagrama de zonas funcionais conectadas por condutos controlados: Zona Corporativa, DMZ Industrial, Supervisão, Controle e Sistema Instrumentado de Segurança. Mostrar fronteiras, fluxos permitidos e pontos de aplicação de controles, sem atribuir automaticamente um SL às zonas nem sugerir que a criticidade sempre cresce em direção ao centro.*

8.4.2 Security Levels (SL)

A IEC 62443 descreve níveis de segurança (**SL**) associados à capacidade de resistência requerida para um sistema ou componente. A seleção do **SL-alvo** resulta da análise de risco e dos requisitos de segurança; não se atribui um nível apenas pelo nome ou posição da zona. SL 0 indica ausência de requisitos específicos, e SL 1 a 4 representam capacidades progressivamente maiores contra ameaças intencionais.

SL	Proteção contra	Descrição
SL 1	Violações casuais ou acidentais	Erros e uso indevido não deliberado
SL 2	Ataques intencionais com meios simples e recursos limitados	Ameaça com competências e motivação limitadas
SL 3	Ataques intencionais com meios sofisticados e recursos moderados	Ameaça com competências e motivação moderadas
SL 4	Ataques intencionais com meios sofisticados e recursos extensos	Ameaça com competências e motivação elevadas

---

8.5 Boas Práticas de Segurança em SCADA

8.5.1 Segmentação de Redes

Figura

[Figura 8.3 – Segmentação em zonas, com o tráfego permitido e o bloqueado]
Diagrama de blocos com zonas corporativa, DMZ industrial, supervisória e de controle. Mostrar apenas os fluxos explicitamente permitidos e bloqueados, os pontos de inspeção (firewalls e proxies) e os protocolos/portas do cenário ilustrado. Não pré-atribuir SL às zonas: indicar que os alvos dependem de análise de risco.

- Segmentar a rede OT da rede corporativa com **firewall industrial** e regras explícitas de comunicação; isolamento físico pode ser apropriado em certos casos, mas não substitui a análise dos caminhos de manutenção e troca de dados.
- Criar **DMZ Industrial** para sistemas que precisam compartilhar dados com TI (historiador, servidor web de relatórios).
- Usar **diodos de dados** (*data diodes*) quando a arquitetura exigir fluxo unidirecional. Eles impõem uma direção de comunicação no enlace protegido, mas não tornam invulneráveis os sistemas conectados nem eliminam outros caminhos de retorno.

8.5.2 Controle de Acesso Remoto

- **VPN com MFA** (*Multi-Factor Authentication*) para todos os acessos remotos.

- **Jump Server / Bastion Host:** toda conexão remota passa por um servidor intermediário — nunca acesso direto a CLPs.
- **PAM** (*Privileged Access Management*): registro e gravação de todas as sessões de manutenção remota.
- **Tempo limitado:** acessos remotos habilitados apenas durante janelas de manutenção, com aprovação.

Dica

Dica: Nunca exponha a porta RDP (3389) ou SSH (22) de servidores SCADA diretamente para a internet. Uma busca no Shodan.io revela milhares de sistemas SCADA acessíveis publicamente — a maioria sem autenticação adequada.

8.5.3 Gestão de Vulnerabilidades e Patches

- **Inventário de ativos:** manter catálogo atualizado de todos os equipamentos OT (fabricante, modelo, firmware, protocolos).
- **Avaliação de vulnerabilidades passiva:** ferramentas como **Claroty**, **Dragos** ou **Tenable OT** identificam vulnerabilidades sem enviar pacotes ativos (que poderiam impactar o processo).
- **Patch em janelas de manutenção:** atualizações testadas em ambiente de homologação antes da produção.
- **EOL Management:** equipamentos sem suporte (End-of-Life) devem ser isolados e substituídos no prazo.

8.5.4 Backup e Recuperação

- Backup regular de: configurações de CLPs, configurações do SCADA (tags, telas, scripts), banco de dados histórico.
- Teste periódico de restauração.
- Plano de **DRP** (*Disaster Recovery Plan*) específico para o ambiente OT.

8.5.5 Treinamento e Cultura de Segurança

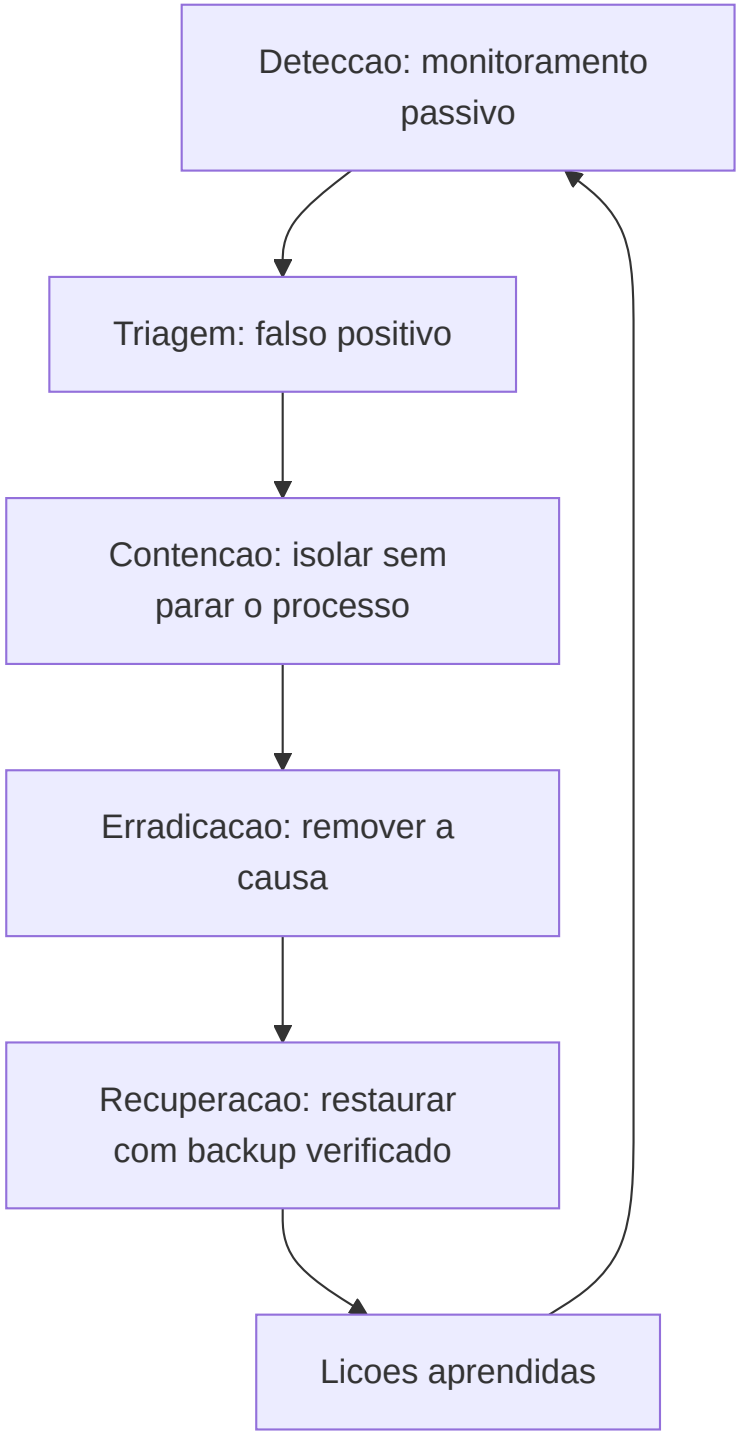
- Operadores e técnicos de manutenção são o principal vetor de ataque (phishing, USB).
- Treinamentos regulares de **segurança cibernética industrial** para toda a equipe OT.
- Política clara sobre uso de dispositivos pessoais e mídias removíveis na planta.

Nota

Nota: No Brasil, o setor de energia elétrica é regulado pela **Resolução Normativa ANEEL nº 964/2021**, que estabelece requisitos mínimos de segurança cibernética para o Ambiente de TI e OT de agentes do setor elétrico, baseados na IEC 62443 e no NIST Cybersecurity Framework.

8.6 Monitoramento e Resposta a Incidentes (SOC OT)

Responder a um incidente em rede industrial é diferente de responder em rede corporativa: não se pode simplesmente desligar o equipamento suspeito se ele estiver operando o processo.



8.6.1 Monitoramento Passivo da Rede OT

Ferramentas de monitoramento OT analisam o tráfego de rede de forma **passiva** — sem injetar pacotes — para:

- Criar inventário automático de ativos (dispositivo, firmware, protocolo).
- Detectar comunicações anômalas (CLP falando com IP desconhecido, volume atípico de dados).
- Alertar sobre tentativas de varredura de rede.

Soluções especializadas em OT:

Ferramenta	Empresa	Diferencial
Claroty	Claroty	Amplo suporte a protocolos OT; integração com SIEM
Dragos	Dragos	Threat intelligence focado em OT (grupos de ameaça)
Tenable OT	Tenable	Integração com gestão de vulnerabilidades TI
Microsoft Defender for IoT	Microsoft	Integrado ao Azure Sentinel
Nozomi Networks	Nozomi	IA para detecção de anomalias OT

8.6.2 Plano de Resposta a Incidentes OT

Em caso de incidente cibernético em ambiente OT:

1. **Detecção:** sistema de monitoramento alerta sobre atividade suspeita.
2. **Avaliação:** equipe de segurança avalia se é falso positivo ou ameaça real.
3. **Contenção:** isolar segmento afetado sem parar o processo (se possível).
4. **Erradicação:** remover o malware ou fechar o vetor de ataque.
5. **Recuperação:** restaurar sistemas a partir de backup limpo; testar antes de retornar à produção.
6. **Lições aprendidas:** documentar o incidente; atualizar procedimentos.

8.7 Estudo de Caso — Segmentação por zonas e condutos em uma planta de médio porte

Cenário. Planta de processamento contínuo com 340 funcionários. A rede de automação cresceu por acumulação ao longo de 15 anos: CLPs, IHM, supervisório, servidor de histórico, estações de engenharia e o Wi-Fi da manutenção dividem hoje o mesmo conjunto de *switches*, com o roteador corporativo ligado a um deles. Não há documentação de endereçamento.

Diagnóstico pelo modelo de zonas e condutos (IEC 62443). O primeiro produto do trabalho não é um firewall: é o **inventário de ativos com função e criticidade**. Só depois as zonas aparecem.

Zona proposta	Ativos	Nível de criticidade
Zona de controle	CLPs, IHM locais, *drives*	Crítica — para a produção se parar
Zona de supervisão	Servidores SCADA e historiador, estações de operação	Crítica
Zona de engenharia	Estações de programação, *notebooks* de fornecedor	Alta
Zona corporativa	ERP, e-mail, estações administrativas	Média
Zona de acesso remoto	VPN de fornecedores e acesso de manutenção	Alta, com sessão auditada

Regras de conduto que fizeram a diferença. A segmentação só reduz risco quando o tráfego entre zonas é **explícito**:

- Supervisão **lê** do controle; o controle **não inicia** conexão para a supervisão.
- Engenharia acessa o controle apenas em janela agendada, com sessão gravada.
- Corporativo **nunca** alcança o controle diretamente — inclusive o tráfego de atualização de *antivírus*, que costuma ser o furo silencioso das segmentações.
- Nenhum acesso remoto entra sem duplo fator e sem registro de sessão.

O que observar no resultado. A segmentação **não** exige trocar CLP nem parar a produção: as zonas são implementadas em VLANs e regras de firewall, com janelas de manutenção para repassar a rede. O ganho maior, porém, costuma ser indireto: o inventário de ativos gerado na etapa 1 é o que permite, depois, aplicar correções e detectar anomalias de tráfego.

Limitações do arranjo. Segmentar sem monitorar deixa o sistema mais difícil de operar e não mais seguro — a rede fica opaca, e cada diagnóstico passa a depender de alguém com acesso ao firewall. A segmentação precisa vir acompanhada de registro de tráfego nas fronteiras de zona (Seção 8.6).

8.8 Resumo

- Em OT, a análise de risco deve considerar disponibilidade, integridade, confidencialidade e, sobretudo, as consequências à segurança de pessoas e ao processo; não há uma ordenação universal que sirva a todo sistema.
- Sistemas SCADA foram projetados para disponibilidade, não para segurança; a convergência TI/OT os expôs a ameaças digitais.
- A **IEC 62443** estrutura a segurança industrial em zonas e condutos e define níveis de segurança; os SL-alvo são selecionados a partir da análise de risco.
- Boas práticas essenciais: segmentação de rede, DMZ industrial, VPN com MFA, monitoramento passivo e treinamento de operadores.
- Ferramentas especializadas como Claroty e Dragos permitem monitoramento OT sem impactar o processo.

8.9 Questões de Revisão

Conceituais:

1. Por que a prioridade de segurança em OT (Disponibilidade > Integridade > Confidencialidade) é inversa à de TI?
2. O que são "zonas de segurança" e "condutos" na IEC 62443? Cite dois exemplos de cada um.
3. Por que o antivírus convencional de TI pode ser inadequado para servidores SCADA?

Práticas:

4. Uma empresa petroquímica conectou sua rede OT diretamente à rede corporativa para facilitar o acesso ao historiador a partir do ERP. Identifique os riscos dessa arquitetura e proponha uma solução com DMZ industrial.
5. Um técnico de manutenção precisa acessar remotamente um CLP para realizar ajuste de parâmetros fora do horário comercial. Descreva o procedimento seguro de acesso remoto, incluindo autenticação, registro e controle de sessão.

Desafio:

6. Com base no incidente do Stuxnet (2010), analise: (a) Qual era o objetivo do ataque? (b) Qual vetor de infecção foi usado? (c) Como ele manipulou os CLPs Siemens? (d) Quais medidas de segurança da IEC 62443 poderiam ter dificultado ou impedido o ataque?

8.10 Referências

- IEC 62443: *Security for Industrial Automation and Control Systems* (séries 1 a 4; consultar a edição vigente de cada parte aplicável no catálogo IEC).
- NIST SP 800-82 Rev. 3: *Guide to OT Security*. NIST, 2023.
- ANEEL Resolução Normativa nº 964/2021. Disponível em: aneel.gov.br.
- LANGNER, R. *Stuxnet: Dissecting a Cyberwarfare Weapon*. IEEE Security & Privacy, 2011.
- CLAROTY. *State of XIoT Security Report*. Disponível em: `claroty.com/research`.

Capítulo 9

Tendências: Cloud SCADA e Digital Twin

Objetivos de Aprendizagem

Ao final deste capítulo, o leitor será capaz de:

- Descrever as vantagens e desafios do Cloud SCADA versus SCADA on-premises.
- Identificar as principais plataformas de Cloud SCADA disponíveis no mercado.
- Compreender como o Digital Twin se integra aos sistemas SCADA para otimização.
- Analisar o impacto da IA/ML e do low-code/no-code no futuro dos sistemas supervisórios.

9.1 Cloud SCADA: Conceito e Motivações

O **Cloud SCADA** é a evolução natural dos sistemas supervisórios tradicionais: em vez de servidores físicos instalados na própria planta (*on-premises*), o software SCADA é hospedado em infraestrutura de nuvem pública (AWS, Azure, Google Cloud) ou privada.

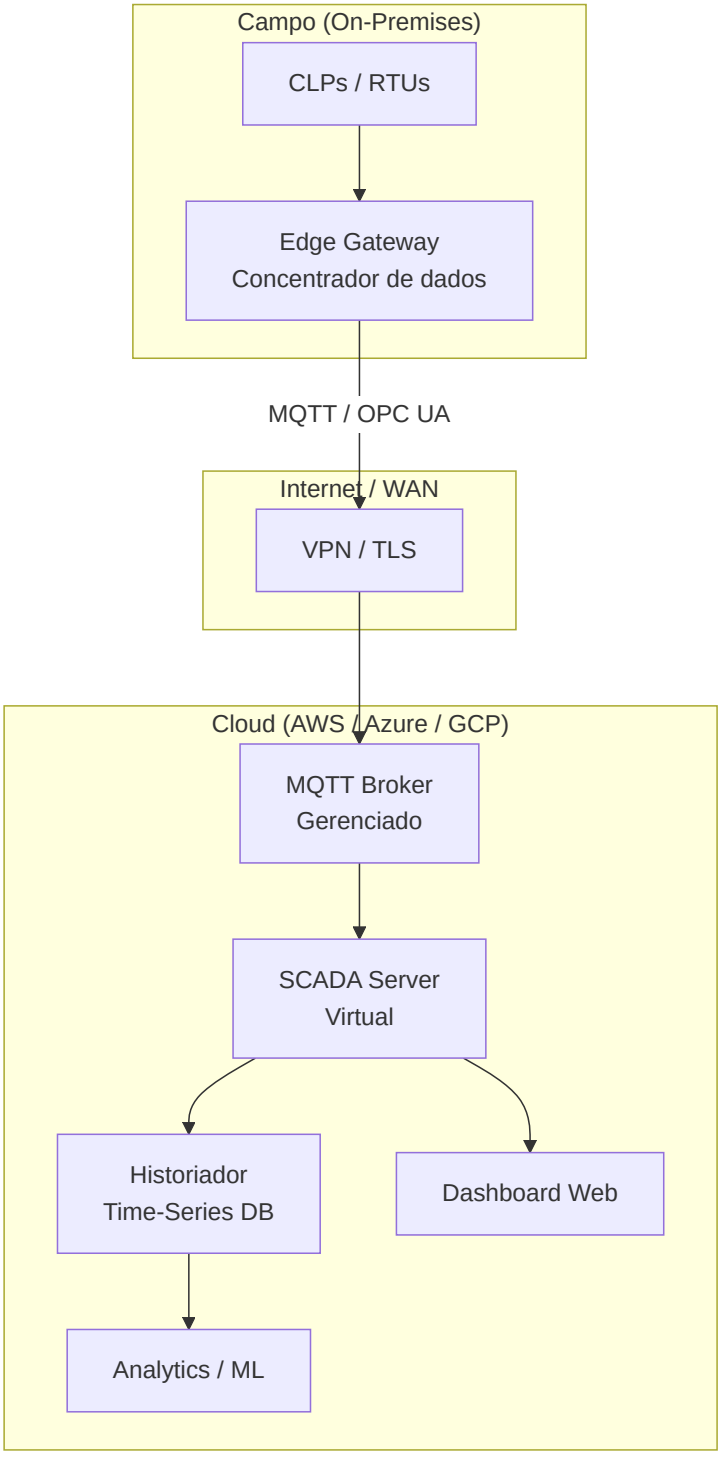
9.1.1 Por que migrar para a nuvem?

Tabela 9.1 – SCADA On-Premises × Cloud SCADA

Critério	SCADA On-Premises	Cloud SCADA
Infraestrutura	Servidores físicos na planta	Instâncias virtuais gerenciadas
Custo inicial (CapEx)	Alto (hardware + licenças)	Baixo ou zero
Custo operacional (OpEx)	Equipe de TI, manutenção, energia	Pagamento por uso (pay-as-you-go)
Escalabilidade	Limitada (hardware físico)	Elástica (escala automaticamente)
Acesso remoto	Via VPN, complexo	Nativo via browser, global

Critério	SCADA On-Premises	Cloud SCADA
Atualizações	Manual, risco de incompatibilidade	Automáticas, sem downtime
Latência	Mínima (rede local)	Depende da conectividade WAN
Disponibilidade garantida	Depende do investimento local	SLAs de 99,9% a 99,99%
Compliance / Soberania de dados	Total controle	Depende da região do datacenter

9.1.2 Arquitetura Cloud SCADA



Figura

[Figura 9.1 – Arquitetura Cloud SCADA com Edge Gateway] *Diagrama colorido mostrando: (esquerda) campo industrial com CLPs e edge gateway; (centro) conexão segura via VPN/TLS pela internet; (direita) nuvem com os componentes SCADA virtualizados. Destacar que o controle em tempo real permanece no CLP local — a nuvem é supervisão e analytics.*

Atenção

Atenção: Em Cloud SCADA, o **controle em malha fechada permanece nos CLPs locais**. A latência da nuvem (decenas a centenas de ms) inviabiliza controle PID via cloud. A nuvem é responsável pela supervisão, historização, analytics e acesso remoto — não pelo controle direto do processo.

9.2 Plataformas de Cloud SCADA

9.2.1 Ignition Cloud Edition (Inductive Automation)

O **Ignition** foi pioneiro no modelo de licenciamento cloud-native:

- Hospedado na AWS ou Azure.
- Mesma interface do Ignition on-premises — sem curva de aprendizado adicional.
- Módulo de borda (*Ignition Edge*) instalado na planta coleta dados e sincroniza com a cloud.
- Ideal para empresas com múltiplos sites que precisam de visão unificada.

9.2.2 AWS IoT SiteWise

O **AWS IoT SiteWise** é a solução da Amazon para modelagem e supervisão de ativos industriais:

- **Asset Models:** define hierarquia de ativos (planta → unidade → equipamento → componente).
- **Asset Properties:** variáveis de processo com cálculos e transformações configuráveis.
- **SiteWise Monitor:** dashboards de supervisão sem código.
- **SiteWise Edge:** módulo local para operação sem conectividade.

9.2.3 Azure IoT Hub + Azure Digital Twins

A combinação **Azure IoT Hub** (ingestão de dados) + **Azure Digital Twins** (modelagem) + **Azure Time Series Insights** (histórico) + **Power BI** (dashboards) forma uma stack completa de Cloud SCADA na Microsoft.

9.2.4 AVEVA Insight

O **AVEVA Insight** é a solução cloud da AVEVA (herdeira do PI System e Wonderware):

- Historização de dados na nuvem com a mesma interface do PI System.
- KPIs e dashboards web sem necessidade de servidor local.
- Integração nativa com System Platform on-premises existentes.

Dica

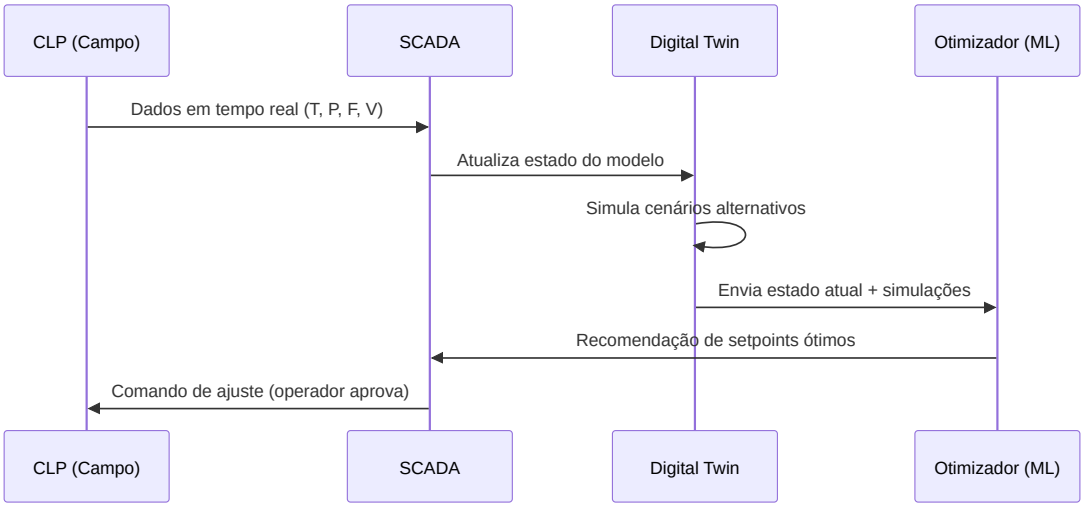
Dica: Para plantas industriais brasileiras com dados sensíveis de processo, verifique a conformidade com a **LGPD** (Lei Geral de Proteção de Dados) e a **soberania de dados**: dados de processo podem conter informações estratégicas. Priorize regiões de datacenter no Brasil (AWS São Paulo, Azure Brasil Sul) ou considere nuvem privada

(*on-premises cloud*).

9.3 Digital Twin nos Sistemas SCADA

A integração do **Digital Twin** com o SCADA cria um sistema de supervisão que não apenas monitora — mas também simula, prevê e otimiza.

9.3.1 Como o SCADA alimenta o Digital Twin



Figura

[Figura 9.2 – Integração SCADA e Digital Twin: ciclo de otimização] *Diagrama do ciclo acima com representações visuais: planta física com sensores → SCADA → modelo 3D digital twin → algoritmo de otimização → recomendação ao operador no SCADA → ajuste no processo real. Ciclo contínuo indicado por setas.*

9.3.2 Caso de Uso: Otimização de Destilação

Em uma coluna de destilação de uma refinaria, o Digital Twin permite:

1. **Modelo de primeiro princípio** (equações de equilíbrio termodinâmico) calibrado com dados reais do SCADA.
2. Simulação em tempo real de múltiplos setpoints alternativos de temperatura e refluxo.
3. Identificação do ponto ótimo que maximiza rendimento do produto e minimiza consumo de energia.
4. Recomendação ao operador com justificativa técnica e previsão de impacto.

Resultado típico: ganho de 2–5% no rendimento do destilado com redução de 8–12% no consumo de vapor.

9.4 Inteligência Artificial Aplicada a Sistemas Supervisórios

9.4.1 Manutenção Preditiva (PdM)

A **manutenção preditiva** usa modelos de ML para prever falhas antes que ocorram, com base em dados históricos e em tempo real:

Algoritmo	Aplicação	Indicadores Monitorados
LSTM (Long Short-Term Memory)	Degradação de bombas e compressores	Vibração, temperatura de mancal, corrente
Isolation Forest	Detecção de anomalias em sensores	Desvios estatísticos multivariados
Random Forest	Classificação de modo de falha	Padrões de alarmes históricos
Redes Neurais Convolucionais	Análise de vibração espectral	FFT de sinal de acelerômetro

9.4.2 Controle Avançado com IA

- **MPC** (*Model Predictive Control*): controlador baseado em modelo que prevê o comportamento futuro do processo e otimiza ações de controle considerando restrições.
- **RL** (*Reinforcement Learning*): agente aprende a operar o processo por tentativa e erro no Digital Twin — depois é implantado na planta real.

9.4.3 Processamento de Linguagem Natural (NLP) em SCADA

Interfaces conversacionais permitem que operadores consultem o SCADA por voz ou texto:

- *"Qual foi a temperatura máxima do Reator 3 na última semana?"*
- *"Mostre o histórico de alarmes do Compressor A12 nos últimos 30 dias."*
- *"Qual bomba teve o maior consumo de energia no turno da manhã?"*

9.5 Low-Code / No-Code em Supervisórios

A tendência **low-code/no-code** está democratizando o desenvolvimento de sistemas supervisórios:

Plataforma	Abordagem	Ideal para
Ignition	Configuração por objetos, scripts Python/Jython	Integradores e engenheiros de automação
Node-RED	Flow-based visual, zero código	Prototipagem, técnicos, pequenas plantas

Plataforma	Abordagem	Ideal para
AWS IoT SiteWise Monitor	Dashboards sem código	Operadores, gerentes de planta
Microsoft Power Apps + Power BI	Low-code apps e dashboards	Integração com dados industriais no Office 365
Grafana	Configuração visual de dashboards	Analytics, visualização avançada

Figura

[**Figura 9.3 – Espectro low-code / no-code em supervisórios**] *Gráfico de eixo horizontal: de "No-Code" (esquerda) a "Full-Code" (direita). Posicionar as ferramentas: AWS SiteWise Monitor e Power BI (no-code), Node-RED e Grafana (low-code), Ignition (low-code/code), AVEVA System Platform e Honeywell Experion (code intensivo). Eixo vertical: nível de customização × complexidade.*

9.6 O Futuro da Profissão: Convergência TI/OT

O engenheiro de automação do futuro precisa transitar entre dois mundos:

Competências OT (tradicionais): - Instrumentação e controle de processo (PID, cascata, ratio). - Programação de CLPs (IEC 61131-3). - Redes industriais (Modbus, Profibus, PROFINET). - Normas de segurança funcional (IEC 61511, IEC 61508).

Competências TI (emergentes): - Redes TCP/IP, cloud computing. - Protocolos IIoT (MQTT, OPC UA, REST). - Análise de dados (Python, pandas, scikit-learn). - Cibersegurança industrial (IEC 62443). - DevOps industrial: containers (Docker), CI/CD para automação.

Figura

[**Figura 9.4 – Roadmap de evolução do SCADA: dos anos 1970 ao futuro**] *Linha do tempo em perspectiva: (1970s) SCADA monolítico com mainframe; (1980s) sistemas proprietários distribuídos; (1990s) Ethernet + OPC; (2000s) SCADA em rede TCP/IP; (2010s) IIoT + cloud; (2020s) AI-SCADA + Digital Twin + 5G; (2030s?) Autonomous Operations. Ícones de tela de cada era.*

9.7 Estudo de Caso — Piloto de Cloud SCADA em uma frota de elevatórias

Cenário. Concessionária com 120 elevatórias de esgoto distribuídas em uma região metropolitana. Cada unidade tem um CLP local, comunicação por 4G e inspeção presencial semanal. A pergunta que o piloto precisa responder não é técnica: **quanto do trabalho de inspeção pode ser trocado por supervisão remota sem aumentar o risco de extravasamento?**

Escopo do piloto — 12 elevatórias, 90 dias. Três decisões de arquitetura definiram o resto:

Decisão	Escolha	Motivo
O que sobe para a nuvem	Telemetria agregada e eventos	Dado bruto de 1 s não agrega valor de gestão
O que fica na borda	Intertravamento, alarme local e corte por nível alto	A segurança não pode depender do link
Comunicação	MQTT com TLS, publicação por exceção	Link 4G intermitente cobra cada byte

Indicadores acompanhados. O piloto só faz sentido se medir o efeito, não a instalação:

- **Número de extravasamentos** por mês, comparado com o histórico das mesmas 12 unidades.
- **Chamados de manutenção** por elevatória/mês.
- **Tempo entre a detecção de uma falha de bomba e a chegada da equipe.**
- **Disponibilidade do enlace**, que costuma ser o fator limitante — e não a plataforma.

O que observar no resultado. A primeira descoberta recorrente é que o gargalo não é a nuvem: é a instrumentação existente. Sem medição confiável de nível e de corrente do motor, a supervisão remota apenas transfere para a tela a mesma incerteza que hoje existe na inspeção presencial.

Limitações do arranjo.

- **Latência e indisponibilidade do link** tornam a nuvem inadequada para comando crítico. A proteção continua no CLP, na borda.
- **Custo recorrente** de conectividade e de plataforma, que precisa entrar no orçamento de operação — não no de projeto, onde costuma ser esquecido.
- **Dependência de fornecedor.** Modelo de dados, formato de exportação e mecanismo de saída dos dados devem ser exigidos no contrato; migrar de plataforma com os dados presos é o cenário mais caro da lista.
- **LGPD e retenção.** Telemetria agregada não identifica pessoas, mas imagens de câmera em instalação e registros de acesso de operadores podem fazê-lo.

9.8 Resumo

- **Cloud SCADA** oferece escalabilidade, acesso remoto e redução de CapEx; o controle em tempo real permanece on-premises.
- As principais plataformas incluem Ignition Cloud, AWS IoT SiteWise, Azure IoT + Digital Twins e AVEVA Insight.
- A integração **SCADA + Digital Twin** cria ciclos de otimização contínua, combinando dados reais com simulações.
- **IA e ML** habilitam manutenção preditiva, controle avançado (MPC, RL) e interfaces conversacionais em SCADA.

- A tendência **low-code/no-code** democratiza o desenvolvimento de supervisórios para operadores e engenheiros sem perfil de programador.
- O profissional de automação moderno precisa dominar tanto competências OT (controle, instrumentação, CLP) quanto TI (cloud, redes, dados, segurança).

9.9 Questões de Revisão

Conceituais:

1. Quais são as três principais vantagens do Cloud SCADA em relação ao SCADA on-premises? Cite também dois desafios.
2. Por que o controle em malha fechada não é realizado na nuvem em arquiteturas Cloud SCADA?
3. Como um Digital Twin se integra ao SCADA para criar um ciclo de otimização? Descreva o fluxo de dados.

Práticas:

4. Uma empresa com 5 plantas industriais em estados diferentes deseja implatar uma visão unificada de supervisão e relatórios gerenciais. Ela já possui SCADA Ignition on-premises em cada planta. Que arquitetura Cloud SCADA você recomendaria? Justifique.
5. Descreva como a manutenção preditiva com ML pode ser integrada a um sistema SCADA existente. Quais dados seriam necessários e como os alertas chegariam ao operador?

Desafio:

6. Elabore um **roadmap tecnológico de 5 anos** para a digitalização de uma planta industrial que atualmente opera com SCADA Wonderware antigo, sem IIoT e sem analytics. O roadmap deve incluir: fases, tecnologias a adotar, casos de uso prioritários, estimativa de benefícios e riscos a gerenciar.

9.10 Referências

- INDUCTIVE AUTOMATION. *Ignition Cloud Edition*. Disponível em: `inductiveautomation.com/ignition/cloud`.
- AMAZON WEB SERVICES. *AWS IoT SiteWise User Guide*. Disponível em: `docs.aws.amazon.com/iot-sitewise`.
- MICROSOFT. *Azure Digital Twins Documentation*. Disponível em: `docs.microsoft.com/azure/digital-twins`.
- AVEVA. *AVEVA Insight Cloud SCADA*. Disponível em: `aveva.com/en/products/insight`.
- GRIEVES, M.; VICKERS, J. *Digital Twin: Mitigating Unpredictable, Undesirable Emergent Behavior in Complex Systems*. Transdisciplinary Perspectives on Complex Systems, 2017.

- LGPD — Lei nº 13.709/2018. *Lei Geral de Proteção de Dados Pessoais*. Disponível em: planalto.gov.br.

Capítulo 10

Gerenciamento de Alarmes

Objetivos de Aprendizagem

Ao final deste capítulo, o leitor será capaz de:

- Explicar por que o gerenciamento de alarmes é uma disciplina de engenharia, e não uma configuração de software.
- Diferenciar alarme, evento e advertência, e distinguir prioridade de severidade.
- Reconhecer o escopo de cada referência normativa: ANSI/ISA-18.2, IEC 62682, EEMUA 191 e NAMUR NA 102.
- Descrever as dez etapas do ciclo de vida de gerenciamento de alarmes.
- Preencher uma ficha de racionalização de alarme.
- Analisar o desempenho de um sistema de alarmes a partir de métricas usuais de mercado.
- Implementar um alarme simples em SCADA e em plataforma IIoT.

10.1 O problema que o gerenciamento de alarmes resolve

Um sistema de alarmes é a última linha de defesa entre uma condição anormal e um acidente. Quando funciona, o operador é avisado exatamente do que importa, no momento em que ainda pode agir. Quando não funciona, o operador recebe centenas de mensagens simultâneas e passa a ignorá-las — ou, pior, desativa o som do sistema.

O sintoma mais conhecido é a **inundação de alarmes** (*alarm flood*): uma perturbação de processo dispara, em poucos segundos, um número de alarmes maior do que um operador consegue ler e processar.

Atenção

Atenção: A inundação de alarmes não é consequência de "operador desatento". É consequência de projeto: alarmes criados sem critério, limites copiados do valor de projeto, ausência de *deadband* e de atraso de confirmação, e nenhuma racionalização. O operador é a vítima, não a causa.

Alguns efeitos medíveis de um sistema de alarmes mal projetado:

- **Perda de priorização:** se 80% dos alarmes são "críticos", nenhum é crítico.
- **Cegueira por ruído:** alarmes repetitivos (*chattering*) escondem o alarme novo.
- **Fadiga de decisão:** cada mensagem irrelevante reduz a capacidade de julgamento.
- **Desconfiança do sistema:** o operador passa a silenciar (*shelve*) em massa, e a proteção desaparece.
- **Perda de rastreabilidade:** sem registro consistente, não há como investigar o incidente depois.

Figura

[Figura 10.1 – Inundação de alarmes: painel de alarmes antes e depois da racionalização] *Lado esquerdo: lista com dezenas de alarmes ativos no mesmo segundo, todos em vermelho, sem ordem de prioridade visível. Lado direito: a mesma perturbação com dez alarmes, ordenados por prioridade, com o alarme de causa raiz destacado. Usar captura de tela genérica de um painel de alarmes; não usar marca de fabricante.*

10.2 Conceitos e taxonomia

A primeira dificuldade prática é terminológica: em muitos projetos, "alarme", "evento" e "aviso" são usados como sinônimos. Não são.

Tabela 10.1 – Termos essenciais de gerenciamento de alarmes

Termo	Definição	Exemplo
Evento	Qualquer mudança de estado registrada pelo sistema	Bomba ligada; operador entrou no sistema
Alarme	Evento que exige **resposta do operador** dentro de um tempo definido	Alta temperatura do mancal acima de 85 °C
Advertência (*alert*)	Informação que não exige resposta imediata, mas pede atenção	Filtro com 70% de saturação
Supressão (*suppression*)	Ocultação temporária por projeto, enquanto a causa está ativa	Alarmes de nível suprimidos durante enchimento
Silenciamento temporário (*shelving*)	Ocultação manual, com prazo e justificativa	Alarme em manutenção por 4 h
Alarme fixo (*standing*)	Permanece ativo por longos períodos sem ação possível	Falha de sensor não reparado
Alarme obsoleto (*stale*)	Não é atualizado ou não é limpo por falha de configuração	Alarme ativo cuja condição já cessou

A definição operacional de alarme tem duas partes que não podem ser separadas: **existe uma resposta possível** e **existe tempo para executá-la**. Se não há ação

a tomar, não é alarme — é registro. Se o operador tem 30 segundos para agir e a tela mostra o alarme em uma página que ele abre em 40 segundos, também não é alarme: é informação histórica.

10.2.1 Prioridade e severidade não são a mesma coisa

Essa confusão aparece em quase toda especificação de SCADA.

- **Severidade** descreve a **consequência** da condição anormal: o que acontece se ninguém agir.
- **Prioridade** descreve a **urgência da resposta** do operador: em quanto tempo ele precisa agir.

Uma falha de instrumento pode ter consequência alta (perde-se a medição de vazão de um reator) e urgência baixa (há redundância e o processo continua seguro por horas). Prioridade alta, portanto, não é sinônimo de "coisa grave": é "aja agora".

Tabela 10.2 – Exemplo de categorias de prioridade por tempo de resposta

Prioridade	Tempo de resposta esperado	Referência de apresentação
Crítica	Segundos a poucos minutos	Destaque máximo, som distinto, exige reconhecimento
Alta	Minutos	Destaque visual forte, som padrão
Média	Dezenas de minutos	Destaque visual moderado
Baixa	Horas ou próximo turno	Apenas registro em lista, sem som

A distribuição de prioridades também é um critério de qualidade: em um sistema maduro, a maior parte dos alarmes é de prioridade baixa e média, e a prioridade crítica é rara. Um sistema em que metade dos alarmes é crítica está mal priorizado, por definição.

Dica

Dica: antes de discutir prioridade, responda por escrito, para cada alarme: *"qual é a ação esperada do operador e em quanto tempo ela precisa começar?"* Alarmes que não sobrevivem a essa pergunta são candidatos a remoção, não a reclassificação. Os nomes das prioridades, os tempos e os sinais visuais desta tabela são um exemplo didático. Cada planta deve defini-los na filosofia de alarmes, considerando consequências, tempo disponível e capacidade de resposta do operador; não são limites universais impostos pela ISA-18.2 ou pela IEC 62682.

10.2.2 Ajustes que definem o comportamento

Três parâmetros determinam se um alarme é útil ou ruído:

- **Limite** (*setpoint*): valor que dispara a condição.
- **Faixa morta** (*deadband*): diferença necessária para limpar o alarme, evitando oscilação em torno do limite.

- **Atraso de confirmação** (*on-delay*): tempo que a condição precisa persistir para que o alarme seja gerado. Elimina disparos fugazes (*fleeting*) causados por transitórios de partida.

Figura

[Figura 10.2 – Limite, faixa morta e atraso de confirmação] *Gráfico de tendência com a variável cruzando o limite três vezes em poucos segundos. Marcar: (a) disparo no cruzamento com atraso de 3 s; (b) zona de faixa morta impedindo o disparo/limpeza repetitivo; (c) os alarmes que existiriam sem esses ajustes, em linha tracejada. Incluir eixo de tempo legível.*

10.3 Normas e o escopo de cada uma

Quatro referências dominam o assunto, e elas **não são equivalentes**.

As edições abaixo são as identificadas na bibliografia desta apostila. Como não foi possível consultar nesta revisão os catálogos oficiais ISA, IEC, EEMUA e NAMUR, elas não são declaradas aqui como as edições atualmente vigentes; confirme status, edição e escopo antes de especificar ou contratar.

Tabela 10.3 – Comparação das referências normativas

Referência	Natureza	Escopo e uso típico
ANSI/ISA-18.2	Norma (ISA)	Gerenciamento de alarmes na indústria de processo; a edição identificada nesta bibliografia é a de 2016 (1ª edição: 2009)
IEC 62682	Norma (IEC)	Gerenciamento de alarmes na indústria de processo; edição identificada nesta bibliografia: 2ª edição, 2022
EEMUA 191	Guia de engenharia (EEMUA)	Projeto, gestão e contratação de sistemas de alarme; a edição identificada nesta bibliografia é a 3ª, de 2013
NAMUR NA 102	Recomendação (NAMUR)	Gerenciamento de alarmes na indústria química e farmacêutica; a edição identificada nesta bibliografia é de 2003

Notas de uso, verificadas na documentação pública dos organismos:

- ISA-18.2 e IEC 62682 são **quase idênticas** — a versão IEC foi modificada a partir da ISA. Como normas, dizem **o que** deve existir em um programa de gerenciamento de alarmes, não **como** fazer.

- EEMUA 191 é um **guia**, não uma norma: cobre parcialmente o "como" e é muito usada como referência de desempenho.
- A série de **relatórios técnicos ISA-18.2** (sete documentos, alinhados à norma) aprofunda o "como" em cada etapa.
- ISA-18.2 foi escrita para **não conflitar** com EEMUA 191 e NAMUR NA 102: o objetivo foi criar terminologia e atividades consistentes entre as referências.

Nota

Atualização tecnológica: o conceito de gerenciamento de alarmes não muda com IIoT — muda o **alcance**. Em SCADA clássico, os alarmes vivem no servidor de supervisão. Em arquiteturas atuais, plataformas IIoT possuem entidade de alarme própria (por exemplo, o tipo *alarm* do ThingsBoard, com severidade, estado e detalhes) e podem gerar alarmes na borda, antes da nuvem. A disciplina (filosofia, racionalização, prioridade, monitoramento) permanece a mesma.

Atenção

Atenção: não atribua requisitos de ISA-18.2/IEC 62682 a normas de outra família, como **IEC 60839** (alarme de segurança eletrônica/patrimonial e sistemas de detecção de intrusão) ou a normas de detecção e alarme de incêndio. São escopos distintos: alarme de **processo** exige resposta de operador sobre uma variável de processo; alarme de **segurança patrimonial** trata de detecção de intrusão. Se o material antigo do curso fizer essa associação, corrija e explique a diferença.

10.4 Ciclo de vida

Normas e guias organizam o trabalho em etapas. O ciclo é iterativo: a saída da auditoria alimenta a filosofia.

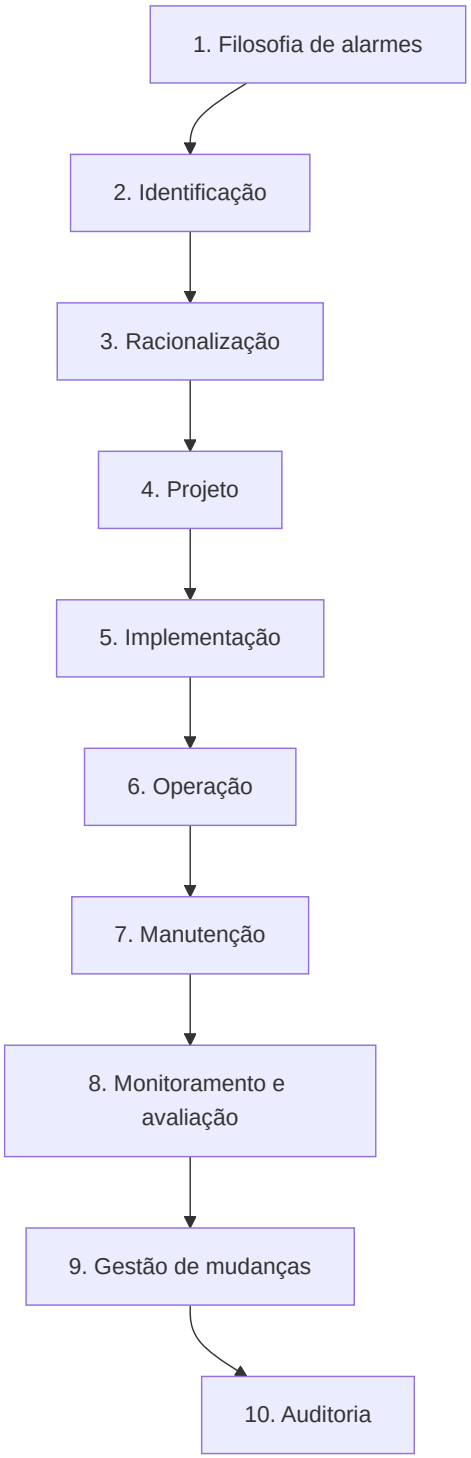


Tabela 10.4 – O que cada etapa entrega

Etapa	Pergunta que responde	Produto
Filosofia	Quais são os princípios, papéis e critérios de desempenho?	Documento de filosofia de alarmes
Identificação	Que condições precisam de alarme?	Lista de eventos candidatos

Etapa	Pergunta que responde	Produto
Racionalização	Cada alarme é necessário, único e acionável?	Ficha de racionalização por alarme
Projeto	Como o alarme se apresenta e se comporta?	Especificação de limite, prioridade, deadband, atraso
Implementação	Está configurado como especificado?	Alarmes configurados e testados
Operação	O operador consegue responder?	Rotina de operação e tratamento de shelving
Manutenção	O sistema continua íntegro?	Correção de alarmes fixos, obsoletos e falhas
Monitoramento e avaliação	O desempenho é adequado?	Relatórios de métricas
Gestão de mudanças	A alteração está justificada e registrada?	Registro de mudança aprovado
Auditoria	O programa funciona como previsto?	Relatório de auditoria e plano de ação

Dica

Dica: a etapa que separa plantas maduras das demais é a de **gestão de mudanças**. Mudar um limite de alarme é alterar um projeto: precisa de justificativa, aprovação e registro. Sem esse controle, o sistema degrada em poucos meses — é o chamado *drift* de configuração.

10.5 Racionalização na prática

Racionalizar é documentar, alarme por alarme, por que ele existe e o que o operador deve fazer. A saída é uma ficha (no jargão em inglês, *Alarm Basis Form* ou *rationalization record*).

Tabela 10.5 – Campos de uma ficha de racionalização

Campo	Conteúdo
Identificação	Tag do instrumento, descrição e tipo de alarme
Causa	O que provoca a condição anormal
Consequência	O que acontece se ninguém agir (e em quanto tempo)
Ação esperada	O que o operador deve fazer, em passos objetivos
Tempo de resposta	Tempo disponível para agir
Prioridade	Derivada da consequência e do tempo
Limite, deadband e atraso	Valores justificados por processo, não copiados do projeto

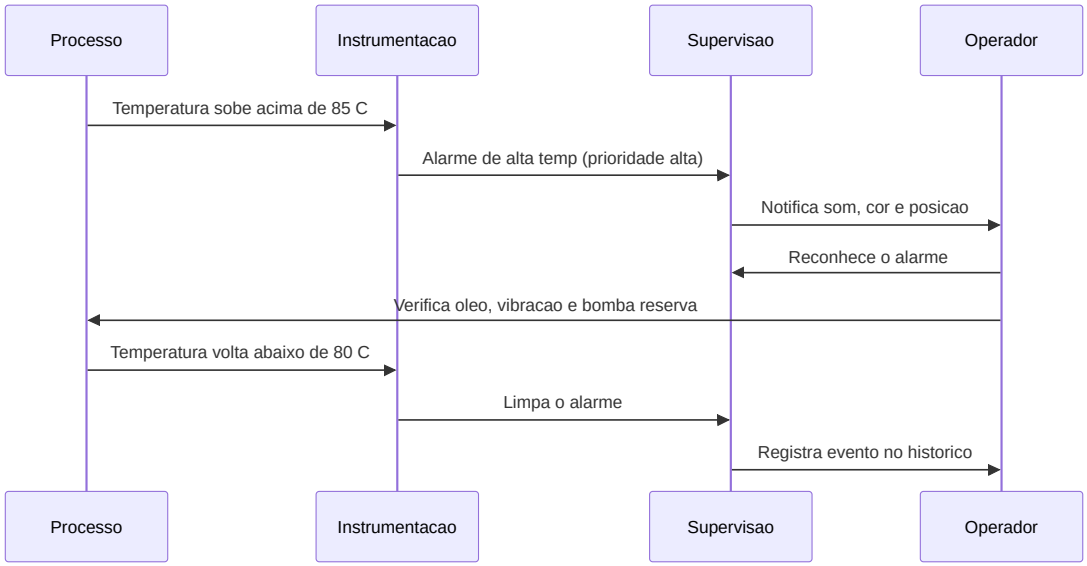
Campo	Conteúdo
Classificação	Prioridade, tipo (alarme/trip) e agrupamento na tela
Responsável e revisão	Quem aprovou e quando revalidar

Um teste rápido de qualidade: se a coluna “ação esperada” está vazia ou diz “verificar o processo”, o alarme ainda não foi racionalizado.

10.5.1 Um exemplo completo

Considere um alarme de **alta temperatura** no mancal de uma bomba de processo.

- **Causa:** perda de lubrificação, desalinhamento ou falha de resfriamento.
- **Consequência:** desgaste acelerado e falha do mancal em dezenas de minutos; parada não programada.
- **Ação esperada:** verificar vazamento de óleo, checar vibração, avaliar acionamento da bomba reserva.
- **Prioridade:** alta (há tempo de minutos, mas a ação é obrigatória).
- **Limite:** 85 °C; **deadband:** 5 °C; **atraso:** 5 s.
- **Justificativa:** a faixa normal é 45–70 °C; 85 °C indica perda de lubrificação; o *deadband* evita alternância de estado perto do limite; o atraso elimina a leitura espúria de partida.



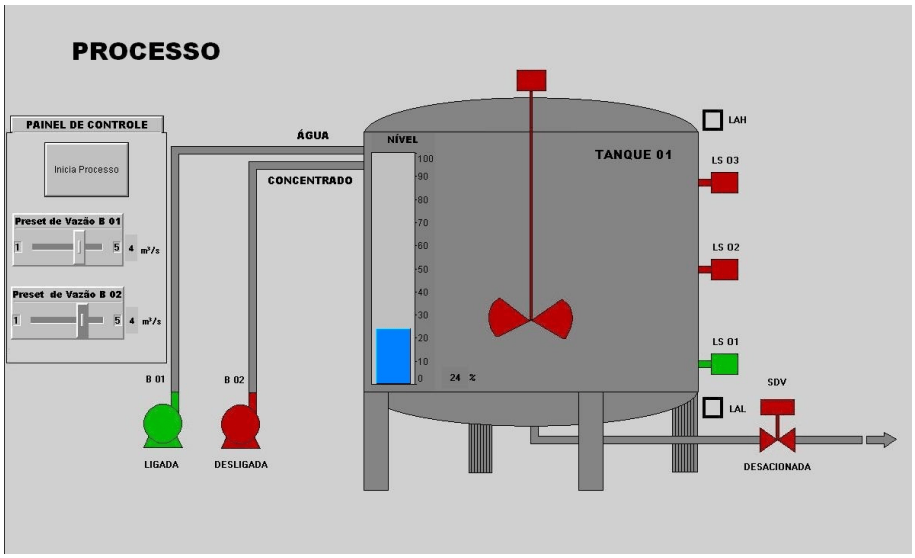


Figura 10.3 – Sinalização de estado no sinótico, com o alarme fora da tela de processo. **Fonte:** *Livro SCADA — versão para análise*, p. 135 (Machado, Pontes e Vianna, reproduzida com crédito).

A tela separa três coisas que não podem se misturar: o **sinótico** com o estado do equipamento (cor e forma das válvulas, altura do nível), o **painel de controle** com os *setpoints* e os indicadores de estado discreto (LAH, LS-01 a LS-03) e a **barra de navegação** que leva à lista de alarmes. O alarme não é anunciado dentro do desenho do processo; ele recebe uma tela própria, com horário, prioridade e botão de reconhecimento — a separação que a Seção 10.6 justifica.

10.6 Apresentação ao operador e desempenho

10.6.1 Como o alarme chega ao operador

A ergonomia de telas e a sinalização de alarmes podem ser orientadas pela **ISA-101** (*Human-Machine Interfaces for Process Automation Systems*), referência de engenharia para interfaces de sistemas de automação de processo. Ela está no material de origem desta apostila (inclui o simpósio ISA-101 realizado em São Paulo, com aplicação em saneamento).

Nota

Nota: a ISA-101 **não é um software SCADA**. É um conjunto de diretrizes e padrões de engenharia para criar telas eficientes: o pacote de supervisão é escolhido à parte, e a norma diz **como** a informação deve ser apresentada ao operador. Comprar um SCADA não garante uma IHM de alta performance — projetá-la, sim.

O que define uma IHM/SCADA de **alta performance** pela ISA-101:

- **Foco cognitivo:** elimina poluição visual, gráficos 3-D excessivos e cores chamativas sem função. Cada elemento que sobra compete com o que importa.

- **Uso estratégico de cores:** o processo em condição normal é desenhado em tons neutros (fundo cinza/apagado), e as cores vibrantes ficam reservadas para alarmes e anomalias. Se tudo é colorido, nada se destaca.
- **Hierarquia da informação:** as telas são organizadas em níveis navegáveis, para que o operador entenda o problema rapidamente sem se perder em excesso de dados — a hierarquia de níveis 1 a 4 detalhada no Capítulo 11.
- **Tomada de decisão rápida:** é o resultado dos três itens anteriores — reduz o tempo de resposta a situações anormais e amplia a **consciência situacional** do operador.

O gerenciamento de alarmes tratado nas Seções 10.1 a 10.5 é o outro lado da mesma moeda: a ISA-101 cuida de **como** a informação chega ao operador, enquanto a ISA-18.2 e a IEC 62682 definem **quando** um alarme deve existir.

Aplicando esses princípios à tela de alarmes:

- **Cor consistente:** seguir a filosofia da planta, não usar cor apenas como decoração e oferecer outros sinais de prioridade além da cor.
- **Lista previsível:** apresentar horário, estado e critério de ordenação de modo consistente; mudanças de ordenação não devem ocultar alarmes novos ou dificultar o acompanhamento.
- **Som projetado para a operação:** definir avisos sonoros conforme a filosofia e evitar diferenças que acrescentem ruído ou fadiga; registrar o silenciamento quando aplicável.
- **Agrupamento:** a tela precisa indicar a **área** afetada, não apenas a tag.
- **Taxa de atualização coerente** com a dinâmica do processo — atualizar 10 vezes por segundo um valor que muda por hora só consome rede e desvia atenção.

10.6.2 Métricas de desempenho

O acompanhamento é o que dá objetividade à discussão. As métricas mais usadas:

Tabela 10.6 – Métricas usuais e valores de referência citados

Métrica	Como se lê	Referência usualmente citada
Taxa média de alarmes	Alarmes por hora por operador	Máximo da ordem de 10 alarmes a cada 10 minutos
Taxa média de alarmes	Alarmes por hora por operador	Picos da ordem de 10 alarmes em 10 minutos indicam sobrecarga a investigar

Nota: os números da tabela são referências orientativas, não critérios universais de aceitação. Confirme a fonte, a edição e o contexto antes de compará-los ou incluí-los em contrato; as taxas toleráveis dependem da situação operacional e da estratégia de resposta. | Volume diário | Alarmes por dia por operador | Cerca de 150 alarmes/dia como meta de sistema maduro | | Alarmes repetitivos | Percentual que chattering | Pequena fração do total; reduzir com deadband e atraso | | Alarmes fixos | Ativos por mais de 24 h | Tendência a zero | | Distribuição por prioridade | Fração de críticos | Críticos como minoria clara do total | | Tempo de resposta | Intervalo entre alarme e ação | Compatível com o tempo de resposta definido na ficha |

Nota

Nota: esses valores são referências de mercado difundidas por EEMUA 191 e ISA-18.2. Antes de usá-los em contrato ou relatório, **verifique a redação vigente da edição atual** da referência escolhida — a redação e a apresentação das metas mudam entre edições, e a norma não deve ser citada por aquilo que ela não afirma.

10.7 Alarmes em SCADA e em plataformas IIoT

10.7.1 No SCADA clássico

O servidor de supervisão mantém o motor de alarmes:

- Avaliação cíclica dos limites, com deadband e atraso por tag.
- Estado por alarme: ativo/não reconhecido, reconhecido, limpo, suprimido, silenciado, fora de serviço.
- Registro em base de dados de eventos, com **timestamp de origem** e **timestamp de detecção** distintos — a diferença entre eles é um dado de engenharia valioso.
- Área de auditoria: quem reconheceu, quem silenciou, quem alterou limite, quando e por quê.

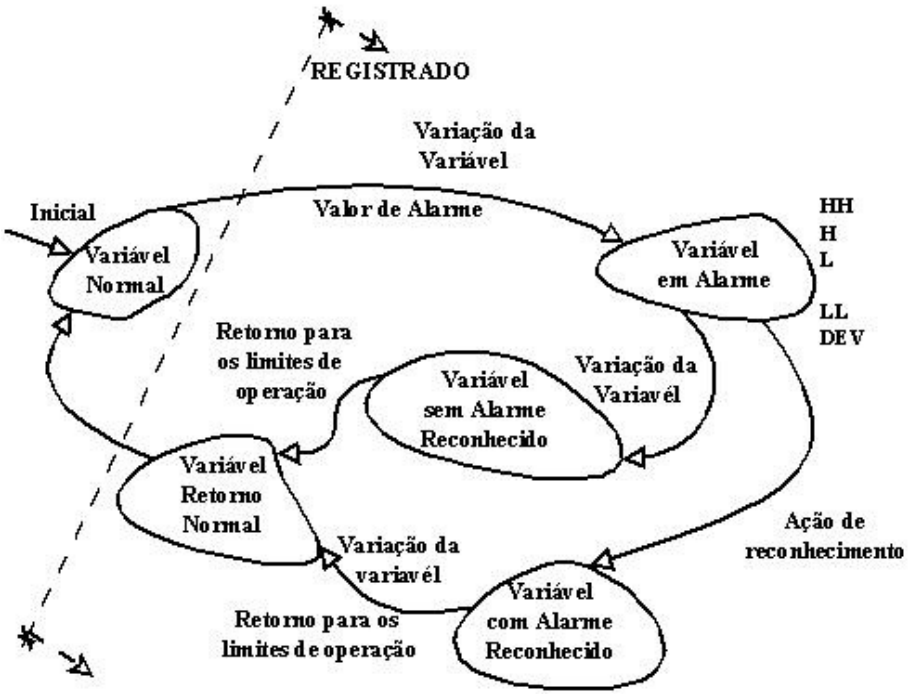


Figura 10.4 – Diagrama de estados de uma variável de alarme. **Fonte:** *Livro SCADA — versão para análise*, p. 61 (Machado, Pontes e Vianna, reproduzida com crédito).

Este é o modelo de estados do alarme, e ele explica por que **reconhecer** e **normalizar** não são a mesma coisa. A variável parte de *Normal*, entra em *alarme* ao cruzar o limite e passa a *alarme reconhecido* quando o operador age. Se a condição persistir, ela **volta** ao estado de alarme — agora com o reconhecimento registrado. Só o retorno da variável aos limites de operação devolve o estado *normal*. Um sistema que oferece apenas “alarme

ativo/inativo" perde essa distinção e não consegue auditar quem agiu, quando e sobre qual condição.

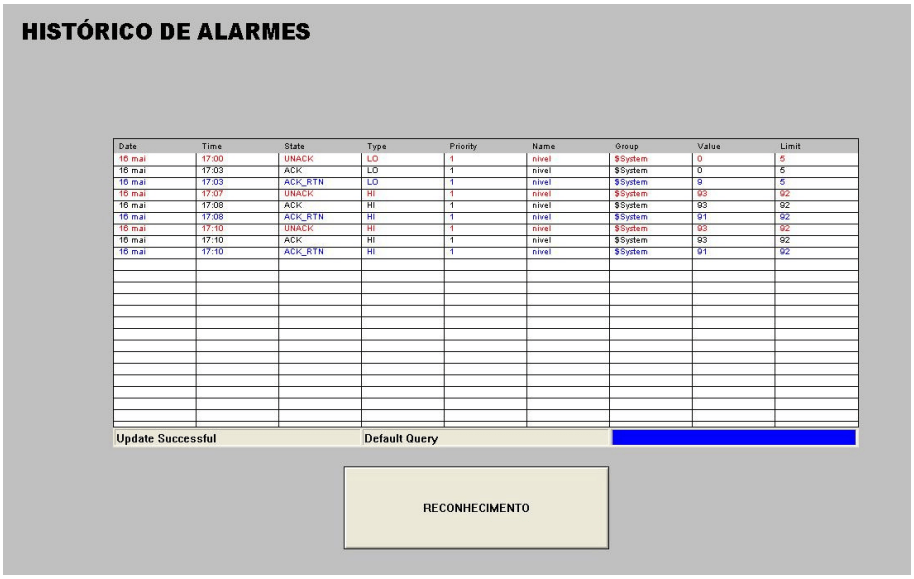


Figura 10.5 – Registro de eventos de alarme com estado, prioridade, valor e limite. **Fonte:** *Livro SCADA — versão para análise*, p. 116 (Machado, Pontes e Vianna, reproduzida com crédito).

A janela de histórico é o lado do sistema que **não** é apagado pelo reconhecimento. As colunas são as da ficha de racionalização transformadas em dado: *state* (UNACK/ACK/RTN), *type*, *priority*, *name*, *group*, *value*, *limit*. Repare no botão de reconhecimento separado da consulta: quem consulta é auditoria, quem reconhece é operação — e as duas ações exigem perfis de acesso diferentes.

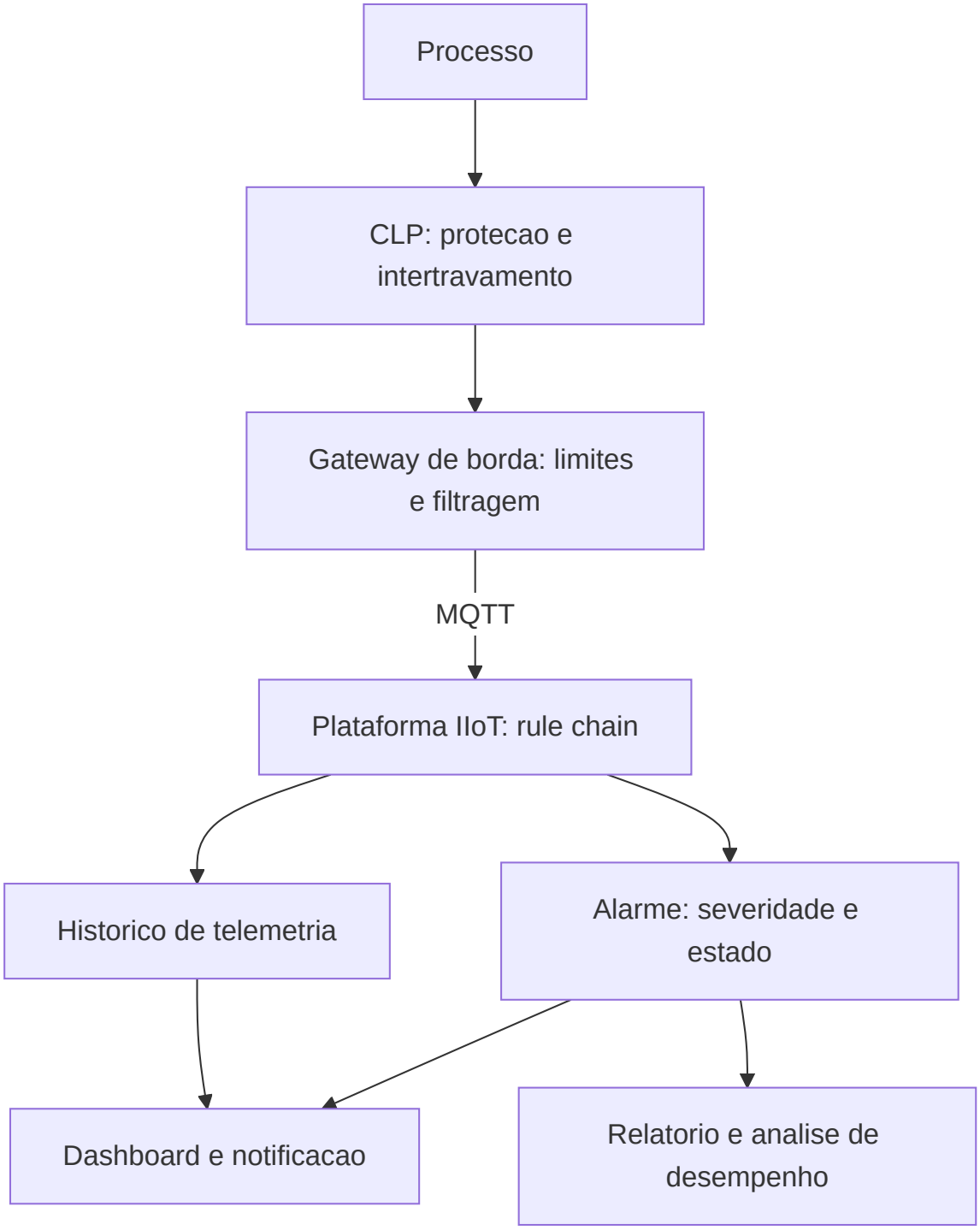
Atenção

Atenção: alarme gerado a partir de dado com qualidade ruim (*bad quality*) é uma armadilha frequente. Se o valor é inválido por falha de comunicação, o sistema não deve gerar "temperatura baixa" — deve gerar **falha de comunicação**. Tratar qualidade de dado como dado é uma das causas mais comuns de alarmes espúrios.

10.7.2 Na borda e na plataforma

Em arquiteturas com MQTT e plataformas IIoT, o alarme pode ser gerado em três lugares: no CLP (intertravamento e proteção), no *gateway* de borda (primeira avaliação de limites) e na plataforma (correlação, notificação, acompanhamento e relatório).

No ThingsBoard, o alarme é uma **entidade** com tipo, severidade e estado (*active unacknowledged*, reconhecido, limpo), criada e encerrada por nós de uma *rule chain*. Os campos observáveis na documentação oficial incluem tipo, severidade, origem, instante de início e de fim, estado de reconhecimento e um campo de detalhes com valores do momento.



A divisão de responsabilidade recomendada:

Onde	O que faz	Nunca deve fazer
CLP	Proteção, intertravamento, trip	Depender da rede para segurança
Borda	Filtragem, agregação, alarme local	Assumir lógica de segurança funcional

Onde	O que faz	Nunca deve fazer
Plataforma	Correlação, notificação, histórico, relatório	Ser o único caminho para uma ação crítica
SCADA	Supervisão e comando do operador	Ser o único repositório de eventos

10.8 Estudo de caso — alarmes de uma estação de bombeamento

Cenário. Estação com quatro bombas, uma em operação e três em espera, controlando vazão para um reservatório a 12 km. Instrumentação: pressão de sucção e descarga, temperatura de mancal, corrente do motor, nível do reservatório. Comunicação: CLP local com o SCADA por rádio; telemetria para a nuvem por MQTT via *gateway*.

Situação inicial. 420 alarmes por dia por operador; nos primeiros minutos após a partida de uma bomba, 70 alarmes em 90 segundos. Operadores mantinham o som silenciado.

Diagnóstico. Os alarmes de pressão de sucção e de corrente disparavam durante a partida (transitório de aceleração), o que caracteriza alarme fugaz. Todos os alarmes tinham prioridade alta. Quatro alarmes de temperatura permaneciam ativos há semanas, sem ação possível — alarmes fixos indicando sensor com falha não reparada.

Ações.

1. **Racionalização** dos 180 alarmes do sistema: 62 removidos (não acionáveis), 41 tiveram prioridade reclassificada, 23 tiveram limite e deadband revistos.
2. **Atraso de 10 s** nos alarmes de partida de bomba, com supressão automática por 20 s durante a sequência de partida (supressão por projeto, não manual).
3. **Correção dos alarmes fixos** com ordem de serviço para os sensores.
4. **Separação de falha de comunicação** de alarme de processo: falta de dado no rádio passou a gerar alarme de comunicação único por estação, não um por tag.
5. **Monitoramento mensal** com relatório de taxa, alarmes fixos, chattering e distribuição de prioridade.

Resultado (após 90 dias). Taxa média de 6 alarmes a cada 10 minutos, cerca de 90 alarmes/dia por operador, nenhum alarme fixo, e o silenciamento permanente substituído por silenciamento com prazo registrado.

Leitura de engenharia. Nenhuma alteração exigiu troca de software. O ganho veio de **racionalização, atraso, deadband, prioridade e disciplina de gestão de mudanças** — exatamente o que o ciclo de vida propõe.

10.9 Atividade prática

Em grupos de três, com a lista de alarmes fictícia fornecida pelo professor (ou extraída de um projeto real, anonimizado):

1. Classifique cada item como **alarme**, **advertência** ou **registro**, justificando.
2. Preencha a ficha de racionalização (Tabela 10.5) para os cinco itens de maior criticidade.
3. Defina limite, deadband e atraso, justificando cada escolha com a dinâmica do processo.
4. Proponha a apresentação na tela do operador: cor, prioridade, som, agrupamento.
5. Liste as métricas que serão usadas para avaliar o sistema em 90 dias e o critério de sucesso.

10.10 Resumo

- Alarme é um evento que exige **resposta do operador em tempo definido**; sem ação possível, é registro.
- **Prioridade** (urgência da resposta) e **severidade** (consequência) são dimensões diferentes.
- Limite, **deadband** e **atraso de confirmação** definem se o alarme é útil ou ruído.
- **ANSI/ISA-18.2** (edição vigente de 2016) e **IEC 62682** (edição vigente de 2022) são normas de mesmo escopo; **EEMUA 191** (3ª ed., 2013) é guia de engenharia e referência de desempenho; **NAMUR NA 102** (2003) é recomendação de indústria química/farmacêutica.
- O **ciclo de vida** tem dez etapas, da filosofia à auditoria, e é iterativo.
- A **racionalização** documenta causa, consequência, ação esperada, tempo de resposta e prioridade.
- A **gestão de mudanças** é o que impede a degradação do sistema ao longo do tempo.
- Em arquiteturas IIoT, o alarme pode ser gerado no CLP, na borda e na plataforma — sem transferir à nuvem nenhuma função de proteção.

10.11 Questões de Revisão

1. Explique a diferença entre alarme e evento e dê um exemplo de cada em um processo de tratamento de água.
2. Um alarme de nível alto tem severidade alta e prioridade baixa. Isso é contraditório? Justifique.
3. Qual é a função da faixa morta (*deadband*) e o que acontece se ela for configurada com valor muito alto?
4. Por que um atraso de confirmação reduz alarmes fugazes? Em que situação ele seria prejudicial?
5. Compare ISA-18.2, IEC 62682, EEMUA 191 e NAMUR NA 102 quanto a natureza (norma ou guia) e escopo.
6. Descreva três etapas do ciclo de vida de gerenciamento de alarmes e o produto de cada uma.
7. Um sistema tem 40% dos alarmes classificados como críticos. O que se pode concluir e que ação tomar?
8. O que é *alarm flooding* e quais ajustes de projeto reduzem sua ocorrência?

9. Explique por que a qualidade do dado (*quality*) deve ser tratada separadamente do valor no motor de alarmes.
10. Em uma arquitetura com CLP, *gateway* e plataforma em nuvem, qual camada deve concentrar a proteção do processo e por quê?

10.12 Referências

- INTERNATIONAL SOCIETY OF AUTOMATION. **ANSI/ISA-18.2 — Management of Alarm Systems for the Process Industries**. Research Triangle Park: ISA, edição vigente de 2016 (1ª edição: 2009).
- INTERNATIONAL ELECTROTECHNICAL COMMISSION. **IEC 62682 — Management of alarm systems for the process industries**. Genebra: IEC, edição vigente de 2022 (substitui a edição de 2014).
- ENGINEERING EQUIPMENT AND MATERIALS USERS ASSOCIATION. **EEMUA 191 — Alarm Systems: A Guide to Design, Management and Procurement**. Londres: EEMUA, 3ª edição, 2013.
- NAMUR. **NA 102 — Alarm Management**. Leverkusen: NAMUR, 2003 (recomendação).
- ISA. *Alarm management questions that everyone asks*. InTech, 2020. Disponível em: `isa.org/intech-home/2020/march-april/features/alarm-management-questions-that-everyone-asks`.
- INTERNATIONAL SOCIETY OF AUTOMATION. **ISA-101 — Human-Machine Interfaces for Process Automation Systems**. Principal referência global de projeto de IHM/SCADA de **alta performance** (foco cognitivo, uso estratégico de cores, hierarquia da informação e tempo de resposta); **não** é um software SCADA. Material de origem desta apostila: `ISA101.pdf` e simpósio ISA-101 de novembro de 2016.
- MATERIAL DE ORIGEM: *Livro SCADA — versão para análise* (capítulos sobre supervisão e alarmes, figuras dos autores reproduzidas com crédito); *ISA boas práticas SCADA/PIMS (2017).*

Capítulo 11

Dashboards e Visualização de Dados

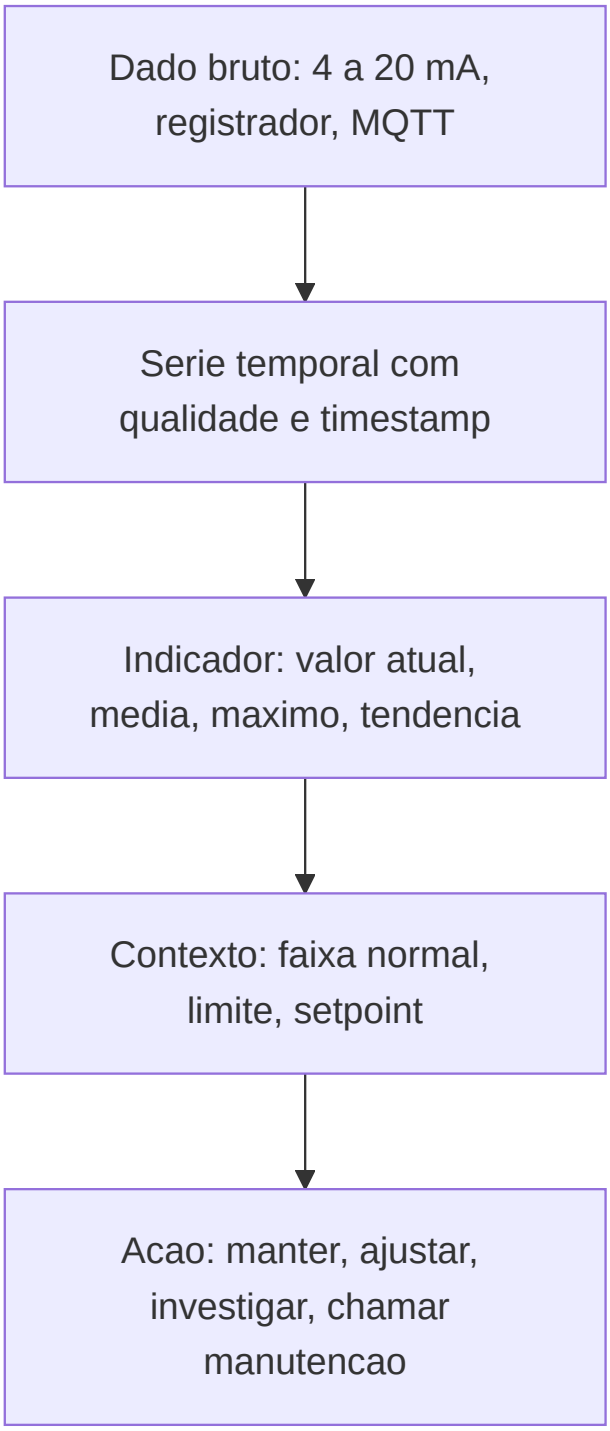
Objetivos de Aprendizagem

Ao final deste capítulo, o leitor será capaz de:

- Explicar a diferença entre dado, indicador e decisão, e o papel do dashboard nessa cadeia.
- Projetar uma hierarquia de telas coerente com a tarefa do operador, gestor e engenheiro.
- Escolher o tipo de painel adequado a cada tipo de dado.
- Aplicar princípios de cor, escala, precisão e atualização em painéis industriais.
- Comparar SCADA HMI, Node-RED Dashboard, ThingsBoard e Grafana sem tratá-los como substitutos.
- Configurar fontes de dados, variáveis e *alerting* no Grafana, incluindo provisionamento por arquivo.
- Montar um dashboard de processo com pressão, temperatura, vazão e nível.

11.1 Do dado ao indicador

Um dashboard não é um repositório de números: é um instrumento de decisão. A cadeia que leva do dado bruto à ação tem quatro elos, e cada elo tem sua responsabilidade.



A falha mais comum em dashboards industriais é pular o elo do **contexto**. Um painel que mostra 72,4 não diz nada; o mesmo valor com a faixa normal (45–70), o limite de alarme (85) e a tendência de 30 minutos permite decidir em segundos.

Tabela 11.1 – Vocabulário mínimo do dashboard

Elemento	Função	Cuidado
Valor instantâneo	Situação atual	Sem contexto vira número decorativo
Tendência	Comportamento no tempo	Janela de tempo tem de ser escolhida, não fixa
Indicador de estado	Ligado/desligado, aberto/fechado, manual/automático	Não usar cor de alarme para estado normal
KPI de processo	Vazão média, consumo específico, disponibilidade	Precisa de fórmula e período explícitos
Alarme	Condição que exige ação	Não despejar a lista completa de alarmes no painel de gestão
Meta/limite	Referência de comparação	Marcar graficamente no próprio painel

Dica

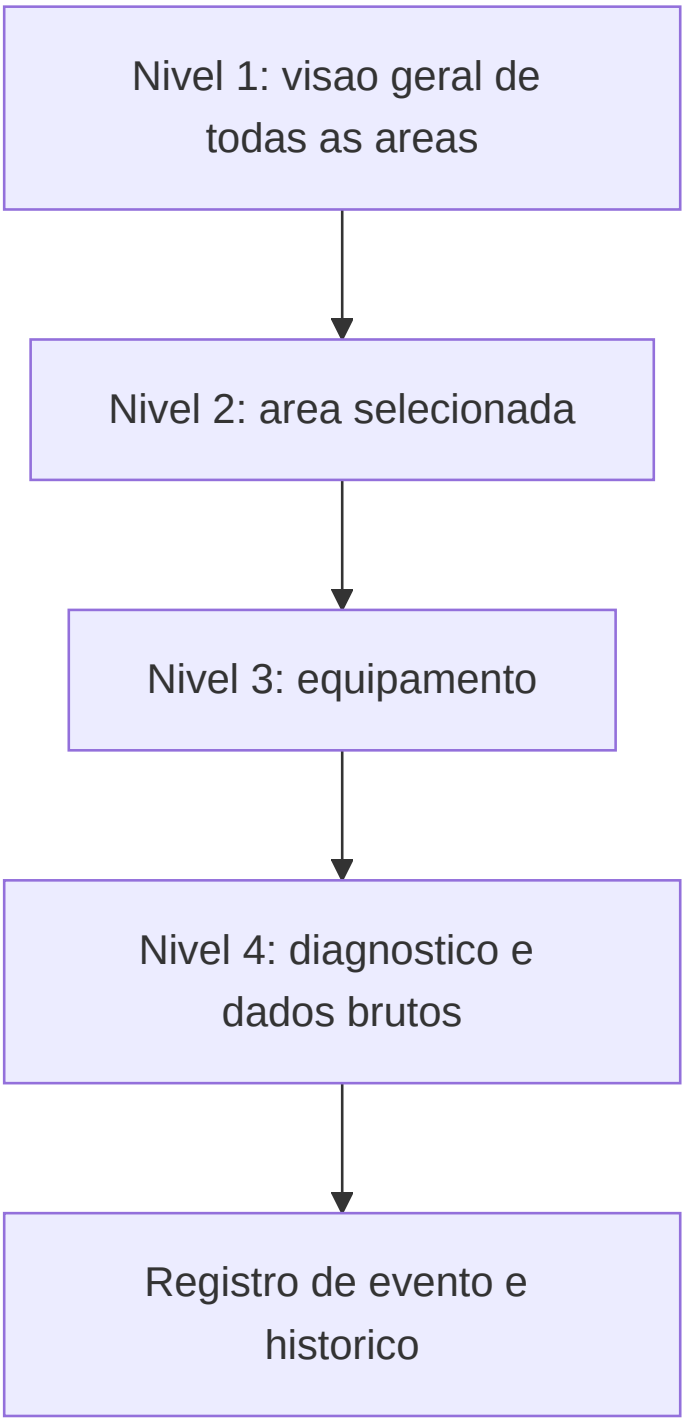
Dica: antes de desenhar qualquer tela, escreva a pergunta que ela responde. "Esta tela responde: a estação está entregando a vazão contratada?" Se a pergunta não estiver clara, o painel será uma coleção de widgets.

11.2 Hierarquia de telas

A ISA-101 — principal referência global para o projeto de **IHM de alta performance** (Seção 10.6.1) — organiza as telas de operação em níveis de detalhe. A mesma ideia vale para dashboards web de processo: fundo neutro, cor reservada a alarme e anomalia, e cada nível respondendo a uma pergunta.

Tabela 11.2 – Níveis de detalhe e responsabilidade

Nível	Público	Pergunta que responde	Densidade
1 — Visão geral	Gerência, operação	A planta inteira está saudável?	Muito baixa, uma informação por área
2 — Área/processo	Operador	Esta área está dentro do esperado?	Média, sinótico com valores-chave
3 — Equipamento	Operador e manutenção	A bomba 2 está em condição normal?	Alta, todas as variáveis relevantes
4 — Diagnóstico	Manutenção e engenharia	Por que o mancal está aquecendo?	Máxima, tendências, eventos, dados brutos



A regra prática é **navegação a dois cliques**: do nível 1 ao dado que explica a anomalia em no máximo dois cliques. Se o engenheiro precisa abrir três telas e consultar um banco separado, o dashboard não cumpriu sua função.

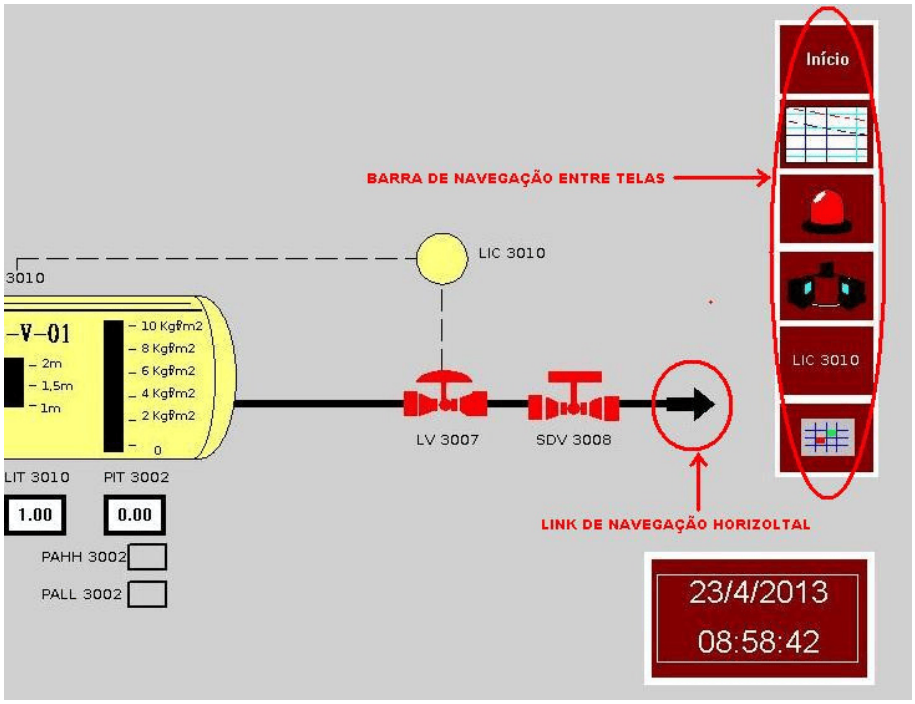


Figura 11.1 – Navegação entre telas: barra de navegação e link horizontal no sinótico. **Fonte:** *Livro SCADA — versão para análise*, p. 83 (Machado, Pontes e Vianna, reproduzida com crédito).

A figura mostra os dois mecanismos de navegação que sustentam a hierarquia da Tabela 11.2. A **barra de navegação** (à direita) é persistente e dá acesso direto aos níveis 1 e 4 — processo, tendência e alarmes. O **link horizontal** (no corpo do sinótico, apontando para o equipamento) é contextual: leva ao nível 3 daquele ativo específico. O relógio fixo no canto inferior direito não é decoração: sem hora de referência, o operador não consegue comparar a tela com o registro de eventos.

11.3 Tipos de painel e quando usar cada um

A escolha do painel depende do tipo de dado e da pergunta.

Tabela 11.3 – Tipo de painel por tipo de dado

Tipo de dado	Painel adequado	Painel inadequado
Analógico contínuo (temperatura, pressão)	Tendência + valor com faixa	Gauge de ponteiro para 30 variáveis
Estado discreto (liga/desliga)	Indicador de estado com rótulo textual	Gráfico de linha
Contador acumulativo	Valor com unidade e período	Média do contador
Evento esporádico	Tabela ou linha do tempo de eventos	Média móvel
Comparação entre unidades	Barra ordenada ou mapa de calor	Pizza com mais de cinco fatias

Tipo de dado	Painel adequado	Painel inadequado
Distribuição no tempo	Histograma e curva de duração	Valor instantâneo
Posição geográfica	Mapa com estado por ponto	Lista sem ordenação

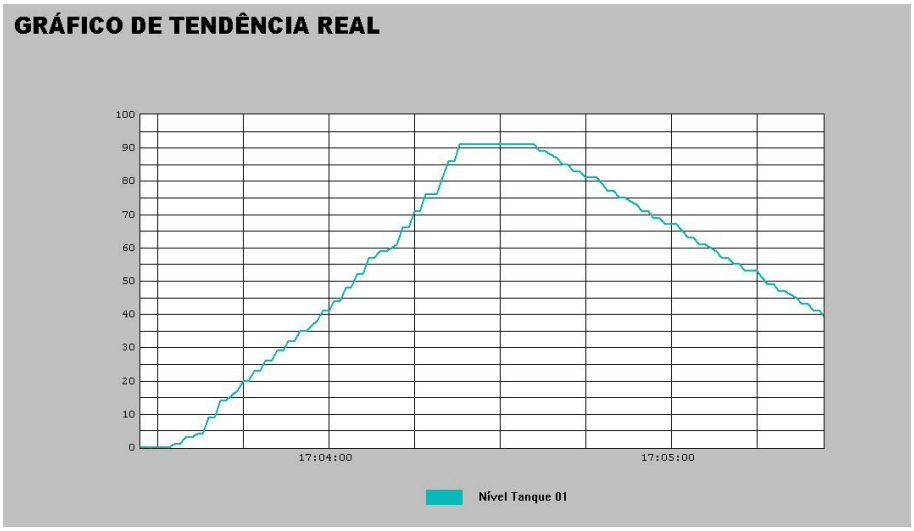


Figura 11.2 – Tendência de processo com escala fixa e legenda da variável. **Fonte:** *Livro SCADA — versão para análise*, p. 114 (Machado, Pontes e Vianna, reproduzida com crédito).

O painel de tendência é o tipo mais subestimado do dashboard industrial: ele responde "para onde está indo" e não apenas "quanto está". Repare nos três elementos que fazem a leitura funcionar — escala **fixa** de 0 a 100, eixo de tempo com hora explícita e legenda nomeando a variável (`Nível Tanque 01`). Um valor instantâneo de 45 seria idêntico em uma subida ou em uma descida; a tendência distingue os dois casos em um relance.

11.3.1 Cor, escala e precisão

- **Cor é semântica, não decoração.** Vermelho, âmbar e verde ficam reservados para condição anormal, atenção e normal — se a paleta da empresa usar vermelho no cabeçalho, o operador perde a referência.
- **Escala fixa em painéis de processo.** Escala automática faz um desvio de 2% parecer catástrofe. Fixe mínimo e máximo coerentes com a faixa de operação.
- **Precisão significativa.** Registrar 24,8735 °C quando o transmissor tem incerteza de 1 °C é ruído numérico. Duas casas decimais já é exagero para a maioria das variáveis de processo.
- **Atualização coerente com a dinâmica.** Nível de reservatório muda por minuto; atualizar a cada segundo consome rede e chama atenção para o que não muda.
- **Unidade sempre visível.** Um número sem unidade é um erro esperando para acontecer.
- **Nada piscando sem motivo.** Pisca é reservado para alarme não reconhecido de prioridade alta.

Tabela 11.4 – Antipadrões frequentes e a correção

Antipadrão	Por que é ruim	Correção
Sinótico como papel de parede	Reproduz o desenho do processo, não a informação	Reduzir ao essencial, uma informação por elemento
Dezenas de gauges de ponteiro	Exigem leitura lenta e comparação ruim	Tendências agrupadas com faixa fixa
Escala automática em painel de alarme	Amplifica ruído e assusta	Escala fixa com limites marcados
Dashboard de gestão com dados de 1 s	Ruído e custo de rede	Agregar por período e documentar a agregação
Muitas métricas sem hierarquia	Nada se destaca	No máximo cinco indicadores principais por tela
Filtro sem indicação de contexto	Usuário não sabe o que está vendo	Cabeçalho com período, ativo e unidade

11.4 Comparando plataformas de dashboard

Não existe vencedor: existe tarefa. A comparação útil é por função.

Tabela 11.5 – Comparação de plataformas de visualização

Plataforma	Função principal	Fonte de dados	Perfil de uso	Limitação típica
SCADA / HMI	Operação e comando em tempo real	Drivers industriais, CLP	Sala de controle, ação crítica	Custo por licença e pouca flexibilidade analítica
Node-RED Dashboard	Prototipagem e painéis operacionais simples	MQTT, HTTP, OPC UA, banco	Bancada, laboratório, integração rápida	Recursos limitados de autenticação e escala
ThingsBoard	Gestão de dispositivos e telemetria	MQTT, HTTP, CoAP	Frotas de dispositivos, alarme e RPC	Curva de aprendizado no motor de regras
Grafana	Análise e monitoramento sobre séries temporais	InfluxDB, Prometheus, SQL, MQTT	Engenharia, manutenção, gestão	Não é sistema de comando: não substitui o SCADA

A leitura correta dessas linhas: o SCADA **comanda**, o Grafana **analisa**, o ThingsBoard **gerencia dispositivos e telemetria**, o Node-RED **integra e prototipa**. Em uma planta real, os quatro podem coexistir, cada um no seu papel.

Atenção

Atenção: dashboard web **não** é interface de segurança nem de comando crítico. Comando de válvula, partida de equipamento e intertravamento pertencem ao CLP e à IHM/SCADA, com requisitos de disponibilidade e latência que um painel em nuvem não garante. Se a rede cai, o painel cai — o processo não pode depender dele.

11.5 Grafana na prática

O Grafana é uma plataforma de visualização que consulta fontes externas: ele não armazena os dados de processo (com exceção de configuração e estado de alerta). Isso define o desenho da arquitetura: primeiro o banco de séries temporais, depois o painel.

11.5.1 Fonte de dados: InfluxDB

A configuração de fonte de dados pode ser feita por interface ou por arquivo de provisionamento. Os dois formatos abaixo são os documentados oficialmente para InfluxDB 1.x e 2.x.

```
# InfluxDB 1.x - banco e modo HTTP
apiVersion: 1
datasources:
  - name: InfluxDB_SCADA
    type: influxdb
    access: proxy
    url: http://influxdb:8086
    jsonData:
      dbName: processos
      httpMode: GET
    secureJsonData:
      password: ${INFLUX_SENHA}
```

```
# InfluxDB 2.x - Flux, organizacao e bucket padrao
apiVersion: 1
datasources:
  - name: InfluxDB_V2
    type: influxdb
    access: proxy
    url: http://influxdb:8086
    jsonData:
      version: Flux
      organization: iffluminense
      defaultBucket: processo
    secureJsonData:
      token: ${INFLUX_TOKEN}
```

Observações de engenharia:

- O `access: proxy` faz o Grafana consultar o banco; o navegador do operador não fala direto com o banco — melhor para segurança e para redes segmentadas.
- Credenciais vão em `secureJsonData`, nunca em `jsonData`. Em ambiente real, use variáveis de ambiente ou cofre de segredos, como indicado no `${...}`.
- No InfluxDB 1.x o campo recomendado é `dbName` (o campo `database` é legado); no 2.x o idioma de consulta é o Flux.

11.5.2 Variáveis de dashboard

Variáveis transformam um dashboard em um instrumento reutilizável: em vez de uma tela por área, uma tela com seletor de área, bomba e período.

```
"templating": {
  "list": [
    {
      "name": "area",
      "type": "query",
      "datasource": "InfluxDB_V2",
      "query": "tag_values(processo, \"area\")",
      "includeAll": true,
      "multi": true,
      "refresh": 1
    }
  ]
}
```

O campo `query` é resolvido pela fonte de dados (por exemplo `tag_values` para InfluxDB) e `refresh` controla quando a lista é recarregada. Um cuidado de desempenho: variável com `refresh: 1` (a cada carregamento) em painel com muitos usuários gera consultas repetidas — considerar `refresh: 2` (sob demanda).

11.5.3 Alertas: a limitação que pega todo mundo

Atenção

Atenção: regras de alerta do Grafana são avaliadas no **backend**, sem o contexto do dashboard. Por isso, **uma regra de alerta não pode usar variáveis de template** (`$area`, `$bomba`): a consulta falha ou não retorna o esperado. Se a mesma consulta precisa existir no painel e no alerta, mantenha **duas versões**: uma com variáveis, para o painel, e outra com valores fixos, para o alerta. É a causa mais comum de "o alerta nunca dispara".

11.5.4 Provisionamento como código

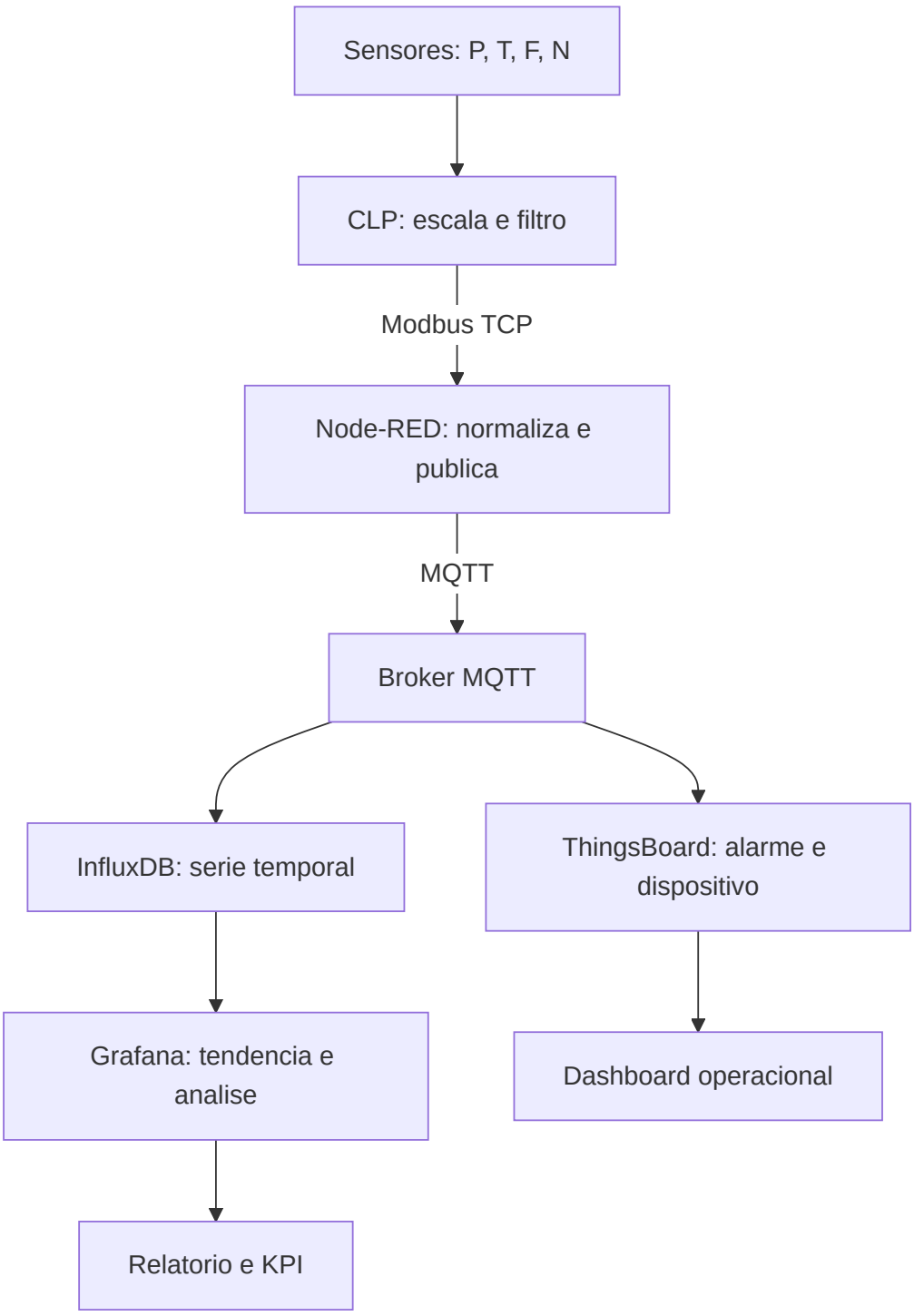
Configurar dashboards à mão não escala e não é auditável. O Grafana aceita provisionamento por arquivo em quatro tipos de recurso: **fontes de dados, plugins, dashboards e alertas**. Usuários **não** são provisionados por arquivo — isso é feito por integração com provedor de identidade.

Na prática: dashboards versionados em Git, provisionados por arquivo, revisados por *pull request*. É o equivalente, para a camada de visualização, da gestão de mudanças discutida no capítulo de alarmes.

11.6 Estudo de caso — dashboard de pressão, temperatura, vazão e nível

Cenário. Estação elevatória com quatro variáveis principais: pressão de descarga (0–16 bar), temperatura do mancal (0–120 °C), vazão (0–420 m³/h) e nível do reservatório (0–6

m). Telemetria publicada em MQTT a cada 5 s; persistida em InfluxDB; visualizada em Grafana.



Esquema de medição (InfluxDB). Um *measurement* por variável, *tags* para localização e *fields* para valor e qualidade:

```
measurement: processo
  tags:   area=estacao-1, equipamento=bomba-2, variavel=pressao
  fields: valor=11.4, qualidade=1
```

A **tag** identifica o ativo (cardinalidade baixa e estável); o **field** carrega o número. Trocar isso — usar valor na tag — destrói o desempenho do banco. E a **qualidade** entra como campo, não como comentário: painel que ignora qualidade mostra valor inválido como se fosse bom.

Painéis do dashboard operacional.

Painel	Conteúdo	Configuração relevante
Faixa de estado geral	Quatro indicadores com faixa normal	Escala fixa, unidade, limites marcados
Tendências	P, T, F e N com janela de 8 h	Sobreposição apenas de variáveis correlatas
Nível	Tendência longa (7 dias) + marcador de meta	Agregação por hora na janela longa
Consumo	KPI de m^3/h médio e energia por m^3	Período explícito no título
Eventos	Lista de alarmes e mudanças de estado	Ordenada por tempo, com reconhecimento
Diagnóstico	Vibração e corrente do motor	Nível 4, acesso restrito

Consultas (exemplos em Flux, InfluxDB 2.x).

```
from(bucket: "processo")
  |> range(start: -8h)
  |> filter(fn: (r) => r._measurement == "processo")
  |> filter(fn: (r) => r.variavel == "pressao")
  |> filter(fn: (r) => r._field == "valor")
  |> aggregateWindow(every: 1m, fn: mean, createEmpty: false)
```

```
from(bucket: "processo")
  |> range(start: -7d)
  |> filter(fn: (r) => r.variavel == "nivel")
  |> filter(fn: (r) => r._field == "valor")
  |> aggregateWindow(every: 1h, fn: max)
```

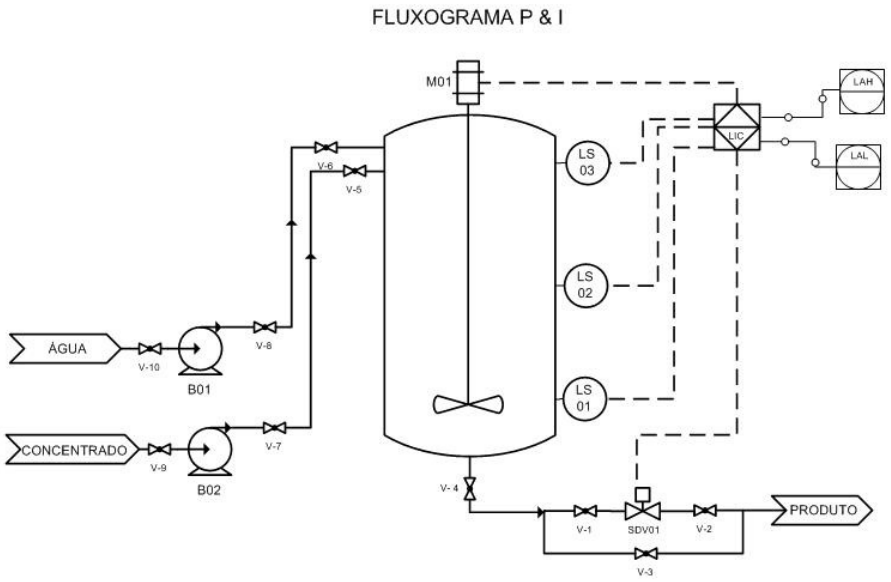


Figura 11.3 – Fluxograma P&I do processo usado no estudo de caso. **Fonte:** *Livro SCADA — versão para análise*, p. 89 (Machado, Pontes e Vianna, reproduzida com crédito).

Antes de desenhar qualquer painel, é preciso ter o **P&I** do processo. O fluxograma acima é a referência de verdade do estudo de caso: tanque misturador com agitação, duas linhas de alimentação (água e concentrado) com bombas B01 e B02, linha de produto com a válvula de controle V-1 e o conjunto de instrumentos que gera cada variável do dashboard — FT (vazão), LT (nível), TT (temperatura) e o controlador de vazão. O painel só pode mostrar o que o P&I instrumenta; o resto é expectativa.

Resultado de engenharia. O painel de gestão passou a mostrar consumo específico e disponibilidade; o painel do operador ficou com quatro variáveis e a lista de eventos; o nível 4 ficou restrito à manutenção. O tempo de leitura da condição geral caiu de cerca de dois minutos (consulta em três telas diferentes) para menos de dez segundos.

11.7 Atividade prática

Com os dados históricos disponibilizados em um arquivo CSV (ou em um InfluxDB local) contendo pressão, temperatura, vazão e nível de um período de sete dias:

1. Importe os dados para o banco de séries temporais e defina *tags* e *fields* com justificativa.
2. Crie três dashboards: visão geral (nível 1), área (nível 2) e diagnóstico (nível 4).
3. Escolha o tipo de painel para cada variável, justificando com a Tabela 11.3.
4. Use escala fixa nos painéis de processo e explique os limites escolhidos.
5. Crie uma variável de dashboard para selecionar o período e teste a navegação.
6. Crie **uma** regra de alerta, respeitando a limitação de variáveis de template, e explique a solução adotada.
7. Exporte os dashboards como JSON, versione os arquivos e descreva o fluxo de provisionamento.

11.8 Resumo

- Dashboard é instrumento de decisão: sem **contexto** (faixa normal, limite, período), o valor não informa.
- A hierarquia de telas (níveis 1 a 4, alinhada aos princípios da ISA-101) organiza o detalhe por público e por pergunta, com navegação a dois cliques.
- A ISA-101 define a **IHM de alta performance**: foco cognitivo, fundo neutro com cor só para anomalia, informação hierarquizada e decisão rápida — o mesmo vale para dashboards web.
- O tipo de painel deve seguir o tipo de dado: tendência para analógico, indicador de estado para discreto, tabela para evento.
- Cor, escala fixa, precisão significativa, unidade visível e atualização coerente são requisitos, não preferências.
- SCADA, Node-RED, ThingsBoard e Grafana têm papéis distintos e complementares; nenhum substitui o comando crítico do SCADA.
- No Grafana, a fonte de dados carrega o dado (InfluxDB 1.x por banco, 2.x com Flux), variáveis tornam o painel reutilizável, o provisionamento por arquivo traz auditabilidade e o *alerting* não aceita variáveis de template.
- Um dashboard industrial mínimo tem quatro variáveis de processo, contexto visual e uma lista de eventos — não trinta gauges.

11.9 Questões de Revisão

1. Explique por que um dashboard que mostra apenas o valor instantâneo é insuficiente para a operação.
2. Descreva os quatro níveis de detalhe de tela e o público de cada um.
3. Qual painel você escolheria para o estado ligado/desligado de oito bombas? Justifique.
4. Por que escala automática é desaconselhada em painéis de processo? Em que caso ela é aceitável?
5. Descreva três antipadrões de dashboard industrial e a correção de cada um.
6. Compare SCADA HMI, Node-RED Dashboard, ThingsBoard e Grafana quanto à função principal e à limitação típica.
7. Qual a diferença entre `jsonData` e `secureJsonData` na configuração de uma fonte de dados no Grafana?
8. Por que uma regra de alerta do Grafana não pode usar variáveis de template? Como resolver?
9. Em um banco de séries temporais, por que o valor medido deve ser *field* e não *tag*?
10. Um gestor pede um dashboard com dados de 1 segundo de todas as 40 variáveis da planta. Apresente duas objeções técnicas e uma proposta alternativa.

11.10 Referências

- GRAFANA LABS. **Documentação oficial do Grafana** — provisionamento de fontes de dados (InfluxDB 1.x e 2.x), modelo JSON de dashboards, variáveis de template e limitações de *alerting* com variáveis. Disponível em: `grafana.com/docs`. Consulta: 2026.

- INFLUXDATA. **InfluxDB — documentação oficial** (modelo de dados, *tags* e *fields*, linguagem Flux).
- INTERNATIONAL SOCIETY OF AUTOMATION. **ISA-101 — Human-Machine Interfaces for Process Automation Systems** (principal referência de projeto de IHM de **alta performance**: foco cognitivo, uso estratégico de cor, hierarquia de telas e tempo de resposta).
- MATERIAL DE ORIGEM: `ISA101.pdf`; *ISA-101 — III Simpósio ISA São Paulo / Sabesp, novembro de 2016*; *ISA boas práticas SCADA/PIMS, 2017*; `Node-Red_InterfaceDeSupervisão.pdf`; `IIoT e suas Tecnologias Aderentes.pdf`.
- OPC FOUNDATION. **OPC UA** — modelo de informação e qualidade de dado (*quality*), usados no contexto de dashboards de processo.

Capítulo 12

ThingsBoard

Objetivos de Aprendizagem

Ao final deste capítulo, o leitor será capaz de:

- Explicar o papel do ThingsBoard em uma arquitetura de supervisão e onde ele **não** deve ser usado.
- Descrever o modelo de entidades da plataforma: locatário, cliente, dispositivo, ativo e perfil.
- Distinguir telemetria de atributos e os escopos de atributo.
- Publicar telemetria por MQTT e por HTTP e explicar o formato do *payload*.
- Construir uma *rule chain* que persiste telemetria e cria e limpa um alarme.
- Distinguir RPC do lado do servidor e do lado do dispositivo.
- Integrar CLP e Node-RED a uma instância ThingsBoard, inclusive por *gateway*.
- Montar um dashboard com telemetria, atributos e alarmes.

12.1 O que é o ThingsBoard e onde ele se encaixa

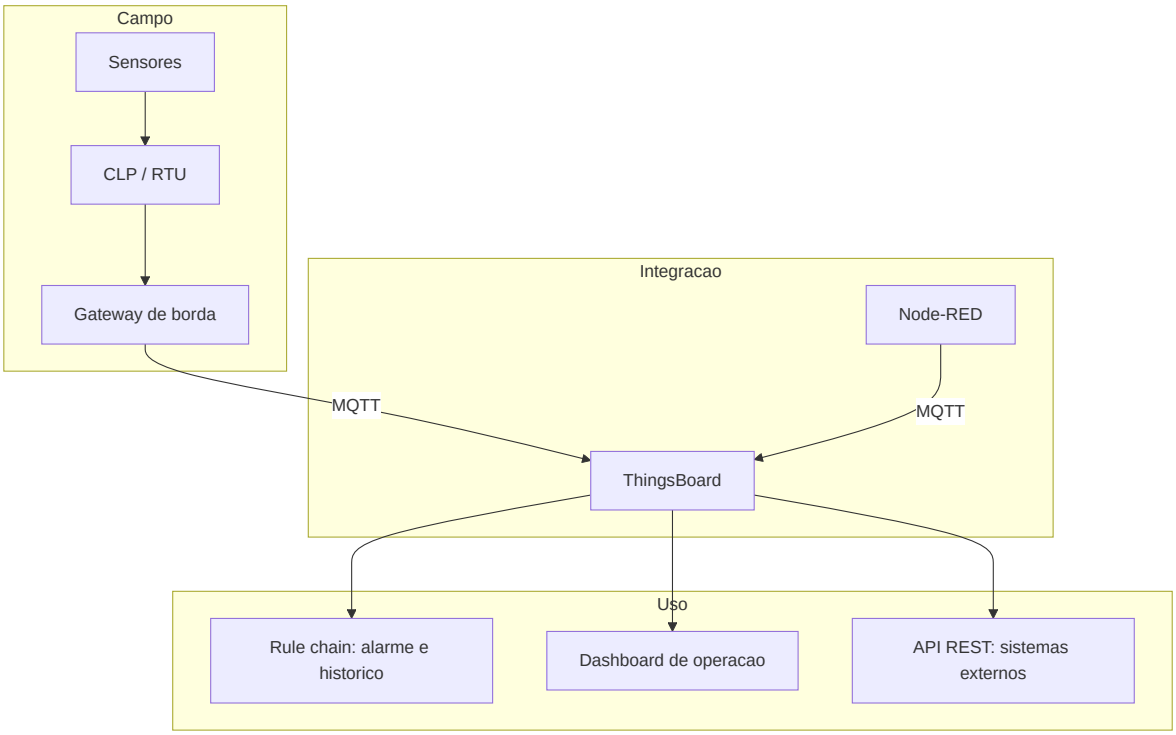
O ThingsBoard é uma plataforma de código aberto para **coleta, processamento, visualização e gestão de dados de dispositivos IoT e IIoT**. Ele oferece três modos de uso:

Tabela 12.1 – Modos de uso

Modo	O que é	Quando faz sentido
Community Edition (CE)	Código aberto, instalado pelo usuário	Estudo, laboratório, piloto, projeto com controle total da infraestrutura
Professional Edition (PE)	Distribuição com recursos adicionais de gestão e integração	Operação comercial com requisitos de suporte e escala
ThingsBoard Cloud	Serviço gerenciado	Prova de conceito rápida, sem infraestrutura própria

A documentação oficial concentra a maior parte dos exemplos em três domínios equivalentes: o *site* principal (`thingsboard.io/docs`), a versão profissional (`/docs/pe/`) e a nuvem (`/docs/paas/`). A API de dispositivo, no entanto, é a mesma — o que aprende no laboratório vale na operação.

Onde ele se encaixa. O ThingsBoard entra na camada de **integração, telemetria, alarme informativo e visualização** de frotas de dispositivos. Ele não substitui o CLP (controle e proteção) nem o SCADA (operação e comando do processo em tempo real).



Atenção

Atenção: o ThingsBoard **não** é sistema de comando de processo. Comandos enviados por *dashboards* em nuvem dependem de rede, proxy e disponibilidade da plataforma — nenhum deles é adequado a uma função de segurança ou de controle em malha fechada. Use a plataforma para supervisão, diagnóstico, manutenção e análise; mantenha proteção e controle no CLP.

12.2 Modelo de entidades

Entender o modelo de entidades é o que separa "usar a plataforma" de "usar a plataforma corretamente".

Tabela 12.2 – Entidades principais

Entidade	Papel	Exemplo na planta
Locatário (*tenant*)	Instância do cliente, raiz dos dados	A concessionária
Cliente (*customer*)	Subdivisão de negócio dentro do locatário	Município atendido
Dispositivo (*device*)	Ativo que publica dados	Gateway da estação, transmissor
Ativo (*asset*)	Agrupamento lógico de dispositivos	Estação elevatória 1, adutora norte
Perfil de dispositivo (*device profile*)	Modelo com comportamento comum	Perfil "transmissor de pressão"
Grupo de entidades (*entity group*)	Agrupamento para permissões e consultas	Bombas críticas

A hierarquia recomendada para processo industrial é: **ativo** representa a instalação física (estação, subestação, adutora) e **dispositivo** representa o equipamento conectado. Relações explícitas entre ativos e dispositivos permitem consultar "todas as variáveis da estação 1" sem armazenar isso no nome de cada dispositivo.

Dica

Dica: nomes de entidade são interface. `EST1-BOMBA2-TEMP-MANCAL` informa; `device_0017` não. Em projeto com mais de vinte dispositivos, um padrão de nomenclatura definido antes do cadastro vale mais que qualquer recurso da plataforma.

12.3 Telemetria e atributos

A distinção é conceitual e tem consequência direta em custo de armazenamento e desempenho.

Tabela 12.3 – Telemetria × atributos

Aspecto	Telemetria	Atributos
Natureza	Série temporal: cada valor é um ponto no tempo	Valor de estado: o último valor substitui o anterior
Exemplo	Temperatura, pressão, vazão	Número de série, versão de firmware, faixa de medição
Armazenamento	Histórico com todos os pontos	Apenas valor atual por chave
Escopo	Único	Cliente, compartilhado e servidor

Os **escopos de atributo** definem quem lê e quem escreve:

- **Atributo de servidor** (*server scope*): gerenciado apenas pela plataforma e pela API; o dispositivo não vê. Útil para dados de gestão, contrato e configuração central.

- **Atributo compartilhado** (*shared scope*): definido na plataforma e **enviado ao dispositivo**. É o mecanismo de configuração remota, como intervalo de amostragem ou limite de alarme.
- **Atributo de cliente** (*client scope*): publicado pelo dispositivo e visível na plataforma. Útil para metadados do próprio equipamento.

12.3.1 MQTT: tópicos e formato

O dispositivo autentica com o **token de acesso** e publica em tópicos definidos.

Tabela 12.4 – Tópicos MQTT de dispositivo

Operação	Tópico padrão	Forma curta
Publicar telemetria	`v1/devices/me/telemetry`	`v2/t`
Publicar atributo de cliente	`v1/devices/me/attributes`	—
Assinar atributo compartilhado	`v1/devices/me/attributes`	—
Assinar RPC do servidor	`v1/devices/me/rpc/request/+`	—
Requisição RPC do dispositivo	`v1/devices/me/rpc/request/$id`	`v2/r/req/$id`
Resposta de RPC do dispositivo	`v1/devices/me/rpc/response/$id`	`v2/r/res/+`

Formatos de *payload* aceitos para telemetria:

```
{"temperature": 22.5, "humidity": 61}
```

```
{"ts": 1757429087063, "values": {"temperature": 22.5}}
```

```
[
  {"ts": 1757429080000, "values": {"temperature": 22.5}},
  {"ts": 1757429090000, "values": {"temperature": 22.7}}
]
```

O primeiro formato usa o horário de recebimento na plataforma. O segundo e o terceiro usam **timestamp do dispositivo** (`ts`, em milissegundos Unix) — indispensável quando a publicação pode atrasar, como em enlace de rádio ou *store and forward* na borda.

Atenção

Atenção: sem *timestamp* do dispositivo, todo dado que chega atrasado é gravado com a hora de chegada. Em enlace instável isso produz séries temporais que mentem: uma leitura de duas horas atrás aparece como atual. Se o seu *gateway* tem fila local, **sempre** envie `ts`.

12.3.2 HTTP e REST

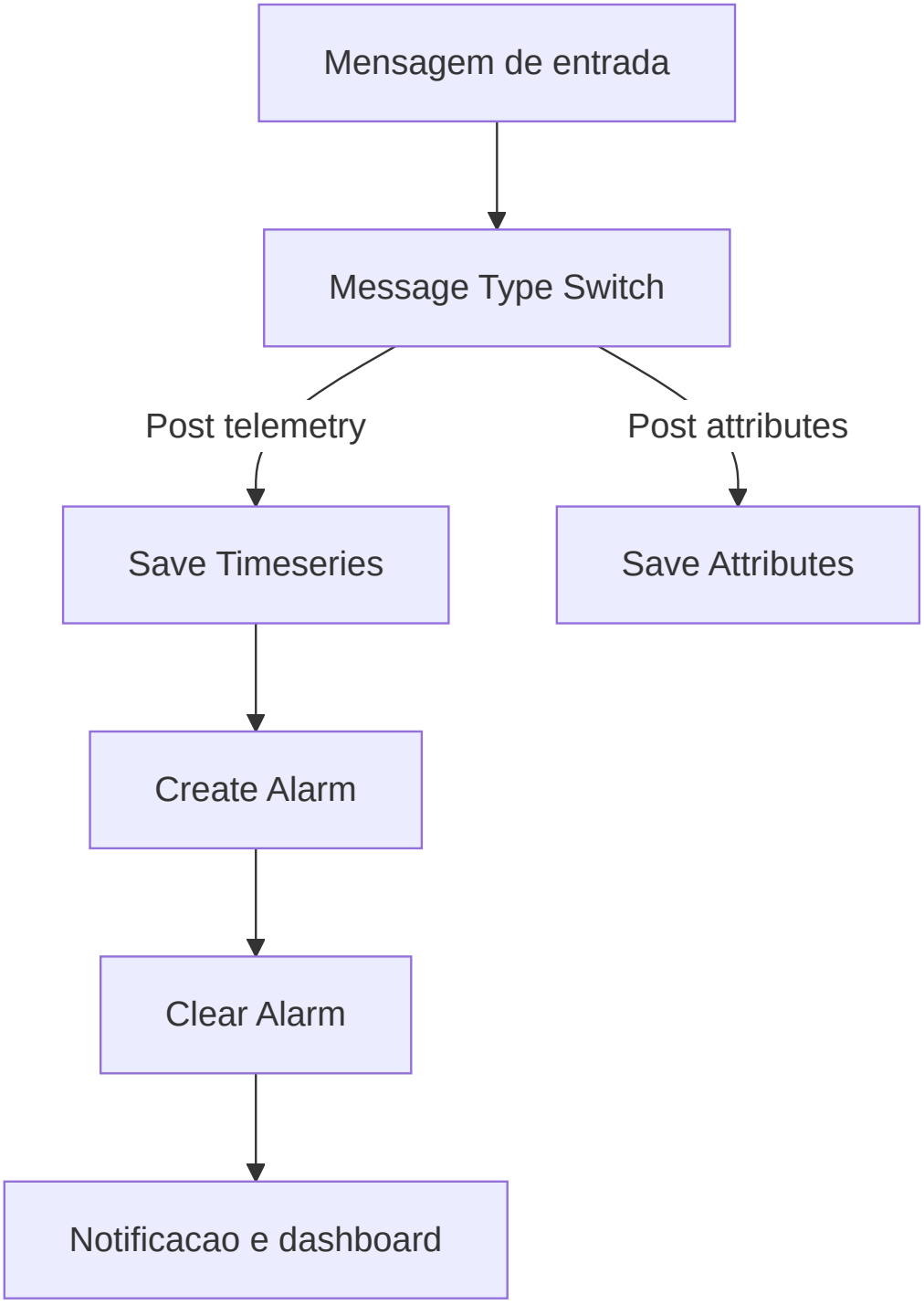
Para dispositivos sem MQTT ou para integração com sistemas externos, a plataforma expõe:

```
POST https://<host>/api/v1/$ACCESS_TOKEN/telemetry
POST /api/plugins/telemetry/{tipoEntidade}/{id}/attributes/{escopo}
```

A mesma estrutura de *payload* é aceita. O segundo *endpoint* é usado por aplicações com credencial de usuário (API REST administrativa), não pelo dispositivo.

12.4 Rule engine

O motor de regras é o núcleo de processamento: cada mensagem (telemetria, alteração de atributo, evento de ciclo de vida, chamada RPC) percorre uma **rule chain** (*cadeia de regras*), composta por nós conectados.



Nós mais usados em um projeto de supervisão:

Nó	Função
Message Type Switch	Encaminha conforme o tipo da mensagem (telemetria, atributo, RPC)
Save Timeseries	Persiste a telemetria com retenção configurável

Nó	Função
Save Attributes	Grava atributos no escopo definido
Create Alarm	Cria ou atualiza um alarme a partir da condição avaliada
Clear Alarm	Encerra o alarme quando a condição cessa
Fetch Device Info	Enriquece a mensagem com dados da entidade de origem
Kafka / MQTT / REST	Publica a mensagem em sistemas externos

Figura

[**Figura 12.1 – Editor de rule chain com o fluxo de alarme**] *Captura do editor com quatro nós conectados: Message Type Switch, Save Timeseries, Create Alarm e Clear Alarm. Anotar as saídas "Post telemetry", "Success" e "Alarm created"; destacar onde ficam o tipo do alarme e a severidade. Interface em inglês, captura genérica.*

Um nó comum é o **Message Type Switch**, que separa o fluxo: mensagens `Post telemetry`, `Post attributes` e `RPC Request` seguem caminhos diferentes na mesma cadeia.

12.4.1 Alarme como entidade

No ThingsBoard o alarme é uma **entidade** com estado. Os campos relevantes observáveis na documentação oficial:

- **Tipo** (*type*), como `High Temperature`.
- **Severidade** (*severity*): `CRITICAL`, `MAJOR`, `MINOR`, `WARNING`, `INDETERMINATE`.
- **Origem** (*originator*): dispositivo ou ativo que gerou.
- **Estado**: ativo não reconhecido (`ACTIVE_UNACK`), ativo reconhecido, limpo não reconhecido, limpo reconhecido.
- **Instantes** de início (`startTs`) e fim (`endTs`).
- **Detalhes** (*details*): valores do momento, definidos por script.

```
{
  "scriptLang": "TBEL",
  "alarmType": "High Temperature",
  "severity": "CRITICAL",
  "alarmDetailsBuildTbel": "return { temperature: msg.temperature };"
}
```

O nó **Clear Alarm** encerra o alarme quando a condição normaliza — e o **reconhecimento** é dado operacional, não de configuração: é o registro de que alguém assumiu o alarme.

Nota

Atualização tecnológica: o alarme em plataforma IIoT **não substitui** o gerenciamento de alarmes de processo do capítulo 10. Ele cobre o ciclo operacional de notificação, reconhecimento e histórico de uma frota de dispositivos, mas a filosofia de alarmes, a matriz de prioridade, a racionalização e o controle de mudanças continuam sendo o que dá qualidade ao sistema. Sem essas disciplinas, a plataforma apenas automatiza a produção de alarmes em excesso.

12.5 RPC — comando e consulta

RPC (*Remote Procedure Call*) permite trocar comandos e respostas entre plataforma e dispositivo.

- **RPC do lado do servidor** (*server-side*): a plataforma (ou um dashboard) envia um comando ao dispositivo. Exemplos: alterar intervalo de amostragem, solicitar leitura imediata, acionar reinício controlado.
- **RPC do lado do dispositivo** (*client-side*): o dispositivo solicita algo à plataforma por um tópico de requisição e recebe a resposta em outro.

```
Dispositivo publica:  {"method": "getCurrentTime", "params": {}}
em v1/devices/me/rpc/request/42
Plataforma responde:  {"currentTime": 1757429087063}
em v1/devices/me/rpc/response/42
```

Cuidado de engenharia: RPC é um canal de **comando**, e comando exige autorização e auditoria. Antes de habilitar RPC de escrita em um dispositivo de processo, defina quem pode chamar, qual o efeito de um comando repetido (idempotência) e como o evento aparece no registro de auditoria.

12.6 Dashboards e visualização

Os dashboards da plataforma oferecem painéis de série temporal, indicadores, tabelas, mapas e widgets de alarme. O que muda em relação ao capítulo 11 é a **origem do dado**: aqui o painel consulta a própria plataforma, e não um banco externo.

Diretrizes que continuam valendo integralmente:

- Escala fixa em painéis de processo; unidade visível; precisão significativa.
- Poucos indicadores por tela, com hierarquia clara.
- Cor reservada para significado, não para estética.
- Lista de alarmes separada do painel de indicadores — no painel de operação, o alarme entra como destaque e como contagem por severidade.

Figura

[Figura 12.2 – Dashboard de operação com telemetria, atributos e alarmes]
Tela com quatro indicadores de processo com faixa normal, duas tendências (24 h), um widget de estado de alarme por severidade e uma tabela de dispositivos com último contato. Destacar o uso de escala fixa e a ausência de cor decorativa. Captura genérica, sem marca de terceiros.

12.7 Integração com CLP e com Node-RED

12.7.1 Direto do CLP

CLPs com função de cliente MQTT publicam telemetria diretamente, desde que o *payload* siga o formato esperado e o token esteja protegido. Como token em CLP é difícil de rotacionar, o padrão mais defensável é **não** expor a credencial no controlador.

12.7.2 Por gateway de borda

O *gateway* concentra credenciais, fila local e conversão de protocolo. A plataforma define tópicos específicos para esse papel:

Tabela 12.5 – Tópicos MQTT de gateway (prefixo `v1/gateway/`)

Tópico	Direção	Uso
`connect` / `disconnect`	Publica	Anunciar conexão e desconexão de dispositivo subordinado
`telemetry`	Publica	Enviar telemetria de um ou vários dispositivos
`attributes`	Publica / Assina	Enviar atributos de cliente e receber atributos compartilhados
`attributes/request` e `attributes/response`	Publica / Assina	Solicitar valores de atributo
`rpc`	Assina / Publica	Receber comandos do servidor e enviar respostas
`claim`	Publica	Iniciar processo de reivindicação de dispositivo

Esse desenho é o que viabiliza o caso clássico: **um gateway para N dispositivos**, com uma única credencial e um único ponto de filtragem e buffer.

Figura

[Figura 12.3 – Gateway de borda concentrando dispositivos para a plataforma] *Diagrama com três CLPs e quatro transmissores de campo à esquerda; no centro, o gateway de borda com fila local e conversão de protocolo; à direita, a plataforma recebendo um único fluxo MQTT. Marcar com tracejado o caminho que permanece local quando o enlace cai. Sem logomarca de fabricante.*

12.7.3 Com Node-RED

O Node-RED atua como camada de integração e normalização — papel discutido no capítulo 7:

```
CLP -> Node-RED (Modbus TCP) -> normaliza -> MQTT -> ThingsBoard
```

Vantagens: conversão de unidades e escalas no fluxo, agregação, cálculo de indicadores antes da publicação (reduz volume na nuvem) e buffer em caso de queda de enlace.

Atenção

Atenção: publique na nuvem a **taxa que a decisão exige**, não a taxa que o CLP produz. Publicar 50 variáveis a 100 ms gera gigabytes por mês de dado que ninguém vai analisar. Agregue na borda (média, máximo e mínimo por minuto) e mantenha a alta taxa apenas para diagnóstico sob demanda.

12.8 Estudo de caso — monitoramento de frota de transmissores

Cenário. Uma estação de tratamento possui 24 transmissores de pressão e nível em campo, conectados a dois CLPs. Objetivo: acompanhar falhas de instrumento e condições anormais sem depender do operador para perceber o problema.

Arquitetura. CLP lê os transmissores; Node-RED na borda lê o CLP por Modbus TCP, normaliza escalas, calcula média de 1 minuto e publica em MQTT; um único *gateway* ThingsBoard encaminha 24 dispositivos; a *rule chain* persiste telemetria, cria alarme de desvio e de falha de comunicação e notifica a manutenção.

Regras implementadas.

Condição	Alarme	Severidade
Pressão fora da faixa por mais de 5 min	Desvio de processo	MAJOR
Valor com qualidade ruim por mais de 2 min	Falha de comunicação	CRITICAL
Sem mensagem por mais de 10 min	Dispositivo silencioso	CRITICAL
Bateria abaixo de 20% (onde aplicável)	Manutenção preventiva	WARNING

Decisões de projeto que fizeram diferença.

1. **Timestamp do dispositivo** mantido em todos os pontos, porque o enlace de rádio enfileira mensagens em dias de chuva.
2. **Alarme de comunicação separado** de alarme de processo — o mesmo erro discutido no capítulo 10.
3. **Agregação na borda**: 24 variáveis a 10 s na coleta, 1 minuto na publicação; volume na nuvem reduzido em cerca de 80%.
4. **Limite de alarme como atributo compartilhado**, ajustável pela plataforma sem reconfigurar o CLP — com registro de quem alterou.
5. **Autorização por papel**: operação reconhece alarmes; engenharia altera limites; nenhum perfil de dashboard pode acionar RPC de escrita.

Limitações reconhecidas. A plataforma não participa de proteção nem de intertravamento; o reconhecimento de alarme na plataforma não substitui o registro do SCADA; e nenhum comando crítico depende de disponibilidade de Internet.

12.9 Atividade prática

1. Suba uma instância ThingsBoard local (Docker) ou use uma instância de teste fornecida pelo professor.
2. Cadastre um **ativo** representando uma estação e dois **dispositivos** (transmissor de pressão e transmissor de nível) com perfil adequado.

3. Publique telemetria por MQTT com e sem `ts` e compare os horários registrados. Explique a diferença.
4. Defina um **atributo compartilhado** de limite de alarme e altere seu valor; descreva o efeito.
5. Monte uma *rule chain* que salve telemetria, crie alarme quando a pressão exceder o limite compartilhado e limpe o alarme na normalização; teste os dois caminhos.
6. Configure um dashboard com dois indicadores, uma tendência de 24 h e um widget de alarmes.
7. Simule falha de comunicação (pare de publicar por 10 min) e verifique o alarme correspondente.
8. Entregue um relatório de uma página com a arquitetura, a lista de alarmes configurados com severidade e justificativa, e um parágrafo sobre o que **não** deve ser feito por essa plataforma.

12.10 Resumo

- ThingsBoard é plataforma de **coleta, processamento, gestão e visualização** de dados de dispositivos — não é CLP nem SCADA, e não participa de proteção ou controle crítico.
- O modelo de entidades (locatário, cliente, dispositivo, ativo, perfil) precisa de padrão de nomenclatura e hierarquia definidos antes do cadastro.
- **Telemetria** é série temporal; **atributo** é estado. Escopos: servidor, compartilhado (desce para o dispositivo) e cliente.
- A API MQTT usa `v1/devices/me/...` (com formas curtas `v2/...`); o *gateway* usa o prefixo `v1/gateway/.`
- Sempre envie **timestamp do dispositivo** quando houver fila ou enlace instável.
- A **rule chain** processa cada mensagem; alarme é uma entidade com severidade, estado, instantes e detalhes, criada por Create Alarm e encerrada por Clear Alarm.
- **RPC** é canal de comando: exige autorização, idempotência e auditoria.
- A integração típica é CLP → Node-RED → MQTT → ThingsBoard, com **agregação na borda** e limites ajustáveis por atributo compartilhado.
- A plataforma automatiza o ciclo operacional do alarme, mas não substitui a disciplina de gerenciamento de alarmes.

12.11 Questões de Revisão

1. Em que camada da pirâmide de automação o ThingsBoard atua e por que ele não substitui o SCADA?
2. Explique a diferença entre *device* e *asset* e dê um exemplo de hierarquia para uma estação elevatória.
3. Qual a diferença entre telemetria e atributo? Por que o histórico completo só existe para telemetria?
4. Descreva os três escopos de atributo e dê um caso de uso para cada um.
5. Um dispositivo publica sem `ts` e o enlace de rádio fica 30 minutos fora. O que aparece no gráfico? Como corrigir?

6. Escreva o tópico MQTT e um *payload* válido para publicar temperatura e umidade com carimbo de tempo.
7. Para que serve o prefixo `v1/gateway/` e qual problema ele resolve?
8. Descreva o papel dos nós Create Alarm e Clear Alarm e o que significa o estado `ACTIVE_UNACK`.
9. Qual a diferença entre RPC do lado do servidor e do lado do dispositivo? Cite um risco de segurança de cada um.
10. Um projeto pretende usar ThingsBoard em nuvem para fechar uma válvula automaticamente quando a pressão exceder 12 bar. Avalie tecnicamente a proposta e apresente uma alternativa.

12.12 Referências

- THINGSBOARD. **Documentação oficial** — conceitos de entidades, API MQTT de telemetria e atributos, API de *gateway*, motor de regras (nós Create Alarm e Clear Alarm, exemplo de *rule chain*), RPC de dispositivo e API REST de atributos. Disponível em: `thingsboard.io/docs` (versões CE, PE e PaaS). Consulta: 2026.
- MATERIAL DE ORIGEM: `IIoT e suas Tecnologias Aderentes.pdf`; `IoT.pdf`; `Node-Red_InterfaceDeSupervisão.pdf`; *Livro SCADA — versão para análise* (arquitetura de integração).
- OASIS. **MQTT Version 3.1.1 e 5.0** — especificação do protocolo de mensagens usado na integração.
- ECLIPSE FOUNDATION. **Sparkplug 3.0** — especificação de tópicos e estado para integração MQTT industrial.

Capítulo 13

OPC UA, MQTT e Sparkplug B

Objetivos de Aprendizagem

Ao final deste capítulo, o leitor será capaz de:

- Explicar por que a integração de sistemas é um problema de **semântica**, não apenas de protocolo.
- Descrever o modelo de informação do OPC UA e a função do espaço de endereços.
- Distinguir os modos de segurança do OPC UA e escolher uma política adequada.
- Diferenciar o OPC UA cliente-servidor do OPC UA PubSub e dizer quando cada um se aplica.
- Explicar o modelo publish-subscribe do MQTT, seus níveis de QoS, *retained messages* e *last will*.
- Descrever o que o Sparkplug B acrescenta ao MQTT e por que isso importa em IIoT.
- Projetar uma integração CLP → borda → SCADA → plataforma combinando os três.
- Reconhecer os erros de projeto mais comuns nessas integrações.

13.1 O problema: conectar é fácil, entender é difícil

Um CLP que expõe 4.000 registradores Modbus está "conectado". Mas o que é o registrador 40012? É a pressão de descarga da bomba 2? Em que unidade? Está escalado? Qual o valor que significa "sem medição"?

Essa é a diferença entre **transporte** e **semântica**:

- **Transporte** leva bytes de um ponto a outro: Modbus, TCP, MQTT, HTTP.
- **Semântica** diz o que os bytes significam: identificação do ativo, unidade, escala, tipo de dado, qualidade, relação entre variáveis.

Modbus e outros protocolos clássicos resolvem o transporte e deixam a semântica para a documentação — que envelhece, se perde e diverge entre projetos. OPC UA e Sparkplug B são respostas diferentes ao mesmo problema: **levar significado junto com o dado**.

Tabela 13.1 – Onde cada solução atua

Camada	O que define	Quem resolve bem
Comunicação	Caminho e serviços de troca de dados	TCP/IP (transporte); Modbus, MQTT e OPC UA (protocolos de aplicação)
Identificação do dado	Qual variável, de qual ativo	OPC UA (espaço de endereços); Sparkplug B (tópico e métricas, conforme a convenção de nomes adotada)
Significado do dado	Tipo, unidade, escala, faixa	OPC UA (modelo de informação); Sparkplug B (métricas tipadas; convenções de unidade e escala precisam ser acordadas)
Qualidade e tempo	Valor válido? De quando?	OPC UA (*status code*, *source timestamp*); Sparkplug B (*timestamp* por métrica)
Comportamento	Como ler, escrever, assinar, descobrir	OPC UA (serviços); Sparkplug B (estado, nascimento/morte)

Dica

Dica: antes de escolher o protocolo, escreva a **lista de variáveis com unidade, faixa e regra de qualidade**. Metade dos projetos de integração falha porque essa lista não existe e a outra metade porque ela existe apenas na cabeça de quem escreveu o CLP.

13.2 OPC UA

A **OPC UA** (*OPC Unified Architecture*) foi lançada pela OPC Foundation em **2008** e é normalizada como a série **IEC 62541** (verifique a edição vigente antes de citá-la em projeto). Ela sucedeu o OPC clássico (DCOM) resolvendo três limitações que inviabilizavam seu uso em redes industriais modernas: dependência de Windows/DCOM, dificuldade de atravessar *firewalls* e ausência de um modelo de informação padronizado.

Nota

Referência histórica: o **OPC clássico** (OPC DA, 1996) organizava o acesso a dados em três níveis — **servidor**, **grupo** e **item**. O grupo existia por uma razão prática: era a unidade de *scan rate* e de ativação, o que permitia ler mais devagar o que não mudava. Referência atualmente aplicável: **OPC UA (IEC 62541)**. Motivo da atualização: o OPC clássico depende de DCOM/Windows e não atravessa *firewall* corporativo de forma segura.

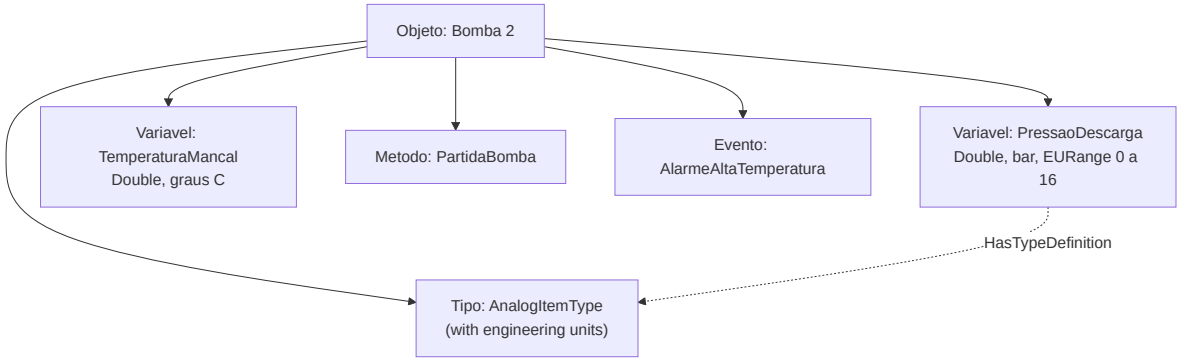


Figura 13.1 – Arquitetura lógica do OPC clássico: servidor, grupos e itens. **Fonte:** *Livro SCADA — versão para análise*, p. 48 (Machado, Pontes e Vianna, reproduzida com crédito).

Vale entender essa hierarquia porque o OPC UA **manteve a ideia** e trocou a implementação: o que no OPC clássico era "grupo com taxa de leitura" virou **sessão com assinatura** (`Subscription` + `MonitoredItem`), e o "item" virou **nó** de um espaço de endereços com tipo, unidade e qualidade. A herança explica por que tantos clientes OPC UA ainda expõem a leitura "por grupo de itens": é a tradução direta do modelo antigo.

13.2.1 Modelo de informação

O OPC UA não expõe "registradores": expõe um **espaço de endereços** (*address space*) organizado em **nós** ligados por **referências** hierárquicas.



Cada nó tem **atributos** (valor, tipo, *status code*, *source timestamp*, descrição) e **referências** para outros nós. A hierarquia é definida pelo servidor, não pelo cliente — o cliente **descobre** a estrutura com `Browse` em vez de receber um mapa de registradores.

Os **tipos** vêm de *ObjectTypes* e *VariableTypes* padronizados, e a OPC Foundation publica **especificações de acompanhamento** (*companion specifications*) por domínio, com numeração própria (séries OPC 3xxxx e 4xxxx). Quando existe uma especificação de acompanhamento para o seu setor, usá-la é o caminho mais curto para interoperar com terceiros.

13.2.2 Serviços

Os serviços são agrupados em conjuntos. Os que aparecem em todo projeto:

Serviço	Para que serve
`Browse`	Descobrir a estrutura e os nós disponíveis
`Read` / `Write`	Ler e escrever valores
`Subscribe` + `MonitoredItem`	Assinar variações: o servidor envia quando muda (menos tráfego que *polling*)
`Call`	Chamar métodos no servidor (comandos)
`HistoryRead`	Ler histórico do próprio servidor
`CreateSession` / `ActivateSession`	Estabelecer e autenticar a sessão

Dois detalhes de engenharia que mudam o resultado do projeto:

- **Assinatura com *sampling interval* e *deadband*.** Assinar sem *deadband* inunda o cliente com variações irrelevantes da última casa decimal. O *deadband* (absoluto ou percentual) resolve o mesmo problema que a faixa morta de um alarme.
- `StatusCode` **obrigatório.** Todo valor lido carrega um código de status. Um valor "0,0 bar" pode ser leitura válida de pressão zero **ou** indicar que a variável está com qualidade ruim. Tratar os dois casos como iguais é a origem de diagnóstico errado (e de alarme espúrio, como discutido no Capítulo 10).

13.2.3 Segurança

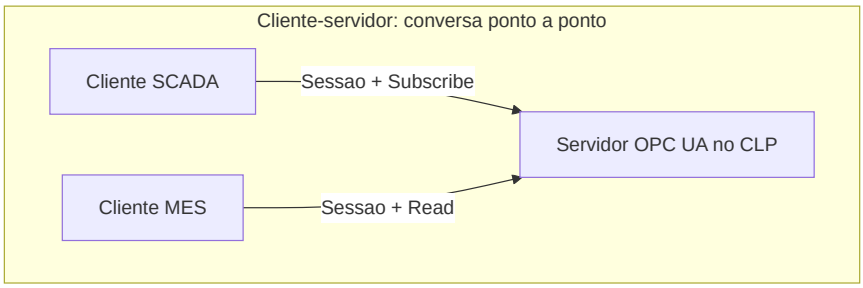
O OPC UA traz segurança no próprio protocolo, o que nenhum protocolo clássico da camada de campo oferece:

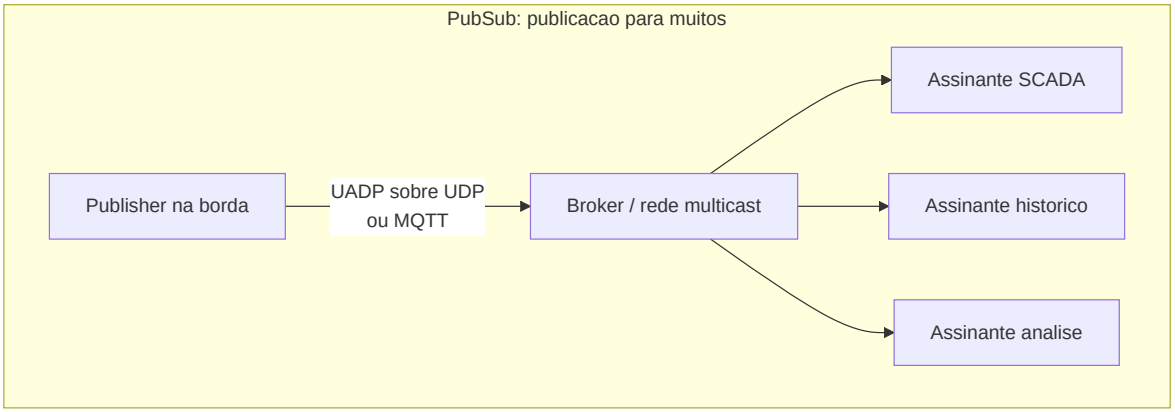
- **Endpoints e políticas de segurança.** Um servidor anuncia vários *endpoints* com políticas diferentes — de `None` (nenhuma) até `Basic256Sha256` e variantes com AES/SHA-256. A política define assinatura e cifragem das mensagens.
- **Modos de mensagem**: `None`, `Sign` (integridade) e `SignAndEncrypt` (integridade e confidencialidade).
- **Certificados X.509** para aplicação e para usuário, com troca e validação de confiança (*trust lists*).
- **Autenticação de usuário** por senha, certificado ou *token* de identidade.

Atenção

Atenção: aceitar `SecurityPolicy: None` "porque é mais fácil" em rede compartilhada significa expor leitura e escrita de qualquer variável. Em rede de laboratório, documente a exceção; em rede de campo, não faça. E lembre: certificado expirado derruba a comunicação **exatamente como uma falha de rede** — coloque a renovação no calendário de manutenção.

13.2.4 Cliente-servidor e PubSub





Critério	Cliente-servidor	PubSub
Modelo	Sessão entre cliente e servidor, ponto a ponto	Publicação em rede, muitos assinantes
Transporte	`opc.tcp` (porta 4840), HTTPS, WebSocket	UADP sobre UDP, MQTT, AMQP
Escala	Boa para supervisão e engenharia	Melhor para muitos consumidores do mesmo dado
Latência	Baixa e determinística	Depende do transporte (MQTT acrescenta broker)
Uso típico	Diagnóstico, comando, configuração, engenharia	Telemetria em massa, IIoT, integração com nuvem

A regra prática: **cliente-servidor para quem conversa com o ativo; PubSub para quem consome o dado em escala.**

Figura

[Figura 13.2 – Espaço de endereços de um servidor OPC UA] *Captura de um cliente OPC UA genérico mostrando a árvore de nós: Objects → Servidor → Bomba 1 → variáveis com tipo, unidade e valor; ao lado, o painel de atributos do nó selecionado, com StatusCode e SourceTimestamp visíveis. Destacar que a estrutura é descoberta pelo cliente, não digitada em planilha.*

13.3 MQTT

O **MQTT** (*Message Queuing Telemetry Transport*) é um protocolo publish-subscribe padronizado pela OASIS — versão **3.1.1** e versão **5.0** —, pensado para enlaces instáveis e dispositivos com poucos recursos.

O desenho é simples: dispositivos **publicam** em um **tópico**; o **broker** entrega a quem **assinou** aquele tópico (ou um padrão de tópicos). Publisher e subscriber não se conhecem.

13.3.1 Tópicos e curingas

Os tópicos são hierárquicos, separados por `/`:

```
planta/estacao-1/bomba-2/pressao
planta/estacao-1/bomba-2/temperatura
```

Curinga	Alcance	Exemplo
`+`	Um nível	`planta/estacao-1/+/pressao`
`#`	Todos os níveis abaixo (só no fim)	`planta/estacao-1/#`

O tópico é **contrato de interface**: renomear um nível quebra todos os assinantes silenciosamente. Trate a estrutura de tópicos como se fosse o mapa de registradores do seu Modbus — documentada e versionada.

13.3.2 Recursos que definem o comportamento

Recurso	O que faz	Cuidado prático
QoS 0	Entrega “no máximo uma vez” (pode perder)	Aceitável para telemetria de tendência, não para evento
QoS 1	Entrega “ao menos uma vez” (pode duplicar)	O assinante precisa ser idempotente
QoS 2	Entrega de acordo com o fluxo de protocolo de maior garantia do MQTT	Não garante, por si só, execução exatamente uma vez do efeito da aplicação
Retained message	O broker guarda a última mensagem do tópico e a entrega a novos assinantes	Excelente para “último valor”; perigoso para dado temporal
Last will and testament (LWT)	Mensagem que o broker publica se detectar desconexão inesperada do cliente	Ajuda a sinalizar uma desconexão; não substitui monitoramento de disponibilidade ponta a ponta

Atenção

Atenção: `SecurityPolicy: None` não protege a mensagem com assinatura ou criptografia; o controle de acesso a nós depende também da autenticação e das permissões configuradas no servidor. Em rede compartilhada, evite esse modo e valide a política e a lista de confiança. Certificados expirados ou não confiáveis podem impedir a conexão, portanto inclua sua gestão na manutenção. | Sessão persistente | Assinaturas e fila sobrevivem à desconexão | No 5.0 a expiração de sessão é explícita |

A versão 5.0 acrescentou o que faltava para uso industrial: códigos de motivo (*reason codes*), propriedades de usuário, *shared subscriptions* (vários consumidores dividindo a carga de um tópico), apelidos de tópico e expiração de sessão.

Tabela 13.2 – MQTT 3.1.1 × 5.0 (o que muda em projeto industrial)

Aspecto	3.1.1	5.0
Motivo de falha	Limitado	Códigos de motivo e propriedades
Sessão	Binária (limpa ou persistente)	Expiração configurável
Escala de consumo	Um assinante por mensagem	*Shared subscriptions*
Diagnóstico	Difícil	Propriedades de usuário e fluxo de motivo

Atenção

Atenção: MQTT puro não diz **o que** está no *payload* nem **de quando** ele é. Um JSON `{"t": 23.4}` não informa unidade, ativo nem instante. Sem convenção, cada integração inventa a sua — e o resultado é o que se vê em muitas plantas: dado na nuvem que ninguém consegue interpretar sem perguntar a quem escreveu o *gateway*.

13.4 Sparkplug B: MQTT com regras

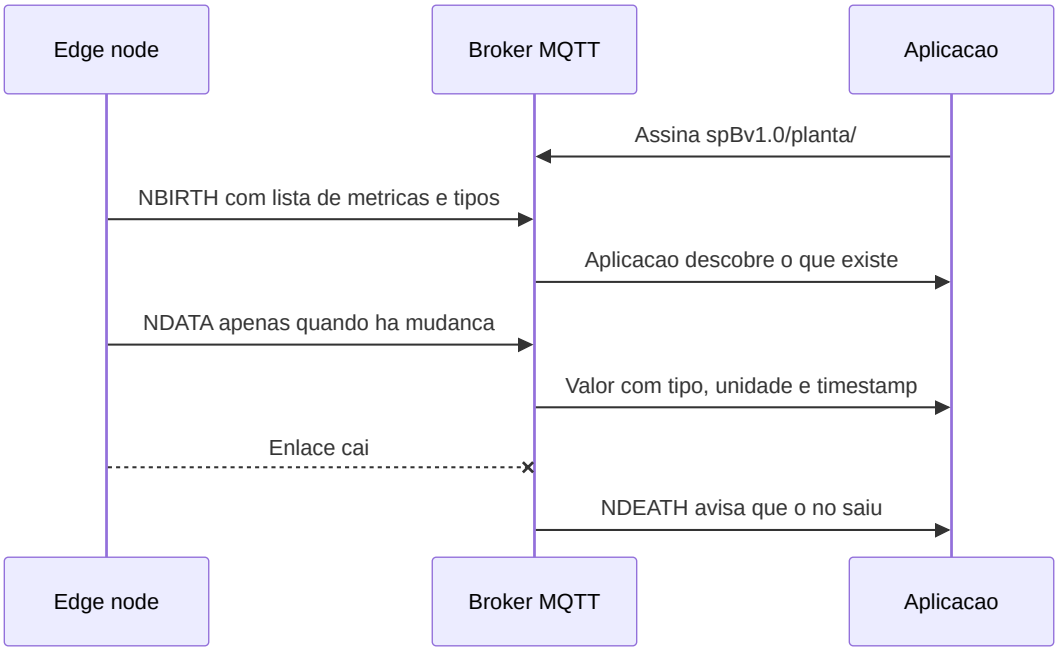
O **Sparkplug B** é uma especificação mantida pela **Eclipse Foundation** (versão **3.0**) que **define a convenção** que faltava no MQTT industrial: estrutura de tópicos, formato de *payload* e uma máquina de estados para os dispositivos.

13.4.1 Estrutura de tópicos

```
spBv1.0/<grupo>/<tipo de mensagem>/<edge node>/[device]
```

Tipo de mensagem	Significado
`NBIRTH` / `NDEATH`	Nascimento e morte do *edge node*: o que ele publica e quando ele deixa de existir

Tipo de mensagem	Significado
`DBIRTH` / `DDEATH`	Nascimento e morte de um dispositivo subordinado
`NDATA` / `DDATA`	Dados do nó e do dispositivo
`NCMD` / `DCMD`	Comandos para o nó e para o dispositivo
`STATE`	Estado do próprio sistema hospedeiro (online/offline)



O ganho é direto:

- `NBIRTH` **entrega o dicionário de dados.** O consumidor descobre nomes, tipos e unidades das métricas sem planilha externa.
- `NDEATH`/`DDEATH` resolvem a detecção de dispositivo offline de forma explícita (é o LWT organizado).
- **Publicação por exceção** (*report by exception*): publica quando muda, reduzindo tráfego de enlace caro.
- **Payload com tipo e *timestamp* por métrica** — resolve o problema do "número sem contexto".
- **Número de sequência** permite detectar perda de mensagens.

13.4.2 O que o Sparkplug não resolve

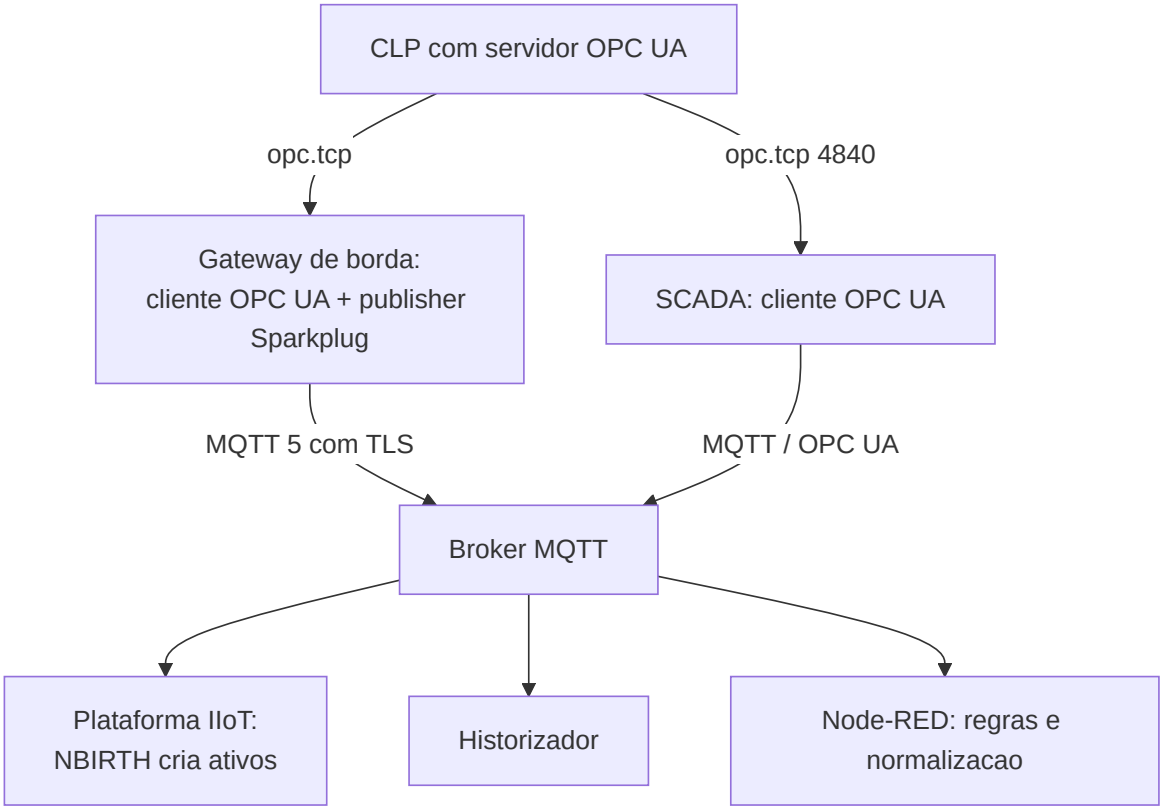
- Não é controle: não é protocolo de malha fechada nem de segurança funcional.
- Não substitui o OPC UA para descoberta rica de estrutura, métodos e histórico em equipamento que já fala OPC UA.
- Não dispensa a disciplina de modelagem: um sistema Sparkplug mal nomeado continua sendo um sistema difícil de integrar.

13.5 Combinando as três peças

Na prática, os protocolos se sobrepõem em papéis diferentes:

Tabela 13.3 – Comparação para decisão de projeto

Critério	OPC UA cliente-servidor	MQTT 5.0	Sparkplug B sobre MQTT
Descoberta de estrutura	Completa (espaço de endereços)	Nenhuma	Dicionário no NBIRTH
Semântica do dado	Tipos, unidades, *status code*	Definida pelo usuário	Métrica tipada no *payload*
Segurança	Nativa (X.509, políticas)	TLS no transporte + credenciais	Herdada do MQTT/TLS
Comando	Serviços e métodos	Tópico de comando (livre)	NCMD/DCMD padronizados
Escala de consumidores	Sessão por cliente	Alta, com *shared subscriptions*	Alta
Melhor uso	Equipamento, engenharia, comando	Telemetria, integração, nuvem	IIoT padronizado, frota de *gateways*
Limitação típica	Mais pesado na borda	Sem semântica	Menos flexível que OPC UA



Nota

Atualização tecnológica: este capítulo não existia no material original, que tratava OPC UA e MQTT apenas como "protocolos modernos" em um subtópico. A separação entre transporte, semântica e comportamento — e o papel do Sparkplug B como convenção — refletem a prática atual de projetos de integração industrial. As fontes estão citadas ao final.

Figura

[**Figura 13.3 – Arquitetura de integração com OPC UA e Sparkplug B**] *Diagrama em camadas: à esquerda o CLP com servidor OPC UA (endpoint e política de segurança anotados); no centro o gateway de borda com as duas funções destacadas (cliente OPC UA e publisher Sparkplug); à direita o broker e os consumidores (SCADA, historizador, plataforma, Node-RED). Anotar o sentido das setas e marcar com cadeado o trecho cifrado.*

13.6 Estudo de caso — integração de uma frota de gateways

Figura

[**Figura 13.4 – Tela de configuração do publisher Sparkplug no gateway de borda**] *Captura de uma interface de configuração de gateway (genérica, sem marca) com: endereço do broker e porta, TLS habilitado e certificado carregado,* `group id` *e* `edge node id`*, intervalo de publicação e* deadband* por métrica, e a lista de métricas mapeadas com nome, tipo e unidade. Destacar a área onde aparece o `NBIRTH` publicado e o estado da conexão.*

Cenário. Cinco subestações de distribuição de água, cada uma com um CLP e um *gateway* de borda. O SCADA central precisa de supervisão em tempo real; a engenharia precisa de histórico; a gestão precisa de indicadores; e a manutenção precisa saber quando um *gateway* cai. O enlace é 4G, com franquia de dados.

Decisão de arquitetura.

Necessidade	Solução	Por quê
Supervisão do CLP em tempo real	OPC UA cliente-servidor, assinatura com *deadband*	Descoberta de estrutura, *status code* e controle de tráfego
Telemetria agregada para a nuvem	Sparkplug B sobre MQTT 5 com TLS	Dicionário no `NBIRTH`, publicação por exceção, detecção de queda
Detecção de *gateway* offline	`NDEATH` + monitoramento do broker	Não depende de alguém perceber o dado parado
Comando de *setpoint*	OPC UA `Write` autenticado, com registro de auditoria	Comando exige controle de acesso e trilha

Necessidade	Solução	Por quê
Histórico	Historizador assinando OPC UA e consumindo MQTT	Duas fontes, mesma modelagem de nomes

Regras adotadas.

1. **Um único dicionário de nomes.** O mesmo identificador de ativo e de variável vale no espaço de endereços do OPC UA, no tópico Sparkplug e no historizador. Sem isso, o relatório da gestão e o sinótico do operador falam línguas diferentes.
2. **Política de segurança obrigatória** no OPC UA e **TLS** no MQTT, com certificados renovados no calendário de manutenção.
3. **Agregação na borda:** amostragem de 1 s no CLP, publicação por exceção com *deadband* — consumo de dados reduzido a menos de um décimo do que seria com publicação periódica de tudo.
4. `StatusCode` **propagado até a tela.** Variável com qualidade ruim aparece como tal, e não como valor zero.
5. **Sem comando pela nuvem.** O comando de *setpoint* sai do SCADA por OPC UA; a nuvem é somente leitura. É a aplicação direta da regra do Capítulo 12.

Resultado. A manutenção passou a ver a queda de um *gateway* em segundos (pelo `NDEATH`), em vez de descobrir a falha no dia seguinte pela ausência de dados. O diagnóstico de sensor com defeito deixou de ser confundido com "pressão zero". E o consumo de dados caiu o suficiente para manter o enlace 4G dentro da franquia.

13.7 Atividade prática

1. Suba um servidor OPC UA de teste (por exemplo, um simulador público de servidor OPC UA) e navegue pelo espaço de endereços com um cliente genérico.
2. Escolha três variáveis e monte uma assinatura com *sampling interval* de 1 s e *deadband* de 0,5%. Compare o número de notificações com e sem *deadband*.
3. Verifique quais políticas de segurança o servidor anuncia e tente conectar com `SecurityPolicy: None`. Registre o que aconteceu e o risco correspondente.
4. Publique a mesma telemetria em MQTT com QoS 0 e QoS 1; simule queda de enlace e compare o que o assinante recebe.
5. Estruture os tópicos conforme Sparkplug B para um *edge node* com dois dispositivos e simule `NBIRTH`, `NDATA` e `NDEATH`.
6. Escreva, em uma página, o dicionário de dados (ativo, variável, tipo, unidade, faixa, regra de qualidade) usado nos itens anteriores. Esse documento é o produto mais importante da atividade.

13.8 Resumo

- Conectar é transporte; **integrar é semântica**. A maior parte das falhas de integração é falta de modelagem, não de protocolo.

- **OPC UA** (OPC Foundation, 2008; série IEC 62541) oferece espaço de endereços, tipos, *status code*, assinatura com *deadband*, métodos e segurança nativa com X.509.
- Modos de segurança `None`, `Sign` e `SignAndEncrypt`: escolher `None` em rede compartilhada é expor leitura e escrita.
- **Cliente-servidor** atende quem conversa com o ativo; **PubSub** atende muitos consumidores do mesmo dado.
- **MQTT** (OASIS 3.1.1 e 5.0) é publish-subscribe com QoS 0/1/2, *retained*, *last will* e, na versão 5.0, códigos de motivo e *shared subscriptions* — mas **não define o conteúdo**.
- **Sparkplug B 3.0** (Eclipse Foundation) define tópicos, *payload* tipado, `NBIRTH`/`NDEATH` e publicação por exceção: é o MQTT com regras de interoperabilidade.
- A combinação típica é OPC UA para equipamento e comando, MQTT/Sparkplug para telemetria em escala, e um **dicionário de nomes único** atravessando SCADA, historizador e plataforma.
- Nenhuma dessas tecnologias substitui CLP e SCADA em controle, proteção e segurança funcional.

13.9 Questões de Revisão

1. Explique a diferença entre transporte e semântica, com um exemplo de falha causada pela ausência de semântica.
2. O que é o espaço de endereços do OPC UA e por que o cliente descobre a estrutura em vez de receber um mapa?
3. Qual a função do `StatusCode` e o que acontece se ele for ignorado na tela do operador?
4. Compare `SecurityPolicy: None`, `Sign` e `SignAndEncrypt` quanto à garantia oferecida.
5. Em que situação o OPC UA PubSub é preferível ao cliente-servidor? Justifique com um exemplo de planta.
6. Descreva o papel do broker MQTT e por que publisher e subscriber não precisam se conhecer.
7. Explique QoS 0, 1 e 2 com um caso de uso industrial para cada, indicando o risco de escolher mal.
8. O que um *retained message* resolve e qual é o seu risco quando o dado é temporal?
9. O que o Sparkplug B acrescenta ao MQTT? Cite três elementos concretos da especificação.
10. Uma planta quer publicar 500 variáveis a cada 100 ms na nuvem, por enlace 4G com franquia. Critique a proposta e proponha uma arquitetura alternativa com OPC UA e Sparkplug.

13.10 Referências

- OPC FOUNDATION. **OPC Unified Architecture** — conceitos, modelo de informação, serviços, segurança e especificação PubSub. Disponível em: opcfoundation.org. Consulta: 2026.
- INTERNATIONAL ELECTROTECHNICAL COMMISSION. **IEC 62541 — OPC Unified Architecture** (série; verificar edição vigente).
- OASIS. **MQTT Version 3.1.1** (2014) e **MQTT Version 5.0** (2019) — especificações do protocolo. Disponível em: docs.oasis-open.org.

- ECLIPSE FOUNDATION. **Sparkplug Specification 3.0** — estrutura de tópicos, *payload* e máquina de estados. Disponível em: `eclipse.org/tahu`.
- MATERIAL DE ORIGEM: *Livro SCADA — versão para análise* (integração de sistemas e protocolos; figura da arquitetura lógica OPC dos autores, reproduzida com crédito); `Node-Red_InterfaceDeSupervisão.pdf` (cliente OPC UA e MQTT no Node-RED); `IoT.pdf` e `IIoT e suas Tecnologias Aderentes.pdf` (protocolos de IIoT).

Capítulo 14

Tags, Variáveis, Estados e Eventos

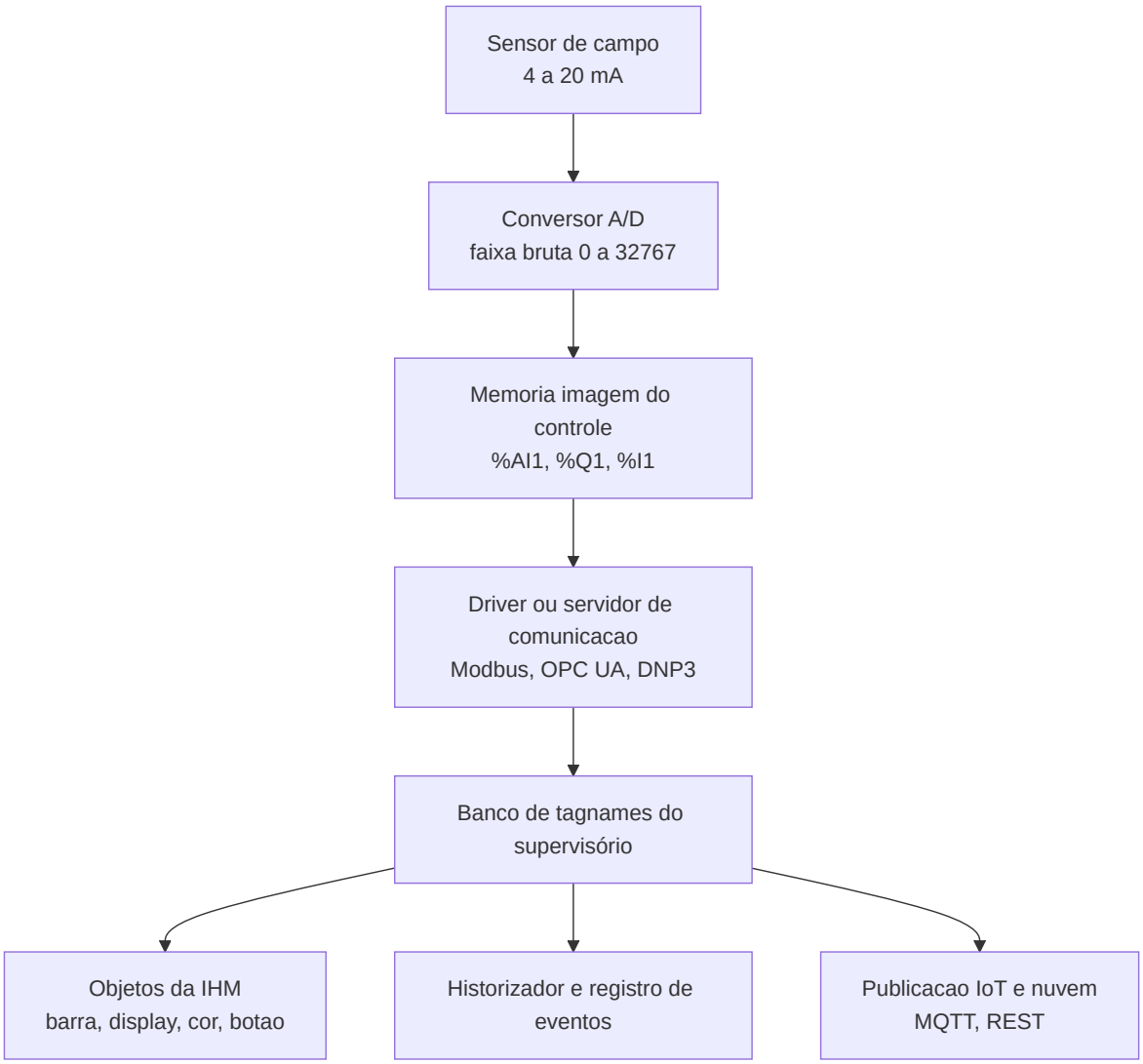
Objetivos de Aprendizagem

Ao final deste capítulo, o leitor será capaz de:

- Definir *tag* (*tagname*) e distinguir tag interno de tag de I/O.
- Descrever os campos de um cadastro de tag e montar um mapa de memória entre o CLP e o supervisório.
- Escalonar um valor bruto para unidade de engenharia e justificar a taxa de amostragem escolhida.
- Diferenciar **valor**, **estado**, **evento** e **alarme** — e explicar por que confundir os quatro degrada a operação.
- Modelar o estado de um equipamento e usá-lo como fonte de evento na IHM, no historizador e na nuvem.

14.1 O tag é o contrato entre o campo e a tela

Um sistema supervisório não "lê instrumentos": ele lê **tags**. Entre o transmissor de nível instalado no tanque e o número que aparece na tela do operador existem vários elos, e cada um deles guarda uma representação diferente do mesmo dado.



Tag (ou *tagname*) é a variável criada no software de supervisão: um espaço de memória destinado a guardar um valor de um tipo determinado. Ele é o **contrato** do sistema — o nome pelo qual engenharia, operação, manutenção e sistemas vizinhos passam a se referir ao mesmo dado. Quando o contrato é cumprido, o sinótico, o histórico, o relatório de produção e o painel na nuvem mostram o mesmo número; quando não é, cada área passa a calcular o seu próprio indicador.

Quanto à **origem da atualização**, o tag se classifica em duas classes:

Tabela 14.1 – Tag interno e tag de I/O

Classe	Onde o valor é atualizado	Comportamento	Uso típico
Interno (memória ou RAM)	No próprio software de supervisão	Sem vínculo de atualização com outro software	Cálculos, contadores, estado da aplicação, textos de receita

Classe	Onde o valor é atualizado	Comportamento	Uso típico
De I/O (comunicação)	No servidor de comunicação (*driver*), que lê e escreve a memória do hardware de controle	Vinculado a uma variável do servidor	Medições, posições de válvula, comandos, *setpoints*

O conjunto dos cadastros forma o **banco de dados de tagnames**, que é a estrutura que o supervisório consulta para receber, manipular e atualizar valores, e de onde os objetos das telas tiram o que exibir.

Nota

Nota: um mesmo software pode ser cliente e servidor ao mesmo tempo. Nesse caso, um tag interno também é legível por outro cliente que se conecte — o que é útil para compartilhar cálculos entre estações, mas exige disciplina de nomenclatura, porque o cálculo passa a ser um dado público do sistema.

14.2 Anatomia de um cadastro de tag

O cadastro é curto, mas cada campo tem consequência prática na operação. Os campos essenciais são:

Tabela 14.2 – Campos de um cadastro de tag

Campo	O que define	Exemplo
Nome	Identificador único; de preferência derivado do TAG do instrumento de campo	`LT100`
Tipo	Natureza do valor armazenado	Discreto (*bit*), numérico (inteiro ou real), texto
Contexto	De onde o valor vem	Interno (memória) ou I/O (comunicação com o CLP)
Endereçamento	Só para tag de I/O: servidor de comunicação, endereço e grupo	`ServCom` — `%AI1`
Faixa	Para analógicos: unidade de engenharia, mínimo, máximo e os limites correspondentes na memória do controle	0 a 100 % / 4 a 20 mA

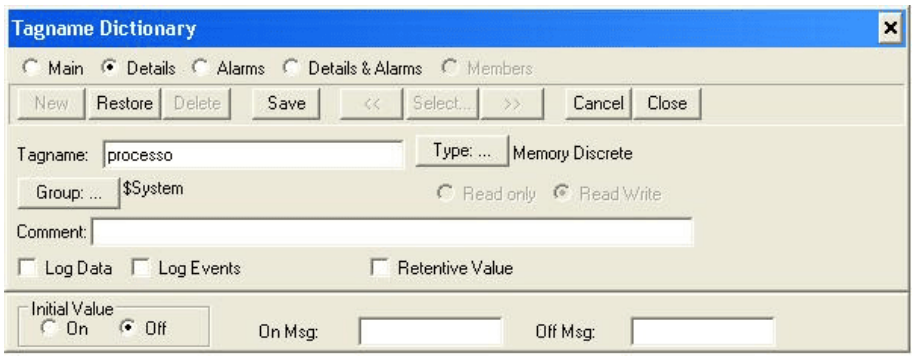


Figura 14.1 – Dicionário de tagnames: cadastro com tipo, contexto, modo de leitura e ponto inicial. **Fonte:** *Livro SCADA — versão para análise*, p. 99 (Machado, Pontes e Vianna, reproduzida com crédito).

Na janela acima, o campo `Type` é o que fixa a natureza do dado (`Memory Discrete`), o campo `Group` define o contexto (`$System`, ou seja, interno) e os modificadores `Read only`/`Read Write` estabelecem se o operador pode escrever. Repare nos marcadores `Log Data` e `Log Events`: eles ligam o tag ao historizador e ao registro de eventos — dois destinos diferentes para o mesmo valor, como se verá na seção 14.7.

O cadastro é a metade da história. A outra metade é o **mapa de memória** (ou tabela de alocação): o documento que relaciona o tagname ao endereço real do hardware de aquisição e ao driver de comunicação.

Tabela 14.3 – Exemplo de mapa de memória

TAG	Tagname	Driver	Protocolo	Endereço	Descrição
LT100	LT100	ServCom	proprietário	`%AI1`	Transmissor de nível
LSH100	LSH100	ServCom	proprietário	`%I1`	Chave de nível alto

Nota

Fonte: *Sistemas Supervisórios para Processos Industriais — parte 2*, p. 10 (Vianna), reescrita como tabela.

Atenção: o mapa de memória é o primeiro documento a ser consultado quando um valor "não atualiza na tela". Sem ele, o diagnóstico vira tentativa: não se sabe se o problema está no instrumento, na faixa da entrada analógica, no endereço do driver ou no cadastro do tag. Mantenha o mapa versionado junto do projeto do CLP — quando alguém insere uma entrada analógica no meio do programa e desloca os endereços seguintes, esse documento é a diferença entre uma hora e um dia de parada.

14.3 Tipos, resolução e escalonamento

Os tags primitivos fundamentais são três: **discreto** (*bit*, *bool*, *discrete*), **numérico** (inteiro ou real) e **caracter** (mensagem ou texto). Os compostos são associações de primitivos — uma estrutura de bomba com estado, corrente e horas de operação, por exemplo.

Nos controladores, a família de tipos vem da linguagem de programação definida pela **IEC 61131-3** (4ª edição, maio de 2025), que padroniza `BOOL`, `INT`, `DINT`, `REAL`, `LREAL`, `TIME`, `STRING` e as variantes sem sinal. O supervisório não precisa suportar todos eles: ele precisa **não perder informação** na conversão. Um contador de 32 bits lido como inteiro de 16 bits estoura em 32.768 e volta a zero — e o histórico passa a mostrar um consumo que "reinicia" sozinho.

14.3.1 Da faixa bruta à unidade de engenharia

A entrada analógica de um CLP não entrega graus Celsius nem litros por segundo: entrega um número inteiro proporcional ao sinal elétrico. Com resolução de 15 bits, o conversor A/D divide a faixa de tensão em $2^{15} = 32768$ degraus. Para uma entrada de 0 a 5 VCC, a correspondência é direta: 0 V vira 0 e 5 V viram 32767.

O supervisório faz o resto do caminho, convertendo a faixa bruta na **faixa de engenharia** do instrumento:

```
EU = EU_min + (bruto - bruto_min) * (EU_max - EU_min)
                     / (bruto_max - bruto_min)

Exemplo: LT100, entrada 4 a 20 mA em 12 bits (0 a 4095)
  bruto = 2047   (metade da faixa)
  EU    = 0 + (2047 - 0) * (100 - 0) / (4095 - 0) = 50,0 %
```

Tabela 14.4 – Faixas brutas usuais e a resolução correspondente

Resolução	Faixa bruta	Degraus	Degrau em 4 a 20 mA (faixa de 100 %)
12 bits	0 a 4095	4096	0,024 %
15 bits	0 a 32767	32768	0,003 %
16 bits	0 a 65535	65536	0,0015 %

Dica

Dica: a resolução raramente é o fator limitante — o ruído do sinal é. Aumentar a resolução do cartão de entrada não melhora a leitura de um transmissor cujo sinal chega com 1 % de ruído. Antes de trocar o hardware, verifique aterramento, blindagem e a origem da alimentação.

O complicador aparece quando a faixa configurada no CLP não é a mesma do instrumento. Um transmissor de 0 a 200 mbar cadastrado no supervisório com faixa de 0 a 250 mbar informa valores **plausíveis e errados** — o pior tipo de defeito, porque não gera alarme nem mensagem de erro. A verificação é simples e obrigatória na partida: compare a leitura com um valor conhecido (tanque vazio, tanque cheio, *setpoint* de teste) e confira os dois extremos.

14.4 O tag na tela: representação de discretos e analógicos

A variável discreta corresponde a 1 bit e assume apenas dois estados. Ela é apresentada por três formas básicas:

- **Visibilidade de texto:** exibe o status da variável — LIGADO/DESLIGADO, ABERTO/FECHADO, AUTOMÁTICO/MANUAL, LOCAL/REMOTO.
- **Mudança de cor** (ou de outro atributo) do objeto, conforme o valor da variável.
- **Visibilidade do objeto:** o objeto aparece ou não no sinótico segundo o valor do tag — usado para representar a presença de um equipamento ou de um trecho de processo.

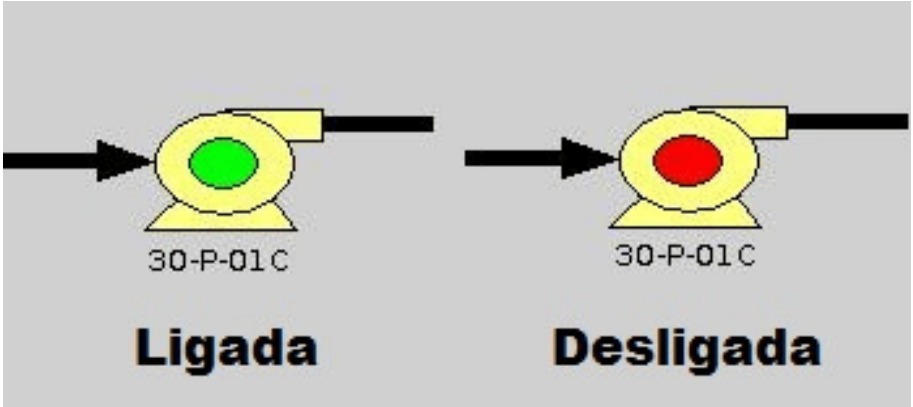


Figura 14.2 – Representação de uma variável discreta: a mesma bomba ligada e desligada. **Fonte:** *Livro SCADA — versão para análise*, p. 55 (Machado, Pontes e Vianna, reproduzida com crédito).

A variável analógica é apresentada por **objetos dinâmicos**, cada um com uma finalidade distinta. A escolha do objeto é uma decisão de ergonomia, não de estética.

Tabela 14.5 – Formas de apresentação instantânea da variável analógica

Objeto dinâmico	O que representa	Quando usar	Cuidado
Display numérico	O valor do tag, com casas decimais	Registro, anotação, conferência	Exige leitura; não comunica proporção
Barra gráfica	Valor em percentual de enchimento	Nível, pressão, carga	Precisa de escala visível
Ponteiro de deslocamento	Translado do objeto proporcional ao valor	Válvulas, comportas, posição	Não serve para valores muito ruidosos
Mostrador circular	Simula *gauges* e dials convencionais	Recuperar a leitura de painéis antigos	Ocupa muito espaço na tela

A mudança de cor do display é o recurso mais usado para codificar os limites da variável: muito baixa (LL), baixa (L), normal, alta (H) e muito alta (HH). Isso tem um custo que a apostila já discutiu no Capítulo 11: a mesma cor que informa também compete com as outras telas do sistema. Uma tela em que quase tudo é colorido não informa nada.

14.5 Amostragem, varredura e qualidade do dado

Todo valor que chega ao supervisório é uma **amostra**, não um contínuo. Quem define a frequência dessa amostragem é a taxa de varredura do protocolo (o *polling*), no caso de leitura periódica, ou o intervalo de amostragem da assinatura, no caso de leitura por exceção.

Na tendência real, cada curva guarda seus valores em um vetor na memória RAM da estação de supervisão. O tamanho do vetor determina o tempo máximo registrado: taxa de amostragem multiplicada pelo número de amostras. Com 100 ms e 1024 amostras, a janela é de 1,7 minuto; com 5 s e a mesma memória, a janela passa a 85 minutos. Não existe almoço grátis — ou se amplia a janela, ou se aumenta a taxa.

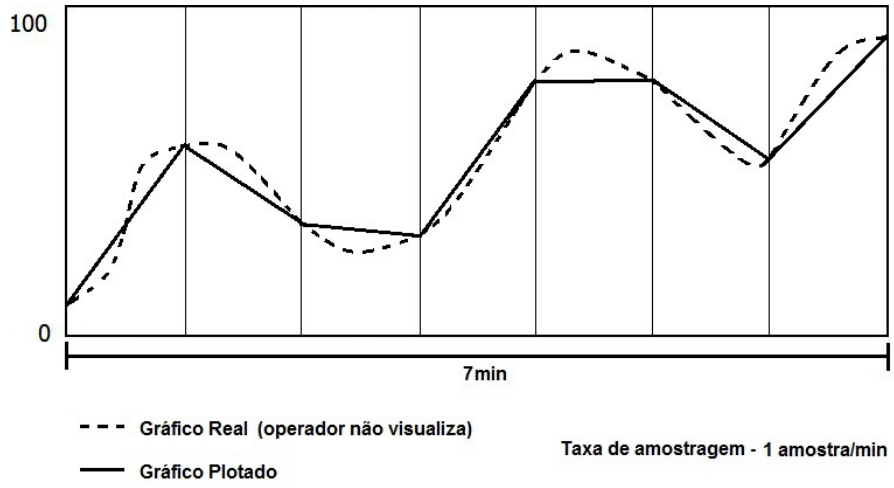


Figura 14.3 – Taxa de amostragem inadequada: a curva plotada difere da variação real. **Fonte:** *Livro SCADA — versão para análise*, p. 68 (Machado, Pontes e Vianna, reproduzida com crédito).

A figura é o argumento mais direto a favor de se escolher a taxa de amostragem pela dinâmica da variável e não pela capacidade do enlace. A curva pontilhada é o comportamento real do processo; a curva cheia é o que o operador vê com uma amostra por minuto. A variável oscilou quatro vezes no intervalo e o operador enxergou uma oscilação lenta — informação suficiente para tomar a decisão errada. Vazão responde mais rápido que temperatura, e a taxa de amostragem deve refletir isso.

14.5.1 Qualidade: o valor sozinho não basta

Todo valor lido carrega, na arquitetura moderna, um **código de qualidade** ao lado do número. No OPC UA (Capítulo 13), esse campo é o `StatusCode`; em plataformas IIoT, é um atributo de qualidade ou um simples indicador de *timestamp* antigo.

Tabela 14.6 – O que a qualidade indica

Situação	O que o número significa	O que a tela deve mostrar
`Good`	Leitura válida do campo	O valor e a unidade
`Uncertain`	Valor calculado, em inicialização ou com precisão degradada	O valor, marcado como incerto

Situação	O que o número significa	O que a tela deve mostrar
`Bad`	Sem comunicação, sem alimentação, fora de faixa ou falha do sensor	Traço, e não zero

Atenção

Atenção: tratar qualidade ruim como valor zero é uma das causas mais comuns de decisão errada na sala de controle. "Pressão 0 bar" e "pressão sem dado" levam a ações opostas: a primeira sugere linha despressurizada, a segunda sugere instrumento fora do ar. O Capítulo 10 já tratou esse ponto do lado dos alarmes; aqui ele aparece do lado do dado, que é a origem do problema.

Outro requisito é o **timestamp de origem**. Se o valor é carimbado na chegada, todo dado atrasado (em rádio, em enlace 4G ou em fila na borda) é gravado como se fosse atual, e a série temporal passa a existir em ordem diferente da que aconteceu no processo.

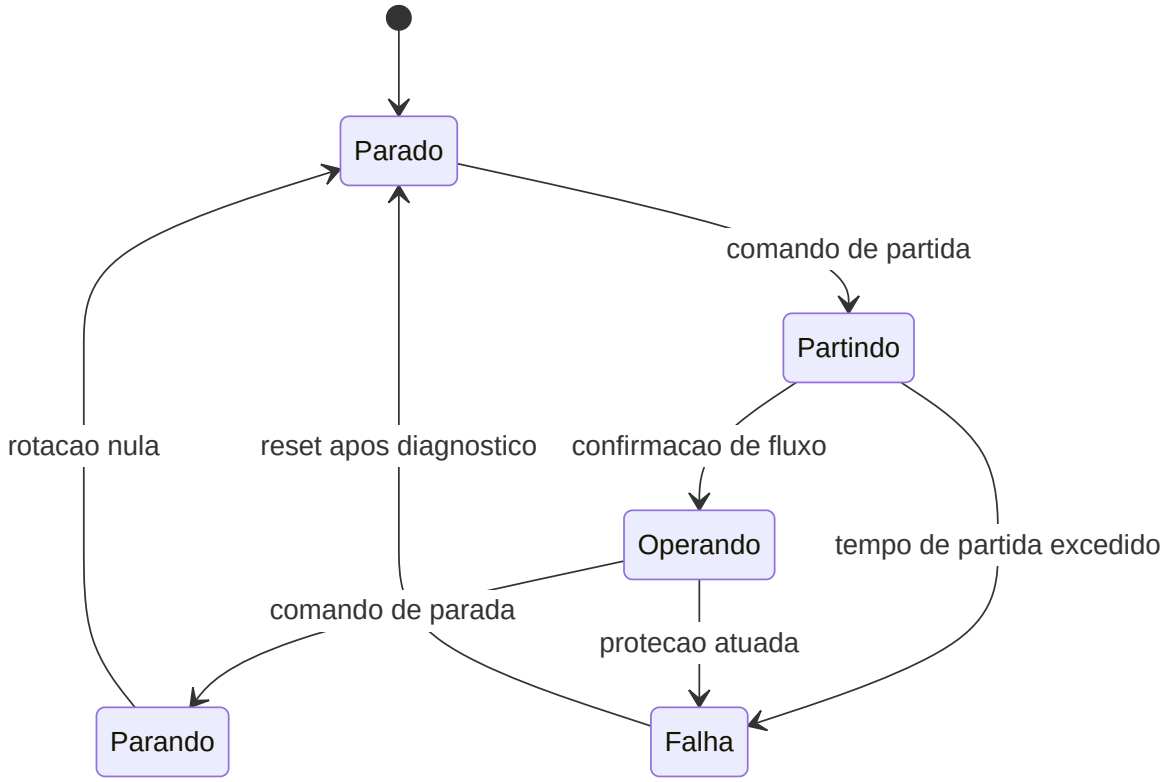
Nota

Atualização tecnológica: no material original, a qualidade do dado era tratada apenas indiretamente, pela indicação de falha de comunicação do supervisório. Com OPC UA, Sparkplug B e gateways de borda, o código de qualidade passou a ser um campo obrigatório que atravessa toda a cadeia, do CLP até o painel na nuvem. A recomendação atual é propagá-lo até a tela, em vez de substituí-lo por um valor padrão.

14.6 Estados: quando o valor não conta a história

Saber que a bomba está ligada não diz se ela está funcionando. Esse é o limite do tag discreto: ele responde "ligado" ou "desligado", mas a operação precisa saber se o equipamento está **partindo**, **operando**, **parando** ou em **falha**. Essas são condições qualitativamente diferentes, com alarmes, intertravamentos e procedimentos diferentes.

A solução é separar **valor** de **estado** e construir a máquina de estados explicitamente — no CLP, no supervisório ou em ambos.



A máquina de estados muda três coisas na prática:

1. **A IHM deixa de deduzir.** Em vez de o desenvolvedor combinar dois ou três tags na tela para adivinhar o estado, o estado é um dado publicado, com nome e valor conhecidos.
2. **O evento ganha significado.** "Partindo → Operando" e "Partindo → Falha" são transições distintas, e é a transição — não o valor — que interessa ao registro de eventos e à estatística de disponibilidade.
3. **O alarme fica mais preciso.** Alarmes condicionados ao estado (avisar somente se a bomba está em operação) eliminam uma classe inteira de alarmes espúrios durante a partida e a parada.

Quando o controle é feito por estado, é útil adotar as convenções já consolidadas em vez de inventar as suas. A identificação de instrumentos e de funções de controle segue a prática de projeto de instrumentação — letras para a variável medida e para a função (LT para transmissor de nível, LSH para chave de nível alto, FT para transmissor de vazão). A definição de estados de equipamento e de procedimento aparece nos modelos de automação de batelada e de automação de procedimento (ISA-88/IEC 61512 e ISA-106, verificar edição vigente antes de citar em projeto). Adotar o vocabulário já conhecido pela equipe custa menos do que defender um vocabulário próprio.

Dica

Dica: escreva a máquina de estados em uma tabela antes de desenhar a tela. Para cada estado, registre o que a operação vê, qual alarme é possível, qual comando é aceito e qual transição é proibida. Essa tabela é o que impede o clássico defeito de dois caminhos que deveriam ser exclusivos acabarem ligados ao mesmo tempo.

14.7 Eventos e alarmes: a diferença que evita a sobrecarga

O supervisório precisa distinguir duas coisas que o senso comum trata como uma só:

- **Evento** é uma ocorrência com importância para o operador, mas que **não exige ação corretiva**. Exemplo: um equipamento entrou em operação, um operador assumiu o turno, uma receita foi carregada.
- **Alarme** é uma situação que **exige intervenção corretiva** do operador. Exemplo: nível acima do limite alto, falha de comunicação com o CLP.

Essa distinção não é preciosismo de nomenclatura. É o que separa um sistema de alarmes utilizável de um sistema em que o operador abandona a tela: se tudo for tratado como alarme, o operador perde a capacidade de identificar o que exige ação.

Os eventos têm três proveniências, e cada uma delas dispara ações diferentes no supervisório:

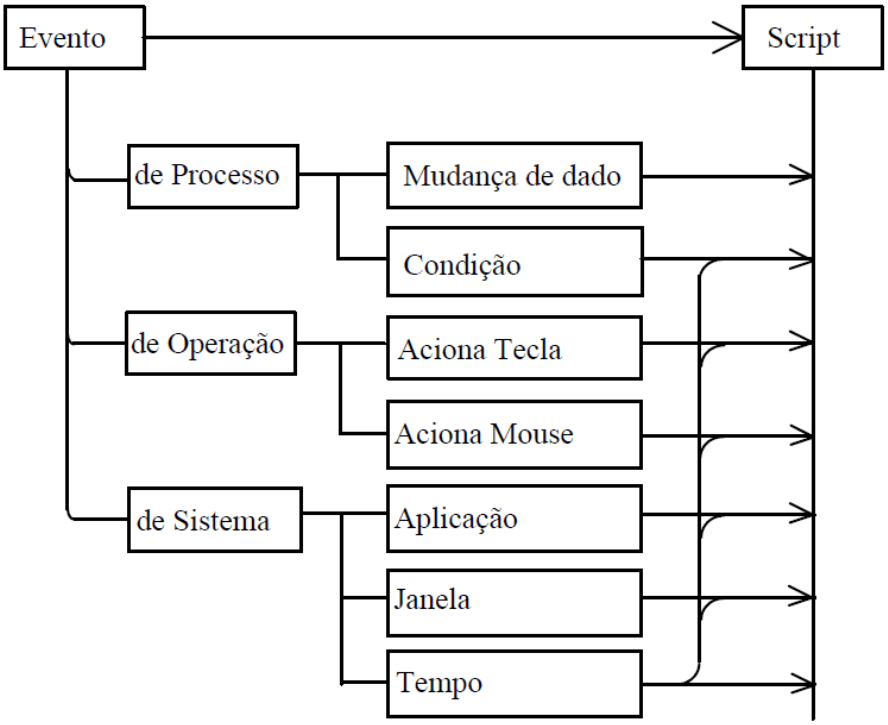


Figura 14.4 – Eventos de processo, de operação e de sistema que disparam scripts. **Fonte:** *Livro SCADA — versão para análise*, p. 72 (figura de Seixas, 2002, reproduzida com crédito na obra de Machado, Pontes e Vianna).

A decisão de classificar uma ocorrência como evento ou como alarme pode ser organizada em uma sequência de perguntas:

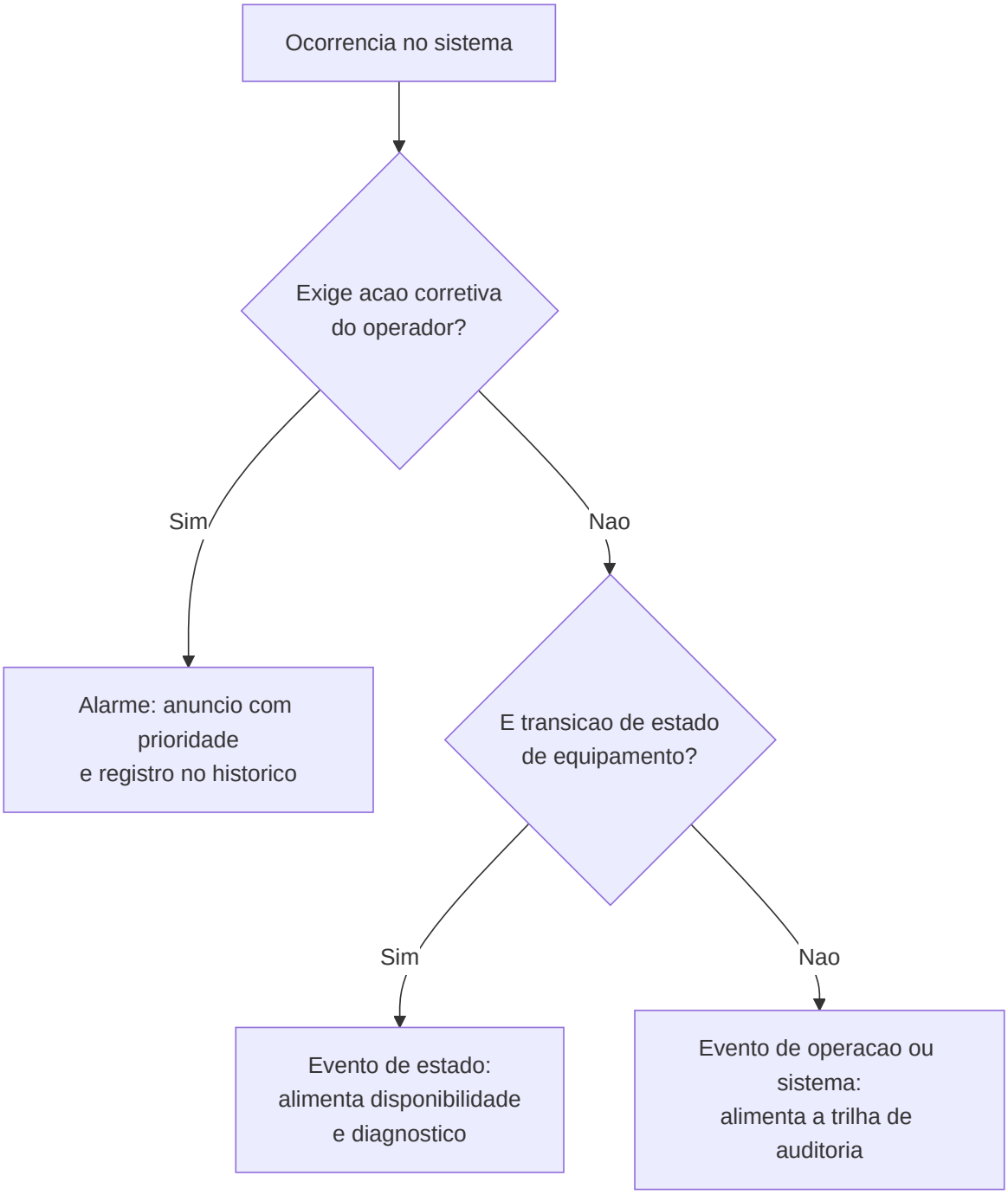


Tabela 14.7 – Tipos de evento e quando a rotina é executada

Proveniência	Evento	Tipo de disparo
De processo	Mudança de dado	`On true`
De processo	Condição verdadeira ou falsa	`On true`, `On false`, `While true`, `While false`
De operação	Acionar teclado (ex.: `Ctrl + H`)	`On key up`, `On key down`, `While down`
De operação	Acionar mouse sobre um objeto ativo	`On up`, `On down`, `Double click`

Proveniência	Evento	Tipo de disparo
De sistema	Aplicação entra ou sai de execução	`On startup`, `On shutdown`, `While run`
De sistema	Janela aberta, fechada ou em exibição	`On show`, `On hide`, `While show`

Nem todo software de supervisão oferece todos esses eventos, e o conjunto de tipos varia entre fabricantes. O que não varia é a consequência do registro: um evento gravado com carimbo de tempo e com o nome de quem o reconheceu é a matéria-prima da análise de causas e da apuração de responsabilidade operacional.

O destino do dado também difere. O **sumário** de alarmes vive na memória primária e mostra somente o que está em andamento agora; o **histórico** é gravado em memória secundária e permite recuperação posterior. Um tag pode estar ligado ao historizador de séries temporais sem estar ligado ao registro de eventos, e vice-versa — foi o que se viu nos marcadores `Log Data` e `Log Events` da Figura 14.1.

Nota

Referência histórica: o material de origem desta apostila citava quatro referências para o gerenciamento de alarmes: EEMUA 191, ISA S18.02, IEC 60839 e NAMUR NA 102. **Referência atualmente aplicável:** para alarmes de processo, **ANSI/ISA-18.2**, **IEC 62682**, **EEMUA 191** e **NAMUR NA 102**. **Motivo da atualização:** a IEC 60839 trata de sistemas de alarme de segurança eletrônica e patrimonial — outro escopo, que não define os requisitos de gerenciamento de alarmes de processo. O detalhamento está no Capítulo 10.

14.8 Comparativo: tag clássico, nó OPC UA e métrica IoT

O conceito de tag sobreviveu a três gerações de tecnologia. O que mudou foi onde a informação adicional — tipo, unidade, faixa, qualidade, carimbo de tempo — é declarada e quem a mantém.

Tabela 14.8 – O mesmo dado em três modelos

Tecnologia	Função	Protocolo	Aplicação	Vantagens	Limitações
Tag de supervisório	Variável do banco de tags	Driver proprietário, Modbus, DNP3	Supervisão e comando local	Simples, consolidado, independe do modelo de informação do equipamento	Estrutura digitada à mão; tipo e unidade vivem em planilha; sem qualidade padronizada

Tecnologia	Função	Protocolo	Aplicação	Vantagens	Limitações
Nó OPC UA	Nó de um espaço de endereços	`opc.tcp` (IEC 62541)	Integração equipamento–supervisão–MES	Tipos, unidade, `StatusCode` e carimbo de tempo no próprio dado; descoberta automática	Exige servidor no equipamento; mais pesado na borda
Métrica Sparkplug B	Métrica tipada em payload	MQTT 5 com TLS	Telemetria em escala e nuvem	Dicionário declarado no `NBIRTH`; detecção de queda pelo `NDEATH`	Semântica dependente da convenção; histórico depende do consumidor

Nota

Nota: a convivência é a regra, não a exceção. Um projeto típico mantém o tag clássico no supervisório, expõe o mesmo dado por OPC UA para integração e publica uma versão agregada por MQTT para a nuvem. O que não pode divergir é o **nome e o significado** do dado nas três pontas — é por isso que o dicionário de dados, e não o protocolo, é o artefato central do projeto.

14.9 Estudo de Caso — Mapa de tags de uma estação de bombeamento

Cenário. Uma estação de bombeamento de água opera com três conjuntos motobomba e um painel de supervisão local. A equipe herdou um projeto em que a tela do supervisório foi montada por tentativa: os nomes dos tags são abreviações próprias (`b1_lig`, `pressao2`, `nivel_novo`), a faixa dos transmissores foi cadastrada em percentual e o mapa de memória existe apenas como anotação à mão em uma folha dentro do painel.

Diagnóstico. Três sintomas apareceram no primeiro mês de operação:

1. O histórico mostra a vazão "reiniciando" a cada poucas horas. O contador é de 32 bits no CLP e foi cadastrado como inteiro de 16 bits no supervisório.
2. O operador relatou "pressão zero" em uma madrugada; a manutenção trocou um transmissor saudável. O que havia era queda de comunicação com o CLP naquela entrada, apresentada como valor zero.
3. O relatório de disponibilidade das bombas não fecha com o horário real de operação. A tela deduz o estado a partir de dois tags combinados, e a combinação `ligado + sem fluxo` tanto pode ser partida quanto falha — a estatística somou as duas coisas.

Decisão de engenharia.

Problema	Ação adotada	Resultado esperado
Estouro de contador	Cadastro corrigido para inteiro de 32 bits; faixa bruta reconferida	Fim do "reinício" no histórico
Valor zero em falha	Qualidade propagada até a tela; valor exibido como traço quando `Bad`	Fim da troca de transmissor saudável
Estado ambíguo	Máquina de estados publicada pela lógica: `Parado`, `Partindo`, `Operando`, `Falha`	Disponibilidade passa a ser contada por estado, com transições registradas
Nomes ilegíveis	Renomeação para o padrão do projeto (`P101_ESTADO`, `PT101_PV`, `FIT101_PV`)	Rastreabilidade entre CLP, supervisório, histórico e nuvem
Mapa inexistente	Mapa de memória em planilha versionada, com TAG, endereço, driver e faixa	Diagnóstico em minutos em vez de horas

Limitações do arranjo. A renomeação de tags exige atualizar o histórico já gravado por meio de um dicionário de tradução, e não por alteração retroativa — sob pena de o histórico virar uma coleção de curvas sem nome. A máquina de estados foi implementada na lógica do CLP, o que limita a mudança de comportamento a uma janela de manutenção. E a estação continua sem redundância de servidor: o ganho foi de diagnóstico e de confiabilidade do dado, não de disponibilidade do sistema.

14.10 Resumo

- **Tag** é a variável do supervisório e funciona como contrato de nomes entre CLP, IHM, histórico, relatórios e nuvem.
- Tag **interno** é atualizado no próprio software; tag de **I/O** é atualizado pelo driver, ligado à memória do hardware de controle.
- Um bom cadastro define **nome, tipo, contexto, endereço e faixa**; o **mapa de memória** liga o tagname ao endereço real do CLP.
- A conversão de faixa bruta para **unidade de engenharia** é aritmética simples, mas a divergência entre a faixa do instrumento e a cadastrada produz valores plausíveis e errados.
- A **taxa de amostragem** deve seguir a dinâmica da variável: amostrar devagar mostra uma curva que não existe.
- **Qualidade** é dado, não enfeite: `Bad` deve aparecer como ausência de dado, nunca como zero.
- **Valor** e **estado** são informações diferentes; a máquina de estados explícita melhora a IHM, os alarmes e o cálculo de disponibilidade.
- **Evento** é ocorrência sem ação corretiva exigida; **alarme** exige intervenção. Misturar os dois destrói a credibilidade do sistema de alarmes.
- O conceito de tag sobreviveu ao OPC UA e ao MQTT: o que mudou foi o lugar onde tipo, unidade e qualidade são declarados.

14.11 Questões de Revisão

1. Explique a diferença entre tag interno e tag de I/O e dê um exemplo de uso legítimo para cada um.
2. Cite os cinco campos essenciais de um cadastro de tag e explique a consequência de preencher mal cada um deles.
3. Um transmissor de 0 a 400 mmHO foi cadastrado no supervisório com faixa de 0 a 250 mmHO. Descreva o que o operador vê e por que esse defeito é mais perigoso que uma falha franca de comunicação.
4. Calcule o valor em unidade de engenharia de um tag de nível com entrada de 15 bits, faixa de 0 a 5000 L e leitura bruta de 19660. Apresente o cálculo.
5. Um gráfico de tendência real é configurado com taxa de amostragem de 10 s e 512 amostras. Qual é a janela de tempo registrada? O que se ganha e o que se perde ao reduzir a taxa para 2 s?
6. Por que a qualidade do dado deve ser exibida na tela em vez de substituída por zero? Relacione a resposta com o diagnóstico de falha de instrumento.
7. Descreva uma máquina de estados para um misturador com motor e válvula de alimentação. Indique ao menos quatro estados, as transições permitidas e um alarme possível em cada estado.
8. Explique a diferença entre evento e alarme com um exemplo de cada em um processo de tratamento de água, e diga o que acontece com a operação quando essa diferença é ignorada.
9. Um projeto publica a mesma variável no supervisório, no servidor OPC UA e no broker MQTT com três nomes diferentes. Descreva dois problemas concretos que essa escolha cria e proponha uma correção.

14.12 Referências

- MACHADO, R.; PONTES, W.; VIANNA, W. **Livro SCADA — versão para análise**. Material didático do curso (capítulos sobre recursos do software SCADA, tagnames, alarmes, eventos e tendências). Figuras reproduzidas com crédito.
- VIANNA, W. S. **Sistemas Supervisórios para Processos Industriais — parte 2**. Material didático do curso (conceitos de tagname, banco de dados de tagnames e mapa de memória).
- INTERNATIONAL ELECTROTECHNICAL COMMISSION. **IEC 61131-3 — Programmable controllers: Programming languages**. 4ª edição, maio de 2025 (a 3ª edição é de 2013; verificar a edição vigente antes de citar em projeto).
- INTERNATIONAL SOCIETY OF AUTOMATION. **ANSI/ISA-18.2 — Management of Alarm Systems for the Process Industries**. Edição vigente de 2016 (1ª edição: 2009).
- INTERNATIONAL ELECTROTECHNICAL COMMISSION. **IEC 62682 — Management of alarm systems for the process industries**. Edição vigente de 2022.
- NAMUR. **NA 102 — Alarm Management**. Leverkusen: NAMUR, 2003 (recomendação).
- SEIXAS, F. **Diagrama de eventos de um sistema SCADA**. 2002 (figura reproduzida no *Livro SCADA — versão para análise*, p. 72).

- ISA-88 / IEC 61512 (modelo de controle de batelada) e ISA-106 (automação de procedimento): citados apenas como vocabulário de estados de equipamento e procedimento — verificar edição e escopo vigentes antes de usar como requisito.
- OPC FOUNDATION. **OPC Unified Architecture** — atributos de nó, `StatusCode` e carimbo de tempo. Disponível em: opcfoundation.org. Consulta: 2026.

Capítulo 15

Aquisição e Tratamento de Dados

Objetivos de Aprendizagem

Ao final deste capítulo, o leitor será capaz de:

- Descrever a cadeia completa de aquisição, do fenômeno físico ao valor gravado no histórico.
- Escolher o tipo de sinal e a forma de conexão adequados a cada medição.
- Explicar resolução, exatidão e ruído e distinguir um do outro na prática da instalação.
- Justificar as taxas de varredura de cada elo e reconhecer quando um evento se perde entre eles.
- Aplicar tratamento de dados: filtração, validação de faixa, faixa morta e publicação por exceção.
- Projetar o registro da primeira falha de uma parada de planta.

15.1 A cadeia de aquisição

Nenhum dado industrial nasce pronto. Entre o fenômeno físico e o número que o historiador grava existe uma cadeia de elos, e cada elo degrada, atrasa ou descarta informação. Conhecer a cadeia é o que permite diagnosticar por que o valor na tela não corresponde ao que aconteceu no processo.

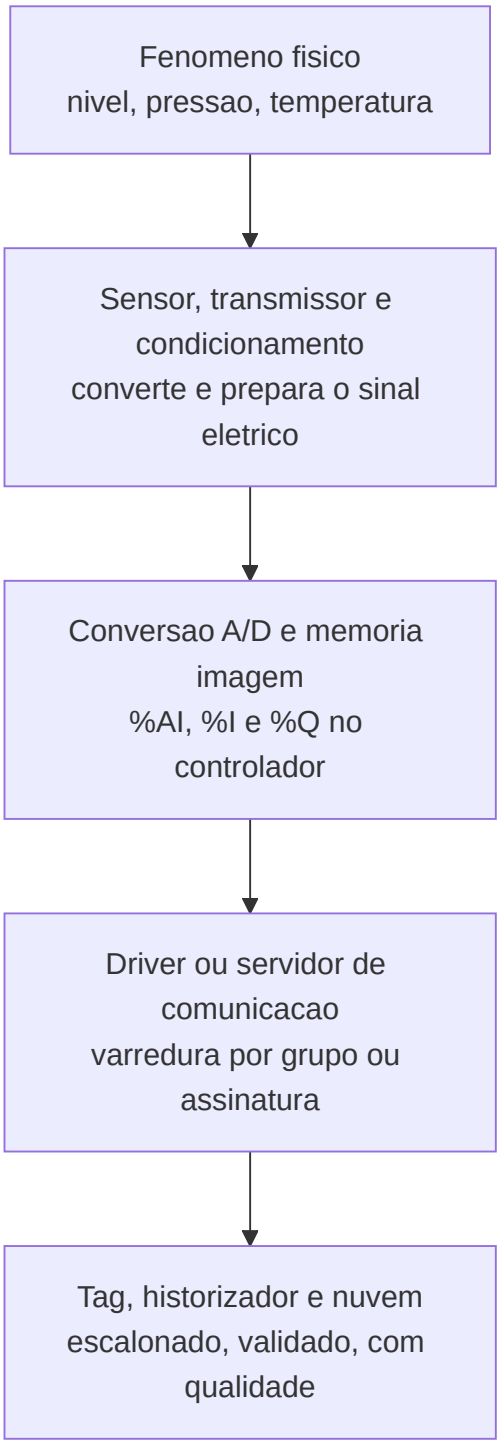


Tabela 15.1 – O que cada elo pode fazer com o dado

Elo	O que ele acrescenta	Como ele degrada
Sensor e transmissor	Converte a grandeza física em sinal elétrico	Erro de exatidão, deriva, tempo de resposta do próprio sensor

Elo	O que ele acrescenta	Como ele degrada
Condicionamento	Filtra ruído, isola, protege contra surto	Filtro mal dimensionado atrasa a resposta e suaviza o evento
Conversor A/D	Quantifica o sinal em degraus discretos	Perde resolução se a faixa utilizada for muito maior que o sinal
Memória imagem	Mantém o valor atual no controle	É sobrescrita a cada varredura: o que não é copiado, desaparece
Driver e rede	Transporta em blocos, com taxa configurável	Atraso e perda sob congestionamento; ordem de chegada fora de sequência
Tag e historizador	Escalona, valida, filtra e arquiva	Descarta por faixa morta, por validação ou por política de retenção

Nota

Nota: o elo mais barato de corrigir quase nunca é o transmissor. Antes de trocar um instrumento, verifique a faixa configurada, o filtro do canal, o endereço no mapa de memória e a taxa de varredura — os mesmos quatro itens que explicam a maior parte das reclamações de "leitura errada".

15.2 Tipos de sinal e forma de conexão

A escolha do sinal determina o custo do cabeamento, a imunidade a ruído e o que é possível fazer com o dado depois. As formas mais usadas:

Tabela 15.2 – Comparativo dos sinais de campo

Sinal	Natureza	Vantagem	Limitação	Uso típico
4 a 20 mA	Analógico de corrente	Imune a queda de tensão no cabo; o 4 mA denuncia cabo rompido	Um par por variável; exige resistor de precisão na entrada	Pressão, nível, vazão, posição
0 a 10 V	Analógico de tensão	Simples de gerar e de medir	Sensível a queda de tensão e a ruído indutivo	Sinais curtos, bancada, comandos
RTD (Pt100)	Resistência	Estável e preciso em faixa larga	Exige 3 ou 4 fios para compensar o cabo	Temperatura de processo

Sinal	Natureza	Vantagem	Limitação	Uso típico
Termopar	Tensão em microvolts	Faixa ampla de temperatura	Exige cabo de extensão e junta de referência corretas	Temperatura alta, fornos
Pulso / contagem	Frequência	Totaliza volume e energia sem perda acumulada	Exige entrada de alta velocidade dedicada	Hidrômetros, medidores de energia
Discreto 24 VCC	Dois estados	Simples, robusto, barato	Um ponto por informação	Fim de curso, chave, partida de motor

Atenção

Atenção: o **4 mA** da faixa de 4 a 20 mA existe para distinguir "valor zero" de "sem sinal". Se a leitura de um transmissor de 4 a 20 mA chega como zero na tela, há um erro de configuração ou de hardware em algum lugar — o zero é justamente o valor que o sinal *não* pode assumir em condição normal. Tratar isso como "pressão zero" leva à ação errada (Capítulo 14).

15.3 Digitalização: resolução, exatidão e ruído

Três grandezas diferentes costumam ser confundidas em uma só frase:

- **Resolução** — o menor degrau que o conversor consegue representar. Um A/D de 15 bits divide a faixa em 32.768 degraus (dois elevado a quinze); em 4 a 20 mA, isso dá menos de 0,004 % por degrau.
- **Exatidão** — o quanto a leitura se aproxima do valor verdadeiro. É definida pelo transmissor e pela instalação, não pelo conversor.
- **Ruído** — a variação aleatória em torno do valor. É o que, na prática, limita o que se enxerga.

Um cartão de 16 bits lendo um transmissor com 1 % de erro continua tendo 1 % de erro: a resolução não substitui a exatidão. E um sinal com ruído de 0,5 % não melhora nada ao passar de 12 para 16 bits — apenas passa a mostrar o ruído com mais casas decimais.

```
Exemplo de proporção entre as três grandezas:

  Faixa de 0 a 100 %
  A/D de 12 bits -> degrau de 0,024 %
  Transmissor de classe 0,5 % -> erro de ate 0,5 %
  Ruido observado na instalacao -> 0,3 % de variacao pico a pico

  Conclusao: o gargalo e o transmissor e a instalacao, nao o cartao.
```

O tratamento começa no **filtro analógico** do canal de entrada e continua no **filtro digital** do controlador ou do supervisório. Um filtro de média móvel de n amostras reduz o ruído na proporção da raiz quadrada de n, mas atrasa a resposta — e um atraso grande em uma variável de proteção é pior que o ruído que ele resolve.

Dica

Dica: antes de configurar qualquer filtro, meça. Com o processo estável, colete dois minutos de amostra crua e observe quanto a leitura varia sem que nada aconteça na planta. Esse número é o seu limite inferior de faixa morta e o seu critério para decidir se o problema é filtro, blindagem ou aterramento.

15.4 Varredura: quem decide o ritmo

Cada elo da cadeia tem o seu próprio ritmo, e eles são muito diferentes:

Tabela 15.3 – Ordens de grandeza típicas das taxas de atualização

Elo	Faixa típica	Quem define
Ciclo de varredura do controlador	Poucos milissegundos a dezenas de milissegundos	Fabricante e tamanho do programa
Taxa de varredura do driver	Centenas de milissegundos a segundos, por grupo de tags	Engenharia de configuração do servidor de comunicação
Atualização dos objetos na IHM	Décimos de segundo a poucos segundos	Desempenho do conjunto (rede, estação, driver)
Intervalo de amostragem da assinatura	Configurável por variável, com faixa morta	Projeto (OPC UA; ver Capítulo 13)
Gravação no historiador	Por exceção ou por período, conforme política	Projeto do histórico (ver Capítulo 16)



Figura 15.1 – Tempo de resposta de uma tarefa: o intervalo entre a solicitação e a resposta realizada. **Fonte:** *Livro SCADA — versão para análise*, p. 33 (Machado, Pontes e Vianna, reproduzida com crédito).

O tempo de resposta de um sistema SCADA é a **soma** dos tempos de cada subsistema envolvido — sensoriamento, ciclo do controlador, driver, rede, processamento na estação e desenho na tela. Como as estações de supervisão não são determinísticas (*Windows* ou *Linux* com carga variável), o valor exato não pode ser calculado a priori: ele é estimado, configurado e medido em campo.

O erro clássico de projeto é dimensionar a comunicação como se toda a banda configurada estivesse disponível para dados. Em uma serial de 19.200 bps, cada *frame* gasta bits de partida, de parada e de verificação: sobra cerca de **dois terços** da banda para o dado útil. Em um intervalo de atualização de 100 ms, isso significa algo em torno de 1.280 bits de dado — suficiente para poucas dezenas de variáveis, não para o cadastro inteiro.

15.4.1 O evento que se perde entre as varreduras

Há uma consequência mais grave que o atraso: o controlador pode **ver** um sinal que o supervisório **não registra**. Se um pressostato atua por 20 ms e provoca o desligamento da planta, o CLP executa o intertravamento no seu ciclo de varredura de poucos milissegundos — mas a atualização da IHM pode levar um segundo, e o alarme nunca aparece. A planta parou, e a tela não mostra por quê.

Esse é o tipo de falha mais difícil de investigar, porque não deixa rastro. A solução não é reduzir a taxa do sistema inteiro — é **capturar o evento no controlador**, onde a resolução é suficiente, e transferir o resultado, e não o sinal.

Figura

[Figura 15.2 – Configuração de tópicos com tempos de varredura distintos em um servidor de comunicação] *Captura de tela de configuração de driver com três grupos de tags e a respectiva taxa: grupo rápido (200 ms) com variáveis de intertravamento e estado de bombas; grupo médio (1 s) com medições analógicas de processo; grupo lento (10 s) com totalizadores, temperaturas de mancal e diagnóstico. Ao lado, o campo de faixa morta por tag. Destacar que a classificação por dinâmica, não por área da planta, é o critério da divisão.*

15.5 Tratamento: o que se faz com o dado antes de mostrá-lo

Tratamento não é maquiagem: é a parte do projeto que separa informação de ruído. As operações usuais, na ordem em que costumam ser aplicadas:

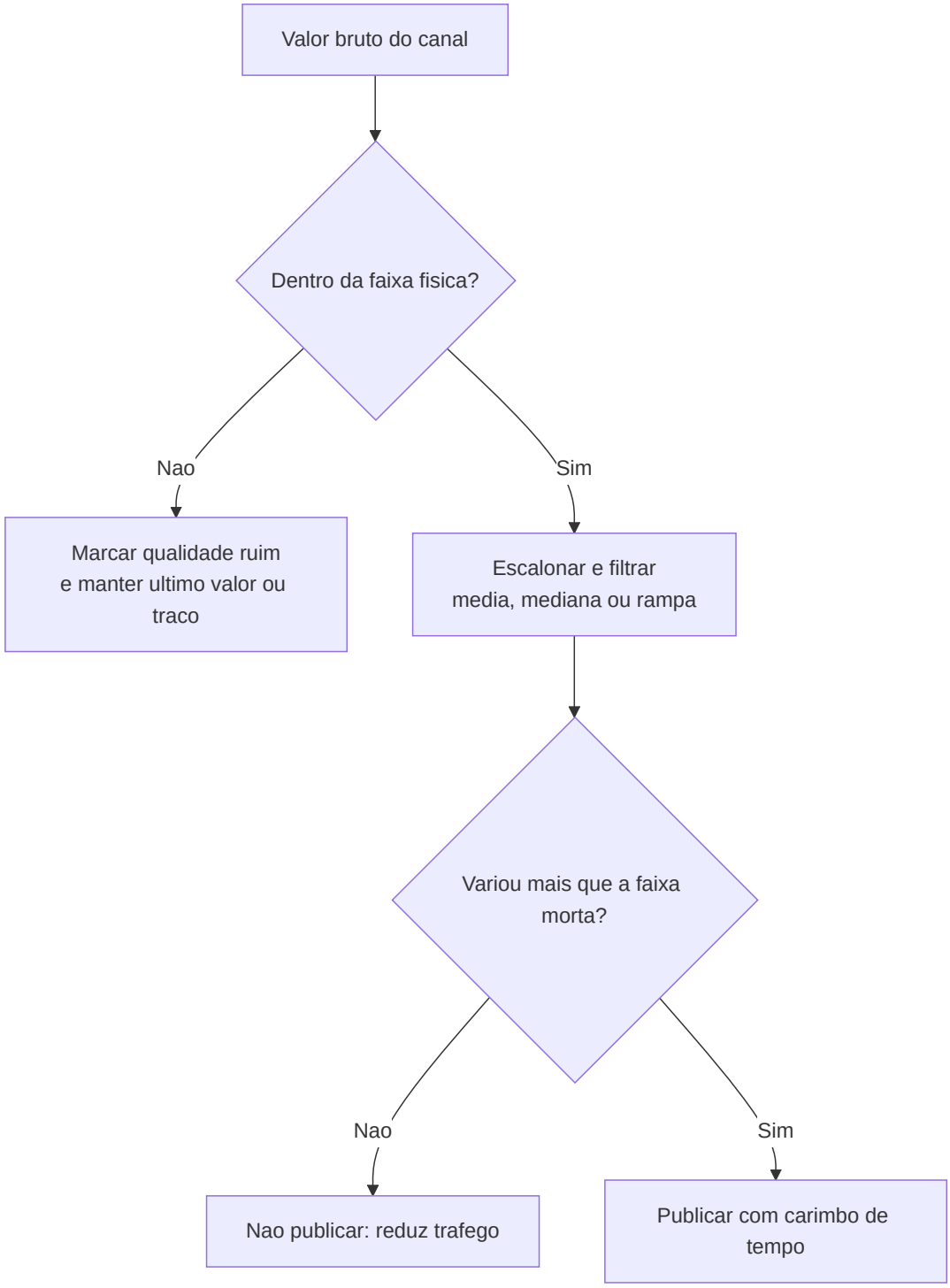


Tabela 15.4 – Operações de tratamento e o risco de cada uma

Operação	Para que serve	Efeito colateral
Validação de faixa	Rejeitar leituras fisicamente impossíveis (fora da faixa do sensor)	Faixa mal cadastrada rejeita dado bom

Operação	Para que serve	Efeito colateral
Escalonamento	Levar o valor bruto à unidade de engenharia	Divergência de faixa entre instrumento e cadastro gera erro silencioso
Média móvel	Reduzir ruído aleatório	Atrasa a resposta; mascara transitório real
Mediana móvel	Remover picos espúrios isolados	Custa mais processamento que a média
Faixa morta (*deadband*)	Evitar publicação de variação irrelevante	Faixa morta alta esconde a tendência lenta
Publicação por exceção	Reduzir tráfego na rede e no enlace remoto	Exige *refresh* periódico para não deixar a tela “congelada”

O critério para escolher entre eles não é estético. Em uma variável de proteção, o filtro deve ser o menor possível ou inexistente; em uma variável de indicação lenta, como temperatura de mancal, um filtro largo e uma faixa morta generosa reduzem tráfego sem prejuízo.

Atenção

Atenção: filtro e faixa morta se somam. Um canal com média de 10 amostras e faixa morta de 2 % pode levar mais de dois minutos para anunciar uma variação real de 1,5 % — e o operador só vai descobrir isso no dia em que precisar da resposta rápida. Documente, para cada tag, o filtro e a faixa morta configurados.

15.6 Registro da primeira falha

Quando uma planta para, quase sempre vários instrumentos atuam em sequência: os intertravamentos abraem, as bombas desligam, os pressostatos comutam. Se todos os eventos chegam ao supervisório com carimbo de tempo do próprio supervisório, a ordem aparente é apenas a ordem de chegada — e a causa raiz se perde na avalanche.

O **registrador de primeiro evento** resolve isso dentro do controlador: cada linha de intertravamento ganha, logo abaixo, um bloco que grava o código do sensor em um endereço de memória e aciona um bit de bloqueio. Do segundo sensor em diante, o bloqueio impede a sobrescrita. O supervisório lê apenas um endereço — o código do primeiro sensor que atuou.

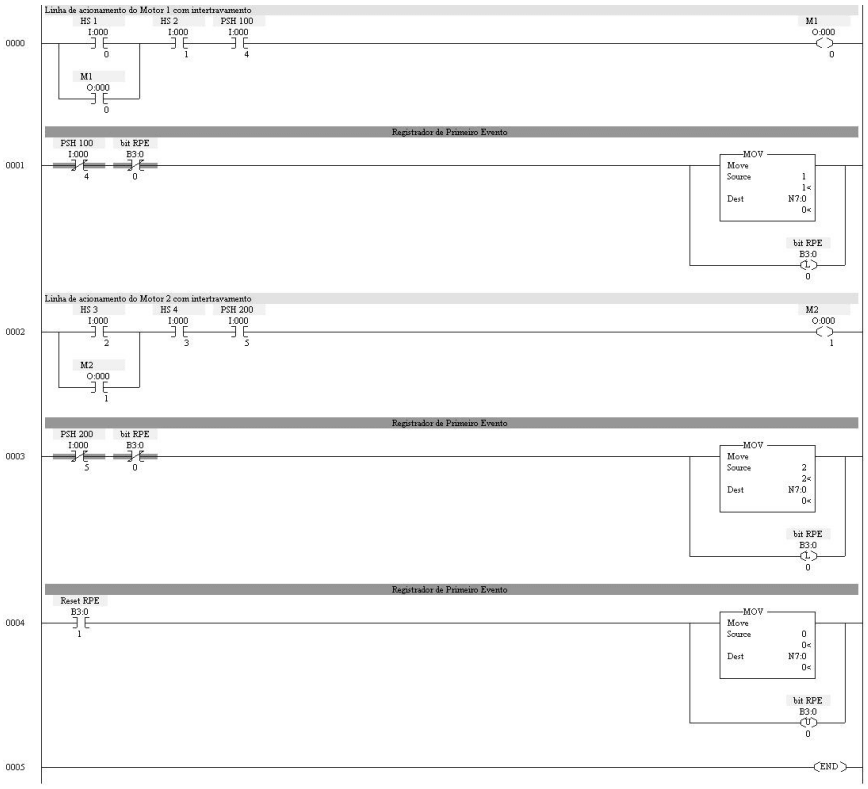


Figura 15.3 – Lógica em Ladder do registrador de primeiro evento, com bit de bloqueio e blocos Move. **Fonte:** *Livro SCADA — versão para análise*, p. 36 (Machado, Pontes e Vianna, reproduzida com crédito).

Tabela 15.5 – Tabela de alocação do registrador de primeiro evento

Endereço	Tagname	Descrição
`I:000/0`	`HS 1`	Chave de acionamento do motor 1 (NA)
`I:000/1`	`HS 2`	Chave de desacionamento do motor 1 (NF)
`I:000/4`	`PSH 100`	Chave de pressão alta (NF)
`I:000/5`	`PSH 200`	Chave de pressão alta (NF)
`B3:0/0`	`bit RPE`	Bit de bloqueio que impede alteração do valor gravado
`B3:0/1`	`Reset RPE`	Botão que reinicia o registrador pela IHM
`N7:0`	`Codigo RPE`	Número do sensor que originou a parada

Três cuidados determinam se a lógica funciona na prática:

1. **A linha de registro fica imediatamente abaixo do intertravamento correspondente.** Se ela for deslocada, o tempo de varredura pode fazer o código do segundo sensor chegar antes do primeiro.
2. **Existe uma lista de relação entre número e evento**, homologada e disponível para a operação — sem ela, o código gravado na memória é um número sem

significado.

3. **O registrador é reiniciado pela IHM**, com registro de quem reiniciou e quando.

Nota

Atualização tecnológica: o registrador em Ladder continua válido e é a solução mais barata em controladores que não têm cartão de sequência de eventos. Em arquiteturas atuais, a mesma função é obtida com **RTUs e controladores com SOE** (*Sequence of Events*), que carimbam cada entrada com resolução de milissegundo na origem, e com **sincronização de tempo** por NTP ou PTP (IEEE 1588) em toda a planta. A escolha depende do que se quer provar: a lógica em Ladder diz *qual* foi o primeiro a atuar; o SOE com carimbo na origem diz *quando*, com resolução suficiente para reconstruir a sequência inteira.

O carimbo de tempo é o outro lado do problema. Dados carimbados na chegada ao supervisório perdem a ordem real, sobretudo em enlaces com atraso variável. A regra é simples: **carimbe na origem** sempre que o equipamento permitir, e mantenha os relógios de todos os nós sincronizados — um histórico com relógios discrepantes é pior que um histórico sem relógio, porque produz conclusões erradas com aparência de precisão.

15.7 Comparativo das arquiteturas de aquisição

Quatro arranjos resolvem a aquisição, e a escolha depende do que existe no campo e do que se pretende fazer com o dado.

Tabela 15.6 – Comparativo das arquiteturas de aquisição

Tecnologia	Função	Protocolo	Aplicação	Vantagens	Limitações
CLP com cartões de I/O	Aquisição e controle no mesmo equipamento	Modbus, OPC UA, PROFINET, EtherNet/IP	Processo com lógica de controle local	Lógica e aquisição juntas; varredura rápida; diagnóstico de canal	Custo por ponto maior em instalações muito distribuídas
Sistema DAQ	Aquisição dedicada, sem controle	Modbus, OPC, proprietário	Bancada, ensaio, monitoramento de máquina	Alta resolução e taxa; pronto para ensaio	Sem controle, sem intertravamento
RTU	Aquisição remota com transmissão	DNP3, IEC 60870-5-101/104, Modbus	Campo remoto, baixo consumo	Baixo consumo, robustez, comunicação por rádio ou celular	Menor capacidade de lógica; programação menos padronizada

Tecnologia	Função	Protocolo	Aplicação	Vantagens	Limitações
Gateway de borda	Aquisição, normalização e publicação	Modbus, OPC UA, MQTT, REST	Integração com nuvem e múltiplos protocolos	Concentra protocolos, faz *store and forward*, reduz tráfego	É mais um nó para manter, atualizar e proteger

A tendência de projeto não é escolher um deles para toda a planta, e sim **usar cada um onde ele é melhor**: CLP onde há controle e intertravamento, RTU onde há campo remoto, gateway onde há integração. O Capítulo 18 trata da integração entre eles.

15.8 Estudo de Caso — A planta que parou e a tela não mostrou por quê

Cenário. Uma planta de tratamento de água com duas bombas de recalque e duas chaves de pressão alta. Em três semanas, o conjunto parou quatro vezes por atuação da proteção. Em todas as ocorrências, o histórico de alarmes registrou, com diferença de poucos segundos, o alarme de "pressão alta" e o de "bomba desligada" — sem indicação confiável de qual veio primeiro. A manutenção trocou um pressostato, verificou o outro e não encontrou defeito.

Diagnóstico. Três problemas somados:

1. O carimbo de tempo de todos os alarmes era atribuído pelo supervisório, na chegada — não na origem.
2. A varredura do driver para as chaves de pressão era de 1 s, maior que a duração do pulso de pressão alta no instante da atuação.
3. Os dois pressostatos atuavam em sequência por projeto (um é proteção, o outro é alarme), o que torna indistinguível a "primeira" atuação se os dois eventos chegam com o mesmo carimbo.

Decisão de engenharia.

Medida	Efeito
Registrador de primeiro evento no CLP, com código por sensor e lista homologada	Passa a existir um único registro confiável de primeira atuação
Chaves migradas para o grupo de varredura rápido (200 ms) do driver	A atuação passa a ser observada, e não apenas inferida
Sincronização dos relógios do CLP, da estação e do servidor	Os carimbos de tempo passam a ser comparáveis
Alarme de "primeira atuação da proteção" distinto do alarme de processo	A operação deixa de receber dois alarmes concorrentes sem hierarquia

Resultado. Na ocorrência seguinte, o registro apontou PSH 100 como primeira atuação, nove segundos antes do desligamento — a causa era uma válvula de retenção que fechava

lentamente e provocava golpe de aríete. O defeito era de processo, não do instrumento, e nenhuma das quatro ocorrências anteriores permitiria essa conclusão.

Limitações do arranjo. O registrador de primeiro evento informa *qual* foi o primeiro, não *quando* com precisão de milissegundo. Para análise de sequência real, a solução é cartão de SOE com carimbo na origem e PTP. Além disso, a lógica em Ladder precisa ser revista sempre que um novo intertravamento é acrescentado — e essa manutenção precisa estar no procedimento de gestão de mudanças, não na memória de quem programou.

15.9 Resumo

- A aquisição é uma **cadeia de elos**, e cada um acrescenta erro, atraso ou descarte.
- O **4 mA** da faixa de 4 a 20 mA existe para separar "valor zero" de "sem sinal".
- **Resolução, exatidão e ruído** são grandezas distintas; a resolução do conversor raramente é o gargalo.
- Cada elo tem o seu ritmo: o ciclo do controlador é de milissegundos, a atualização da IHM é de décimos de segundo a segundos.
- O tempo de resposta é a **soma** dos subsistemas e não é determinístico nas estações de supervisão.
- Eventos curtos podem atuar no intertravamento e **não aparecer** no supervisório — a solução é capturar no controlador.
- **Tratamento** (validação, escalonamento, filtração e faixa morta) separa informação de ruído, mas cada operação tem custo: atraso e perda de transitório.
- A **primeira falha** de uma parada precisa ser registrada na origem; lógica em Ladder resolve o "qual", o SOE com carimbo resolve o "quando".
- Relógios sincronizados são pré-requisito para qualquer análise de sequência.

15.10 Questões de Revisão

1. Descreva a cadeia de aquisição de uma medição de nível, citando o que cada elo pode fazer com o dado.
2. Por que a faixa de 4 a 20 mA começa em 4 mA e qual a consequência prática de ler zero nessa entrada?
3. Explique a diferença entre resolução, exatidão e ruído, usando o exemplo de um transmissor de 0,5 % ligado a um cartão de 16 bits.
4. Um sistema tem ciclo de varredura de 10 ms no CLP, varredura de driver de 1 s e atualização de IHM de 2 s. Um sinal permanece alto por 300 ms. Descreva o que cada elo registra e qual o risco para a operação.
5. Por que o tempo de resposta de um sistema SCADA não pode ser calculado com precisão antes da instalação? Cite três fatores que o influenciam.
6. Explique como funciona o registrador de primeiro evento e por que a linha de registro precisa ficar logo abaixo do intertravamento correspondente.
7. Compare filtro de média e faixa morta quanto ao efeito sobre ruído, atraso e volume de tráfego.
8. Um histórico de alarmes mostra dois eventos com o mesmo carimbo de tempo. Que deficiências de projeto podem explicar isso e como corrigi-las?

9. Compare CLP, sistema DAQ, RTU e gateway de borda quanto à função, protocolo e limitação principal, e indique um cenário em que os quatro convivem na mesma planta.

15.11 Referências

- MACHADO, R.; PONTES, W.; VIANNA, W. **Livro SCADA — versão para análise**. Material didático do curso (seções sobre desempenho do sistema de automação SCADA, sistemas de tempo real e registrador de primeiro evento). Figuras reproduzidas com crédito.
- VIANNA, W. S. **Sistema SCADA Supervisório**. IFF, 2008 (capítulos sobre componentes de hardware e software, drivers de comunicação e elementos dinâmicos).
- FARINES, J. M.; FRAGA, J. S.; OLIVEIRA, R. S. **Sistemas de Tempo Real**. São Paulo: Escola de Computação, 2000 (definição de sistema de tempo real citada na obra de origem).
- INTERNATIONAL ELECTROTECHNICAL COMMISSION. **IEC 61131-3 — Programmable controllers: Programming languages**. 4ª edição, maio de 2025.
- INSTITUTE OF ELECTRICAL AND ELECTRONICS ENGINEERS. **IEEE 1588 — Precision Clock Synchronization Protocol for Networked Measurement and Control Systems** (PTP; verificar edição vigente).
- INTERNATIONAL SOCIETY OF AUTOMATION. **ANSI/ISA-5.1 — Instrumentation Symbols and Identification** (identificação de instrumentos por letras; verificar edição vigente antes de citar em projeto).
- ISA. **ISA-101 — Human-Machine Interfaces for Process Automation Systems** (referência geral de apresentação de dados ao operador; verificar edição vigente).
- OPC FOUNDATION. **OPC Unified Architecture** — *subscription*, intervalo de amostragem e *deadband* por item monitorado. Disponível em: opcfoundation.org. Consulta: 2026.

Capítulo 16

Historiadores e Séries Temporais

Objetivos de Aprendizagem

Ao final deste capítulo, o leitor será capaz de:

- Distinguir tendência real de tendência histórica e dimensionar a janela de registro.
- Reconhecer por que bancos relacionais convencionais não atendem à historização de processo.
- Explicar o efeito da compressão por exceção e a consequência de configurá-la mal.
- Definir política de retenção, agregação e descarte por classe de dado.
- Separar histórico de processo de histórico de eventos e de trilha de auditoria.
- Comparar soluções de historização pela função e pela limitação, sem apelo de fabricante.

16.1 A pergunta que o histórico responde

O histórico de processo atende a três públicos que fazem perguntas diferentes:

Público	Pergunta	O que precisa ser preservado
Operação	"O que está acontecendo agora e nos últimos minutos?"	Alta resolução no presente, com atualização contínua
Engenharia e manutenção	"Por que parou, o que mudou antes da falha, qual a tendência da variável?"	Resolução suficiente para reconstruir o transitório
Gestão e conformidade	"Quanto produzimos, qual o consumo, como estava a emissão naquele dia?"	Integridade, retenção longa e agregação confiável

As três exigências são incompatíveis entre si quando o armazenamento é tratado como um só. Guardar tudo com máxima resolução por anos é caro; guardar tudo comprimido o suficiente para caber em disco destrói a resposta que a engenharia precisa. Todo sistema de historização real é uma negociação entre essas demandas — e essa negociação é projeto, não parâmetro deixado no padrão.

16.2 Tendência real e tendência histórica

São dois objetos diferentes, com armazenamento diferente.

- A **tendência real** exibe os últimos valores em função do tempo, de forma dinâmica. Cada curva guarda os valores em um **vetor na memória** da estação de supervisão.
- A **tendência histórica** recupera valores gravados em memória secundária (disco, SSD) e exibe o intervalo escolhido pelo operador. É estática, aceita mais variáveis e permite recuperar qualquer período.

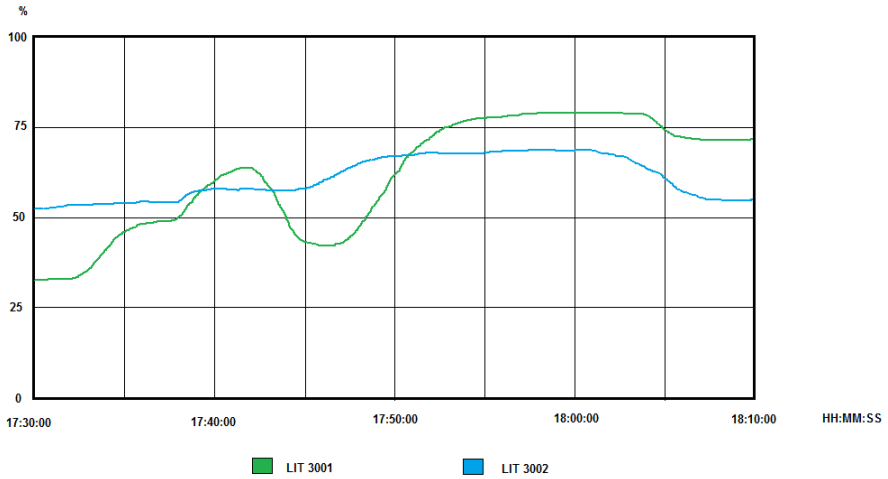


Figura 16.1 – Tendência real com duas variáveis de nível, escala fixa e legenda. **Fonte:** *Livro SCADA — versão para análise*, p. 67 (Machado, Pontes e Vianna, reproduzida com crédito).

Na tendência real, o tamanho do vetor é o recurso limitado, e a aritmética é implacável:

```
janela de tempo = taxa de amostragem x numero de amostras

  100 ms x 1024 amostras = 102,4 s   (1,7 minuto)
  1 s    x 1024 amostras = 1024 s    (17 minutos)
  5 s    x 1024 amostras = 5120 s    (85 minutos)
  10 min / 5 s           = 120 amostras no vetor
```

Não existe configuração que amplie a janela e melhore a resolução ao mesmo tempo: aumentar uma obriga a reduzir a outra, ou a ampliar memória. É por isso que a tendência real é organizada por **grupos de variáveis com dinâmica parecida** — vazão e pressão em um grupo, temperatura de mancal em outro.

Tabela 16.1 – Tendência real × tendência histórica

Aspecto	Tendência real	Tendência histórica
Armazenamento	Memória RAM da estação	Disco rígido, SSD ou servidor de histórico
Comportamento	Dinâmica, atualizada continuamente	Estática, recuperada sob consulta
Janela	Limitada pelo tamanho do vetor	Limitada pela política de retenção

Aspecto	Tendência real	Tendência histórica
Número de variáveis	Normalmente poucas (1 a 8 penas por gráfico)	Muitas, com filtro por grupo e período
Perda ao reiniciar	Sim: o vetor é volátil	Não: o dado está gravado
Uso típico	Acompanhar a operação corrente	Diagnosticar, comparar, relatar, auditar

16.3 Os parâmetros que decidem o que você vai ver depois

Quatro parâmetros definem o resultado de um gráfico de tendência, e todos os quatro são decididos muito antes de alguém precisar do histórico.

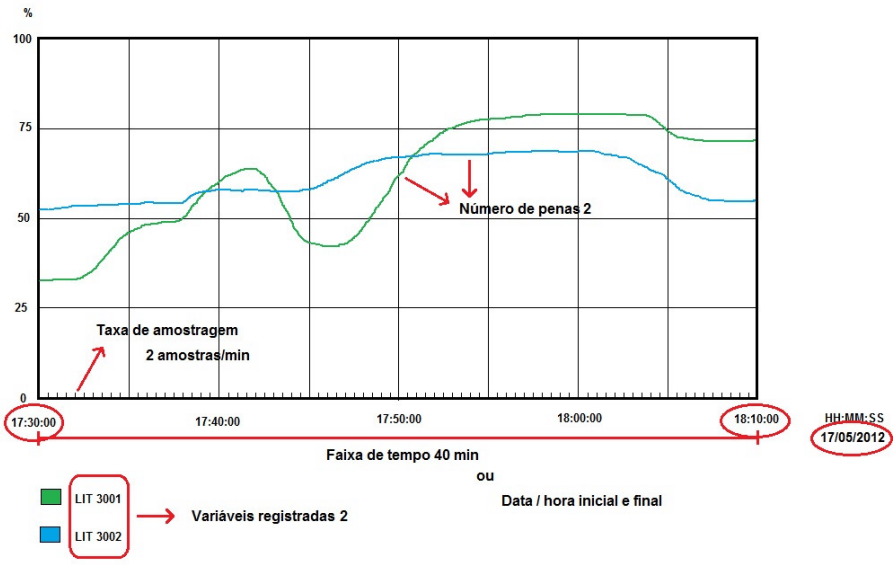


Figura 16.2 – Parâmetros de configuração de um gráfico de tendência. **Fonte:** *Livro SCADA — versão para análise*, p. 70 (Machado, Pontes e Vianna, reproduzida com crédito).

Tabela 16.2 – Parâmetros do gráfico de tendência

Parâmetro	O que define	Consequência de errar
Taxa de amostragem	Intervalo entre amostras gravadas	Subamostragem esconde o transitório (Capítulo 14)
Número de penas	Quantas variáveis na mesma escala	Acima de poucas penas, as cores deixam de ser distinguíveis
Variáveis registradas	Quais tags entram no vetor	Tag fora do vetor não tem histórico no período

Parâmetro	O que define	Consequência de errar
Faixa de tempo e escala	Janela exibida e limites do eixo	Escala automática esconde comparação; escala fixa esconde a variação

Dica

Dica: escolha as variáveis de um mesmo gráfico pela **dinâmica**, não pela área da planta. Misturar na mesma escala uma vazão que oscila a cada segundo com a temperatura de um tanque que varia em horas produz um gráfico em que uma das curvas é uma linha reta — e o operador aprende a ignorá-la. Quando a comparação entre dinâmicas diferentes for necessária, use eixos separados.

Atenção

Atenção: a escala automática é confortável e enganosa. Ela sempre preenche a tela, fazendo uma variação de 0,1 % parecer uma oscilação dramática. Em painel de operação, escala fixa com faixa conhecida é o padrão; escala automática é ferramenta de investigação, não de vigilância.

16.4 Por que banco relacional não é historiador

Um banco de dados relacional resolve bem cadastro, venda, financeiro. Ele não foi projetado para o padrão de carga da indústria, que tem três características incômodas:

1. **Volume alto e previsível.** Uma planta com 5.000 tags coletadas a cada segundo produz 432 milhões de pontos por dia — cerca de 158 bilhões em um ano. Com 16 bytes por ponto, isso passa de 2 TB por ano em armazenamento bruto, sem contar índices.
2. **Escrita sequencial e contínua.** O padrão é *append*: quase nunca se atualiza um ponto antigo. Um banco relacional com índices em todas as colunas paga caro por uma flexibilidade que não é usada.
3. **Consulta por intervalo e por série.** Quase toda pergunta é "valores da variável X entre T1 e T2". Isso é um *range scan* sobre uma série temporal, não um *join*.

Um banco de dados de séries temporais (*TSDB*) existe para esse padrão. O que ele faz de diferente:

Tabela 16.3 – O que um TSDB acrescenta ao banco convencional

Recurso	Para que serve
Índice temporal nativo	Consulta por intervalo sem varrer a tabela inteira
Compressão por série	Estabilidade do sinal vira compressão; rampa é gravada ponto a ponto
Agregação contínua (*rollup*)	Pré-calcula média, mínimo e máximo por hora e por dia

Recurso	Para que serve
Política de retenção	Reduz a resolução ou descarta o dado antigo automaticamente
Resolução de tempo nativa	Carimbo de tempo com precisão declarada e sem ambiguidade

O modelo de dados continua sendo o mesmo de sempre — **série, tempo, valor e qualidade** —, e é exatamente o que o Capítulo 14 descreveu do lado do tag. O que muda é como esse modelo é armazenado e consultado.

Nota

Nota: o "melhor banco" não existe. Existe o banco adequado ao volume, à janela de retenção, ao ferramental de consulta e à competência da equipe que vai mantê-lo. Um historian embarcado no próprio supervisório é a escolha certa para quem precisa de 30 dias de dado e já tem o software; um TSDB em contêiner é a escolha certa para quem precisa integrar processo e nuvem com consulta programável.

16.5 Compressão: o que o histórico decide esquecer

Historiador sem compressão é caro; historiador com compressão mal configurada é pior que caro, porque produz uma série que parece verdadeira e não é.

O mecanismo mais difundido é a **compressão por exceção**, também chamada de *swinging door*: o software só grava um novo ponto quando o valor se afasta do último ponto gravado mais do que uma tolerância definida.

```
Sinal estavel em 50,0 %  -> 1 ponto, mais um refresh periodico
Rampa de 0 a 100 %       -> pontos a cada desvio de tolerancia
Degrau instantaneo       -> gravado como degrau, se a tolerancia
                            for menor que o degrau

tolerancia de compressao = erro maximo da serie reconstruida
```

Daí decorrem três consequências que precisam estar no projeto:

- A **tolerância de compressão é o erro máximo** que a curva reconstruída terá em relação ao sinal real. Declará-la é obrigatório em qualquer análise que dependa de precisão.
- Quanto **maior a tolerância, mais liso** o gráfico fica e menor o volume gravado — o que é ótimo para tendência de longo prazo e péssimo para diagnosticar transitório.
- A compressão é **irreversível**. O ponto descartado na gravação não volta.

Atenção

Atenção: o pico é a primeira coisa que a compressão descarta, e é justamente o que a engenharia procura quando investiga uma falha. Em variáveis ligadas a proteção, a tolerância de compressão deve ser a menor possível, e vale avaliar o registro explícito de mínimo e máximo por intervalo — recurso que preserva a envoltória do sinal mesmo com poucos pontos gravados.

16.6 Retenção, agregação e conformidade

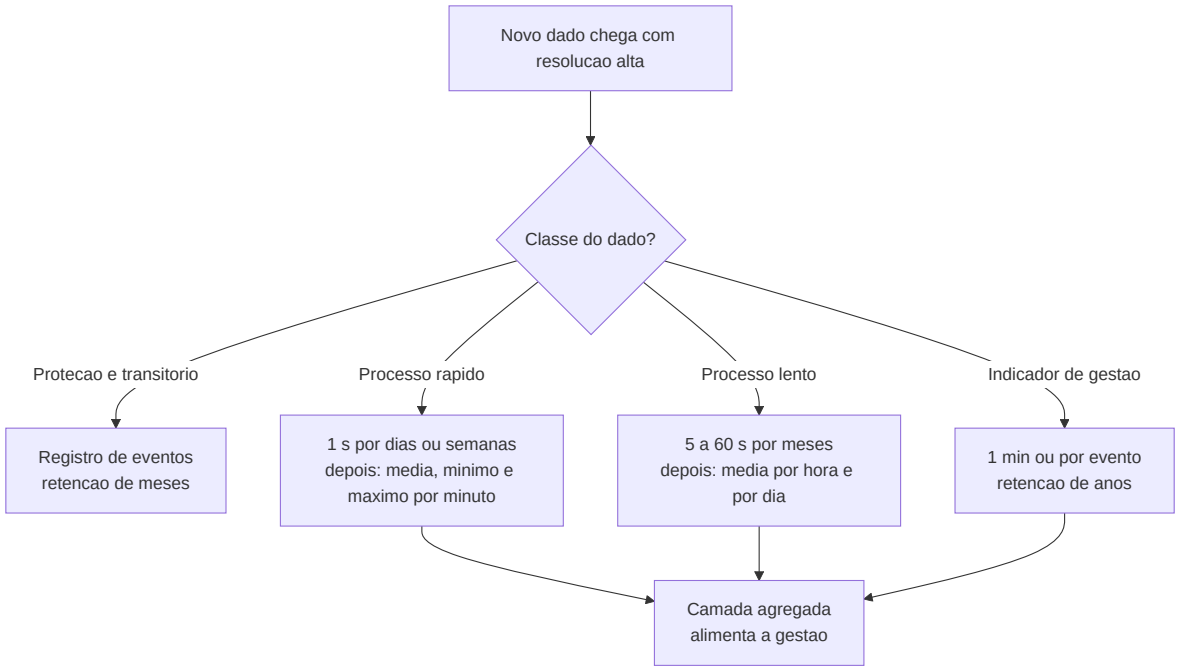
Reter tudo para sempre não é política de arquivo: é ausência de política. A prática é **reduzir resolução com o tempo**, mantendo a informação estatística do período.

Tabela 16.4 – Política de retenção por classe de dado

Classe	Exemplos	Resolução inicial	Retenção típica	Agregação
Transitório e proteção	Chaves de intertravamento, primeira falha	Na origem, por exceção	Meses	Nenhuma: eventos vão para o registro de eventos
Processo rápido	Vazão, pressão	1 s	Dias a semanas	Média, mínimo e máximo por minuto
Processo lento	Temperatura, nível de tanque	5 a 60 s	Meses a anos	Média por hora e por dia
Indicadores de gestão	Produção, consumo, energia	1 min ou por evento	Anos	Totais diários e mensais

Três cuidados completam a política:

- **Agregar preservando extremos.** Média por hora sem mínimo e máximo apaga o pico da hora — e é exatamente o pico que interessa em consumo, vazão e temperatura.
- **Backup com restauração testada.** Histórico que existe em um só disco não é histórico; é um arquivo temporário.
- **Expurgo planejado.** O Livro SCADA de origem já alertava para a necessidade de critério de esvaziamento e *backup* dos arquivos de registro, "a fim de evitar ocupação total da mídia". Em sistemas atuais, isso é resolvido pela própria política de retenção do banco, mas a decisão de quanto tempo guardar continua sendo da engenharia — e frequentemente responde a exigências regulatórias e contratuais, não a preferências técnicas.



Figura

[Figura 16.3 – Consulta de histórico com agregação e comparação de períodos] *Captura de uma tela de consulta a séries temporais com três controles visíveis: seleção de variáveis por ativo, intervalo de tempo (data e hora inicial e final) e função de agregação (média, mínimo, máximo, por hora ou por dia). O gráfico exibe duas curvas sobrepostas de períodos diferentes, com legenda identificando cada período e uma tabela resumida abaixo com mínimo, média e máximo de cada série. Destacar que a agregação está explícita na consulta e não escondida no painel.*

16.7 Histórico de processo × histórico de eventos

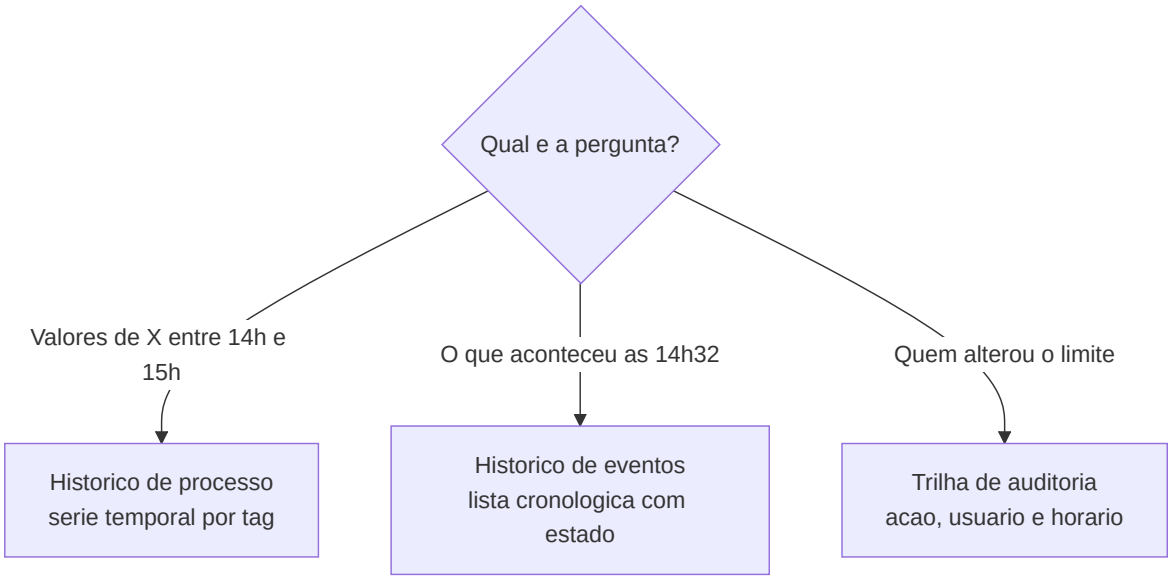
São dois registros, com dois propósitos, e misturá-los prejudica os dois.

Tabela 16.5 – Comparativo dos dois registros

Aspecto	Histórico de processo	Histórico de eventos e alarmes
Conteúdo	Valor de variável ao longo do tempo	Ocorrência com carimbo, estado, prioridade e reconhecimento
Formato	Série temporal por tag	Lista cronológica de registros
Volume	Milhões de pontos por dia	Centenas a milhares de registros por dia
Consulta típica	“Valores de X entre 14h e 15h”	“O que aconteceu entre 14h e 15h e quem reconheceu”

Aspecto	Histórico de processo	Histórico de eventos e alarmes
Uso	Diagnóstico, otimização, relatório de processo	Investigação de causa, auditoria de operação, indicador de desempenho

A **trilha de auditoria** é um terceiro registro, ainda distinto: quem comandou, quem mudou *setpoint*, quem alterou limite de alarme, quem reiniciou o registrador de primeira falha. Esse registro responde à pergunta "quem fez", que nem o histórico de processo nem a lista de eventos respondem sozinhos — e é requisito de qualquer sistema sujeito a auditoria (Capítulo 8).



Dica

Dica: antes de escolher um historian, escreva as cinco consultas que você precisa responder. "Consumo diário por setor, últimos 12 meses", "todas as partidas da bomba 2 com duração acima de 30 s", "temperatura média e máxima do mancal na semana da falha". A ferramenta que responde às suas cinco consultas com o esforço aceitável é a ferramenta certa — o resto é argumento de catálogo.

16.8 Comparativo das soluções de historização

Tabela 16.6 – Famílias de solução

Tecnologia	Função	Protocolo ou interface	Aplicação	Vantagens	Limitações
Historizador do próprio supervisório	Série temporal integrada à aplicação de supervisão	Nativo, OPC DA/UA	Planta única, escopo de operação	Já integrado, sem projeto de infraestrutura	Retenção e consulta limitadas; integração externa mais difícil
Historizador de processo dedicado	Série temporal industrial em escala de planta e corporação	OPC DA/UA, HDA, API própria	Múltiplas plantas, indicadores corporativos	Compressão madura, consulta rápida, ferramental consolidado	Licença por ponto; projeto longo
Banco de séries temporais de código aberto	Série temporal com consulta por linguagem própria	HTTP, linha de comando, *drivers*	Integração com dashboards e nuvem	Custo de licença baixo, integração ampla, retenção configurável	Exige operação de infraestrutura e disciplina de modelagem
Plataforma IIoT	Série temporal com entidades, alarmes e regras	MQTT, HTTP, REST	Frota de dispositivos e telemetria	Já traz alarme, painel e gestão de dispositivos	Modelo de dados próprio; histórico de alta resolução exige configuração
Arquivo plano (CSV, registrador de carta digital)	Registro simples para conferência	Arquivo	Bancada, ensaio pontual, laudo	Simples e portátil	Sem consulta, sem índice, sem escala

A combinação mais comum em projetos atuais é **duas camadas**: o historian junto ao processo, com alta resolução e escopo curto, e um TSDB ou plataforma na camada de integração, com resolução reduzida e retenção longa. As duas camadas não se substituem — a primeira responde ao diagnóstico, a segunda responde à gestão.

16.9 Estudo de Caso — O pico que o histórico não viu

Cenário. Uma estação de bombeamento registrava a pressão de recalque em um historian configurado com taxa de amostragem de 30 s e tolerância de compressão de 2 % da faixa. Em três meses, dois motores falharam por sobrecarga em partida. A investigação pediu as curvas de pressão dos dez minutos que antecederam cada partida — e encontrou uma reta.

Diagnóstico. Quatro decisões de projeto, tomadas em momentos diferentes, se somaram:

1. A taxa de 30 s foi escolhida quando o historian ainda rodava em um servidor antigo, para caber no disco. A limitação deixou de existir, a configuração ficou.
2. A tolerância de compressão de 2 % era adequada para relatório mensal e destruía o transitório de partida, que dura entre 3 e 8 segundos.
3. A variável estava no grupo de amostragem lenta do driver, com 10 s de varredura — os primeiros segundos da partida nunca chegavam ao historian.
4. Não havia registro de mínimo e máximo por intervalo, o que teria preservado ao menos a envoltória do pico.

Decisão de engenharia.

Medida	Efeito
Pressão de recalque e corrente do motor migradas para grupo rápido (1 s) no driver	O transitório passa a ser observável
Tolerância de compressão reduzida para 0,2 % nas variáveis de processo crítico	O pico deixa de ser descartado
Ativação do registro de mínimo e máximo por minuto	A envoltória é preservada mesmo no dado antigo
Cópia agregada (média, mínimo, máximo por hora) para a camada de gestão	Retenção longa sem custo de armazenamento alto

Resultado. A terceira partida monitorada mostrou elevação de pressão de 40 % acima do normal nos primeiros 4 segundos, seguida de fechamento lento da válvula de retenção — a mesma causa identificada no estudo de caso do Capítulo 15, agora visível na curva.

Limitações do arranjo. A mudança não recupera o histórico já comprimido: os dois primeiros eventos continuarão sem explicação no dado. E o aumento de resolução tem custo de armazenamento que precisa ser dimensionado — a solução não é migrar tudo para 1 s, e sim escolher as variáveis em que o transitório tem significado.

16.10 Resumo

- O histórico atende **operação, engenharia e gestão**, com exigências de resolução e retenção que não coincidem.
- **Tendência real** vive em memória e tem janela limitada pelo vetor; **tendência histórica** vive em disco e responde por período.
- A janela da tendência real é o produto da taxa de amostragem pelo número de amostras: não há como ampliar as duas ao mesmo tempo.
- Banco relacional não é historian por causa do volume, do padrão de escrita sequencial e do tipo de consulta.
- **Compressão por exceção** define o erro máximo da série reconstruída e é irreversível: o pico é o primeiro dado a ser descartado.
- Política de retenção deve **reduzir resolução com o tempo**, preservando mínimo e máximo nas agregações.

- Histórico de processo, histórico de eventos e trilha de auditoria são três registros distintos.
- A escolha da ferramenta sai das **cinco consultas** que o projeto precisa responder.

16.11 Questões de Revisão

1. Explique a diferença entre tendência real e tendência histórica quanto a armazenamento, comportamento e uso.
2. Uma tendência real tem vetor de 2.048 amostras. Qual a janela de tempo com taxa de 500 ms e com taxa de 5 s? Que compromisso você está fazendo em cada caso?
3. Por que um banco de dados relacional não é a escolha adequada para historização de processo? Cite as três características do padrão de carga industrial.
4. Estime o volume anual de pontos de um sistema com 2.000 tags gravadas a cada 2 s e discuta se o armazenamento bruto é aceitável.
5. Explique o que é compressão por exceção e por que a tolerância de compressão é o erro máximo da série reconstruída.
6. Uma variável de proteção e uma temperatura ambiente serão historizadas. Proponha taxa de amostragem, tolerância de compressão e retenção para cada uma, justificando.
7. Descreva uma política de retenção em três camadas para uma planta de saneamento, indicando o que se guarda em cada camada e por quanto tempo.
8. Qual a diferença entre histórico de processo, histórico de eventos e trilha de auditoria? Dê um exemplo de pergunta que só a trilha de auditoria responde.
9. Um operador afirma que "a pressão nunca passou de 8 bar" e a engenharia mede 11 bar com um registrador independente. Que decisões de configuração do historian podem explicar a divergência?

16.12 Referências

- MACHADO, R.; PONTES, W.; VIANNA, W. **Livro SCADA — versão para análise**. Material didático do curso (seções sobre gráficos de tendência real e histórica, parâmetros de configuração, relatórios e esvaziamento de arquivos de registro). Figuras reproduzidas com crédito.
- VIANNA, W. S. **Sistema SCADA Supervisório**. IFF, 2008 (seções sobre gráficos de tendência e gerador de relatórios).
- INFLUXDATA. **InfluxDB Documentation** — modelo de dados, políticas de retenção e tarefas de agregação contínua. Disponível em: docs.influxdata.com. Consulta: 2026.
- TIMESCALE. **TimescaleDB Documentation** — hipertabelas, compressão e agregação contínua sobre PostgreSQL. Disponível em: docs.timescale.com. Consulta: 2026.
- GRAFANA LABS. **Grafana Documentation** — consulta a séries temporais, agregação na consulta e painéis. Disponível em: `grafana.com/docs`. Consulta: 2026.
- OPC FOUNDATION. **OPC UA Part 11 — Historical Access**: leitura de histórico no próprio servidor (`HistoryRead`). Disponível em: opcfoundation.org. Consulta: 2026.

- AVEVA. **PI System** — visão geral de historização industrial e compressão de séries temporais. Disponível em: aveva.com. Consulta: 2026 (citado por função, sem comparativo de produto).

Capítulo 17

Edge Computing na Indústria

Objetivos de Aprendizagem

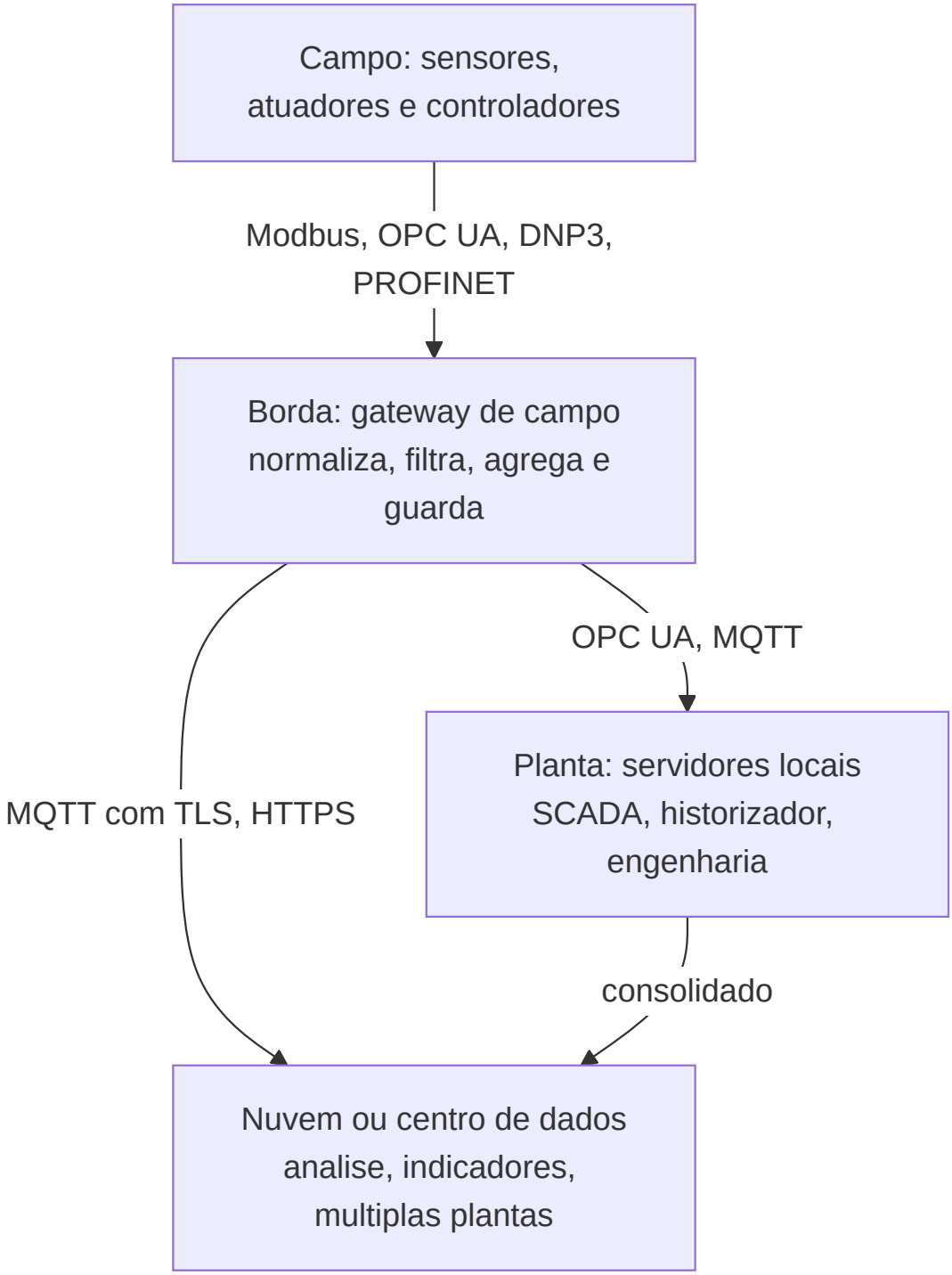
Ao final deste capítulo, o leitor será capaz de:

- Explicar por que nem todo dado industrial deve ser enviado integralmente à nuvem.
- Descrever as funções de uma borda de campo e o que acontece quando cada uma falta.
- Dimensionar *store and forward*, agregação e filtro na borda.
- Reconhecer os limites do que se deve decidir na borda — e o que nunca deve sair do controlador.
- Projetar a operação de uma frota de gateways: provisionamento, atualização e monitoramento.

17.1 Por que a borda existe

A pergunta que cria a borda não é tecnológica, é econômica: **vale transportar este dado?** Enviar todas as variáveis de uma planta, na resolução do controlador, para um servidor distante custa enlace, armazenamento e energia — e a maior parte desse volume nunca é consultada. Ao mesmo tempo, há exigências que a nuvem não atende: latência previsível, funcionamento durante a queda do enlace e residência do dado dentro da planta.

Computação na borda (*edge computing*) é o processamento feito próximo à fonte — no próprio dispositivo ou em um equipamento de campo —, em vez de no servidor central ou na nuvem. O critério de projeto não é "onde é moderno processar", e sim qual decisão precisa ser tomada **onde** e **em quanto tempo**.



Três forças explicam a borda industrial:

1. **Economia de enlace.** Publicar por exceção em vez de publicar por período reduz o volume em uma ordem de grandeza em variáveis estáveis — e enlace 4G com franquia é custo recorrente.
2. **Continuidade.** O processo não pode parar porque o enlace caiu. A borda guarda o dado e o entrega quando a comunicação volta, sem perder a série.
3. **Escala de protocolos.** Cada equipamento fala um dialeto. Concentrar a tradução em um ponto reduz o número de integrações de $n \times m$ para $n + m$ (Capítulos 5 e 13).

Nota

Nota: "borda" e "névoa" (*fog*) são usados como sinônimos na prática. Quando se distingue, *edge* é o processamento no próprio dispositivo ou no gateway imediatamente ao lado dele, e *fog* é a camada intermediária da planta — servidores locais que agregam vários gateways antes da nuvem. Para efeito de projeto, o que importa é a hierarquia de agregação, não o nome da camada.

17.2 O que uma borda faz — e o que acontece quando falta

Uma borda de campo acumula três funções distintas. Tratá-las como uma só é a origem da maioria dos projetos frustrados.

Tabela 17.1 – As três funções da borda

Função	O que faz	Sintoma da ausência
Aquisição e normalização	Conversa com o campo por vários protocolos, escala, valida e publica em um modelo único	Cada sistema integra com o campo do seu jeito; nomes e unidades divergem
Continuidade (*store and forward*)	Guarda o dado em fila local quando o enlace cai e reenvia depois, preservando o carimbo de origem	Buraco na série temporal exatamente nos períodos de problema — quando o dado mais importava
Decisão local	Filtra, agrega, detecta condição e dispara ação local sem depender de ida e volta à nuvem	Latência e indisponibilidade da nuvem contaminam a operação da planta

Atenção

Atenção: a terceira função tem limite claro. **Proteção, intertravamento e segurança funcional permanecem no controlador** — nunca na borda, nunca na nuvem. Uma parada de emergência que depende de rede, de broker ou de servidor remoto é um acidente esperando a hora certa. A borda pode decidir publicar, agregar e até ajustar um *setpoint* dentro de limites já validados; ela não executa a função de segurança do processo.

17.3 O gateway na prática

O gateway é o equipamento que materializa a borda. Suas duas faces têm nomes próprios:

- **Face sul** (*southbound*): conversa com o campo — Modbus RTU/TCP, OPC UA, DNP3, IEC 60870-5-104, PROFINET, entradas digitais e analógicas locais. É onde

está o **mapa de pontos**: a relação entre o endereço no equipamento e o nome publicado.
- **Face norte** (*northbound*): publica para cima — MQTT com TLS, HTTP/REST, OPC UA, Sparkplug B. É onde se define a **taxa de publicação** e a **faixa morta** por ponto.

O mapa de pontos da borda é o primo do mapa de memória do Capítulo 14, e sofre do mesmo mal: quando não é documentado, ninguém sabe de onde veio o número que aparece no painel. A diferença é que, na borda, o modelo de dados pode ser **descoberto** (OPC UA) em vez de digitado, o que diminui o erro de digitação mas não elimina a necessidade de revisão.

Figura

[**Figura 17.1 – Painel de configuração de um gateway de campo**] *Captura de tela de um gateway com duas abas visíveis: à esquerda, a lista de conexões face sul (duas conexões Modbus TCP e uma OPC UA, com endereço, porta e estado da conexão); à direita, a tabela de pontos com colunas nome publicado, endereço de origem, tipo, unidade, faixa, intervalo de leitura e faixa morta. Destacar o indicador de fila local (itens pendentes) e o carimbo de tempo da última publicação bem-sucedida.*

17.3.1 Store and forward: o teste que separa projeto de improviso

Guardar dados quando o enlace cai parece simples e não é. Quatro decisões definem se a fila local serve para alguma coisa:

- **Volume da fila.** Quanto tempo de dado cabe em caso de queda prolongada? Uma semana de dado agregado é viável em memória local; uma semana de dado bruto, raramente.
- **Carimbo de tempo.** O dado entra na fila com o **carimbo de origem**, não com o horário da chegada (Capítulo 14). Sem isso, a série reenviada mente sobre quando as coisas aconteceram.
- **Política de descarte.** Se a fila enche, o que se perde: o mais antigo ou o mais novo? A resposta depende do uso — para diagnóstico, o recente importa mais; para relatório de conformidade, a sequência completa importa.
- **Proteção da memória.** Fila em cartão sujeito a desgaste por escrita exige cuidado com frequência de gravação e com o que se grava.

Dica

Dica: o teste de aceitação de qualquer borda é simples e quase nunca é feito. Desligue o enlace por duas horas, com o processo em operação. Quando o enlace voltar, verifique três coisas: se o dado chegou, se chegou com o carimbo correto e se a plataforma o aceitou como histórico — e não como se tudo tivesse acontecido no instante da reconexão.

Figura

[**Figura 17.2 – Comportamento da fila local durante a queda e o restabelecimento do enlace**] *Gráfico de duas séries no mesmo eixo de tempo: a vazão medida*

no processo (linha contínua) e o número de itens pendentes na fila do gateway (linha tracejada). No trecho de enlace ativo a fila fica em zero; no início da queda a fila cresce em rampa; ao final, cai até zero em um patamar diferente do de subida, mostrando o dreno controlado. Marcar a área do gráfico com o rótulo "sem perda de dado" e anotar que a série recuperada mantém o carimbo de origem.

17.4 Reduzir na borda: filtro, agregação e evento

O ganho da borda aparece quando ela **reduz** o dado antes de publicá-lo. As técnicas são as mesmas do Capítulo 15, aplicadas na publicação:

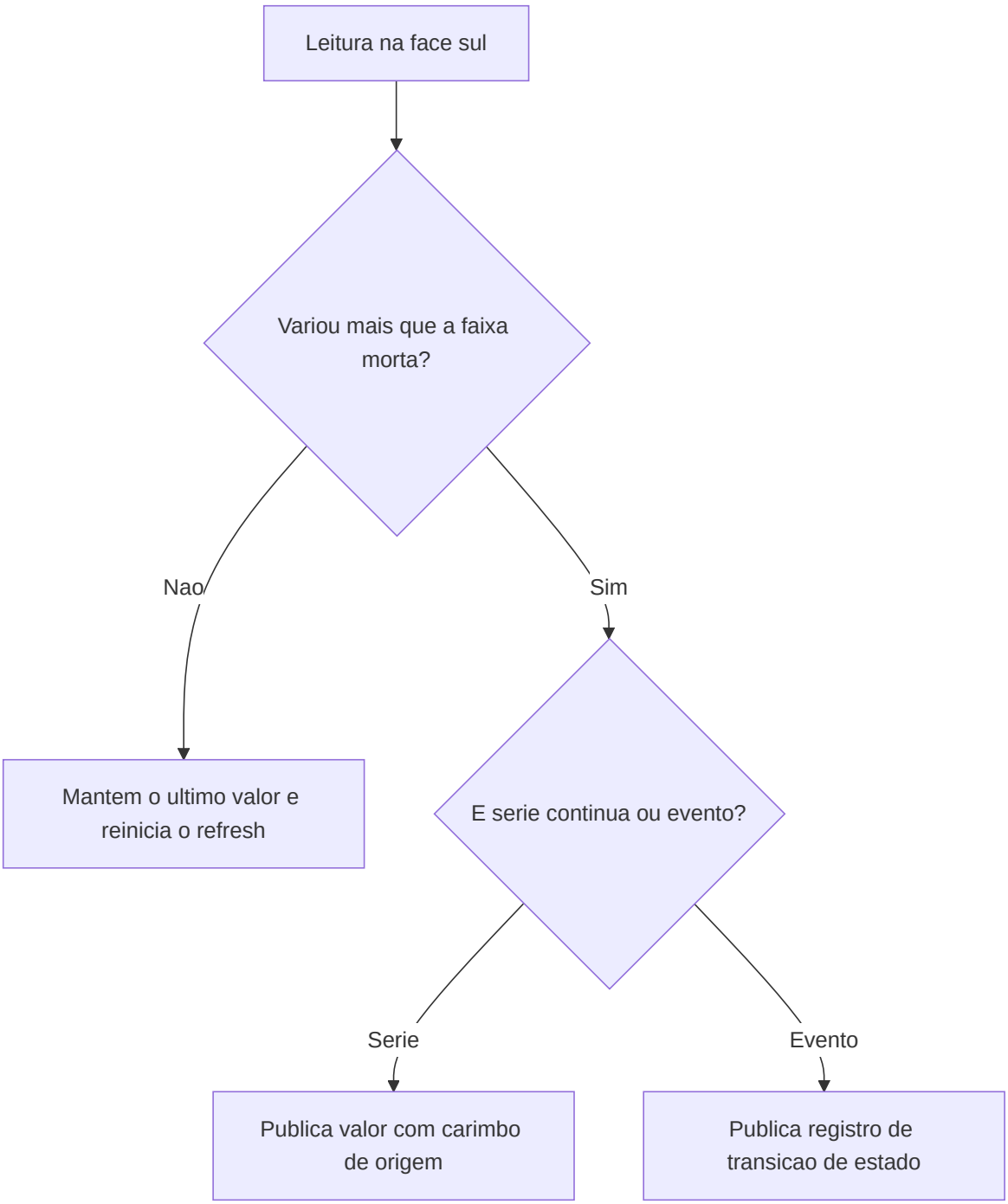


Tabela 17.2 – Estratégias de redução na borda

Estratégia	Como funciona	Ganho típico	Risco
Publicação por exceção	Só publica quando o valor varia mais que a faixa morta	Alto em sinais estáveis	Esconde a tendência lenta se a faixa morta for grande

Estratégia	Como funciona	Ganho típico	Risco
Refresh periódico	Publica o valor atual mesmo sem variação, a cada intervalo longo	Mantém a tela viva	Consumo fixo de enlace
Agregação local	Publica média, mínimo e máximo de um intervalo	Alto em variáveis rápidas	Perde o instante exato do pico (mitigado por mínimo/máximo)
Evento em vez de série	Publica um registro quando a condição ocorre, não a série que a originou	Muito alto	Perde o contexto da série anterior ao evento
Detecção local de condição	Compara com limites e publica somente a transição de estado	Alto, e reduz alarme espúrio	Exige que os limites estejam sincronizados com o SCADA

Atenção

Atenção: limite configurado na borda e limite configurado no SCADA precisam ser o **mesmo** número, e mudar um sem o outro é defeito garantido. A prática recomendada é que o limite venha do sistema de gestão como configuração (atributo compartilhado, no Capítulo 12) e seja apenas aplicado na borda — não digitado duas vezes.

17.5 Edge analytics: o que cabe e o que não cabe

Há um entusiasmo previsível em rodar análise preditiva no gateway. Vale separar o que é razoável do que é propaganda:

- **Cabe na borda:** filtros digitais, cálculo de indicadores simples (média, desvio, energia, horas de operação), detecção de condição por regra, comparação com *baseline* fixo, contagem de eventos. Tudo isso é aritmética leve, determinística e auditável.
- **Cabe com restrição:** inferência de modelos pequenos (classificação por árvore, regressão simples, detecção de anomalia por distância) — desde que a memória e o processamento disponíveis suportem, e desde que a atualização do modelo possa ser feita sem parar a aquisição.
- **Não cabe bem na borda:** treinamento de modelo, análise de séries longas, correlação entre ativos, otimização global de processo. Essas tarefas precisam de volume e de histórico — e o histórico consolidado vive na planta ou na nuvem.

Nota

Atualização tecnológica: a forma atual de empacotar essa lógica na borda é a **contêiner** — a aplicação de aquisição, o motor de regras e o modelo são imagens versionadas, atualizáveis sem trocar o sistema operacional do gateway. Isso resolve o problema histórico de "não posso atualizar o gateway porque ele tem outras coisas rodando" e permite que a mesma automação de integração contínua usada em software

corporativo chegue à borda com segurança.

17.6 Operar uma frota de gateways

Um gateway isolado é fácil. Cinquenta gateways espalhados por uma cidade são um problema de operação, não de eletrônica. O que muda:

Tabela 17.3 – Requisitos de operação em escala

Requisito	Prática
Identidade por dispositivo	Um certificado por gateway, nunca um certificado compartilhado entre a frota
Provisionamento repetível	Configuração versionada e aplicada em lote; gateway novo entra em serviço por *script*, não por digitação
Atualização de firmware e de aplicação	Janela planejada, com *rollback* possível e sem interromper a aquisição
Monitoramento da própria borda	O gateway publica o seu próprio estado: fila, uso de CPU e memória, tempo desde a última coleta, erros de protocolo
Watchdog e recuperação	Reinício automático de serviço travado; registro do motivo do reinício
Consolidação de alarmes da borda	Queda de gateway é evento de primeira ordem, não ruído de rede

O item mais ignorado é o monitoramento da própria borda. Sem ele, a queda de um gateway se manifesta como **ausência de dado** — e ausência de dado não gera alarme. A detecção precisa de um mecanismo ativo: ou o gateway publica periodicamente o seu estado (o `NDEATH` e o `NBIRTH` do Sparkplug B existem para isso, no Capítulo 13), ou o servidor verifica o silêncio e conclui a queda.

Figura

[**Figura 17.3 – Painel de monitoramento da frota de gateways**] *Captura de painel com uma tabela de 12 gateways e colunas de identificador, última comunicação, estado da fila, uso de memória, versão de firmware e número de reinícios nas últimas 24 h. Linhas em destaque para dois gateways: um com fila acima do limite de atenção e outro sem comunicação há 40 minutos. Ao lado, um indicador do tipo semáforo com a contagem por estado (em operação, atenção, offline).*

Dica

Dica: trate o par *last will* + publicação periódica de estado como requisito obrigatório de projeto. É a diferença entre "o dado parou de chegar" e "o gateway 37 caiu às 14h32, a fila tinha 1.204 itens e a causa provável é o cartão de memória". A primeira frase não permite ação; a segunda, sim.

17.7 Segurança e a borda

A borda aumenta a superfície de ataque: é um computador com sistema operacional, rede, acesso a dispositivos industriais e conexão para fora (Capítulo 8). O que a torna administrável:

- **Sem exposição direta.** O gateway **inicia** as conexões para fora; ninguém entra. Interface de administração em rede segregada, nunca publicada.
- **Certificado por dispositivo** e verificação do certificado do servidor.
- **Contas individuais** para administração, com registro de quem alterou o quê.
- **Serviços mínimos.** Cada serviço ligado é uma porta a defender; se não é usado, desligue.
- **Atualização como processo.** Firmware e aplicação com versionamento, homologação em um gateway piloto e propagação por ondas.

17.8 Estudo de Caso — A borda que não guardava

Cenário. Um sistema de telemetria de 40 unidades consumidoras de água. Cada unidade tem um *gateway* com modem 4G, lendo um medidor de vazão por pulso e a pressão da linha. A empresa de saneamento precisava faturar o volume mensal e detectar vazamentos por comparação entre período noturno e diurno.

Diagnóstico. Depois de três meses, a conciliação acusou divergência de até 18 % entre o volume faturado e o volume real medido em um lote piloto. A investigação mostrou:

1. O gateway não tinha fila local. Em cada queda de sinal 4G — frequente à noite — os pulsos contados durante a queda eram perdidos.
2. O carimbo de tempo era atribuído pelo servidor no instante do recebimento. Quando o enlace voltava, várias horas de leitura apareciam concentradas em minutos, o que falseava a análise de consumo noturno.
3. A publicação era por período, a cada 60 s, o que não capturava pulsos em rajada.

Decisão de engenharia.

Medida	Efeito
Contagem de pulsos acumulada no próprio gateway, com fila local em memória não volátil	Nenhum pulso é perdido durante a queda
Carimbo de tempo na origem, por pulso e por leitura	A série volta a refletir o horário real
Publicação por exceção com *refresh* a cada 15 min	Redução de cerca de 80 % no consumo do enlace
Monitoramento do estado do gateway e alarme de queda	A falha passa a ser vista em minutos, não no fechamento do mês

Resultado. A divergência caiu para menos de 1 %, compatível com o erro do medidor. A detecção de vazamento noturno passou a ser possível porque a série voltou a ser contínua — o que nunca dependeu de inteligência artificial, e sim de a borda ter guardado o dado.

Limitações do arranjo. A fila local tem capacidade finita: uma queda de enlace de muitos dias ainda perde dado, e por isso a política de descarte foi definida — prioriza-se o mais recente. Além disso, o *setpoint* de pressão continua exclusivamente no controlador local; a nuvem apenas observa.

17.9 Resumo

- A borda existe por **economia de enlace, continuidade e escala de protocolos** — não por moda.
- Uma borda tem três funções: aquisição e normalização, continuidade e decisão local. Faltando qualquer uma, o sintoma aparece longe dela.
- **Proteção e intertravamento permanecem no controlador**; a borda observa, filtra, agrega e publica.
- O **mapa de pontos** da borda é tão crítico quanto o mapa de memória do supervisório.
- *Store and forward* só funciona com carimbo de origem, política de descarte definida e capacidade dimensionada.
- Reduzir na borda (por exceção, por agregação, por evento) é o que torna a telemetria sustentável.
- **Limites configurados na borda e no SCADA precisam ser o mesmo número**, gerido em um só lugar.
- Frota de gateways exige identidade por dispositivo, provisionamento repetível, atualização com *rollback* e **monitoramento da própria borda**.
- O teste de aceitação é desligar o enlace por horas e verificar se o dado volta com o carimbo certo.

17.10 Questões de Revisão

1. Explique, com um exemplo numérico de volume de dados, por que nem todo sinal de campo deve ser publicado na nuvem.
2. Quais são as três funções de uma borda de campo e qual o sintoma da ausência de cada uma?
3. Por que proteção e intertravamento não podem ser implementados na borda ou na nuvem?
4. Descreva o papel da face sul e da face norte de um gateway, citando dois protocolos de cada lado.
5. Um gateway publica a cada 60 s e o enlace cai por 8 horas. Descreva o que precisa existir para que nenhum dado se perca e para que o dado chegue com o horário correto.
6. Compare publicação por exceção, agregação local e publicação por evento quanto a ganho de banda e risco de perda de informação.
7. O mesmo limite de pressão está configurado com 6,0 bar no gateway e 6,2 bar no SCADA. Descreva as consequências e proponha a correção.
8. Por que "ausência de dado" não gera alarme por si só? Que mecanismo resolve isso?
9. Liste cinco requisitos de operação de uma frota de 50 gateways e explique o risco de ignorar cada um.

17.11 Referências

- INTERNATIONAL SOCIETY OF AUTOMATION. **ISA-95 / IEC 62264 — Enterprise-control system integration** (níveis hierárquicos de integração; verificar edição vigente).
- VIANNA, W. S. **IIoT e suas Tecnologias Aderentes: Teoria e exemplos práticos**. Material didático do curso (tecnologias aderentes à IIoT, incluindo computação na borda e na nuvem).
- VIANNA, W. S. **Introdução a IoT**. Material didático do curso (conectividade, plataformas de nuvem e análise avançada).
- ECLIPSE FOUNDATION. **Sparkplug Specification 3.0** — NBIRTH, NDATA e NDEATH como mecanismo de detecção de queda de nó de borda. Disponível em: `eclipse.org/tahu`. Consulta: 2026.
- THINGSBOARD. **IoT Gateway** — arquitetura de gateway, conectores de face sul e mapeamento de pontos. Disponível em: `thingsboard.io/docs/iot-gateway`. Consulta: 2026.
- OPC FOUNDATION. **OPC UA** — informação de modelo, *subscription* e PubSub aplicados à borda. Disponível em: opcfoundation.org. Consulta: 2026.
- INTERNATIONAL ELECTROTECHNICAL COMMISSION. **IEC 62443** — requisitos de segurança para sistemas de automação e controle (zonas, conduítes e níveis de segurança; verificar edição das partes vigentes).

Capítulo 18

Integração SCADA + CLP + IoT + Cloud

Objetivos de Aprendizagem

Ao final deste capítulo, o leitor será capaz de:

- Desenhar uma arquitetura de referência que integre controlador, supervisão, borda e nuvem.
- Decidir em qual camada cada função deve morar e justificar a escolha.
- Montar a sequência de dados de integrações típicas, do sensor ao painel.
- Reconhecer as falhas de integração mais comuns e o teste que as revela.
- Aplicar a regra que separa supervisão de comando em arquiteturas híbridas.

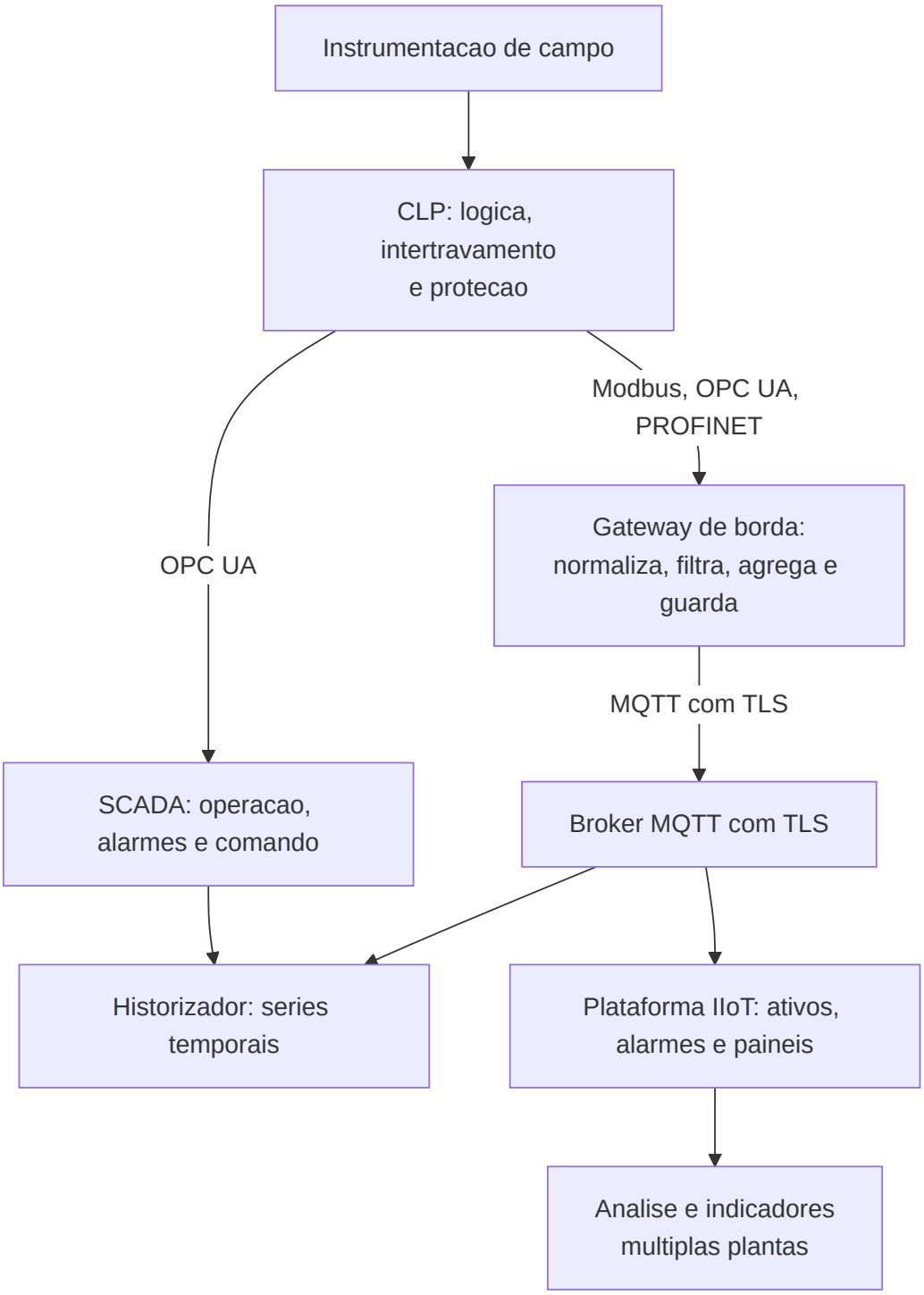
18.1 Quatro mundos, um dado só

O CLP, o SCADA, o mundo IoT e a nuvem têm requisitos que não se parecem:

Tabela 18.1 – Requisitos que precisam conviver

Camada	Precisa de	Não tolera	Horizonte de tempo
CLP	Determinismo e disponibilidade	Depender de rede ou servidor para proteger	Milissegundos
SCADA	Estado atual confiável e comando auditável	Valor mentiroso na tela	Décimos de segundo a segundos
Borda	Reduzir, normalizar e guardar	Perder dado quando o enlace cai	Segundos a horas
Nuvem	Volume, histórico e visão de conjunto	Ser caminho crítico do processo	Minutos a meses

A integração dá certo quando cada camada faz o que sabe fazer, e não quando uma tenta substituir a outra. O Capítulo 6 já estabeleceu que IoT/IIoT não substituem SCADA; aqui a discussão é de projeto: **onde colocar cada peça**.



18.2 Onde cada decisão deve morar

Esta é a tabela que evita a maior parte dos erros de arquitetura. Ela é curta de propósito.

Tabela 18.2 – Alocação de funções por camada

Função	Camada correta	Por quê
Intertravamento, proteção, parada de emergência	CLP (ou SIS)	Precisa de determinismo e de independência de rede
Sequência de partida e controle regulatório	CLP	Ciclo de varredura adequado e disponibilidade local
Supervisão, comando do operador e alarmes de processo	SCADA	Interface, gestão de alarmes e trilha de auditoria (Capítulos 10 e 11)
Sincronização de nomes, unidades e faixas	Uma única fonte, aplicada às demais	Evita o mesmo dado com dois significados
Redução, agregação e *store and forward*	Borda	É onde o dado é volumoso e o enlace é caro (Capítulo 17)
Histórico de alta resolução	Historizador junto ao processo	Consulta rápida e resolução preservada (Capítulo 16)
Histórico longo, indicadores e análise entre plantas	Nuvem	Volume, retenção e visão consolidada
Detecção de anomalia em série longa e modelos	Nuvem ou planta	Exige histórico consolidado (Capítulo 20)

A regra que não se negocia: o **comando** do processo nasce no SCADA ou na IHM local, por protocolo autenticado, com trilha de auditoria. A nuvem **observa**. Quando o cliente pede "quero ligar a bomba pelo celular", a resposta de engenharia é: você pode, se o caminho for autenticado, com autorização por papel, com confirmação de estado de volta e com registro de quem comandou — e mesmo assim só para comandos que não afetem segurança.

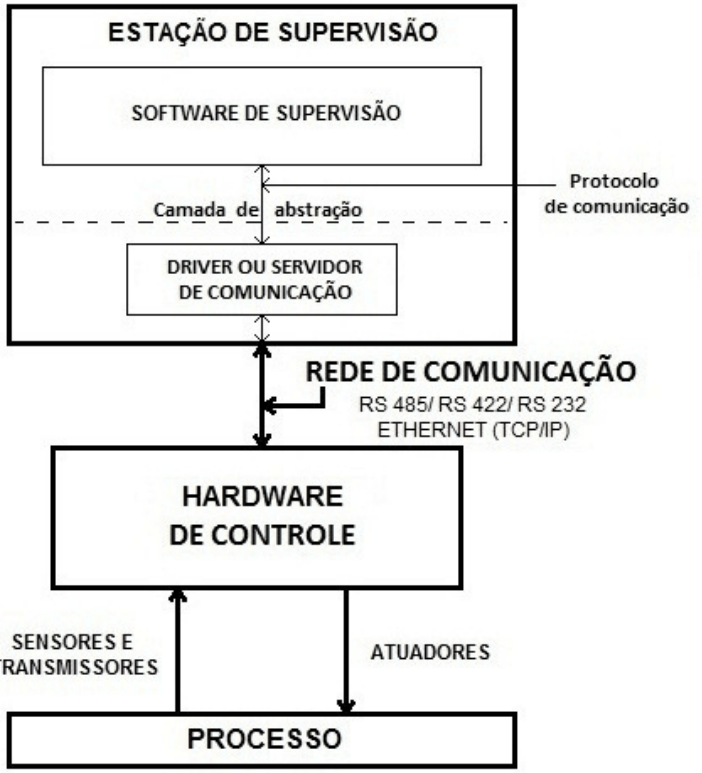


Figura 18.1 – Camada de abstração: onde o software de supervisão se separa do hardware de controle. **Fonte:** *Livro SCADA — versão para análise*, p. 24 (Machado, Pontes e Vianna, reproduzida com crédito).

A figura mostra o conceito que sustenta toda a integração: a **camada de abstração** permite implementar a IHM sem conhecer a tecnologia do hardware de controle, e o **driver** é a peça que pode ser trocada sem reescrever a aplicação. É por isso que o servidor de comunicação pode agregar mais de um driver, para hardware e interfaces diferentes, sem que o software de supervisão precise saber como cada um funciona.

18.3 A sequência de dados, passo a passo

Integração se aprende seguindo o dado. Os cinco arranjos abaixo são progressivos e cada um acrescenta uma peça; todos são usados em plantas reais.

18.3.1 Válvula solenoide comandada por CLP

Sequência: o operador aciona o botão na IHM → o SCADA escreve 1 no tag de comando → o driver grava na memória imagem do CLP → a lógica energiza a saída → a solenoide abre → o sensor de fim de curso confirma → o CLP atualiza a entrada → o driver lê → o tag de estado muda → a IHM anima o objeto.

Configuração mínima: dois tags (comando e estado), faixa morta zero no estado, tempo de retorno esperado.

O que observar no resultado: o atraso entre comandar e ver o estado; se a confirmação não chega no tempo esperado, deve existir alarme de falha de atuação — e não um botão que "não fez nada".

Limitação: o comando pelo supervisório não é proteção. Se a válvula precisa fechar por segurança, quem fecha é o intertravamento, não a IHM.

18.3.2 CLP → SCADA

Sequência: leitura da memória do CLP pelo driver → escalonamento para unidade de engenharia → validação de faixa → avaliação de alarme → atualização do objeto na tela → gravação no historizador.

Configuração mínima: classe de varredura por grupo de tags, faixa e unidade no cadastro, limites de alarme documentados, tendência real com janela dimensionada.

O que observar: a coerência entre o valor exibido e o valor lido localmente no CLP, com o mesmo instrumento e no mesmo instante.

Limitação: o SCADA é não determinístico — o atraso da tela não é erro, mas precisa ser conhecido (Capítulo 15).

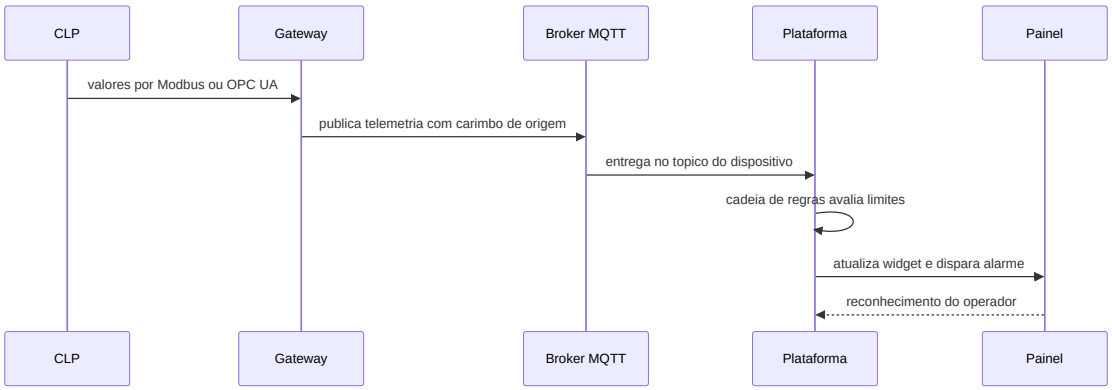
18.3.3 CLP → MQTT → plataforma IIoT

Sequência: a função de cliente MQTT no CLP ou no gateway publica no tópico de telemetria com o identificador do dispositivo → a plataforma recebe, registra a telemetria e dispara a cadeia de regras → a regra cria ou limpa alarme → o painel é atualizado.

Configuração mínima: identificador do dispositivo e credencial por dispositivo, estrutura de tópico, formato do *payload* e carimbo de tempo, além da regra que distingue alarme de registro.

O que observar: o tempo entre a mudança no processo e a atualização do painel; o que acontece com um valor fora da faixa no motor de alarme.

Limitação: a plataforma não executa controle. Ela observa, registra e notifica (Capítulo 12).



18.3.4 Sensor → Node-RED → MQTT → plataforma

Sequência: Node-RED lê a fonte (Modbus, serial, HTTP ou simulador) → normaliza nome e unidade → injeta o carimbo de tempo → publica em MQTT → a plataforma persiste e apresenta.

Configuração mínima: nó de leitura com *timeout*, tratamento de erro explícito, injeção de carimbo de tempo e retenção de última mensagem desativada para dado temporal.

O que observar: o comportamento com a fonte fora do ar — o fluxo precisa **não publicar** valor antigo como se fosse atual.

Limitação: Node-RED é excelente para prototipar e mediar, e não é ambiente de controle nem de garantia temporal (Capítulo 7).

18.3.5 SCADA + historiador + dashboard

Sequência: o SCADA coleta e disponibiliza por OPC UA → o historiador assina as variáveis com intervalo de amostragem e faixa morta definidos → o dashboard consulta o histórico por período e agregação.

Configuração mínima: o que é gravado, com que resolução e por quanto tempo (Capítulo 16); a consulta do painel com agregação explícita.

O que observar: a diferença entre o valor lido na tela do SCADA e o ponto mais próximo no histórico — se for grande, a faixa morta ou o filtro estão escondendo o transitório.

Limitação: painel não é sistema de operação. Ninguém deve comandar o processo por dashboard.

Figura

[**Figura 18.2 – Caminho completo do dado, de ponta a ponta**] *Diagrama em faixas horizontais, no estilo de arquitetura de referência: no alto a instrumentação; abaixo o CLP com a lógica e o intertravamento destacados; em seguida a borda com as duas faces e a fila local; depois, em paralelo, o SCADA com estação de operação e o historizador; por último o broker, a plataforma e a análise. Sobre as setas, anotar protocolo e sentido; nas laterais, marcar com cadeado os trechos cifrados e anotar o carimbo de tempo em cada troca.*

18.4 Onde a integração costuma quebrar

Nenhuma dessas falhas é de protocolo. Todas são de projeto, e todas aparecem no mesmo lugar: no número que está errado na tela de alguém.

Tabela 18.3 – Falhas típicas e sua origem

Sintoma	Causa provável	Prevenção
O mesmo ativo tem três nomes diferentes	Nenhuma fonte única de nomes	Dicionário de dados único, aplicado às três camadas
Valor plausível e errado	Faixa ou unidade divergente entre CLP, cadastro e plataforma	Um cadastro por variável, revisado na partida
Sensor com defeito aparece como zero	Qualidade do dado descartada no caminho	Propagar `StatusCode` até a tela (Capítulos 14 e 15)
Ordem dos eventos não fecha	Carimbo atribuído em pontos diferentes da cadeia	Carimbo na origem e relógios sincronizados
Buraco no histórico em dia de problema	Ausência de fila local ou de *store and forward*	Fila dimensionada e testada com o enlace desligado

Sintoma	Causa provável	Prevenção
Alarmes duplicados no SCADA e na plataforma	Mesma condição avaliada em duas camadas, com limites diferentes	Um lugar avalia, o outro registra
Consumo de dados estourou a franquia	Publicação periódica de tudo	Publicação por exceção com *refresh* (Capítulo 17)

18.5 Como provar que a integração funciona

Integração não se valida lendo configuração; valida-se **injetando perturbação**. O roteiro mínimo:

1. **Valor conhecido.** Injete um valor de teste no instrumento ou simule a fonte e siga o número por todas as camadas até o painel, conferindo unidade, faixa e casas decimais.
2. **Corte de enlace.** Desligue a comunicação por tempo significativo e verifique se o dado volta completo e com o carimbo correto.
3. **Reinício de nó.** Reinicie o gateway, o broker e a estação, um por vez, e observe o que cada reinício faz com a série e com os alarmes.
4. **Valor fora de faixa.** Envie um valor fisicamente impossível e verifique se ele é rejeitado com qualidade ruim em vez de alarmar o operador.
5. **Mudança de horário.** Desloque o relógio de um nó e verifique se a divergência é detectada.
6. **Desligamento da nuvem.** Deixe a plataforma fora do ar e confirme que o SCADA e o CLP continuam operando normalmente.

Dica

Dica: transforme esses seis testes em um procedimento de aceitação com registro de resultado e data. Um sistema integrado sem esse registro é um sistema que ninguém sabe se está funcionando — só se sabe que ainda não falhou de forma visível. [**Figura 18.3 – Registro do teste de corte de enlace**] *Captura de uma tabela de procedimento de aceitação preenchida: colunas de item testado, condição imposta (enlace desligado por 2 h, entre 09h00 e 11h00), resultado esperado, resultado observado e responsável. Duas linhas em destaque: "dado recuperado com carimbo de origem" e "nenhum registro gravado com horário de reconexão". Ao lado, o gráfico da série temporal mostrando o trecho sem recepção e o preenchimento posterior da curva no intervalo correto. —*

18.6 Estudo de Caso — A integração que duplicou os alarmes

Cenário. Uma indústria de alimentos integrou o SCADA existente a uma plataforma IIoT para dar visão de indicadores à gestão. Seis meses depois, a plataforma foi liberada também para a supervisão móvel dos coordenadores de turno, com notificação por mensagem.

Diagnóstico. No primeiro mês de uso, a operação reclamou de "alarme repetido no celular sem nada de errado na tela do SCADA". A análise mostrou três problemas sobrepostos:

1. A plataforma avaliava limites próprios, digitados na configuração dos dispositivos, diferentes dos limites do SCADA — o alarme de temperatura alta da plataforma disparava 2 °C antes.
2. Não havia faixa morta na plataforma, e a variável oscilava em torno do limite: o celular recebia dezenas de mensagens por turno.
3. Os alarmes da plataforma não eram reconhecidos por ninguém: o operador reconhecia no SCADA, e a plataforma continuava notificando.

Decisão de engenharia.

Problema	Medida
Limites divergentes	Limites passaram a ser atributos de configuração, geridos em um só lugar e distribuídos para SCADA e plataforma
Alarme espúrio	Faixa morta e atraso de confirmação configurados na plataforma, iguais aos do SCADA
Reconhecimento sem efeito	Reconhecimento unificado: quem reconhece no SCADA encerra a notificação na plataforma
Papel indefinido	Notificação móvel restrita a alarmes de prioridade alta e de equipamento, com registro de quem recebeu

Resultado. As mensagens por turno caíram de algumas dezenas para menos de cinco, e voltaram a ser lidas. O ganho não veio de tecnologia nova: veio de decidir quem avalia o quê e de um único lugar para os limites.

Limitações do arranjo. O reconhecimento unificado exige um identificador comum de alarme nas duas camadas e não funciona com plataformas que geram o seu próprio identificador sem correlação. E a decisão de restringir a notificação por prioridade deixou a plataforma cega para alarmes de processo de prioridade média — o que foi aceito como decisão de projeto, não como efeito colateral.

18.7 Resumo

- Cada camada tem um **horizonte de tempo** próprio: milissegundos no CLP, segundos no SCADA, horas na borda, meses na nuvem.
- A alocação de funções é o que define o sucesso da integração, e não o protocolo escolhido.
- **Comando** nasce no SCADA ou na IHM local, com autenticação e trilha; a nuvem observa.

- Integração se aprende seguindo o dado: cada arranjo (CLP → SCADA, CLP → MQTT → plataforma, sensor → Node-RED → plataforma, SCADA + historian + dashboard) acrescenta uma peça.
- As falhas de integração são de **modelagem**: nomes divergentes, faixa errada, qualidade descartada, carimbo no lugar errado.
- O mesmo dado não deve ser avaliado em duas camadas com limites diferentes: **um lugar avalia, o outro registra**.
- Integração se valida **injetando perturbação** — corte de enlace, reinício, valor fora de faixa, desligamento da nuvem.
- Nenhuma dessas arquiteturas substitui CLP e SCADA no controle, na proteção e na segurança funcional.

18.8 Questões de Revisão

1. Explique por que o determinismo do CLP e a imprevisibilidade da nuvem exigem uma arquitetura em camadas, e não um único sistema.
2. Preencha a alocação de funções para um projeto de sua escolha, indicando em que camada ficam intertravamento, alarme de processo, agregação e análise preditiva.
3. Descreva a sequência completa de dados de um comando de abertura de válvula pela IHM, identificando onde pode haver atraso.
4. Por que a nuvem não deve comandar o processo? Cite duas exceções aceitáveis e as condições que as tornam seguras.
5. Um operador reclama que o valor de vazão no dashboard é 5 % menor que na tela do SCADA. Liste quatro causas possíveis e como verificar cada uma.
6. Descreva o que deve acontecer com a série temporal quando o enlace é desligado por quatro horas e restabelecido.
7. Explique por que a mesma condição avaliada no SCADA e na plataforma gera alarme duplicado, e proponha a correção.
8. Monte um roteiro de aceitação de integração com seis testes de perturbação e o resultado esperado em cada um.
9. Uma planta quer publicar 800 variáveis por MQTT a cada 5 s em enlace 4G com franquia de 20 GB mensais. Estime o volume, critique a proposta e apresente uma arquitetura alternativa.

18.9 Referências

- MACHADO, R.; PONTES, W.; VIANNA, W. **Livro SCADA — versão para análise**. Material didático do curso (arquitetura básica, servidor de comunicação, camada de abstração e sistema web server). Figura reproduzida com crédito.
- VIANNA, W. S. **Sistema SCADA Supervisório**. IFF, 2008 (arquiteturas típicas de sistemas SCADA, com CLP, fieldbus, controladores dedicados e controle digital direto; tecnologias web).
- INTERNATIONAL SOCIETY OF AUTOMATION. **ISA-95 / IEC 62264 — Enterprise-control system integration** (integração entre níveis de automação e de gestão; verificar edição vigente).
- OPC FOUNDATION. **OPC Unified Architecture** — cliente-servidor e PubSub aplicados à integração entre equipamento, supervisão e nuvem. Disponível em:

opcfoundation.org. Consulta: 2026.

- ECLIPSE FOUNDATION. **Sparkplug Specification 3.0** — convenção de tópicos e estados de sessão para integração IIoT. Disponível em: `eclipse.org/tahu`. Consulta: 2026.
- THINGSBOARD. **Documentação oficial** — telemetria, atributos, motor de regras, alarmes e RPC. Disponível em: `thingsboard.io/docs`. Consulta: 2026.
- GRAFANA LABS. **Grafana Documentation** — consulta a séries temporais e painéis de supervisão. Disponível em: `grafana.com/docs`. Consulta: 2026.

Capítulo 19

Projeto e Implantação de um Sistema Supervisório

Objetivos de Aprendizagem

Ao final deste capítulo, o leitor será capaz de:

- Descrever o ciclo de vida de um projeto SCADA e os produtos de cada fase.
- Executar o levantamento de dados que antecede qualquer tela: entendimento do processo, tomada de dados, banco de dados e lista de alarmes.
- Aplicar critérios de ergonomia ao projeto de IHM e justificar cada decisão.
- Planejar hierarquia de navegação, layout e padronização entre telas.
- Conduzir testes, aceitação, treinamento e manutenção com registro.

19.1 A parte difícil não é a tela

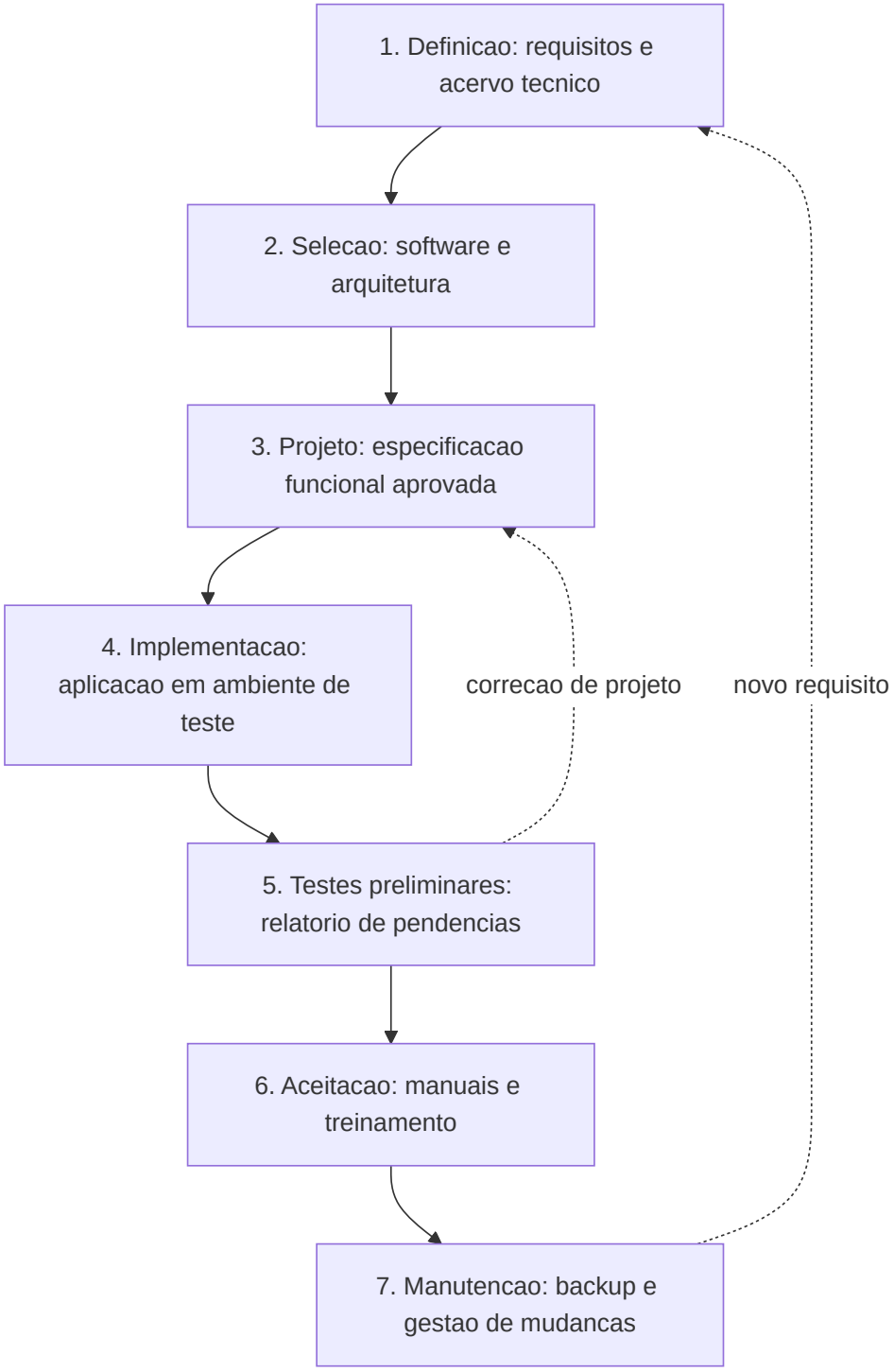
Um projeto de supervisório falha raramente por falta de recurso gráfico. Falha por **levantamento incompleto**: ninguém conversou com quem opera, a lista de variáveis foi montada por semelhança com outro projeto, o mapa de memória chegou depois da implementação e a lista de alarmes foi copiada de um sistema antigo. O resultado é uma IHM bonita que a operação não usa — ou pior, usa com desconfiança.

Este capítulo trata do que precisa ser feito **antes** de desenhar a primeira tela, e do que precisa ser feito depois de entregá-la.

19.2 O ciclo de vida em sete fases

Um projeto SCADA segue um ciclo com fases que se repetem, com inspeção ao final de cada uma. Os produtos de cada fase são o que permite saber se ela terminou.

Tabela 19.1 – Fases do ciclo de vida e produtos



Fase	O que se faz	Produto que encerra a fase
1. Definição	Entrevistas, coleta de dados, fluxogramas P&I, mapa de memória, requisitos de funcionamento e de interface	Documento de requisitos e acervo técnico recebido

Fase	O que se faz	Produto que encerra a fase
2. Seleção	Escolha do software e da arquitetura em função dos requisitos e dos recursos disponíveis	Matriz de decisão com justificativa
3. Projeto	Arquitetura física e lógica, padrões, lista de janelas, fluxograma de navegação, *layout*, mapa de tagnames, lista de alarmes	Especificação funcional aprovada
4. Implementação	Codificação das interfaces, com prototipação quando o escopo do cliente não está claro	Aplicação executável em ambiente de teste
5. Testes preliminares	Verificação de funcionalidades e de comunicação, em ambiente simulado quando não há produção disponível	Relatório de testes com pendências
6. Aceitação e implantação	Homologação pelo cliente, manuais de operação e treinamento da operação	Termo de aceitação e turma treinada
7. Manutenção	Correções pós-entrega e manutenção regular: cópia de segurança de configurações, dados e sistema	Rotina de *backup* e registro de mudanças

As fases não são estanques: uma decisão de projeto frequentemente obriga a voltar à definição. O que não pode acontecer é pular a fase 1 e descobrir na fase 6 que a lista de variáveis estava errada.

Nota

Atualização tecnológica: softwares de supervisão atuais trazem **edição colaborativa e controle de versão**, o que permite que dois ou mais desenvolvedores trabalhem na mesma aplicação sem corromper arquivos. Isso muda a fase 7: a manutenção deixa de ser "guardar uma cópia antes de mexer" e passa a ser ramo, revisão e registro — como em qualquer projeto de software. Se a ferramenta em uso não tem esse recurso, o *backup* datado antes de cada alteração continua obrigatório.

19.3 O levantamento que antecede a tela

O planejamento de uma IHM segue nove etapas. Elas não são burocracia: cada uma elimina uma classe de retrabalho.

Tabela 19.2 – As nove etapas do planejamento e o retrabalho que cada uma evita

Etapa	O que se define	Retrabalho evitado
1. Entendimento do processo	Divisão do processo em etapas nomeadas; o que realmente acontece na planta	Tela que não corresponde à operação real
2. Tomada de dados	Quais variáveis entram, e um limite superior para o número delas	Tráfego excessivo e sistema lento
3. Banco de dados	Nomes, tipos, contexto, grupos e classes de varredura	Nome ilegível e tag sem sentido, que arruína a manutenção futura
4. Alarmes	Condições de acionamento, prioridade, grupos, quem é notificado e qual ação se espera	Inundação de alarmes e perda de credibilidade (Capítulo 10)
5. Hierarquia de navegação	Qual tela abre qual, e por qual caminho	Operador perdido, telas que não se alcançam
6. Desenho das telas	Consistência, clareza, padronização de posições e símbolos	Reaprendizado a cada tela nova
7. Gráficos de tendência	Tendência real ou histórica, quais variáveis e com que escala	Tendência inútil, com dinâmicas incompatíveis na mesma escala
8. Acesso e segurança	Perfis, restrição de comandos, registro de acesso	Comando sem rastro; operador com permissão que não deveria ter
9. Padrão de desenvolvimento	Convenções do projeto, seguindo o ambiente predominante na equipe	Curva de aprendizagem desnecessária

Três documentos sustentam as etapas 2 e 3, e a ausência de qualquer um deles aparece mais tarde:

- **Fluxograma de processo e instrumentação (P&I)** — a fonte da verdade sobre o que existe em campo.
- **Mapa de memória ou tabela de alocação** — a relação entre o endereço no controlador e o tagname (Capítulo 14).
- **Lista de alarmes** — com causa, consequência, ação esperada e prioridade.

Figura

[**Figura 19.1 – Mapa de memória consolidado de um projeto**] *Captura de planilha de alocação com colunas TAG, descrição, tipo de dado, contexto (interno ou I/O), servidor de comunicação, endereço no controlador, faixa em unidade de engenharia, faixa bruta, unidade, classe de varredura e grupo. Linhas agrupadas por área da planta e uma coluna de observações com marcações de revisão e data. Destacar*

que a planilha é versionada e que cada linha tem responsável.

Atenção

Atenção: a tomada de dados tem **limite superior**. Cada variável acrescentada aumenta tráfego, tempo de varredura, tamanho do banco de tags, número de pontos de histórico e volume de alarmes possíveis. Um sistema com 40 % das variáveis que não são usadas por ninguém é mais lento, mais caro de licenciar e mais difícil de manter — e a culpa não é da ferramenta.

19.4 Ergonomia: o operador trabalha na tela

Ergonomia é o conjunto de disciplinas que estuda a organização do trabalho em que há interação entre seres humanos e máquinas. Aplicada à IHM, ela tem quatro metas práticas:

- **Reduzir a sobrecarga.** Um único sistema de unidades de engenharia, tipos de controle padronizados, código de cores e mensagens consistentes. Padronizar reduz erro de leitura e de parametrização.
- **Reduzir a monotonia.** Sinópticos representativos e dinâmicos, com o mínimo de dado tabular.
- **Evitar o cansaço.** Sem excesso de cores extravagantes, com intermitência só quando estritamente necessária e com som de alarme que não agrida.
- **Evitar excesso de informação.** O ser humano consegue processar da ordem de **quatro informações simultâneas**. Um sinóptico deve chamar a atenção para o que interessa — o resto é ruído.

Tabela 19.3 – Critérios práticos de desenvolvimento

Elemento	Critério
Sinóptico	Representa o processo de forma coerente; número de objetos compatível com a capacidade humana; contraste entre objetos, letras e fundo
Sistema gráfico	Resolução suficiente para leitura; uso de texturas e formas com parcimônia
Objetos estáticos	Forma próxima do equipamento real, sem excesso de detalhe; cores sóbrias; nada piscando; mesmo objeto representado do mesmo jeito em todas as telas
Objetos dinâmicos	Redundância na representação da variável (indicação numérica e barra); representação natural, como barra de enchimento para nível
Funções de operação	Ligar, desligar e alterar *setpoint* de forma simples e intuitiva

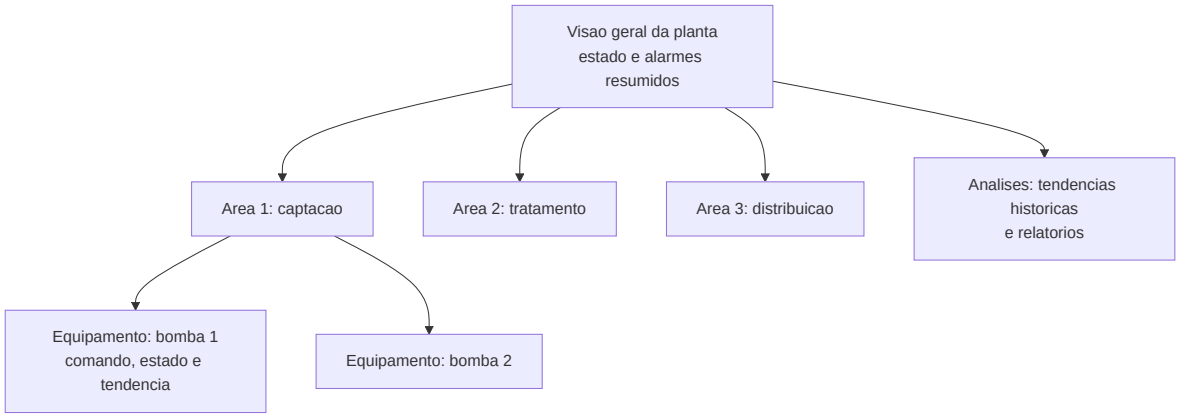
Elemento	Critério
Mensagens	Claras, explícitas e autossuficientes, indicando o que fazer

Dica

Dica: o teste da mensagem é simples. Leia apenas a mensagem, sem olhar a tela, e pergunte: dá para saber o que aconteceu e o que fazer? "Alarme 47" não passa; "TEMPERATURA MUITO ALTA NO RESISTOR 6D4: DESENERGIZE A LINHA 6D" passa.

19.5 Hierarquia de navegação e layout

A navegação de um supervisório se organiza em hierarquia, do geral para o particular, e a boa organização é a que guia o operador até o ponto certo em poucos cliques.



Três regras de projeto tornam essa navegação previsível:

1. **Barra de navegação fixa** (cabeçalho, rodapé ou lateral) presente em todas as telas, sempre no mesmo lugar, com os mesmos ícones.
2. **Navegação horizontal** entre subprocessos, pelas setas que acompanham a direção física do processo — o operador navega "seguindo o fluxo", como ele já pensa.
3. **Botões de retorno** para a tela anterior e para a visão geral, sem exceção.

Para o *layout*, a regra que resolve a maior parte dos problemas é **padronização por cópia**: cada tela nova começa como cópia da anterior, preservando títulos, posição dos nomes de tag e botões de navegação. Consistência vem de repetição, não de inspiração.

Atenção

Atenção: nunca troque a função de um botão entre telas. Se existe um botão de partida na posição inferior direita em uma tela, a posição inferior direita das outras telas não pode ser um botão de parada. O erro de clique é questão de tempo — e em planta, de consequência.

19.6 Implementação, testes e aceitação

Na implementação, o primeiro artefato é a **aplicação**, criada no gerenciador de aplicações do software de supervisão; o segundo, o conjunto de **janelas**, com propriedades definidas.

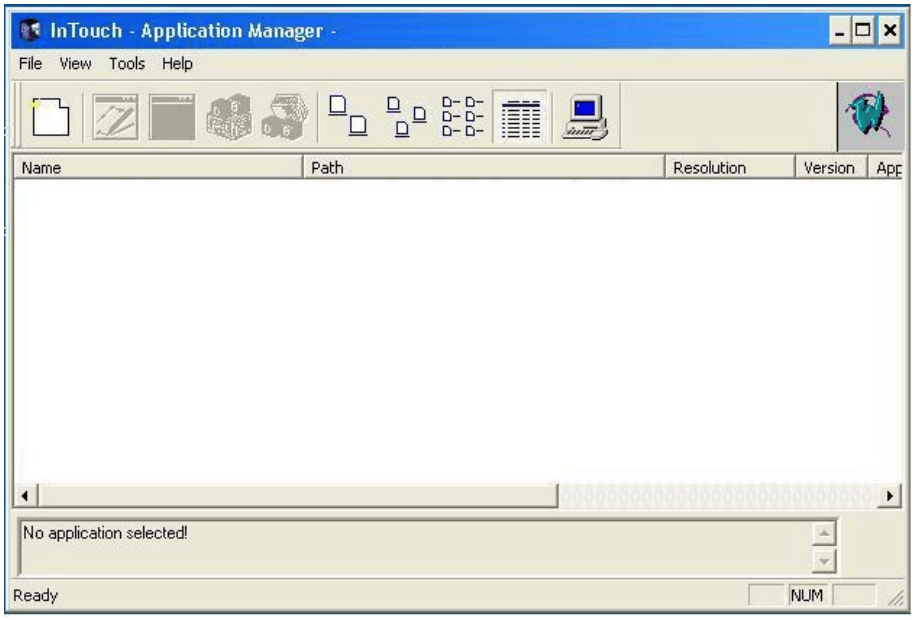


Figura 19.2 – Gerenciador de aplicações: criação e abertura de um projeto de supervisão. **Fonte:** *Livro SCADA — versão para análise*, p. 91 (Machado, Pontes e Vianna, reproduzida com crédito).

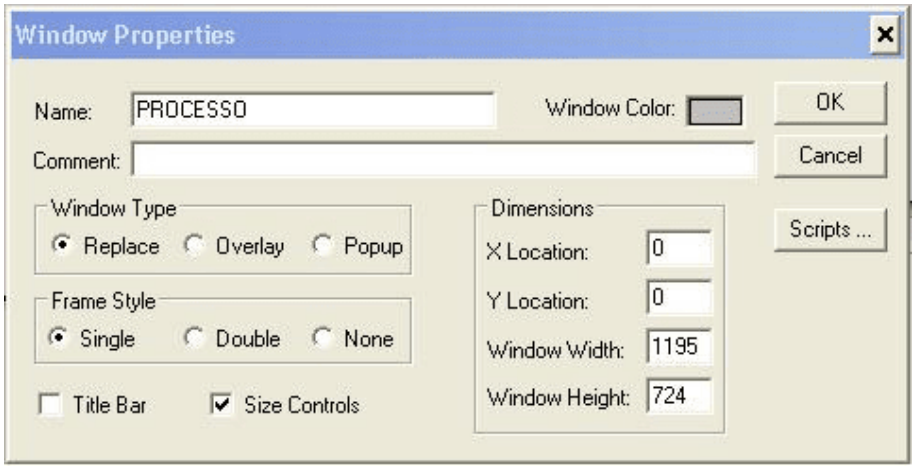


Figura 19.3 – Criação de uma janela e definição de suas propriedades. **Fonte:** *Livro SCADA — versão para análise*, p. 91 (Machado, Pontes e Vianna, reproduzida com crédito).

Depois da aplicação vir os testes — e testar não é abrir a tela e ver se ela abre:

Tabela 19.4 – Roteiro mínimo de testes

Teste	Como
Comunicação	Leitura e escrita em cada grupo de tags, com o hardware real ou simulador
Inicialização	Comportamento com o controlador fora do ar, com valor inválido e na primeira leitura

Teste	Como
Comandos	Cada comando, com confirmação de estado e registro de auditoria
Alarmes	Disparo, reconhecimento, histórico e filtros; verificar o tempo de resposta percebido
Navegação	Todas as telas alcançáveis, e nenhum caminho que leve a lugar nenhum
Tendências	Janela de tempo, escala e variáveis conforme o especificado
Perfis	Cada perfil enxerga e faz exatamente o que foi definido — nada a mais
Falhas de rede	Comportamento com queda de comunicação e no restabelecimento

Dica

Dica: quando o ambiente de produção não pode ser perturbado, monte um **simulador de processo** — uma lógica que reproduz as respostas do controlador, incluindo atrasos e falhas. A prototipação na fase de implementação é o que permite ao cliente ver a tela antes de ela existir em produção, quando mudar ainda é barato.

A aceitação fecha com três entregas que costumam ser esquecidas: **manual de operação** das interfaces, **treinamento** da operação e **plano de manutenção** com rotina de cópia de segurança de configurações, dados e sistema operacional.

19.7 Manutenção: onde o projeto se prova

A fase de manutenção tem duas partes. A **pós-entrega** absorve correções e ajustes solicitados. A **regular** mantém o sistema vivo:

- Cópia de segurança das configurações da aplicação e do sistema operacional, com restauração testada.
- Registro de cada alteração: o que mudou, por que, quem alterou e quando.
- Revisão periódica da lista de alarmes, porque alarme crônico aparece com o tempo (Capítulo 10).
- Revisão de acessos e de usuários, porque gente muda de função e a permissão fica.
- Atualização de *firmware* e de software com janela planejada.

19.8 Estudo de Caso — A IHM que a operação rejeitou

Cenário. Uma indústria contratou a modernização do supervisório de uma linha de envase. A empresa de engenharia tinha um projeto semelhante, de outro cliente, e reaproveitou

80 % das telas. A entrega ocorreu no prazo. Três meses depois, a operação continuava anotando os parâmetros em papel e usando o painel local para comandos.

Diagnóstico. A auditoria de uso mostrou:

1. O vocabulário das telas era o do projeto antigo — nomes de equipamento diferentes dos usados na planta. O operador não reconhecia "TQ-02" onde todos diziam "tanque do xarope".
2. O sinóptico principal exibia 96 objetos com animação, em uma tela de 19 polegadas. Os alarmes prioritários estavam no meio de um painel apertado, sem destaque.
3. A tendência reunia nível, temperatura e vazão na mesma escala percentual: a vazão oscilava, o nível parecia uma reta e a temperatura era invisível.
4. O reconhecimento de alarme exigia três cliques em uma tela de histórico, em vez de um botão no rodapé presente em todas as telas.

Decisão de engenharia.

Problema	Medida	Critério aplicado
Vocabulário alheio à planta	Renomeação com participação dos operadores e da manutenção	Entendimento do processo (etapa 1)
Sinóptico sobrecarregado	Divisão em três telas por área, com no máximo 30 objetos animados cada	Limite da capacidade humana de processar informação
Tendência inútil	Grupos por dinâmica, com escalas separadas e poucas penas por gráfico	Parâmetros de tendência (Capítulo 16)
Reconhecimento difícil	Barra de alarmes no rodapé de todas as telas, com o último alarme e acesso direto	Consistência e acesso imediato

Resultado. O uso do painel local caiu, e a anotação em papel foi abandonada. O custo da correção foi maior que teria sido o levantamento inicial — 14 semanas de retrabalho contra as duas semanas de entrevistas que não foram feitas.

Limitações do arranjo. A renomeação consumiu tempo de operação e exigiu atualizar procedimentos e treinamento. E o reaproveitamento de telas não é condenável em si: o erro foi reaproveitar **o vocabulário** junto com a estrutura — o que dá certo é reaproveitar o padrão gráfico, mantendo os nomes da planta.

19.9 Resumo

- O projeto falha por **levantamento incompleto**, não por falta de recurso gráfico.
- O ciclo de vida tem **sete fases**, e cada uma termina com um produto verificável.
- **Entendimento do processo, tomada de dados, banco de dados e lista de alarmes** são as etapas que mais evitam retrabalho.
- A tomada de dados tem **limite superior**: variável que ninguém usa encarece e atrasa o sistema.

- Ergonomia tem quatro metas: reduzir sobrecarga, monotonia, cansaço e excesso de informação.
- O ser humano processa da ordem de **quatro informações simultâneas**: o sinóptico deve destacar o que interessa.
- **Navegação** hierárquica com barra fixa, navegação horizontal pelo fluxo e retorno sempre disponível.
- **Padronização por cópia** é o que dá consistência ao layout; nunca trocar a função de um botão de posição entre telas.
- Testar é injetar falha, não abrir tela. Aceitação inclui manual, treinamento e plano de manutenção.
- Manutenção regular é *backup* com restauração testada, registro de mudanças e revisão de alarmes e acessos.

19.10 Questões de Revisão

1. Liste as sete fases do ciclo de vida de um projeto SCADA e o produto que encerra cada uma.
2. Por que a fase de definição é a que mais influencia o custo final? Dê um exemplo de retrabalho causado por deficiência nessa fase.
3. Descreva os três documentos que sustentam o levantamento de dados e o que acontece na falta de cada um.
4. Um cliente pede 3.000 variáveis para uma planta de 12 equipamentos. Como você conduz a discussão sobre a tomada de dados?
5. Explique as quatro metas da ergonomia de IHM e dê uma decisão de projeto para cada uma.
6. O que significa dizer que o ser humano processa cerca de quatro informações simultâneas? Como isso se aplica ao sinóptico principal?
7. Descreva a hierarquia de navegação de um sistema com três áreas e dois equipamentos por área, indicando os elementos fixos de navegação.
8. Por que nunca se deve trocar a função de um botão na mesma posição entre telas? Relacione com um risco de operação.
9. Monte um roteiro de testes com oito itens para aceitação de um supervisório de estação de bombeamento.
10. Um projeto entregou a aplicação sem manual e sem treinamento. Descreva as consequências e o que fazer para recuperar a situação.

19.11 Referências

- MACHADO, R.; PONTES, W.; VIANNA, W. **Livro SCADA — versão para análise**. Material didático do curso (capítulo “Desenvolvimento de um projeto SCADA”: ciclo de vida, ergonomia, critérios práticos e planejamento da IHM). Figuras reproduzidas com crédito.
- VIANNA, W. S. **Sistema SCADA Supervisório**. IFF, 2008 (capítulo “Desenvolvimento de um sistema interface homem-máquina”: planejamento em nove etapas, hierarquia de navegação, desenho de telas, tendências, acesso e segurança, tecnologias web e licenciamento).

- MORAES, C. C.; CASTRUCCI, P. L. **Engenharia de Automação Industrial**. Rio de Janeiro: LTC, 2001 (etapas de planejamento de sistema supervisório citadas na obra de origem).
- KIRSHEN, D. S.; WOLLENBERG, B. F. **Intelligent alarm processing**. IEEE, 1992 (filtragem inteligente de alarmes citada na obra de origem).
- INTERNATIONAL SOCIETY OF AUTOMATION. **ISA-101 — Human-Machine Interfaces for Process Automation Systems** (princípios de projeto de IHM; verificar edição vigente).
- ENGINEERING EQUIPMENT AND MATERIALS USERS ASSOCIATION. **EEMUA 191 — Alarm Systems: A Guide to Design, Management and Procurement**. Londres: EEMUA, 3ª edição, 2013.
- INTERNATIONAL SOCIETY OF AUTOMATION. **ANSI/ISA-18.2 — Management of Alarm Systems for the Process Industries**. Edição vigente de 2016.
- ESTADOS UNIDOS. **NIST SP 800-82 — Guide to Operational Technology (OT) Security** (práticas de manutenção, atualização e gestão de mudanças em ambiente de automação; verificar revisão vigente).

Capítulo 20

IA e Analítica Industrial

Objetivos de Aprendizagem

Ao final deste capítulo, o leitor será capaz de:

- Distinguir as quatro famílias de problema que a analítica industrial resolve.
- Verificar se os dados de uma planta estão prontos para um modelo — antes de contratá-lo.
- Estruturar um projeto de manutenção preditiva e de detecção de anomalia.
- Descrever o ciclo de vida de um modelo industrial, do treino ao retreino.
- Reconhecer os modos de falha de um modelo em operação e os limites do seu uso.

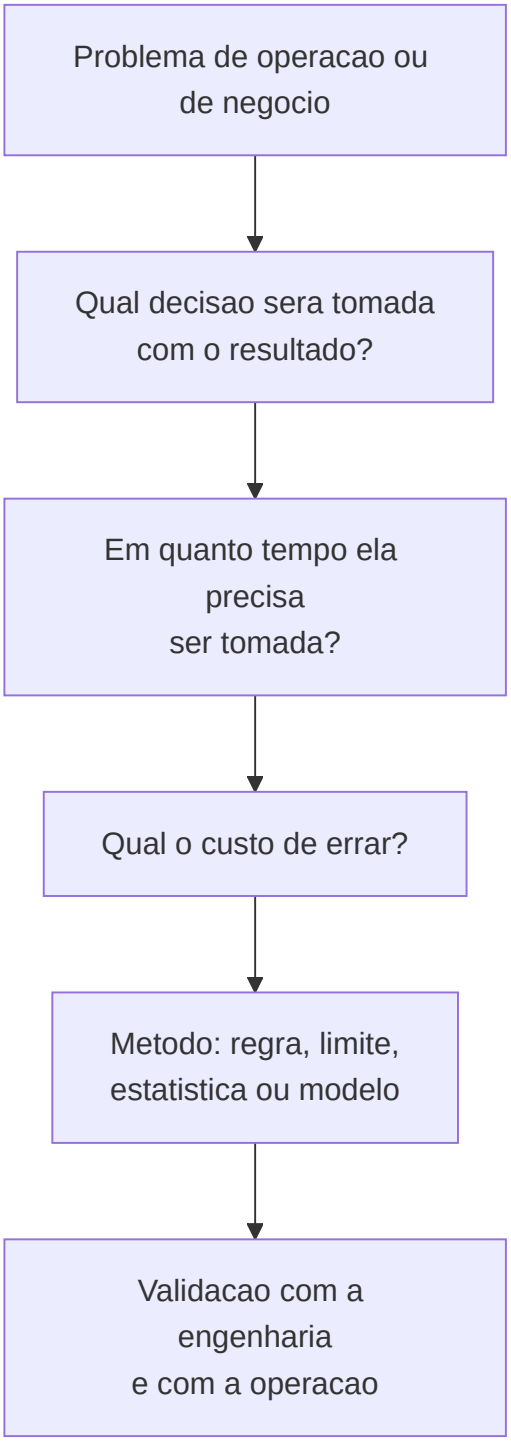
20.1 O que a indústria realmente pede à IA

"Aplicar inteligência artificial" não é um problema de engenharia — é um guarda-chuva. Na prática, os pedidos se organizam em quatro famílias, e cada uma tem pré-requisito diferente:

Tabela 20.1 – As quatro famílias de problema

Família	Pergunta que responde	Exemplo industrial	Pré-requisito principal
Classificação	"Isto é A ou B?"	Peça conforme ou não conforme; motor em falha ou normal	Exemplos **rotulados** de cada classe
Regressão (sensor virtual)	"Quanto vale o que eu não meço?"	Estimar qualidade em linha a partir de variáveis medidas	Relação estável entre entrada e saída
Detecção de anomalia	"Isto está diferente do normal?"	Comportamento atípico de conjunto motobomba	*Baseline* de operação normal, bem definido
Otimização	"Qual o melhor ajuste agora?"	Reduzir consumo de energia mantendo a produção	Modelo do processo e função objetivo acordada

O erro mais comum de projeto é começar pela técnica. A ordem correta é: **definir a decisão** que será tomada com o resultado, o **tempo disponível** para tomá-la e o **custo** de errar. Só então se escolhe o método — que, em boa parte dos casos industriais reais, é uma regra simples com limite bem ajustado.



20.2 O pré-requisito: dado com contexto

Nenhuma técnica compensa dado mal formado. Antes de qualquer modelo, quatro verificações:

1. **Tag com significado completo** — nome estável, unidade, faixa, contexto e qualidade (Capítulos 14 e 15).
2. **Histórico com resolução adequada ao fenômeno** — um modelo de vibração treinado com amostra de 30 s não enxerga nada (Capítulo 16).
3. **Eventos de falha registrados** — com data, hora, equipamento e **causa**. Sem causa registrada, não existe rótulo, e sem rótulo não existe classificação.
4. **Contexto operacional** — regime de carga, produto sendo fabricado, turno, temperatura ambiente. Um modelo treinado em um regime e aplicado em outro aprendeu a reconhecer o regime, não a falha.

Atenção

Atenção: o gargalo mais frequente não é o algoritmo, é a **ordem de serviço**. Em muitas plantas, o registro de manutenção diz "bomba 2 — defeito" sem informar qual componente falhou. Sem essa informação, o histórico técnico mais rico da empresa é inútil para aprendizado supervisionado. Corrigir o formulário de manutenção é, literalmente, o primeiro passo do projeto de IA — e o mais barato.

Figura

[**Figura 20.1 – Diagnóstico de prontidão dos dados para analítica**] *Painel de verificação com quatro blocos de semáforo: cobertura de tags (percentual de variáveis com unidade, faixa e qualidade definidas), resolução do histórico (variáveis com taxa de amostragem compatível com o fenômeno), registro de eventos (número de paradas com causa documentada no último ano) e rotulagem (proporção de ocorrências de manutenção com componente identificado). Abaixo, uma lista dos dez ativos prioritários com o bloco que falta em cada um.*

20.3 Manutenção preditiva

A manutenção evolui em níveis, e cada nível exige mais dado que o anterior:

Tabela 20.2 – Níveis de manutenção

Nível	Quando se intervém	Dado necessário	Limitação
Corretiva	Depois da falha	Nenhum	Parada não planejada e dano secundário
Preventiva	Por calendário ou horas de operação	Contador de horas	Troca de componente bom e falha entre intervalos
Baseada em condição	Quando um indicador cruza um limite	Vibração, corrente, temperatura, análise de óleo	Depende de limite bem escolhido e de tendência confiável

Nível	Quando se intervém	Dado necessário	Limitação
Preditiva	Quando a projeção indica falha em janela útil	Série histórica com eventos rotulados	Exige histórico longo e disciplina de registro
Prescritiva	Antes, com recomendação de ação	Modelo de degradação e de custo	Depende de confiabilidade do modelo e de confiança da operação

Os sinais de campo que alimentam a análise são conhecidos e não exigem instrumentação exótica: valor eficaz de vibração e sua decomposição em faixas, temperatura de mancal, corrente do motor e seu desvio do padrão, pressão diferencial, análise periódica de óleo. O que faz diferença não é o sensor, é a **tendência com baseline**: a mesma leitura de vibração significa coisas diferentes em máquinas diferentes e em regimes diferentes.

Dica

Dica: comece pela manutenção **baseada em condição** com limites extraídos do próprio histórico, antes de partir para modelos preditivos. Ela entrega resultado em semanas, exige só o que já existe e — o mais importante — estabelece a disciplina de registro que o modelo preditivo vai precisar depois.

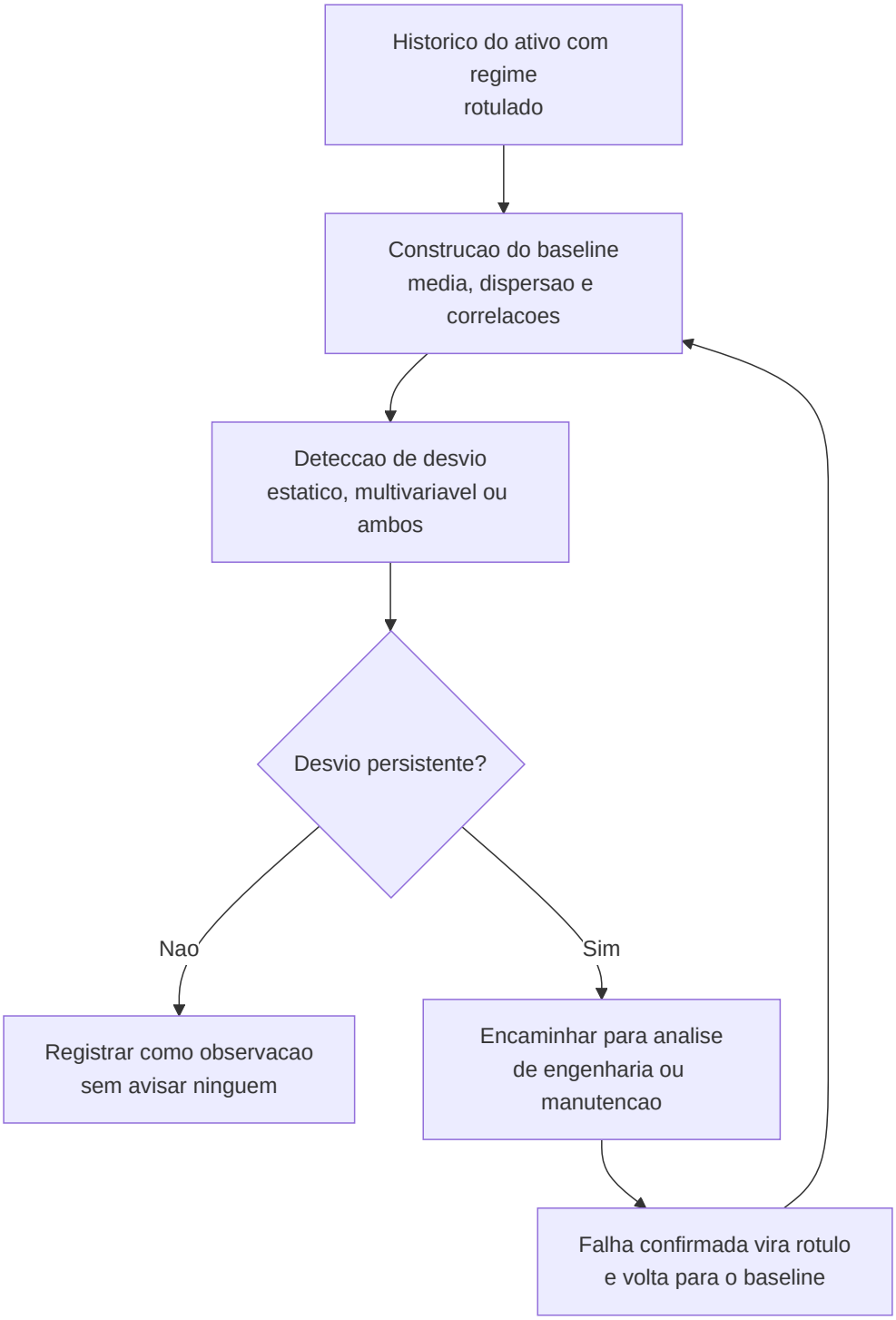
Figura

[Figura 20.2 – Tendência de indicador de condição com desvio progressivo] *Gráfico de linha com a evolução de um indicador de vibração ao longo de seis meses, com faixa de referência sombreada (baseline com dispersão) e uma linha de limite de atenção. No início do período a curva oscila dentro da faixa; a partir de determinado ponto inicia subida gradual, cruza o limite de atenção e continua. Anotar na figura o início da subida, o cruzamento do limite e o instante da intervenção, mostrando a janela de antecedência obtida.*

20.4 Detecção de anomalia

Detecção de anomalia é a técnica mais promissora para a indústria, e a mais fácil de arruinar. Três distinções precisam estar claras:

- **Anomalia não é falha.** Um comportamento diferente pode ser um regime novo, um produto diferente ou uma condição de clima. Anomalia é convite à investigação, não alarme.
- **Limite estático × dinâmico.** Um limite fixo é simples, previsível, auditável e frequentemente suficiente. Um modelo multivariável captura interações que o limite fixo não vê — e cobra o preço em explicabilidade.
- **O inimigo é o falso positivo.** Um detector que avisa dez vezes por dia é desligado na segunda semana. O mesmo raciocínio de racionalização de alarmes do Capítulo 10 se aplica: se ninguém age, o aviso não deveria existir.



O último passo do diagrama é o que separa um projeto vivo de um projeto abandonado: **a falha confirmada volta como rótulo**. Sem esse retorno, o sistema não aprende e o detector continua com o mesmo desempenho do primeiro dia — ou pior, porque o processo mudou.

20.5 Sensor virtual e controle avançado

Duas aplicações de regressão que pagam bem o investimento:

- **Sensor virtual** (*soft sensor*): estimar em tempo real uma variável que só é medida em laboratório ou por analisador de alto custo, a partir de variáveis medidas continuamente. Aplicação típica em qualidade de produto. Requisito: a relação entre entrada e saída precisa ser estável o suficiente, e o modelo precisa saber dizer quando está fora da sua região de validade.
- **Controle avançado**: usar um modelo do processo para calcular a ação considerando restrições e horizonte futuro — o que já era feito, sem aprendizado de máquina, pelo controle preditivo baseado em modelo (MPC). Aqui, a contribuição da analítica está em manter o modelo atualizado e em detectar quando ele deixa de representar o processo.

Em ambos os casos, a saída do modelo **orienta** a decisão; ela não escreve diretamente no atuador sem malha de segurança. Usar uma predição como *setpoint* de proteção é confundir previsão com garantia.

20.6 O ciclo de vida do modelo em operação

Um modelo industrial não termina no treino. Ele tem ciclo de vida, e o ciclo tem modos de falha próprios.

Tabela 20.3 – Etapas do ciclo e os modos de falha

Etapa	O que se faz	Como falha
Preparação	Seleção de variáveis, tratamento de lacunas e de valores ruins	Misturar períodos com configuração diferente sem marcá-los
Treino e validação	Ajuste do modelo e verificação em dados não usados no treino	Vazamento de dados: treinar e validar no mesmo período
Implantação	Publicação do resultado onde a decisão acontece	Modelo certo no lugar errado — resultado em painel que ninguém abre
Monitoramento	Vigilância do erro e das entradas	Ninguém percebe que a variável de entrada mudou de comportamento
Deriva e retreino	Atualização com dado novo	Retreinar sem revisar: o modelo aprende a nova condição anormal como normal

A **deriva** (*drift*) é o modo de falha mais silencioso. Ela acontece por três caminhos: o processo muda (equipamento reformado, produto novo), o sensor muda (descalibração,

substituição por outro modelo) ou a relação muda (a causa deixou de ser a mesma). Nas três, o sintoma na tela é o mesmo: o modelo continua respondendo, com confiança, errado.

Nota

Nota: um modelo em produção é um **ativo com prazo de validade**, e precisa de dono, de indicador de desempenho e de rotina de revisão. Modelo sem dono é passivo técnico: fica na planta até alguém descobrir, em uma investigação de acidente, que a recomendação vinha de um sistema treinado com dados de quatro anos antes.

Figura

[Figura 20.3 – Painel de acompanhamento de um modelo em operação] *Captura de painel com quatro blocos: desempenho do modelo ao longo do tempo (acerto e falsos positivos por mês), distribuição das variáveis de entrada comparada com a do treino, número de recomendações emitidas e quantas geraram intervenção, e lista das últimas recomendações com o desfecho registrado. Destacar o bloco de desvio de entrada, que é o alerta antecipado de deriva.*

20.7 Limites e riscos

Tabela 20.4 – Riscos e a resposta de engenharia

Risco	Descrição	Resposta
Correlação tratada como causa	O modelo aprende que a parada vem junto com um evento que é consequência dela	Validar com engenharia de processo; nunca concluir causa a partir do modelo
Opacidade	Ninguém sabe explicar a recomendação	Preferir modelos interpretáveis quando a decisão for crítica
Viés de registro	Só falhas que já eram observadas estão rotuladas	Reconhecer o limite; não usar o modelo como detector do desconhecido
Uso fora do domínio	Aplicar em equipamento ou regime não representado no treino	Declarar a região de validade e recusar a resposta fora dela
Uso para segurança	Colocar predição no caminho de proteção	Proibido: proteção é do SIS e do intertravamento (Capítulos 8 e 15)
Custo invisível	O modelo melhora o indicador e piora a operação	Medir o efeito na planta, não só a métrica do modelo

Atenção

Atenção: IA e analítica não substituem proteção, intertravamento nem sistema de segurança instrumentado. Um modelo estatístico não tem nível de integridade de segurança, não é certificado para função de proteção e não responde em tempo determinístico. Ele ajuda a decidir; não protege pessoas nem ativos.

20.8 Estudo de Caso — O modelo que acertava e ninguém usava

Cenário. Uma planta de celulose contratou o desenvolvimento de um modelo preditivo de falha para os motores do setor de bombas. O modelo foi entregue com 91 % de acerto na validação, sobre um histórico de três anos de vibração e corrente. A recomendação chegava por painel em nuvem.

Diagnóstico. Seis meses depois, a manutenção não havia mudado uma única intervenção por causa do modelo. Três motivos:

1. **A decisão não passou pelo modelo.** O planejamento da manutenção é semanal; a recomendação chegava por painel, sem entrar na rotina do planejador.
2. **A região de validade não era dita.** Metade dos avisos vinha de dois motores que operavam em regime intermitente, ausente do histórico de treino.
3. **O retorno de rótulo não existia.** As falhas confirmadas nunca voltaram para o modelo, e as substituições preventivas — que evitam a falha — não foram registradas como tal, o que fazia o modelo parecer pior do que era.

Decisão de engenharia.

Problema	Medida
Modelo fora da rotina	Recomendação injetada como item no plano semanal de manutenção, com justificativa e prioridade
Região de validade ignorada	Motores em regime intermitente separados em outro grupo, com baseline próprio
Sem retorno de rótulo	O registro de manutenção passou a exigir causa e componente; o resultado alimenta o retreino
Sem medição de resultado	Indicador de falsos positivos por mês e de acertos que geraram intervenção

Resultado. No semestre seguinte, 11 intervenções foram antecipadas por recomendação do modelo, com redução de paradas não planejadas. O acerto do modelo não mudou — o que mudou foi o sistema em torno dele: a recomendação entrou na rotina, a região de validade foi respeitada e o resultado passou a ser medido.

Limitações do arranjo. O modelo continua dependendo de um baseline revisado a cada mudança de configuração de processo. E os motores em regime intermitente seguem com desempenho inferior, porque o histórico disponível para eles é curto — limitação de dado, não de método.

20.9 Resumo

- A indústria pede quatro coisas distintas: **classificação, regressão, detecção de anomalia e otimização** — cada uma com pré-requisito próprio.
- O projeto começa pela **decisão** a ser tomada e pelo custo de errar, não pela técnica.
- Dado com **contexto** (unidade, faixa, qualidade, regime, causa registrada) é pré-requisito; corrigir o formulário de manutenção costuma ser o primeiro passo.
- A manutenção evolui de corretiva a prescritiva; a **baseada em condição** já entrega resultado e prepara o terreno.
- **Anomalia não é falha**; o inimigo do detector é o falso positivo, e o mesmo raciocínio de racionalização de alarmes se aplica.
- A **falha confirmada precisa voltar como rótulo**, ou o sistema não aprende.
- Sensor virtual e controle avançado orientam a decisão; não escrevem no atuador sem malha de segurança.
- Modelo em produção **tem prazo de validade**, dono, indicador e rotina de revisão — a deriva é o modo de falha mais silencioso.
- **IA não substitui proteção, intertravamento nem SIS**; não tem nível de integridade de segurança.

20.10 Questões de Revisão

1. Descreva as quatro famílias de problema da analítica industrial e dê um exemplo de cada em uma planta de sua escolha.
2. Por que a definição da decisão deve preceder a escolha do método? Ilustre com um caso em que uma regra simples resolve o problema.
3. Liste quatro verificações de prontidão dos dados e explique o que cada uma impede.
4. Por que a ausência de causa no registro de manutenção inviabiliza aprendizado supervisionado? Como corrigir isso na prática?
5. Compare manutenção baseada em condição e manutenção preditiva quanto a dado necessário e limitação.
6. Explique a diferença entre anomalia e falha, e por que o excesso de avisos leva ao abandono do sistema.
7. Descreva o ciclo de vida de um modelo industrial e um modo de falha de cada etapa.
8. O que é deriva de modelo, quais os três caminhos que a produzem e como detectá-la?
9. Explique por que um modelo com 91 % de acerto pode não gerar nenhum benefício. Cite três causas organizacionais.
10. Justifique tecnicamente por que um modelo de aprendizado de máquina não pode substituir o sistema de segurança instrumentado de uma planta.

20.11 Referências

- VIANNA, W. S. **IIoT e suas Tecnologias Aderentes: Teoria e exemplos práticos**. Material didático do curso (Indústria 4.0, inteligência artificial, aprendizado de máquina e tecnologias aderentes).

- VIANNA, W. S. **Introdução a IoT**. Material didático do curso (aprendizado de máquina e análise avançada sobre dados de dispositivos; plataformas de nuvem).
- ORACLE. **O que é IoT** — conceitos de Internet das Coisas, conectividade e análise. Disponível em: `oracle.com/br/internet-of-things`. Consulta: 2026 (referência citada pelo material de origem do curso).
- CISCO. **An Introduction to IoT** (documento de pesquisa traduzido) — fundamentos de IoT e conectividade. Referência citada pelo material de origem do curso.
- INFLUXDATA. **InfluxDB Documentation** — consulta e agregação para análise de séries temporais. Disponível em: docs.influxdata.com. Consulta: 2026.
- GRAFANA LABS. **Grafana Documentation** — painéis de acompanhamento de indicadores e de desempenho de modelos. Disponível em: `grafana.com/docs`. Consulta: 2026.
- THINGSBOARD. **Documentação oficial** — telemetria, atributos e alarmes como fonte de dado para análise. Disponível em: `thingsboard.io/docs`. Consulta: 2026.

Apêndice A

Glossário Técnico — Sistemas SCADA e Supervisórios Industriais

Nota

Como usar: As siglas são expandidas na primeira ocorrência de cada capítulo. Este glossário reúne todos os termos para consulta rápida.

A.1 A

Air Gap
Isolamento físico completo entre uma rede (geralmente OT) e outras redes externas. Historicamente usado em sistemas SCADA críticos para impedir acesso remoto — mas eliminado pela convergência TI/OT.

AMQP (*Advanced Message Queuing Protocol*)
Protocolo de mensageria orientado a filas, similar ao MQTT mas com mais recursos de roteamento. Usado em middlewares enterprise como RabbitMQ.

Analytics Industrial
Conjunto de técnicas de análise de dados aplicadas ao ambiente industrial: estatística descritiva, detecção de anomalias, modelos preditivos.

Alarme
Evento que exige **resposta do operador** dentro de um tempo definido. Se não há ação possível, não é alarme: é registro.

Alarme fixo (*standing alarm*)
Alarme que permanece ativo por longos períodos sem ação possível, geralmente por falha de instrumento ou de configuração.

Alarme obsoleto (*stale alarm*)
Alarme cuja condição já cessou mas que permanece ativo por falha de projeto ou de comunicação.

Alarme fugaz (*fleeting alarm*)
Alarme que dispara e limpa em poucos segundos, normalmente por transitório de partida; combate-se com atraso de confirmação.

Alarm flooding (inundação de alarmes)
Rajada de alarmes em curto intervalo que ultrapassa a capacidade de leitura e de resposta do operador, escondendo a causa raiz.

Anomalia
Comportamento estatisticamente diferente do padrão observado. Não é sinônimo de falha: pode ser regime novo, produto diferente ou condição ambiental. Serve como convite à investigação, não como alarme.

Atributo (ThingsBoard)
Valor de estado associado a uma entidade, em que o último valor substitui o anterior (ao contrário da telemetria, que preserva o histórico). Escopos: servidor, compartilhado e cliente.

Atuador
Dispositivo que converte um sinal elétrico em ação física sobre o processo: válvulas, motores, relés, pistões pneumáticos.

A.2 B

Baseline (linha de base)
Referência de comportamento normal de um ativo ou processo, obtida do próprio histórico, com média, dispersão e correlações. Sem baseline, não há como afirmar que um valor está anormal.

BPCS (*Basic Process Control System*)
Sistema de controle principal de uma planta — geralmente o DCS ou CLP. Distinto do SIS (sistema de segurança).

Broker MQTT
Servidor intermediário no padrão MQTT que recebe mensagens dos publishers e as distribui aos subscribers inscritos nos tópicos correspondentes.

A.3 C

CapEx (*Capital Expenditure*)
Investimento de capital em ativos físicos duráveis — servidores, hardware, licenças perpétuas.

Chattering (alarme repetitivo)
Alarme que dispara e limpa repetidamente em torno do limite, consumindo a atenção do operador; combate-se com faixa morta e atraso de confirmação.

CIA (*Confidentiality, Integrity, Availability*)
Tríade clássica de segurança da informação: Confidencialidade, Integridade e Disponibilidade.

CIP (*Common Industrial Protocol*)
Protocolo de aplicação usado pelo EtherNet/IP e DeviceNet para comunicação industrial Rockwell/ODVA.

Ciclo de vida de um projeto SCADA
Sequência de fases — definição, seleção, projeto, implementação, testes preliminares, aceitação e manutenção — em que cada fase termina com um produto verificável.

CLP (*Controlador Lógico Programável*)
Computador industrial programável para controle de sequências lógicas e processos discretos. Equivalente inglês: PLC (*Programmable Logic Controller*).

Cloud SCADA
Arquitetura em que o software SCADA é hospedado em infraestrutura de nuvem (AWS, Azure, GCP) em vez de servidores físicos na planta.

CoAP (*Constrained Application Protocol*)
Protocolo leve para dispositivos IoT com recursos limitados, similar ao HTTP mas baseado em UDP. Ideal para sensores com baixa energia.

Compressão por exceção (swinging door)
Estratégia de historização que só grava um novo ponto quando o valor se afasta do último gravado mais que uma tolerância definida. A tolerância é o erro máximo da série reconstruída, e o descarte é irreversível.

A.4 D

Contêiner
Forma de empacotar uma aplicação com suas dependências em uma imagem versionada, executável de forma isolada — usada na borda para atualizar aquisição, regras e modelos sem trocar o sistema do gateway.

Dashboard
Painel que reúne indicadores, tendências, estados e alarmes para apoiar decisão. Sem contexto (faixa normal, limite e período) o valor exibido não informa.

DCS (*Distributed Control System* — Sistema de Controle Distribuído*) Sistema integrado de controle distribuído, comum em processos contínuos. Funções de controle, operação e engenharia são organizadas em controladores e estações; topologia, redundância e armazenamento dependem do projeto e do produto.

Deadband (faixa morta)
Diferença necessária entre o valor de disparo e o de limpeza de um alarme, evitando que ele oscile em torno do limite.

Deriva (drift)
Perda de validade de um modelo ou de um baseline em produção, causada por mudança do processo, do sensor ou da relação entre as variáveis. Silenciosa: o sistema continua respondendo, com confiança, errado.

Digital Twin (Gêmeo Digital) Representação digital de um ativo, processo ou sistema, vinculada a dados e modelos do domínio físico. Atualização, sincronização e capacidade de simulação variam conforme o caso; nem todo gêmeo digital opera em tempo real.

DMZ Industrial
Demilitarized Zone — zona de rede intermediária entre a rede corporativa (TI) e a rede industrial (OT), contendo sistemas que precisam trocar dados com ambos os lados (ex.: historiador, servidor web).

DNP3 (*Distributed Network Protocol 3*)
Protocolo de comunicação para sistemas SCADA de infraestrutura crítica (energia, água, óleo e gás). Suporta reporte espontâneo e timestamping de eventos.

DRP (*Disaster Recovery Plan*)
Plano de recuperação de desastres — conjunto de procedimentos para restaurar sistemas críticos após falha grave.

A.5 E

Edge Computing (Computação na Borda)
Processamento de dados realizado próximo à fonte — no próprio dispositivo ou em um gateway de campo — em vez de na nuvem central.

EEMUA 191
Guia de engenharia da EEMUA para projeto, gestão e contratação de sistemas de alarme; 3ª edição, 2013. É guia, não norma, e é referência usual de valores de desempenho.

Escalonamento (de faixa)
Conversão de uma leitura bruta (contagem do conversor A/D) para a unidade de engenharia do instrumento, por regra de três entre os extremos da faixa.

Espaço de endereços (address space)
No OPC UA, a estrutura de nós e referências que o servidor expõe e que o cliente descobre com `Browse` — diferente do mapa fixo de registradores de um protocolo clássico.

Ergonomia
Conjunto de disciplinas que estuda a organização do trabalho na interação entre seres humanos e máquinas. Aplicada à IHM, busca reduzir sobrecarga, monotonia, cansaço e excesso de informação.

ERP (*Enterprise Resource Planning* — Planejamento de Recursos Empresariais*)
Sistema de gestão integrada de negócios: finanças, RH, logística, vendas. Integra-se ao MES via ISA-95.

Estado do equipamento
Condição qualitativa de um ativo (parado, partindo, operando, parando, em falha), modelada explicitamente em vez de deduzida na tela a partir de tags discretos.

EtherNet/IP
Protocolo industrial Ethernet baseado no CIP (*Common Industrial Protocol*), desenvolvido pela ODVA e amplamente usado no ecossistema Rockwell/Allen-Bradley.

Evento
Ocorrência com importância para o operador que **não** exige ação corretiva (partida de equipamento, troca de turno, janela aberta). Distingue-se do alarme, que exige intervenção.

Exatidão
Grau de proximidade entre o valor medido e o verdadeiro. É definida pelo transmissor e pela instalação — não pela resolução do conversor A/D, que apenas determina o tamanho do degrau.

A.6 F

Failover
Comutação automática de um sistema primário para um backup em caso de falha, sem intervenção humana.

FFT (*Fast Fourier Transform*)
Algoritmo de transformada de Fourier para converter um sinal no domínio do tempo para o domínio da frequência — essencial para análise de vibração em manutenção preditiva.

Fieldbus
Barramento digital de comunicação para interligar sensores, atuadores e controladores no nível de campo. Exemplos: Profibus, Foundation Fieldbus, DeviceNet, AS-i.

Fog Computing
Camada de computação intermediária entre o edge (campo) e a nuvem — geralmente um servidor local na planta ou subestação que agrega e pré-processa dados.

Foundation Fieldbus (FF)
Protocolo de fieldbus digital para instrumentação de processo contínuo (petroquímica, refino), com capacidade de executar blocos de função (controle) no próprio instrumento.

A.7 G

Gateway de Protocolo
Dispositivo ou software que converte entre diferentes protocolos de comunicação (ex.: Modbus RTU → OPC UA).

Gateway de borda
Equipamento que materializa a borda de campo: conversa com o processo pela face sul (Modbus, OPC UA, DNP3), normaliza e publica pela face norte (MQTT, REST, Sparkplug B), com fila local para sobreviver à queda do enlace.

Grafana
Plataforma de visualização e monitoramento que consulta fontes externas (séries temporais, bancos SQL, Prometheus). Não armazena o dado de processo e não é sistema de comando.

A.8 H

Hierarquia de navegação
Organização das telas de um supervisório do geral para o particular, com barra de navegação fixa, navegação horizontal pelo fluxo do processo e retorno sempre disponível.

HART (*Highway Addressable Remote Transducer*)
Protocolo que superpõe comunicação digital (FSK — Frequency Shift Keying) sobre o sinal analógico 4-20 mA, permitindo configuração e diagnóstico remoto de instrumentos.

Historiador (historian)
Sistema para armazenar e consultar séries temporais de processo. Pode oferecer compressão por exceção e outras técnicas de redução; distingue-se do histórico de eventos, que registra ocorrências com estado, prioridade e reconhecimento.

HMI (*Human-Machine Interface* — Interface Homem-Máquina*)
Dispositivo ou software que apresenta o estado do processo ao operador e permite intervenções. Pode ser local (painel touchscreen) ou centralizado (estação SCADA).

HA (*High Availability* — Alta Disponibilidade*)
Arquitetura de sistema projetada para minimizar o tempo de inatividade, tipicamente com redundância ativa/passiva ou ativa/ativa.

A.9 I

IEC 61131-3
Norma internacional que especifica linguagens de programação para controladores programáveis. A 4ª edição (2025) mantém LD, FBD, ST e SFC e remove a antiga Instruction List (IL); código legado ainda pode utilizá-la.

IEC 62443
Família de normas para cibersegurança em sistemas de automação e controle industrial (IACS), com requisitos para organizações, sistemas e componentes. Zonas e conduítes apoiam a segmentação; os níveis de segurança-alvo dependem de análise de risco e requisitos do sistema.

IEC 62682
Norma internacional de gerenciamento de alarmes para indústrias de processo, alinhada à ISA-18.2. A 2ª edição foi publicada em 2022; confirme o status e a edição aplicável no catálogo IEC antes de citá-la como vigente.

IHM de alta performance (*High Performance HMI*)
Estilo de projeto de IHM difundido pela ISA-101: fundo neutro, cor reservada a alarme e anomalia, hierarquia de telas navegável, mínimo de elementos decorativos e foco em consciência situacional e tempo de resposta.

IIoT (*Industrial Internet of Things* — Internet Industrial das Coisas*)
Aplicação do paradigma IoT ao ambiente industrial, com requisitos rigorosos de confiabilidade, latência e segurança.

InfluxDB
Banco de dados de séries temporais muito usado com o Grafana. O dado medido entra como *field* e o ativo como *tag* — inverter isso destrói o desempenho.

ISA-18.2
Norma ISA para gerenciamento de sistemas de alarmes em indústrias de processo. A edição citada nesta apostila é a de 2016; confirme no catálogo ISA se há revisão ou reafirmação posterior antes de uso contratual.

ISA-88
Norma ISA para controle de processos em batelada (*batch control*): define modelos de receita, equipamentos e fases de processo.

ISA-95
Norma ISA para integração entre sistemas de controle industrial e sistemas corporativos. O modelo de níveis 0 a 4 distingue processo físico, detecção/atuação, controle/monitoramento, operações de manufatura e planejamento/logística do negócio.

ISA-101
Norma ISA de projeto de interfaces homem-máquina para sistemas de automação de processo. Orienta aspectos como hierarquia de telas, apresentação e interação; não é um software SCADA nem, isoladamente, garante uma IHM de alta performance.

A.10 J–L

Jump Server (Bastion Host)
Servidor intermediário através do qual todos os acessos remotos à rede OT devem passar — nunca acesso direto aos CLPs/SCADA.

KPI (*Key Performance Indicator*)
Indicador-chave de desempenho. Em dashboard industrial, exige fórmula e período explícitos — caso contrário vira número decorativo.

Ladder (LD)
Linguagem de programação de CLP que representa a lógica de controle em forma de diagrama elétrico com contatos e bobinas. É a linguagem mais utilizada na indústria.

LWT (*Last Will and Testament*)
Mecanismo MQTT em que o broker publica automaticamente uma mensagem predefinida quando um cliente desconecta inesperadamente — usado para detecção de falha de dispositivos IIoT.

A.11 M

Máquina de estados
Modelo que descreve os estados possíveis de um equipamento ou de um procedimento e as transições permitidas entre eles, com o evento que provoca cada transição.

Mapa de memória (tabela de alocação)
Documento que relaciona cada tagname do supervisório ao endereço real do hardware de aquisição/controle e ao driver de comunicação responsável pela leitura.

Manutenção preditiva
Nível de manutenção em que a intervenção é programada a partir da projeção de falha em janela útil. Exige histórico longo com eventos rotulados; a baseada em condição registra o desvio e o limite que o caracteriza.

Média móvel
Filtro digital que suaviza o ruído calculando a média das n últimas amostras; reduz o ruído na proporção da raiz quadrada de n, ao custo de atrasar a resposta do sinal.

MES (*Manufacturing Execution System* — Sistema de Execução da Manufatura*)
Sistema que conecta o chão de fábrica ao ERP: rastreabilidade de lotes, qualidade, sequenciamento de ordens, OEE.

MFA (*Multi-Factor Authentication*)
Autenticação multifator — exige dois ou mais fatores de verificação (senha + token + biometria) para acesso a sistemas críticos.

Modbus
Protocolo de aplicação industrial, criado pela Modicon em 1979. Modbus RTU e ASCII são variantes seriais; Modbus TCP transporta mensagens Modbus sobre TCP/IP e não encapsula simplesmente o quadro RTU. O alcance depende da rede.

MPC (*Model Predictive Control*)
Estratégia de controle avançado que utiliza um modelo matemático do processo para prever comportamento futuro e calcular a ação de controle ótima considerando restrições.

MTU (*Master Terminal Unit*)
Servidor central de um sistema SCADA responsável pelo polling das RTUs, banco de dados em tempo real, processamento de alarmes e interface com os clientes HMI.

MQTT (*Message Queuing Telemetry Transport*)
Protocolo leve de mensageria baseado em publicação/assinatura (pub/sub), amplamente utilizado em IIoT. Opera sobre TCP/IP na porta 1883 (ou 8883 com TLS).

A.12 N

NAMUR NA 102
Recomendação da NAMUR (2003) para gerenciamento de alarmes na indústria química e farmacêutica; referência histórica que influenciou ISA-18.2 e IEC 62682.

Node-RED
Plataforma open-source de programação visual *flow-based* para integração de dispositivos, APIs e serviços. Mantida pela OpenJS Foundation.

NTP (*Network Time Protocol*)
Protocolo de sincronização de relógios em rede, com precisão típica de milissegundos. Pré-requisito para comparar carimbos de tempo entre controlador, supervisório e historian.

A.13 O

OEE (*Overall Equipment Effectiveness*)
Indicador composto de eficiência global de equipamentos = Disponibilidade × Desempenho × Qualidade. Padrão SEMI E10.

OpEx (*Operational Expenditure*)
Despesas operacionais recorrentes — energia, manutenção, assinaturas de software, salários.

OPC (*OLE for Process Control*)
Padrão de interoperabilidade entre softwares SCADA e CLPs no ambiente Windows, baseado em COM/DCOM. Predecessor do OPC UA.

OPC UA (*OPC Unified Architecture*)
Arquitetura de interoperabilidade independente de plataforma, com serviços e modelos de informação. Pode usar certificados X.509 e modos de assinatura e criptografia; os recursos dependem da política e da configuração. A porta 4840 é convencional para `opc.tcp`.

OT (*Operational Technology*)
Hardware e software que detecta ou causa mudanças por meio do monitoramento e/ou controle direto de equipamentos físicos, processos e eventos industriais. Contrasta com TI (*Information Technology*).

A.14 P–Q

PAM (*Privileged Access Management*)
Solução de gestão de acessos privilegiados — registra, audita e controla sessões de usuários com permissões elevadas em sistemas críticos.

PID (*Proportional-Integral-Derivative*)
Algoritmo de controle realimentado amplamente utilizado na indústria para manter uma variável de processo no setpoint desejado.

Polling
Método de comunicação em que o mestre (MTU/SCADA) interroga periodicamente cada escravo (RTU/CLP) para coletar dados — contrasta com o reporte espontâneo do DNP3.

Prioridade de alarme
Urgência da resposta do operador, derivada da consequência e do tempo disponível. Não confundir com severidade, que descreve a consequência da condição anormal.

PROFINET
Protocolo Ethernet industrial da Siemens/PI (PROFIBUS & PROFINET International), sucessor do Profibus. Suporta RT (Real-Time) e IRT (Isochronous Real-Time) para motion control.

Profibus
PROFIBUS é uma família de fieldbus serial; DP é usado em automação e PA em instrumentação de processo. Velocidade, topologia e uso em área classificada dependem da variante e do projeto do barramento.

Provisionamento (as code)
Configuração de dashboards, fontes de dados e alertas por arquivo versionado, em vez de ajustes manuais pela interface — o que dá auditabilidade e repetibilidade.

PTP (*Precision Time Protocol*, IEEE 1588)
Protocolo de sincronização com precisão de sub-microssegundo em rede local, usado quando o carimbo de tempo na origem precisa reconstruir a sequência de eventos.

Qualidade do dado (StatusCode)
Código associado a cada valor lido que informa se a leitura é válida (`Good`), incerta (`Uncertain`) ou inválida (`Bad`). Deve ser exibido na tela em vez de substituído por zero.

QoS (*Quality of Service*)
Nível de entrega de mensagens no MQTT: 0 (no máximo uma vez), 1 (pelo menos uma vez) e 2 (intercâmbio QoS 2). QoS 2 não garante execução exatamente uma vez de um efeito externo da aplicação.

A.15 R

Racionalização de alarmes
Processo de documentar, alarme por alarme, causa, consequência, ação esperada, tempo de resposta e prioridade. Alarme sem ação esperada não foi racionalizado.

Registrador de primeira falha (RPE)
Lógica no controlador que grava o código do primeiro sensor a atuar em um endereço de memória e bloqueia as gravações seguintes, preservando a causa raiz em meio a uma avalanche de eventos.

Resolução de entrada analógica
Menor degrau que o conversor A/D consegue representar, dado pelo número de bits da faixa. Não deve ser confundida com exatidão nem com ruído.

Retenção (política de)
Conjunto de regras que define por quanto tempo cada classe de dado é mantida e com que resolução, prevendo agregação, cópia de segurança e descarte planejado.

Rotulagem
Registro da causa e do componente em cada ocorrência de manutenção. É o que transforma o histórico de operação em dado utilizável para aprendizado supervisionado.

RPC (*Remote Procedure Call*)
Chamada de procedimento remoto. Em plataformas IIoT, permite que a plataforma envie comandos ao dispositivo (lado do servidor) ou que o dispositivo solicite algo à plataforma (lado do dispositivo).

RTDB (*Real-Time Database*)
Banco de dados em tempo real — mantido na memória do servidor SCADA com o estado atual de todas as variáveis do sistema.

RTU (*Remote Terminal Unit*)
Dispositivo de campo que coleta dados de sensores, executa lógicas simples e transmite ao servidor SCADA (MTU) via protocolo de comunicação.

Rule chain (cadeia de regras)
No ThingsBoard, sequência visual de nós por onde cada mensagem passa: persistir telemetria, salvar atributos, criar e limpar alarmes, publicar em sistemas externos.

A.16 S

SCADA (*Supervisory Control and Data Acquisition*)
Sistema de controle supervisório e aquisição de dados — monitora e controla remotamente processos industriais e infraestruturas críticas a partir de uma central de operações.

Sensor
Dispositivo que mede uma grandeza física do processo (temperatura, pressão, nível, vazão, posição) e a converte em sinal elétrico padronizado (4-20 mA, 0-10 V, pulso, digital).

Sensor virtual (soft sensor)
Modelo que estima em tempo real uma variável medida apenas em laboratório ou por analisador caro, a partir de variáveis medidas continuamente. Precisa declarar a sua região de validade.

Série temporal
Conjunto de medições indexadas no tempo. Exige carimbo de tempo confiável: sem o *timestamp* do dispositivo, dado atrasado é gravado como atual.

Severidade
Consequência da condição anormal se ninguém agir. Dimensão distinta da prioridade, que trata da urgência da resposta.

SFC (*Sequential Function Chart*)
Linguagem de programação de CLP (IEC 61131-3) baseada em estados e transições — ideal para controle de sequências e processos batch.

Shelving (silenciamento temporário)
Ocultação manual de um alarme por prazo definido e com justificativa registrada. Diferente da supressão, que é definida em projeto.

SIS (*Safety Instrumented System*)
Sistema de segurança funcional que monitora condições críticas do processo e executa ações de proteção automática (shutdown) para evitar acidentes. Normatizado pela IEC 61511.

SL (*Security Level*)
Nível de segurança definido pela IEC 62443: SL1 (proteção contra causas não intencionais) a SL4 (proteção contra atores estatais).

SOE (*Sequence of Events*)
Registro de sequência de eventos com carimbo de tempo na origem, com resolução de milissegundo ou melhor. Exige relógios sincronizados (NTP ou PTP) em todos os nós.

Sparkplug B
Especificação sobre MQTT com esquema de dados padronizado para IIoT industrial — define payloads, timestamps, estados de sessão e gerenciamento de ativos.

StatusCode (OPC UA)
Código que acompanha cada valor lido, indicando se a leitura é válida ou se a qualidade está comprometida. Ignorá-lo faz um sensor com defeito aparecer como "valor zero".

Store and forward
Armazenamento local do dado durante a queda do enlace, com reenvio posterior preservando o carimbo de origem. É o que evita buracos na série temporal justamente nos períodos de problema.

Supressão (suppression)
Ocultação temporária de um alarme por projeto, enquanto a causa que o tornaria irrelevante está ativa (por exemplo, durante o enchimento de um tanque).

A.17 T

Tag
Identificador único de uma variável de processo em um sistema SCADA ou historiador (ex.: `PI-1001` para pressão no ponto 1001). É o nome pelo qual CLP, supervisório, histórico, relatórios e nuvem se referem ao mesmo dado.

Tag de I/O (de comunicação)
Tagname cujo valor é atualizado pelo servidor de comunicação (*driver*), que lê e escreve a memória do hardware de controle. Exige endereçamento informado no cadastro.

Tag interno (de memória ou RAM)
Tagname cujo valor é atualizado no próprio software de supervisão — cálculos, contadores e estado da aplicação —, sem vínculo de atualização com outro software.

Taxa de amostragem
Intervalo entre duas amostras consecutivas de uma variável. Deve seguir a dinâmica do processo: subamostrar produz uma curva que não corresponde à variação real.

Taxa de varredura (scan rate)
Frequência com que o mestre interroga os escravos em uma comunicação por *polling*; define o atraso máximo entre a mudança no campo e sua chegada ao supervisório.

Telemetria
Série temporal publicada por um dispositivo (temperatura, pressão, vazão): cada valor é um ponto no tempo e o histórico é preservado.

Tendência real e tendência histórica
A real exibe os últimos valores a partir de um vetor em memória, com janela limitada; a histórica recupera valores gravados em disco e responde por qualquer período retido.

ThingsBoard
Plataforma de código aberto para coleta, processamento, visualização e gestão de dados de dispositivos IoT/IIoT, com motor de regras, alarmes e RPC.

TI (*Tecnologia da Informação*)
Sistemas computacionais para gestão de informações empresariais: email, ERP, bancos de dados corporativos. Contrasta com OT.

TLS (*Transport Layer Security*)
Protocolo de criptografia para comunicações seguras em rede — sucessor do SSL. Usado para proteger conexões OPC UA, MQTT e HTTPS.

TSDB (*Time Series Database*)
Banco de dados otimizado para séries temporais: índice temporal nativo, compressão por série, agregação contínua e política de retenção própria.

A.18 V–Z

Unidade de engenharia (EU)
Grandeza física em que a variável é apresentada ao operador — graus Celsius, metros cúbicos por hora, percentual —, resultante do escalonamento da faixa bruta do conversor.

Validação de faixa
Tratamento que rejeita leituras fisicamente impossíveis (fora da faixa do sensor) e as marca com qualidade ruim, em vez de apresentá-las como valor válido.

VPN (*Virtual Private Network*)
Rede privada virtual que cria um túnel criptografado sobre a internet pública para comunicação segura entre sites remotos ou usuários e a rede corporativa/industrial.

WirelessHART
Extensão sem fio do protocolo HART, baseada em IEEE 802.15.4 e malha de rádio (*mesh*). Permite adicionar comunicação sem fio a instrumentos de processo industriais.

Zero Trust
Modelo de segurança que pressupõe que nenhuma entidade (usuário, dispositivo ou rede) é confiável por padrão — toda conexão deve ser autenticada e autorizada explicitamente.

Referências